Anatomy of Tor

The Anonymity Protocol, Byte by Byte

Alessandro Greco

First edition — version 1.0.0

Anatomy of Tor — The Anonymity Protocol, Byte by Byte
First edition, version 1.0.0. English translation.

Copyright © 2026 Alessandro Greco.

This work is free: the text, figures, LaTeX sources and laboratory code may be redistributed and modified under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 or (at your option) any later version (GPL-3.0-or-later). The work is distributed in the hope that it will be useful, but *without any warranty*, not even the implied warranty of merchantability or fitness for a particular purpose; see the licence for details, `https://www.gnu.org/licenses/gpl-3.0.html`.

The editable source form (LaTeX sources, TikZ figures, laboratory code and network) is published at `https://github.com/aleff-github/onion-routing-anatomy`.

The book is distributed free of charge; the rights to use and redistribute it are those granted by the licence above.

This translation was generated entirely with artificial-intelligence tools from the original Italian edition, which remains the reference text of the work.

"Tor" and the Tor logo are trademarks of The Tor Project, Inc. This book is an independent work and is not endorsed or sponsored by the Tor Project.

Product and service names mentioned belong to their respective owners and are used for descriptive purposes only.

Contents

Preface **xi**

I History and Foundations **1**

1 Anonymity, Surveillance, and Metadata **3**
1.1 Content and Metadata . . . 3
1.1.1 What Metadata Reveals . . . 3
1.2 Traffic Analysis . . . 4
1.3 A Precise Vocabulary . . . 4
1.4 Who Needs Anonymity? . . . 5
1.4.1 Anonymity Loves Company . . . 6

2 The Roots: Mix Networks, Remailers, Crowds, and Freedom **9**
2.1 Chaum's Mix (1981) . . . 9
2.2 High and Low Latency . . . 10
2.3 Remailers . . . 10
2.4 Toward Low Latency . . . 11

3 From NRL Onion Routing to the Tor Project **13**
3.1 At the Naval Research Laboratory . . . 13
3.2 Three Generations . . . 13
3.3 The Tor Network . . . 15
3.4 An Organization and a Community . . . 15

4 Tor's Threat Model **17**
4.1 Goals . . . 17
4.2 Non-Goals . . . 18
4.3 The Adversary . . . 18
4.3.1 Confirmation and Analysis . . . 18
4.4 What Tor Protects, and What It Does Not . . . 19

II The Low-Level Protocol **21**

5 Essential Cryptography for Tor **23**
5.1 Five Tasks . . . 23
5.2 Ciphers . . . 24
5.2.1 AES in Counter Mode . . . 24
5.2.2 TLS . . . 24
5.3 Hash Functions . . . 24

5.4 MACs and Key Derivation . . . 25
5.4.1 HMAC . . . 25
5.4.2 Key Derivation Functions . . . 25
5.5 Key Exchange: X25519 . . . 26
5.6 Signatures: Ed25519 and RSA . . . 26
5.7 Recent Cryptography . . . 26

6 Network Architecture **29**
6.1 Roles . . . 29
6.2 Protocol Layers . . . 30
6.3 A Web Page from Start to Finish . . . 31
6.4 Implementations and Versions . . . 31
6.4.1 Two Implementations . . . 31
6.4.2 Subprotocols . . . 32

7 Channels and the Link Protocol **35**
7.1 What Is a Channel? . . . 35
7.2 A Relay's Identity . . . 36
7.3 The Handshake at a Glance . . . 37
7.4 The TLS Layer . . . 37
7.5 CERTS: Proving Identity . . . 39
7.5.1 The Cell . . . 39
7.5.2 Tor's Ed25519 Certificate . . . 39
7.5.3 Verification . . . 40
7.5.4 The Legacy RSA Chain . . . 41
7.5.5 What a Relay Actually Sends . . . 41
7.6 AUTH_CHALLENGE and AUTHENTICATE . . . 42
7.6.1 Who Authenticates? . . . 42
7.6.2 The Challenge . . . 42
7.6.3 The Response . . . 42
7.7 NETINFO: Time and Addresses . . . 43
7.8 After the Handshake . . . 44
7.9 Laboratory: Opening a Verified Channel . . . 44

8 Cells **49**
8.1 Why Cells? . . . 49
8.2 From the TLS Stream to Cells . . . 50
8.3 Cell Formats . . . 50
8.3.1 Constants . . . 50
8.3.2 Fixed-Length Cells . . . 51
8.3.3 Variable-Length Cells . . . 51
8.3.4 Distinguishing the Two Formats . . . 51
8.3.5 The Special Case of VERSIONS . . . 52
8.4 CircID . . . 52
8.4.1 Who Chooses the CircID? . . . 53
8.5 Cell Commands . . . 53
8.5.1 DESTROY . . . 54
8.5.2 RELAY_EARLY . . . 55
8.6 Inside RELAY Cells . . . 55
8.6.1 Recognized and Digest: A Two-Stage Filter . . . 56
8.6.2 Relay Message Padding . . . 56

8.6.3 Relay Commands . . . 56
8.7 Toward a New Format: Proposal 340 and CGO . . . 56
8.8 Anatomy of a Session . . . 58
8.9 Laboratory: A Cell Encoder . . . 60

9 Circuit Construction: TAP, CREATE_FAST, ntor, and ntor-v3 65
9.1 One Hop at a Time . . . 65
9.2 Cells and Messages . . . 66
9.2.1 CREATE2 and CREATED2 . . . 66
9.2.2 EXTEND2 and EXTENDED2 . . . 67
9.3 What a Handshake Must Achieve . . . 67
9.4 TAP: The First Handshake . . . 68
9.5 CREATE_FAST: When a Public Key Is Unnecessary . . . 68
9.6 ntor . . . 68
9.6.1 The Ingredients . . . 69
9.6.2 The Protocol . . . 69
9.7 ntor-v3 . . . 69
9.7.1 What Changes . . . 70
9.7.2 The Protocol . . . 70
9.7.3 Extensions . . . 70
9.7.4 Which Handshake Does tor Use Today? . . . 71
9.8 From Negotiated Secrets to Circuit Keys . . . 71
9.9 Laboratory: Building Real Circuits . . . 71

10 The Onion: Relay-Cell Encryption and CGO 77
10.1 The Traditional Scheme . . . 77
10.1.1 Forward: From the Client into the Circuit . . . 78
10.1.2 Backward: From the Circuit to the Client . . . 78
10.1.3 The Circuit as a “Leaky Pipe” . . . 78
10.2 Recognizing a Cell . . . 78
10.3 Weaknesses of tor1 . . . 79
10.3.1 Malleability . . . 79
10.3.2 The Tagging Attack . . . 79
10.3.3 Other Limitations . . . 80
10.4 Counter Galois Onion . . . 80
10.4.1 Properties . . . 80
10.4.2 How It Works, at a High Level . . . 80
10.4.3 CGO in Practice . . . 81
10.5 Laboratory: A Three-Hop Circuit . . . 81

11 Streams, SENDME, and Congestion Control 85
11.1 Streams . . . 85
11.1.1 Opening a stream: BEGIN and CONNECTED . . . 85
11.1.2 Transferring data: DATA . . . 87
11.1.3 Closing a stream: END . . . 87
11.1.4 DNS at the exit . . . 87
11.1.5 Directory streams: BEGIN_DIR . . . 87
11.2 Original Flow Control . . . 87
11.2.1 The problem . . . 87
11.2.2 Windows and SENDME . . . 88
11.2.3 Authenticated SENDME messages . . . 88

11.3 Congestion Control . . . 88
11.3.1 The signal: round-trip time . . . 88
11.3.2 The algorithms . . . 89
11.3.3 The parameters: why 31? . . . 89
11.3.4 No more stream windows: XON and XOFF . . . 89
11.4 Laboratory: Downloading a File Through Tor . . . 90

12 The Directory System **93**
12.1 The Problem . . . 93
12.2 The Participants . . . 94
12.3 The Documents . . . 94
12.4 Voting . . . 95
12.4.1 The schedule . . . 95
12.4.2 Combining the votes . . . 96
12.4.3 Flags . . . 96
12.4.4 Bandwidth . . . 96
12.5 Anatomy of a Consensus . . . 97
12.6 Verifying a Consensus . . . 98
12.7 How a Client Stays Up to Date . . . 99
12.8 Laboratory: Downloading and Verifying a Consensus . . . 99

13 Path Selection, Guards, and Vanguards **103**
13.1 Order and Constraints . . . 103
13.2 Weighting by Bandwidth . . . 104
13.2.1 Why weighting matters . . . 104
13.2.2 Circuit positions are not equivalent . . . 104
13.2.3 Weights on the real network . . . 104
13.3 Guards . . . 105
13.3.1 The problem they solve . . . 105
13.3.2 Three, one, two . . . 105
13.3.3 The algorithm . . . 106
13.4 Vanguards . . . 107
13.5 Laboratory: Examining the Public Network . . . 108

14 Onion Services v3 **111**
14.1 What an Onion Service Provides . . . 111
14.2 The Address . . . 112
14.3 Blinded Keys . . . 113
14.3.1 The problem . . . 113
14.3.2 The solution . . . 113
14.3.3 Credentials and offline keys . . . 113
14.4 Locating the Descriptor . . . 114
14.4.1 Time periods . . . 114
14.4.2 The HSDir ring . . . 114
14.4.3 Anonymous requests only . . . 115
14.5 The Descriptor . . . 115
14.6 Introduction and Rendezvous . . . 116
14.7 Restricted Discovery . . . 117
14.8 Defenses . . . 117
14.9 Laboratory: Finding and Reading a Descriptor . . . 118

15 DoS Defenses and Resource Management **121**
15.1 Why DoS Is an Anonymity Problem . . . 121
15.2 Physical Resources . . . 122
15.2.1 Memory . . . 122
15.2.2 CPU . . . 122
15.2.3 Bandwidth . . . 122
15.3 Per-Address Limits . . . 122
15.3.1 The code and the consensus . . . 123
15.4 Onion Services Under Attack . . . 123
15.4.1 Limits at introduction points . . . 123
15.4.2 Proof of work . . . 124
15.4.3 Linear effort . . . 125
15.4.4 Controlling effort . . . 125
15.5 Laboratory: Triggering the Defenses . . . 125

16 Censorship, Bridges, and Pluggable Transports **129**
16.1 How Tor Is Blocked . . . 129
16.2 Bridges . . . 130
16.2.1 The distribution problem . . . 130
16.2.2 Lox . . . 131
16.3 Pluggable Transports . . . 131
16.3.1 The architecture . . . 131
16.3.2 The protocol between tor and the PT . . . 131
16.3.3 Lyrebird . . . 132
16.4 Strategies . . . 132
16.4.1 obfs4: looking like nothing recognizable . . . 132
16.4.2 meek: behind a large service . . . 133
16.4.3 Snowflake: many ephemeral proxies . . . 133
16.4.4 WebTunnel: looking like a website . . . 133
16.4.5 Conjure: hiding in the address space . . . 133
16.5 Laboratory: An obfs4 Bridge Through a Censor's Eyes . . . 134

III Tor in Practice **137**

17 Installation and torrc **139**
17.1 Which Tor Should You Install? . . . 139
17.2 The torrc File . . . 140
17.2.1 Check Before Use . . . 140
17.3 A Client Configuration . . . 140
17.3.1 Stream Isolation . . . 141
17.3.2 Choosing Relays: Exercise Caution . . . 141
17.3.3 Other Ports . . . 142
17.4 The Control Port . . . 142
17.5 Logs . . . 142

18 Control Port and Stem **145**
18.1 The Control Protocol . . . 145
18.1.1 Circuits as Seen by tor . . . 146
18.1.2 The Main Commands . . . 147
18.2 Stem . . . 147

18.3 Laboratory: Live Stream Isolation . . . 148

19 A Private Tor Network: Chutney and Shadow 151
19.1 Why a Private Network? . . . 151
19.2 Emulation or Simulation? . . . 152
19.3 The Book's Laboratory . . . 152
19.3.1 Requirements . . . 153
19.3.2 Building the Image and Starting the Network . . . 153
19.4 Network Topology . . . 154
19.5 A Test Network Is Not the Real Network . . . 155
19.6 Inside a Node's Directory . . . 156
19.7 Using the Network . . . 156
19.7.1 Browsing Through the Private Network . . . 156
19.7.2 Reaching the Onion Service . . . 157
19.7.3 Observing Circuits with Stem . . . 157
19.7.4 Working Inside the Container . . . 158
19.8 First Contact with a Relay . . . 158
19.9 Shadow: A Network Inside a Simulator . . . 159

20 Observing Traffic: Wireshark and Logs 163
20.1 What an Observer Can See . . . 163
20.2 The Tools . . . 164
20.2.1 Decrypting Your Own TLS . . . 164
20.3 Laboratory: Cells Inside TLS . . . 165
20.4 tor Logs . . . 166

21 Publishing an Onion Service 169
21.1 Your First Onion Service . . . 169
21.1.1 Avoid Revealing the Server . . . 170
21.2 Onion-Location . . . 170
21.3 Restricted Discovery . . . 171
21.4 Other Requirements . . . 173
21.4.1 Vanity Addresses . . . 173
21.4.2 Single-Hop Services . . . 173
21.4.3 Onionbalance . . . 173

22 Running a Relay or a Bridge 175
22.1 Which Type of Node? . . . 175
22.2 Essential Configuration . . . 176
22.2.1 The Life of a New Relay . . . 176
22.2.2 Keys . . . 176
22.3 Exit Policy . . . 177
22.4 Laboratory: Testing an Exit Policy . . . 178
22.5 Relay Families . . . 179
22.6 Running a Bridge . . . 180

23 Tor Browser, Tails, and Whonix 183
23.1 Why a Proxy Is Not Enough . . . 183
23.2 Tor Browser . . . 184
23.2.1 One Circuit per Site . . . 184
23.2.2 Resisting Fingerprints . . . 185

23.2.3 Security Levels . . . 185
23.3 Beyond the Browser . . . 185
23.3.1 Tails . . . 186
23.3.2 Whonix . . . 186

24 Measuring the Network: Tor Metrics and OONI 189
24.1 Data Sources . . . 189
24.2 Counting Anonymous Users . . . 190
24.2.1 Detecting Censorship . . . 191
24.3 Performance . . . 191
24.4 Laboratory: Reading a Real Consensus . . . 191
24.5 OONI . . . 192

25 Legal and Ethical Aspects in Italy and the EU 195
25.1 A Tool and Its Uses . . . 195
25.2 Relay Operators . . . 196
25.2.1 Mere Conduit . . . 196
25.2.2 Civil Liability and Criminal Investigations . . . 196
25.2.3 Data Retention . . . 197
25.2.4 Logs Are Personal Data . . . 197
25.3 Researchers . . . 197

IV Attacks and Defenses 201

26 Traffic Correlation, Fingerprinting, and Sybil Attacks 203
26.1 The Adversary's Position . . . 203
26.2 End-to-End Correlation . . . 204
26.3 Laboratory: Correlating Your Own Traffic . . . 205
26.4 Website Fingerprinting . . . 205
26.5 Sybil: Multiplying Relay Identities . . . 206
26.6 Discovering the Guard . . . 207

27 Real Cases Against the Network 209
27.1 2014: Signaling Through Cell Types . . . 209
27.2 Laboratory: The RELAY_EARLY Limit . . . 210
27.3 2020–2021: KAX17 and Network Health . . . 211
27.4 2022–2023: Denial of Service . . . 211
27.5 Lessons . . . 212

28 Operational Security Mistakes and Deanonymization: Lessons 215
28.1 Four Layers . . . 215
28.2 Application Mistakes . . . 216
28.3 Laboratory: tor Detects a DNS-Leak Warning Sign . . . 216
28.4 Content Mistakes . . . 217
28.5 Behavioral Mistakes: An Anonymity Set That Is Too Small . . . 217
28.6 A Checklist . . . 218

V Research and the Future **221**

29 Academic Research on Tor **223**
29.1 Where Tor Research Is Published . . . 223
29.2 Measuring Anonymity . . . 224
29.3 Four Methods . . . 224
29.4 Laboratory: Time to First Compromise . . . 225
29.5 Conducting Research Safely . . . 227

30 Arti: Tor in Rust **229**
30.1 Why Rewrite tor? . . . 229
30.2 Architecture . . . 230
30.2.1 A Comparison: Writing a Cell . . . 231
30.3 Supported Features . . . 231
30.4 Laboratory: Arti on the Private Network . . . 232

31 Open Problems and Ongoing Projects **235**
31.1 How the Protocol Changes . . . 235
31.2 Quantum Computers . . . 236
31.3 Larger Messages, and Smaller Ones . . . 237
31.4 Performance: Multiple Paths . . . 237
31.5 Scalability: Walking Onions . . . 237
31.6 Other Open Fronts . . . 238
31.7 How to Contribute . . . 238

A Glossary of Tor Terms **241**

B Further Cryptography **245**
B.1 One Curve, Two Forms . . . 245
B.1.1 X25519 Key Clamping . . . 245
B.1.2 The Point at Infinity . . . 246
B.2 From Montgomery to Edwards: The ntor Cross-Certificate . . . 246
B.3 Stream-Cipher Malleability . . . 247

C A Guide to the torspec Specifications **249**
C.1 Where to Find Them . . . 249
C.2 Conventions . . . 249
C.3 Document Map . . . 250
C.4 How to Read a Section . . . 250
C.5 Arti and the Specifications . . . 250

D Exercise Solutions **251**

Bibliography **273**

Index **277**

Preface

Tor is used by millions of people every day, yet it is almost always discussed in terms of its uses, real or presumed: circumventing censorship, protecting sources, the "dark web". This book takes a different approach: it explains how Tor works. It starts with the bytes exchanged by two relays and works its way, chapter by chapter, to design decisions, attacks, real-world cases, and questions that remain open. And it asks readers not to take its word for it: every mechanism described in the technical part is built and tested in a laboratory, on a private Tor network that runs on an ordinary computer.

Origins, Method, and an Invitation to Review

This book draws on study notes and documents I produced over the years, beginning with my bachelor's thesis. To turn those materials into a coherent technical work, I directed the use of artificial-intelligence tools that made substantial contributions to developing the explanations, writing and structuring the text, and creating figures, exercises, code, and laboratories. AI was not merely used to polish my prose, nor does its use amount to independent expert review.

Despite technical checks and my own review, mistakes, simplifications, and outdated information may remain. I invite readers to report problems and propose corrections through the book's repository. I chose to make the work freely available through the project's official channels, with its sources under GNU GPL-3.0-or-later, to support "diffusione dei saperi senza fondare poteri"—the dissemination of knowledge without establishing power. The license governs reuse and redistribution, including commercial redistribution.

Who This Book Is For

This book is for computer science and security students, professionals who encounter Tor in their work, people who operate or want to operate a relay, and researchers. Readers should have a basic understanding of networking (TCP/IP and the idea behind TLS), some familiarity with the command line and, for the laboratories, Python. The necessary cryptography is introduced in chapter 5, with further details in appendix B.

How to Read This Book

The book is divided into five parts. You can read them in order or follow one of these paths:

Historical and conceptual path Part I (Chapters 1–4) and Part V (Chapters 29–31): what anonymity means, where Tor came from, which adversaries it protects against, and where it is heading.

Protocol path Part II (Chapters 5–16), to be read in order alongside the laboratories: this is the core of the book.

Practical path Part III (Chapters 17–25): configuring tor, observing its traffic, publishing an onion service, operating a relay, using Tor Browser, measuring the network, and understanding the legal aspects. Chapter 19 is a prerequisite for all laboratories.

Security path Part IV (Chapters 26–28), preceded by chapter 4: attacks examined through research, real-world cases, and operational mistakes.

Conventions

Each chapter opens with its learning objectives and a *Last verified* line indicating the date and the software and specification versions against which its contents were checked. Tor changes, and readers need to know which version of Tor the text describes. Throughout the book, *Tor* refers to the network and the project, *tor* to the C program, and *Arti* to the Rust implementation. References to specifications take the form [document § section]; protocol change proposals are identified by number. The technical terms used in this edition are defined in appendix A.

Boxes keep the main narrative separate from digressions: *in-depth discussions*, *historical notes*, and *warnings*. Exercises are graded with stars: one for comprehension, two for application, and three for a laboratory task or a small research project; solutions are in appendix D. The laboratories are numbered from 1 to 20. In Part II, they build a small Tor client in Python (`torlab`); the code and laboratory network accompany the book, and an automated test suite can be rerun after updates. All of these resources—the `torlab` package, the twenty laboratories, the Docker test network, and the verification tools—are available in the book's repository: `https://github.com/aleff-github/onion-routing-anatomy`.

An Ethical Note

All experiments in this book take place on the laboratory's private Tor network or use the live network only as an ordinary client, without observing or modifying other people's traffic. This is the principle recommended to researchers by the Tor Research Safety Board, and it is also a legal matter (chapter 25). The chapters on attacks adopt an analytical, defensive perspective: they explain what has been done and how to protect against it, not how to target someone.

Acknowledgments

This book rests on the work of the Tor Project and its community. Open specifications, the source code of `tor` and Arti, protocol change proposals, and documentation provide the basis for every technical claim that follows. Their public availability makes it possible to verify the claims against their sources rather than asking readers to take them on trust. Thanks are also due to the people who develop and maintain the free tools used to write this book and test its laboratories: LaTeX, Python, Docker, Chutney, Wireshark, and the many other projects cited in the text.

Part I

History and Foundations

Chapter 1

Anonymity, Surveillance, and Metadata

Last verified: 23 September 2026 — Tor Metrics data through 20 September 2026

Chapter objectives

By the end of this chapter, you will be able to:

- distinguish confidentiality, anonymity, pseudonymity, unlinkability, and unobservability;
- explain why communication metadata can reveal as much as, and sometimes more than, content;
- describe traffic analysis and explain why encryption alone cannot prevent it;
- identify who uses Tor and why diversity among its users is itself a form of protection.

This book is about a system designed to protect a particular kind of information: not *what* we say online, but *whom* we communicate with. Before examining how the system works, we need to understand the problem it solves, and why that problem differs from the more familiar one addressed by encryption.

1.1 Content and Metadata

When two computers communicate over the Internet, each packet carries two kinds of information. The **content** is what applications exchange: the text of a message or the contents of a web page. **Metadata** describes the communication *itself*: who sends it, to whom, when, how often, and how much data it carries. Some metadata is essential to the network's operation because routers must know where to deliver packets. The destination IP address, and the source IP address to which replies should be sent, travel in plaintext in every packet.

Encryption, now widespread thanks to HTTPS, protects content but leaves metadata visible. Anyone observing the network, from a user's Internet service provider to an operator of a major Internet exchange, can still see that the user connects to a particular website, when the connection occurs, how long it lasts, and how much traffic it generates.

1.1.1 What Metadata Reveals

It is tempting to regard metadata as harmless: "They only know whom I talk to, not what I say." That impression is misleading. A few examples show why:

- someone visits a domestic violence support center's website at two in the morning, then visits the site of a divorce lawyer;

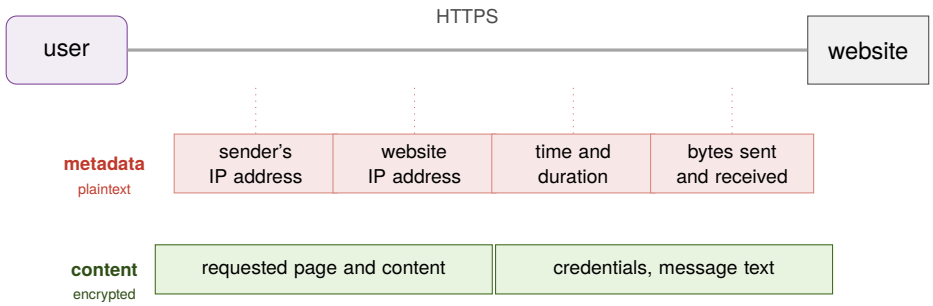


Figure 1.1: What an observer can see on an HTTPS connection. The content is encrypted, but metadata remains visible; protecting this information is precisely what Tor attempts to do.

- in the days before an investigation is published, a journalist exchanges messages with an employee of the ministry the investigation concerns;
- an employee regularly visits a trade union's website, or the site of a patient association for a particular illness.

In none of these cases does anyone need to read the content to learn a great deal. Metadata also has an advantage for those collecting it: it is structured, easy to store and analyze at scale, and can be cross-referenced. General Michael Hayden, a former director of both the NSA and CIA, summarized the point in a remark that became famous during a debate at Johns Hopkins University on April 1, 2014: "We kill people based on metadata", before immediately qualifying it with "but that's not what we do with this metadata" [Jus14]. The qualification does not alter the central point: those collecting metadata regard it as sufficient to inform decisions of that gravity.

1.2 Traffic Analysis

Traffic analysis studies communication metadata to infer information without having to read the content. It is as old as telecommunications. Even during twentieth-century wars, observing how often an enemy radio station transmitted, to whom, and at what pace could reveal the structure of an army without anyone decrypting its messages.

On the Internet, traffic analysis has two main forms:

Address analysis An observer of a link sees the source and destination IP addresses. This is the simplest form, and the one Tor addresses most directly: no single point in the network sees both endpoints.

Volume and timing analysis Even without seeing complete address information, an observer can compare traffic flows by size and timing. If a user downloads 3 MB at 10:15:02 and a server elsewhere in the network sends 3 MB at 10:15:02, the two observations are likely to concern the same communication. This **traffic correlation** is Tor's principal limitation, and we will return to it repeatedly (chapters 4 and 26).

1.3 A Precise Vocabulary

The word "anonymity" is used in several senses, and confusion among them has led to design errors throughout the history of these systems. The scientific literature, particularly the terminology proposed by Pfitzmann and Hansen [PH10], distinguishes several concepts:

Confidentiality An unauthorized party cannot read the content. This is what encryption provides.

Anonymity A subject cannot be identified within a group of subjects, called the **anonymity set**. Anonymity is never absolute: it exists "among" other people. The larger and more diverse the set, the greater the protection.

Pseudonymity A person acts under an identifier other than their own name, potentially a stable one. An account using an invented name is a pseudonym, not a form of anonymity: all its actions can be linked together.

Unlinkability An observer cannot determine whether two items of interest (two messages, a user and a website, or two visits) are related. This is the property Tor tries to provide between users and their destinations.

Unobservability An observer cannot even determine *whether* communication is taking place. This is a much stronger property, which Tor does not provide: an observer watching the user can see Tor being used unless a pluggable transport is employed (chapter 16).

Historical note: Separating identification from routing

The three Naval Research Laboratory researchers who invented onion routing in 1995—David Goldschlag, Michael Reed, and Paul Syverson—preferred not to speak of anonymity. Their goal, as they wrote in a 1996 paper that Syverson later revisited when recalling that period, was to "separate identification from routing" [Syv11]. A user may want to prove their identity to the system they are communicating with, such as their office computer, without every intermediate point in the network learning that someone at the user's location is connecting to that office.

1.4 Who Needs Anonymity?

According to Tor Metrics estimates, between 2.8 and 3.5 million users connected to the Tor network each day in August and September 2026 without using bridges, averaging approximately 3.2 million. From Italy, the estimated daily figure ranged from 44 000 to 83 000, averaging around 60 000. These estimates carry substantial uncertainty because the network estimates users rather than counting them (chapter 24). Who are these users? They belong to very different groups:

- ordinary people who do not want advertisers, Internet service providers, or operators of the networks they use to track them;
- journalists and their sources, activists, and human rights defenders, particularly in countries with authoritarian governments;
- people living under censorship who use Tor to reach blocked websites (chapter 16);
- victims of violence or persecution who need to conceal their location;
- companies that do not want to reveal their research interests to competitors;
- law enforcement and military personnel who need to visit websites or gather information without disclosing their identity: onion routing originated in a US Navy laboratory for precisely this purpose.

A small share of users employ Tor for illegal activities, as happens with any communication tool. Public debate tends to focus on this group. We will examine the subject using evidence and specific cases in Part IV and chapter 25.

1.4.1 Anonymity Loves Company

There is a deeper reason why user diversity matters. An anonymity system used by only one group—the military, for example—would not protect anyone in that group: an observer seeing someone connect to the network would immediately know they were in the military, and anyone observing outgoing traffic would know it came from the military. Syverson makes exactly this point about a network the US Navy could have built exclusively for itself: everything emerging from it would have been recognizable as Navy traffic [Syv11]. Each person's anonymity depends on how many other users there are and how diverse they are. A phrase summarizing this principle, also the title of a paper by Dingledine and Mathewson, is "anonymity loves company" [DM06]. This is why Tor is designed to be available to everyone, free of charge and open: the more people use it, the better it protects everyone.

Summary

- Encryption protects content; metadata (who, with whom, when, and how much) remains visible and can reveal a great deal.
- Traffic analysis infers information from communication addresses, volumes, and timing.
- Anonymity means not being identifiable within a set; it must be distinguished from pseudonymity, unlinkability, and unobservability.
- Tor serves millions of very different users, and that diversity is part of its protection.

Exercises

Exercise 1.1 ⋆ List the metadata an Internet service provider can observe when one of its customers reads an article on an HTTPS news website. Which data remains visible if the customer uses a VPN? What if they use Tor?

Exercise 1.2 ⋆ Classify each of the following as anonymity, pseudonymity, or neither: a social media account under an invented name; an unsigned message left on a public noticeboard; a letter signed with initials; a cash donation placed in a collection box.

Exercise 1.3 ⋆⋆ Over the course of an hour, a website receives visits from 50 different Tor users. One of them posts a comment containing a spelling mistake typical of Italian speakers, and the comment is published at 3:00 a.m. Italian time. How does this change the user's anonymity set? What does this example teach us about the role of user behavior?

Exercise 1.4 ⋆⋆ Compare a commercial VPN with Tor in terms of who can link users to their destinations. Where is trust concentrated in each case?

Further Reading

- A. Pfitzmann, M. Hansen, *A terminology for talking about privacy by data minimization* [PH10]: the reference vocabulary.

- P. Syverson, *A Peel of Onion* [Syv11]: the history of onion routing, told by one of its inventors.
- Tor Metrics, `https://metrics.torproject.org/`: estimated user numbers by country.

Chapter 2

The Roots: Mix Networks, Remailers, Crowds, and Freedom

Last verified: 23 September 2026 — historical sources cited in this chapter

Chapter objectives

By the end of this chapter, you will be able to:

- explain David Chaum's idea of a *mix* and why it underpins anonymity systems;
- distinguish high-latency systems for email from low-latency systems for web browsing;
- trace the evolution of remailers from anon.penet.fi to Mixminion;
- place Crowds, PipeNet, and Freedom in the history leading to Tor.

Tor did not appear out of nowhere. It is, for now, the latest stage in more than forty years of research and deployed systems. Some succeeded and many failed, but each taught us something. This chapter traces the major developments up to the emergence of onion routing.

2.1 Chaum's Mix (1981)

In 1981, David Chaum published a five-page paper that would establish an entire field of research: *Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms* [Cha81]. The problem concerned email: how could a message be sent without an observer linking its sender to its recipient, even if the entire network were under surveillance?

The solution was the **mix**: a server that receives messages encrypted with its public key, decrypts them, collects them in a group (a *batch*), and forwards them together in an order different from their arrival order. Three techniques prevent an incoming message from being linked to an outgoing one:

- an outgoing message looks completely different from its incoming counterpart because the mix has removed a layer of encryption;
- all messages have the same length;
- messages are shuffled and released together, so even their timing does not link them.

Rather than trust just one mix, a sender can pass the message through a **chain** of mixes, encrypting it in layers under each mix's key. Every mix removes its own layer and learns only the identity of the next mix. If even one mix in the chain is honest, the link remains concealed. This is the seed of the onion-routing idea.

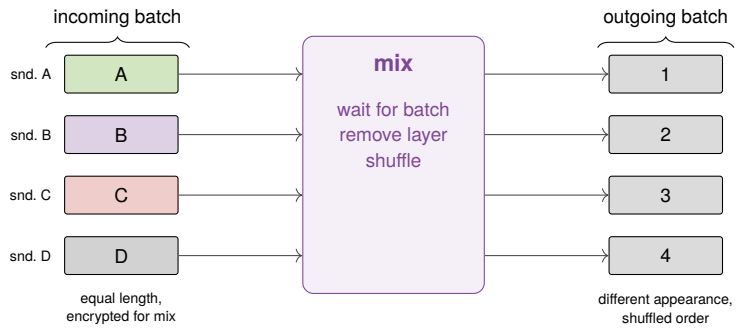


Figure 2.1: A Chaumian mix. An observer watching both input and output cannot link messages: all have the same length, their appearance changes when the mix removes an encryption layer, and they leave together in a different order.

In the same paper, Chaum proposed *anonymous return addresses*, allowing recipients to reply without knowing the sender, as well as digital pseudonyms. In 1988 he added a second mechanism, *DC-nets* [Cha88], which offer unconditional anonymity at very high cost and have had a primarily theoretical influence.

2.2 High and Low Latency

Chaum's mix provides strong protection, but it is **slow**: a message must wait at each mix for a batch to form and may take minutes or hours to arrive. That is acceptable for email, but not for web browsing, where a page needs to arrive within seconds.

This is the field's fundamental divide:

High-latency systems (mixes, remailers) delay and shuffle messages and can withstand an observer watching the entire network, but are useful only for communications that can tolerate delays.

Low-latency systems (onion routing, Tor) forward data as soon as it arrives, making web browsing, chat, and interactive connections possible; however, an observer watching both ends can correlate traffic timing and volume.

Tor deliberately takes the second path: a system that almost nobody uses because it is too slow protects nobody (section 1.4).

2.3 Remailers

In the 1990s, Chaum's ideas became real services called **remailers**: servers that receive an email message and forward it after removing information about its sender.

anon.penet.fi (1993–1996). The best-known remailer was operated in Finland by Johan "Julf" Helsingius from 1993 to 1996. It was not a mix: it assigned each user a *pseudonym* and kept a table mapping pseudonyms to real addresses so that replies could be delivered. Eventually it handled thousands of messages per day. That same table ultimately led to the service's closure. In 1995 and 1996, the Church of Scientology, with the assistance of Finnish police, obtained the identities of some users who had distributed church documents. Helsingius shut down the service in 1996, citing legal uncertainty surrounding the Internet in Finland. The lesson explicitly taken up by the designers of onion routing [Syv11] is that a system retaining

a mapping between users and their identities creates a single point of failure: its operator can be compelled to disclose it.

Cypherpunk, Mixmaster, Mixminion. Meanwhile, the *cypherpunk* community developed remailers without mapping tables: senders encrypted messages in layers for a chain of servers. Successive generations, Mixmaster followed by Mixminion [DDM03], introduced fixed-length messages, genuine batching, and defenses against message replay. Roger Dingledine and Nick Mathewson were among the authors of Mixminion, released in 2003; they were designing Tor at around the same time.

2.4 Toward Low Latency

PipeNet (1995). In a February 1995 message to the cypherpunk mailing list, Wei Dai proposed PipeNet, a low-latency system in which all participants would continuously send traffic at a constant rate, leaving nothing to correlate. It would resist timing analysis, but at an unacceptable cost: if even one participant stopped transmitting, the entire network would have to stop [Syv11]. PipeNet was never implemented, but it remains a reference point for what would be needed to defeat traffic correlation.

Onion routing (1996). That same year, onion routing was taking shape at the Naval Research Laboratory. We will tell that story in chapter 3.

Crowds (1998). Michael Reiter and Aviel Rubin proposed Crowds [RR98]. Users form a "crowd", and each web request is forwarded to a randomly chosen member. With a certain probability, that member sends the request to the website; otherwise, they pass it to another member. The website sees the request arriving from an arbitrary member, and no member can tell whether the person who forwarded a request was its originator. Crowds does not use layered encryption and protects only against the website and other members, not against an observer of the network. It nevertheless introduced a rigorous way to express anonymity in terms of probability, together with an analysis of attacks by colluding members that anticipated guard-related analysis.

Freedom (1999–2001). The largest pre-Tor anonymity network was Freedom, operated by the Canadian company Zero-Knowledge Systems. Announced in late 1998, it operated from late 1999 until late 2001 [Syv11]. It was a commercial service: users paid a subscription, while the servers were run by the company and its partners. It closed because subscriptions did not bring in enough money to sustain it. Its story suggests two lessons that Tor has, in practice, followed, as we will see: an anonymity network is more robust when many independent parties operate it rather than a single company, and its funding model matters as much as its cryptography (section 3.4).

In depth: Many Onions

Numerous other onion-routing systems were proposed in the same period, including the fully decentralized Tarzan and MorphMix, and Cebolla, from which Tor adopted the idea of building circuits one hop at a time using a Diffie–Hellman exchange for each hop [Syv11]. Other projects emerged alongside Tor and took different paths: I2P, designed primarily for internal services and still active, and AN.ON, which originated at Dresden University with the JAP client (and was also offered as a commercial service under the

name JonDonym) and uses fixed chains of mixes operated by known organizations. Each strikes a different balance among decentralization, trust, and performance.

Summary

- Chaum's mix (1981) conceals the connection between incoming and outgoing messages by changing their appearance, standardizing their length, and shuffling their order. A layered-encryption chain of mixes foreshadows the onion.
- High-latency systems resist powerful observers but are slow; low-latency systems are usable for interactive work but vulnerable to timing correlation.
- anon.penet.fi demonstrated the danger of storing user identities; Mixmaster and Mixminion developed the mix approach for email.
- PipeNet, Crowds, and Freedom explored low latency with different trade-offs. Freedom showed the importance of a network operated by multiple parties and a sustainable funding model.

Exercises

Exercise 2.1 $\star$ A mix collects batches of 100 messages. An observer sees all 100 messages enter and all 100 leave. If the observer knows the sender of one incoming message, what is the probability of correctly guessing its outgoing counterpart by choosing at random? What would happen if the observer had sent 99 of the 100 messages?

Exercise 2.2 $\star\star$ Outline a remailer that allows replies to users without keeping a pseudonym-to-address mapping table. Which of Chaum's ideas do you need?

Exercise 2.3 $\star\star$ In Crowds, with forwarding probability $p_f = 3/4$, how many members does a request traverse on average before reaching the website? Why can a member receiving a request not be certain that the member who passed it along originated it?

Further Reading

- D. Chaum, *Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms* [Cha81].
- M. Reiter, A. Rubin, *Crowds: Anonymity for Web Transactions* [RR98].
- G. Danezis, R. Dingledine, N. Mathewson, *Mixminion: Design of a Type III Anonymous Remailer Protocol* [DDM03].
- Free Haven's *Anonymity Bibliography*, `https://www.freehaven.net/anonbib/`: the reference collection of research papers in the field.

Chapter 3

From NRL Onion Routing to the Tor Project

Last verified: 23 September 2026 — Syverson, A Peel of Onion *(2011); the Tor Project's history page*

Chapter objectives

By the end of this chapter, you will be able to:

- trace the origins of onion routing at the Naval Research Laboratory and describe its three generations;
- explain what distinguishes Tor from the preceding generations;
- recount the main stages in the Tor Project's history, from 2002 to the present;
- explain the origin of the name "Tor".

3.1 At the Naval Research Laboratory

In 1995, three researchers at the US Naval Research Laboratory (NRL)—David Goldschlag, Michael Reed, and Paul Syverson—asked whether Internet connections could be established without revealing who was communicating with whom [Thea]. Their goal was not anonymity for its own sake but, as we have seen, to "separate identification from routing" (section 1.3). Their first design was presented at the *Information Hiding Workshop* in May 1996 [GRS96]. A second design followed before the end of that year, separating the user's application, which constructs circuits, from the nodes that carry them [Syv11]. By 1996, the laboratory had already obtained permission to publish the code, before the idea of *open-source* software was widespread. The designers understood that nobody would trust a network whose code could not be examined.

The name comes from the data structure: a message wrapped in encryption layers, each of which is removed by a node, as though peeling an onion.

3.2 Three Generations

Syverson distinguishes three generations of NRL designs [Syv11].

First generation. These were fixed-length circuits of five nodes, designed in part to connect the protected networks of two organizations. With three hops, a hostile intermediate node would immediately see both communicating networks. With four, an attacker who

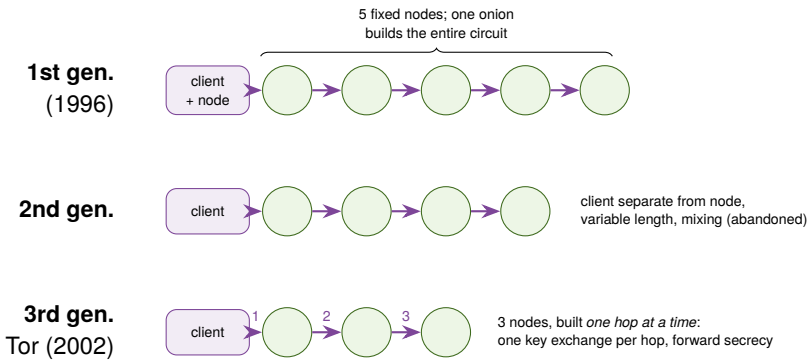


Figure 3.1: The three generations of onion routing. Tor's decisive change was not the number of hops but the way circuits were built: instead of preparing one complete onion at the client, each circuit was extended one hop at a time, with a separate key exchange for each node.

compromised the node adjacent to one endpoint would know which other node to attack to reconstruct the entire circuit. With five, neither shortcut was available without first compromising another node. The client was integrated into the node. The network was static, containing twenty or a hundred nodes operated by stable organizations, with configuration and keys distributed manually. Each application needed its own proxy (for web browsing, remote login, or email), because SOCKS was not yet widespread. A circuit was built as a single onion encrypted under the public keys of all the nodes.

Second generation. The client was separated from the node, circuit length became variable, and mixing and inter-circuit padding mechanisms were introduced before eventually being abandoned. Distributing information about the network was planned but never fully implemented.

Third generation: Tor. The most important change was the construction of circuits **one hop at a time**, with a Diffie–Hellman exchange with each node (section 9.1), an idea adopted from the Cebolla project. This provides forward secrecy and allows each hop to be verified as the circuit is extended. Tor also added directory servers, SOCKS support, hidden services with rendezvous points, and, above all, a design intended for deployment and operation by volunteers around the world.

Historical note: The name

The title of the 2004 paper, *Tor: The Second-Generation Onion Router*, created two misconceptions that Syverson later acknowledged [Syv11]. Tor belongs to the *third* generation, not the second, and the "r" in Tor does not stand for *router*. When Roger Dingledine began working on the NRL project, many onion-routing systems existed. Asked "which one?", he would answer "*the onion routing*", meaning the original one. Rachel Greenstadt noticed that the initials made a good acronym. Dingledine observed that it also worked recursively as *Tor's onion routing* and decided the name should be written "Tor", not "TOR": an ordinary word that also means *gate* in German, as in a city gate. Throughout the rest of this book, we write *Tor* for the network and project, and lowercase *tor* for the program.

3.3 The Tor Network

Roger Dingledine joined the NRL project in the early 2000s, followed shortly afterward by Nick Mathewson. The Tor network went live in **October 2002**, with the code released from the outset under a free-software license. By the end of 2003, it had approximately a dozen volunteer-operated nodes, almost all in the United States and one in Germany [Thea]. In August 2004, Dingledine, Mathewson, and Syverson presented the project at the USENIX Security Symposium [DMS04].

The main milestones from that point onward were as follows [Thea]:

Year	Event
2002	Tor network begins operating, with freely available source code
2004	The Electronic Frontier Foundation begins funding Dingledine and Mathewson's work
2006	The Tor Project, Inc. is founded as a US nonprofit organization (501(c)(3))
2007	Development begins on censorship-resistant bridges (chapter 16)
2008	Development begins on Tor Browser (chapter 23)
2010–2011	Tor becomes an important tool during the Arab Spring uprisings
2013	Edward Snowden's revelations about mass surveillance, in which Tor plays a central role
2017	v3 onion services (chapter 14)
2021	v2 onion services are disabled
2022–2023	Congestion control (section 11.3) and proof of work for onion services (chapter 15)
2024	WebTunnel; migration of bridge distribution to rdsys
2025–2026	CGO in tor (section 10.4); Arti as a client and for onion services (chapter 30)

3.4 An Organization and a Community

The Tor Project, Inc. develops the software, maintains the specifications, operates directory authorities alongside independent operators, measures the network, and supports the community. But it does not own the network: thousands of volunteers—individuals, universities, and associations—operate relays in dozens of countries. This is a design choice informed by the experience of Freedom (section 2.4): no single entity should be able to control the network or be compelled to compromise its users.

The organization's funding has been a recurring topic of public debate. Over the years, it has received funding from government agencies, particularly in the United States, private foundations, and individual donors. The organization publishes its financial statements annually. The question "Can software funded in part by a government be trusted?" is primarily technical rather than political: the code and specifications are public and verifiable, design decisions are discussed openly in proposals (chapter 31), and the system's security does not depend on trusting the Tor Project. Instead, it depends on distributing the network across many independent operators (chapter 4).

In depth: The 2013 revelations

Among the documents released by Snowden in 2013 were internal NSA presentations about Tor. They acknowledged the difficulty of systematically de-anonymizing its users and described targeted attacks against individual users that exploited browser vulnerabilities rather than weaknesses in the network. This indirectly supports a principle we will encounter repeatedly: the most dangerous weaknesses often lie not in the protocol itself but in the systems surrounding it (chapter 28).

Summary

- Onion routing originated in 1995 at the Naval Research Laboratory (Goldschlag, Reed, and Syverson), with the aim of separating identification from routing.
- The first generation used fixed five-node circuits and a static network. Tor, the third generation, builds circuits one hop at a time, with forward secrecy, directories, and rendezvous points.
- The name "Tor" comes from *the onion routing*; only the first letter is capitalized.
- The network has operated since 2002; The Tor Project, Inc. has existed since 2006; independent volunteers operate the relays.

Exercises

Exercise 3.1 ⋆ Why did the first generation use five-node circuits? Reconstruct the argument about three and four hops given in this chapter, and explain why Tor uses only three.

Exercise 3.2 ⋆⋆ Compare building a circuit as a single onion (first generation) with Tor's telescoping construction in terms of forward secrecy and the ability to identify an unresponsive node immediately.

Exercise 3.3 ⋆⋆ Write a short, one-page argument addressing the question "Can Tor be trusted, given who has funded it?" Distinguish between what can be verified technically and what requires trust.

Further Reading

- P. Syverson, *A Peel of Onion* [Syv11].
- D. Goldschlag, M. Reed, P. Syverson, *Hiding Routing Information* [GRS96].
- The Tor Project's history page [Thea].

Chapter 4

Tor's Threat Model

Last verified: 23 September 2026 — Dingledine, Mathewson, Syverson (2004), Section 3

Chapter objectives

By the end of this chapter, you will be able to:

- explain what a threat model is and why every security system needs one;
- list the goals and "non-goals" stated by Tor's designers;
- describe the adversary Tor is intended to protect against and the adversary it cannot withstand;
- distinguish what Tor protects from what remains the responsibility of users and applications.

No security system protects against everything. A **threat model** explicitly describes whom or what a system is intended to protect, which adversaries and threats it is meant to resist, and what assumptions it makes. It is the standard by which the system should be judged: a successful attack within the model reveals a defect, while an attack expressly excluded by the model does not. The latter is nevertheless a limitation that users must understand. This chapter presents Tor's threat model as its designers formulated it in 2004 [DMS04], and describes how it has evolved.

4.1 Goals

The primary goal is to prevent an adversary from **linking** the two parties to a communication, or linking multiple communications from the same user. Four criteria stated in the 2004 paper guided the design:

Deployability The system must work in the real world: it must not cost volunteers too much, expose operators to excessive liability, require operating-system changes, or force websites to install anything.

Usability A difficult system attracts fewer users, and fewer users mean less anonymity. As the authors explain, usability is not merely a convenience but "a security requirement".

Flexibility The protocol must be well specified so it can serve as a foundation for future research.

Simplicity Only well-understood, established techniques should be used. Every additional feature complicates security analysis.

4.2 Non-Goals

Equally important is what Tor explicitly does *not* attempt to do:

It is not peer-to-peer Tor does not try to operate with thousands of short-lived nodes, many of which may be hostile. It uses a set of relays listed and verified by directory authorities (chapter 12).

It does not withstand end-to-end attacks Tor does not claim to defeat timing correlation or intersection attacks by someone who observes both endpoints.

It does not normalize protocols Tor transports application data as it is. If an application reveals the user's identity, Tor cannot stop it. At the time, this required a filtering proxy; today, Tor Browser performs that role (chapter 23).

It is not steganographic Tor does not hide the fact that a user is connected to the network. Pluggable transports, developed later, mitigate this limitation (chapter 16).

4.3 The Adversary

Many theoretical analyses of anonymity systems assume a **global passive adversary**: someone who can observe all Internet traffic. No practical low-latency system protects against such an adversary, and Tor states this openly. Tor instead assumes an adversary with only partial visibility but, in some respects, broader active capabilities [DMS04]. It can:

- observe *part* of the network's traffic;
- generate, modify, delete, or delay traffic;
- operate its own relays;
- compromise some relays operated by others.

With these capabilities, the adversary can attempt many attacks: correlating incoming and outgoing traffic by watching the network's edges; compromising relays or keys; replaying traffic; denying service to honest relays to push users toward malicious ones (chapter 15); marking traffic so it can be recognized elsewhere (section 10.3); giving different users inconsistent views of the network through the directory system (chapter 12); or making the network unreliable enough that users abandon it for less secure systems.

4.3.1 Confirmation and Analysis

Tor's designers distinguish two kinds of traffic-based attack. In a **confirmation** attack, the adversary already suspects a relationship ("Alice communicates with Bob") and observes both endpoints to test that suspicion. Tor, like any low-latency system, cannot provide complete protection against such attacks. In an **analysis** attack, the adversary studies traffic to discover *where* to look—that is, which network points to attack. Tor is designed primarily to resist this latter class. The entire path-selection process, including guards and bandwidth weights (chapter 13), seeks to make it unlikely that an adversary occupies both ends of a circuit.

Warning: The Principle to Remember

Tor protects against someone who observes *part* of the network, not against someone who manages to observe *both ends* of a communication. An adversary that controls or

observes both the user's connection to the guard and the exit's connection to the website can correlate traffic timing and volume. For an organization with access to an entire country's networks, the user and the website may both be located within that country. This is a practical scenario, not just a theoretical one.

4.4 What Tor Protects, and What It Does Not

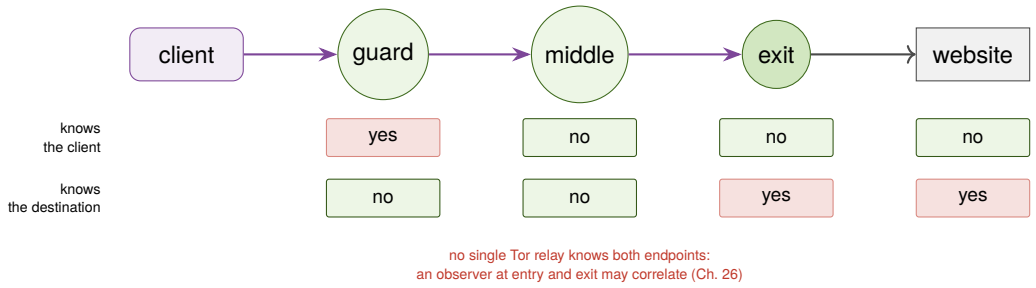


Figure 4.1: What each point on the path knows. Tor's protection comes from separation: the guard knows the client but not the destination; the exit knows the destination but not the client.

Tor protects against	Tor does not protect against
a website trying to learn a visitor's IP address	information the user reveals (logging in to an account or disclosing personal details)
an observer on the local network or an Internet service provider that sees only a connection to Tor	an observer watching both ends of the communication
an individual relay that knows at most one endpoint	an adversary controlling both the guard and exit of a circuit
an exit trying to identify who sent a request	an exit reading or modifying unencrypted traffic (without HTTPS)
IP-address-based censorship when bridges are used	a vulnerable, compromised, or misconfigured application or browser

The model has expanded over time to address developments the 2004 paper could not anticipate: nationwide censorship (chapter 16), browser *fingerprinting* protection (chapter 23), DoS defenses (chapter 15), and the future threat from quantum computers (chapter 31). One principle remains constant: understanding the threat model is the first step toward using Tor safely.

Summary

- A threat model states which adversaries and attacks a system is designed to protect against.
- Tor aims to prevent users and destinations from being linked, prioritizing deployability, usability, flexibility, and simplicity.
- Tor is not peer-to-peer, does not defeat end-to-end attacks, does not normalize application protocols, and does not hide Tor use without pluggable transports.

- The adversary can observe part of the network, operate and compromise relays, and take active measures. Tor is designed primarily to resist analysis attacks and offers less protection against confirmation attacks.

Exercises

Exercise 4.1 ⋆ For each of these scenarios, state whether Tor protects the user and explain why: (a) a café Wi-Fi operator wants to know which websites a customer visits; (b) a website wants to learn its visitors' IP addresses; (c) a user logs in to a personal social-media account through Tor; (d) an agency controlling a country's networks suspects a user is communicating with a newspaper in the same country.

Exercise 4.2 ⋆⋆ Give an example showing why usability is a security requirement, rather than merely a convenience, in an anonymity system.

Exercise 4.3 ⋆⋆ Write a threat model for a journalist using Tor Browser to communicate with a source in another country. Identify possible adversaries, what they can observe, and which aspects remain outside Tor's protection.

Further Reading

- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04], Sections 3 and 7.
- R. Dingledine, N. Mathewson, *Anonymity Loves Company: Usability and the Network Effect* [DM06].

Part II

The Low-Level Protocol

Chapter 5

Essential Cryptography for Tor

Last verified: 23 September 2026 — tor 0.4.9.13; torspec commit `8e1f245`

Chapter objectives

By the end of this chapter, you will be able to:

- identify the cryptographic primitives Tor uses and the role of each;
- distinguish encryption, integrity, key derivation, key exchange, and digital signatures;
- understand why Tor uses several hash families (SHA-1, SHA-2, SHA-3, Blake2b) and how they differ;
- read the formulas in subsequent chapters with confidence.

This chapter is a toolbox, not a cryptography course. It introduces, with only the essential theory, the tools we will encounter in Part II, referring each time to where they are used. Appendix B provides further detail for readers who want to explore the subject more deeply. All the examples can be tried in Python using its standard `hashlib` module and the `cryptography` library, the same tools used in the laboratories.

5.1 Five Tasks

In a protocol such as Tor, cryptography performs five distinct tasks:

Confidentiality Nobody other than the intended recipients should be able to read the data: *ciphers*.

Integrity Nobody should be able to modify the data undetected: *hash functions* and *message authentication codes* (MACs).

Key agreement Two parties that do not share any secret must establish one while communicating publicly: Diffie–Hellman *key exchange*.

Authentication One party must prove possession of a particular identity: *digital signatures*.

Derivation Several independent keys must be obtained from a shared secret: *key derivation functions* (KDFs).

One sound design rule, which Tor follows with increasing rigor, is never to reuse the same key for different tasks or contexts.

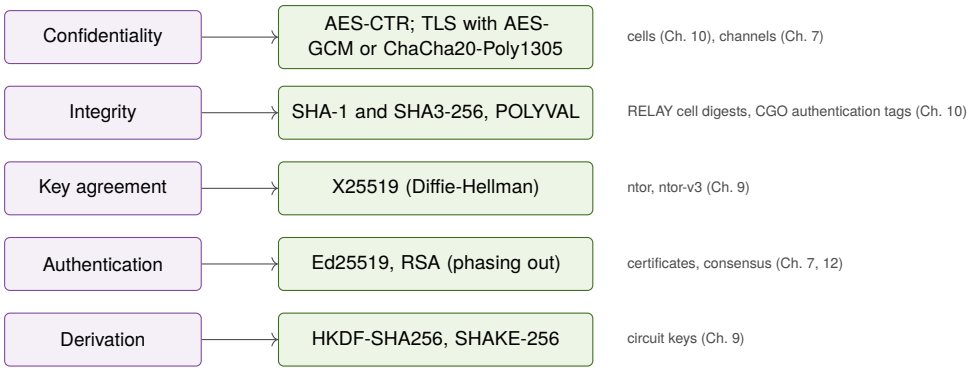


Figure 5.1: The five cryptographic tasks in Tor, the primitives that perform them, and the chapters where we will see them in use.

5.2 Ciphers

5.2.1 AES in Counter Mode

Tor encrypts almost everything with **AES**, the standard block cipher, which has 16-byte blocks and uses 128-bit or 256-bit keys. A block cipher on its own encrypts one block at a time. To encrypt streams of arbitrary length, Tor uses **counter mode** (CTR): AES encrypts a counter that increases by one for every block, and the resulting *keystream* is combined with the data using XOR:

$$C = P \oplus \mathrm{Trunc}_{|P|}\big(\mathrm{AES}_K(IV) \,\|\, \mathrm{AES}_K(IV+1) \,\|\, \cdots\big)$$

Decryption is the same operation. Two properties of counter mode must always be kept in mind:

- the same key–counter pair **must never be reused**: two messages encrypted with the same keystream reveal the XOR of their plaintexts;
- counter mode is **malleable**: changing a ciphertext bit changes the corresponding plaintext bit without requiring knowledge of the key.

AES-128-CTR encrypts relay cells in the traditional scheme (section 10.1); its malleability enables tagging attacks. AES-256-CTR encrypts ntor-v3 messages (section 9.7) and onion-service descriptors (section 14.5). AES-128 is the underlying component of CGO (section 10.4).

5.2.2 TLS

The channel between two nodes is protected by TLS (chapter 7), which negotiates authenticated ciphers such as AES-GCM or ChaCha20-Poly1305 independently. Tor does not, however, rely on TLS for relay identity: relays prove their identities using Tor's own certificates.

5.3 Hash Functions

A cryptographic hash function turns an input of arbitrary length into a fixed-length fingerprint. It is designed to make it computationally infeasible to find two inputs with the same fingerprint (*collision resistance*) or an input matching a given fingerprint. Tor uses four families that reflect its history:

Function	Output	Where we encounter it
SHA-1	20 bytes	relay RSA fingerprints, running digest of relay cells (tor1), KDF-TOR; considered weak against collisions, it survives where that property is unnecessary
SHA-256	32 bytes	ntor and its KDF, Ed25519 and TLS certificates, consensus signatures, microdescriptor fingerprints
SHA3-256, SHAKE-256	32 bytes / variable	ntor-v3, v3 onion services (addresses, blinded keys, HSDir ring, descriptors), rendezvous-circuit digests
Blake2b	variable	onion-service proof of work (chapter 15)

SHA-3 is not an improved version of SHA-2 but an entirely different construction (a *sponge*). SHAKE-256 belongs to the same family and is an extendable-output function: it produces as many bytes as needed, making it convenient for key derivation. In Tor's newer protocols, every use of SHA-3 is prefixed with a domain label (for example, `"ntor3-curve25519-sha3_256-1:kdf_phase1"`). This prevents a hash calculated for one purpose from coinciding with one calculated for another.

In depth: Running Hashes

Many libraries let an application add data to a hash incrementally and read the digest at any point without starting over. Tor uses this capability for the *running digest* of relay cells (section 10.2), which summarizes all cells exchanged with a given hop. In Python, `hashlib.sha1()` with its `update`, `digest`, and `copy` methods does exactly this.

5.4 MACs and Key Derivation

5.4.1 HMAC

A MAC is a fingerprint that also depends on a key: only someone who knows the key can calculate or verify it. **HMAC** constructs a MAC from a hash function. Tor uses HMAC-SHA256 in the ntor handshake (section 9.6) and its key derivation function. In SHA-3-based protocols, a MAC can be obtained more simply by hashing the key, preceded by its length, followed by the message. The length unambiguously identifies where the key ends and the message begins [rend-spec § Cryptographic building blocks]. In ntor-v3, the domain label is also prefixed with its length (section 9.7).

One practical detail: a MAC must always be compared in *constant time* (using `hmac.compare_digest` in Python); otherwise, comparison timing may reveal how many bytes are correct.

5.4.2 Key Derivation Functions

A shared secret must yield several keys: one for forward encryption, one for backward encryption, and two for integrity (section 9.8). A KDF expands a secret into a stream as long as necessary, from which individual keys can be extracted. Tor uses three:

KDF-TOR $\mathrm{SHA1}(K_0\|0) \,\|\, \mathrm{SHA1}(K_0\|1) \,\|\, \ldots$, a historical construction still used by `CREATE_FAST`;

HKDF-SHA256 the standard construction defined by RFC 5869, used by ntor;

SHAKE-256 used directly as a KDF in ntor-v3 and onion services.

5.5 Key Exchange: X25519

Diffie–Hellman key exchange lets two parties establish a shared secret while exchanging only public values. The elliptic-curve version used by Tor is **X25519** (key exchange on *Curve25519*). Each party has a 32-byte private key x and a public key $X = x \cdot G$, where G is a fixed point on the curve and the product denotes multiplication of a point by an integer. Given the keys x, X and y, Y, both parties calculate the same value

$$x \cdot Y = y \cdot X = xy \cdot G,$$

which an observer who knows only X and Y cannot derive. The keys are 32 bytes long and the exchange is very fast. These two advantages led Tor to abandon the 1024-bit modular Diffie–Hellman used by the TAP handshake (section 9.4).

A key used for only one exchange and then deleted is called **ephemeral**. Combining ephemeral and long-term keys is what enables Tor handshakes to provide both authentication and *forward secrecy* (section 9.3).

5.6 Signatures: Ed25519 and RSA

A digital signature is generated with a private key and verified using its corresponding public key. Tor primarily uses **Ed25519**, a signature scheme on a curve equivalent to Curve25519, with 32-byte public keys and 64-byte signatures: for relay identities, Tor certificates (section 7.5), and onion-service descriptors. Ed25519 has a property Tor uses in an unusual way: multiplying a public key A by a factor h produces another valid public key, $A' = h \cdot A$, whose corresponding private key can be calculated by whoever knows the original private key. This is the basis of onion services' blinded keys (section 14.3).

RSA with 1024-bit keys is a legacy of Tor's early years: the legacy identities of relays and the keys with which directory authorities sign the consensus. Authority keys are longer, and the two key types have different requirements: the specification requires at least **3072** bits for the identity key (`KP_auth_id_rsa`) and at least 2048 for the signing key (`KP_auth_sign_rsa`) [dir-spec § Directory authority operation and formats]. Tor is gradually phasing out RSA.

5.7 Recent Cryptography

Two more specialized constructions appear in recent developments:

- **POLYVAL**, a universal hash function (also used in AES-GCM-SIV), forms a building block of CGO alongside AES, the new relay-cell encryption scheme (section 10.4);
- **Elligator2** encodes a Curve25519 point so that it is indistinguishable from a random string; obfs4 uses it to conceal key-exchange public keys (section 16.4).

Post-quantum cryptography, which will need to replace X25519 when quantum computers become a practical threat, is an active research topic for Tor, discussed in chapter 31.

Summary

- AES in counter mode encrypts data; the same keystream must never be reused, and counter mode is malleable.
- SHA-1, SHA-256, SHA3-256/SHAKE-256, and Blake2b coexist in Tor: the choice depends on when the protocol was designed and what the hash is used for.

- HMAC authenticates messages, whereas KDFs (KDF-TOR, HKDF, SHAKE) derive keys; MACs must be compared in constant time.
- X25519 is the key exchange used by all modern handshakes; Ed25519 signs identities, certificates, and descriptors; RSA is being phased out.

Exercises

Exercise 5.1 $\star$ With AES-CTR, an adversary knows the plaintext P_1 of one message and intercepts its ciphertext C_1 and the ciphertext C_2 of another message, mistakenly encrypted with the same keystream. How can the adversary recover P_2?

Exercise 5.2 $\star$ In Python, calculate `hashlib.shake_256(b"Tor").hexdigest(16)` and `hexdigest(32)`. What is the relationship between the two outputs? What does this imply for using SHAKE as a KDF?

Exercise 5.3 $\star\star$ Using the `cryptography` library, generate two X25519 key pairs and verify that they produce identical shared secrets. What happens if one party uses 32 zero bytes as the other's public key? Relate the result to ntor's "point at infinity" check.

Exercise 5.4 $\star\star$ Why do Tor protocols based on SHA-3 prefix every input with a domain label? Describe a possible attack if two distinct uses of the same function were not separated.

Further Reading

- J.-P. Aumasson, *Serious Cryptography* (No Starch Press): a modern, practical introduction to all the primitives in this chapter.
- D. J. Bernstein et al., the original papers on Curve25519 and Ed25519, and RFCs 7748 (X25519) and 8032 (Ed25519).
- The "Preliminaries" section of the Tor specification [Thee], with conventions for ciphers and hashes.

Chapter 6

Network Architecture

Last verified: 23 September 2026 — tor 0.4.9.13; public consensus of 23 September 2026, 15:00 UTC

Chapter objectives

By the end of this chapter, you will be able to:

- list the roles of Tor network nodes and their respective responsibilities;
- describe the protocol layers, from the TLS channel to an application's stream;
- follow, at a high level, everything that happens when someone opens a web page through Tor;
- understand how different implementations and versions coexist on the same network.

Before examining the details in subsequent chapters, we need a bird's-eye view of the network. This chapter is a map: every element will be revisited and explored in greater depth later, with references pointing to the relevant sections.

6.1 Roles

The Tor network consists of programs performing different roles, often more than one at the same time. Figure 6.1 summarizes them.

Client The user's program, such as the one included in Tor Browser. It selects paths, builds circuits, and offers applications a SOCKS port. It does not accept connections from the network and does not authenticate itself (chapter 7).

Relay A node that forwards traffic for others. Depending on the flags assigned by directory authorities and its configuration, it may act as:

- a **guard**: the first hop in a client's circuits, selected once and used for an extended period (section 13.3);
- a **middle**: an intermediate hop that knows neither the source nor the destination;
- an **exit**: the final hop, opening Internet connections according to its exit policy (chapter 11);
- an **HSDir**: storing onion-service descriptors (section 14.4);
- a **directory cache**: redistributing the consensus and descriptors (chapter 12).

Directory authority Nine trusted servers that vote hourly on the network's state and sign the consensus; some also measure relay bandwidth.

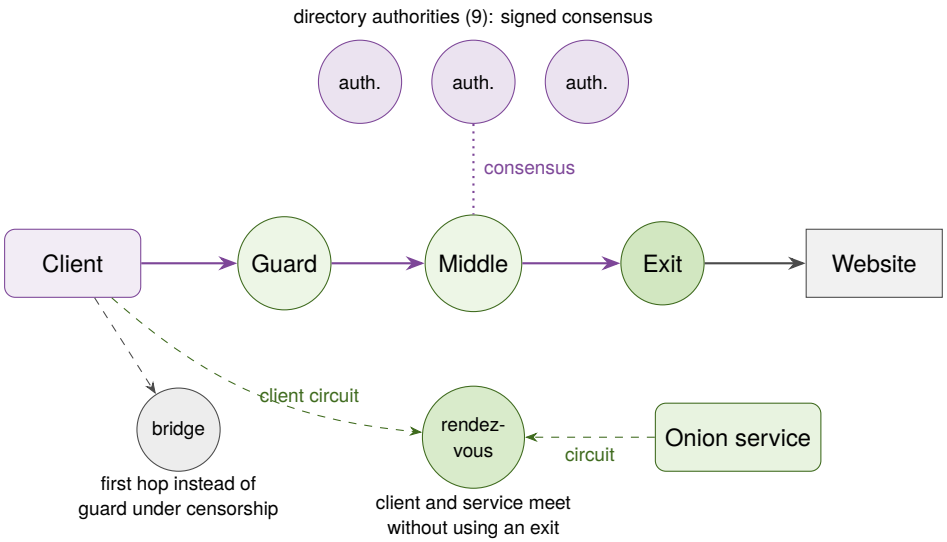


Figure 6.1: Roles in the Tor network. A client builds a circuit through a guard, middle, and exit. Directory authorities publish the consensus listing relays. Bridges take the place of guards for users affected by censorship. Onion services are reached through a rendezvous point without using an exit.

Bridge An unpublished relay used as a first hop by people who cannot access public relays, often behind a pluggable transport (chapter 16).

Onion service A service reachable only through Tor without revealing its location (chapter 14).

In the public consensus of 23 September 2026, there are 9,358 relays: 5,306 with the `Guard` flag and 3,177 with the `Exit` flag (1,615 have both). Guards account for 70 % of total bandwidth, and exits for 33 % (chapter 13).

6.2 Protocol Layers

Tor's protocol can be described as a stack of layers, each built upon the one below (fig. 6.2).

Channel A TLS connection between two adjacent nodes, over which the relay's identity is verified (chapter 7).

Cells The 514-byte transmission units carried by a channel (chapter 8).

Circuit A multi-hop path built one hop at a time using handshakes that derive different keys for each hop (chapter 9).

Layered encryption Relay messages wrapped in one encryption layer for each hop (chapter 10).

Stream Application TCP connections opened by the exit and managed using windows and congestion control (chapter 11).

Alongside this stack are the directory system, which supplies clients with the list of relays, and the onion-service protocol, which uses circuits to meet at a rendezvous point.

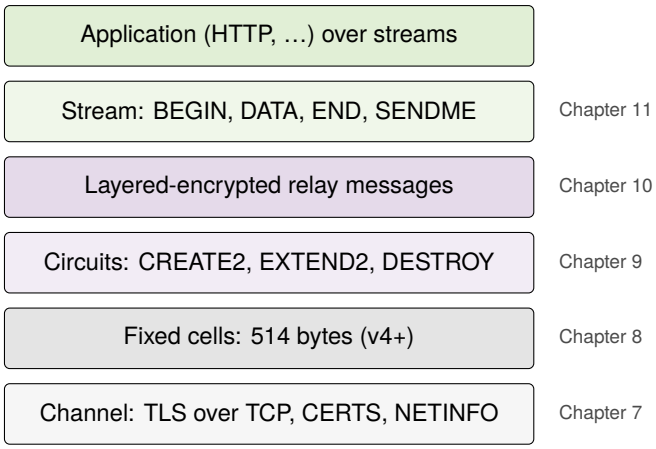


Figure 6.2: The layers of the Tor protocol and the chapters that describe them. Each layer uses the one beneath it: streams travel as relay messages, relay messages are carried in cells encrypted in layers, and cells travel inside TLS channels.

6.3 A Web Page from Start to Finish

Let us follow, at a high level, what happens when someone starts Tor Browser for the first time and opens a page.

1. **Startup.** tor contacts a directory authority or one of approximately two hundred *fallback directories* listed in its source code and downloads the consensus, whose signatures it verifies. It then downloads relay microdescriptors (chapter 12).
2. **Guard selection.** It randomly selects a small sample of guards, weighted by bandwidth, and uses two of them (section 13.3).
3. **Channel.** It opens a TLS connection to a guard and verifies its identity using **`CERTS`** cells (chapter 7).
4. **Circuit.** It selects an exit suitable for the destination and a middle relay, and constructs a three-hop circuit using **`CREATE2`** and `EXTEND2` (chapters 9 and 13). In practice, tor builds some circuits in advance so they are available when needed.
5. **Stream.** Through the SOCKS port, the browser requests a connection to `www.example.com:443`. tor sends a `BEGIN` message to the exit, which resolves the name and opens a TCP connection (chapter 11).
6. **Data.** The browser speaks HTTPS with the website through the stream. The exit sees the website's name and TLS traffic but not who requested it. The guard sees who made the request but not what was requested. The middle relay sees neither.

The entire startup process usually takes only a few seconds. Many mechanisms described in later chapters, including padding, congestion control, and vanguards, operate quietly around this sequence.

6.4 Implementations and Versions

6.4.1 Two Implementations

The historical implementation is **tor**, written in C, which performs all roles: client, relay, authority, and onion service. Alongside it, the Tor Project develops **Arti**, a rewrite in Rust

(chapter 30), now used as a client and for onion services. In the public consensus of 23 September 2026, no relays use Arti: 6,027 run tor 0.4.9.12, 2,520 run 0.4.9.11, and most remaining versions are older releases in the 0.4.9 series. Forty-seven relays run the development version 0.5.0.0-alpha-dev.

6.4.2 Subprotocols

How can a network of thousands of nodes, updated at different times, introduce new features? Tor does not rely on software version numbers for this purpose. Instead, it uses **subprotocols**: each feature belongs to a subprotocol with its own version number, and every relay declares the versions it supports in the consensus's `pr` line [tor-spec § Subprotocol-based versioning]. For example, a relay on the public network declares:

```
pr Conflux=1 Cons=1-2 Desc=1-4 DirCache=2 FlowCtrl=1-2 HSDir=2 HSIntro=4-5
   HSRend=1-2 Link=3-5 LinkAuth=3 Microdesc=1-3 Padding=2 Relay=2-6
```

A client that wants to use ntor-v3 with a relay checks whether the relay advertises `Relay=4`; CGO requires `Relay=6`, and congestion control requires `FlowCtrl=2`. These numbers identify independent *capabilities*, not cumulative versions: `Relay=6` does not formally imply `Relay=4` (Proposal 346, which therefore prefers the term "subprotocol capabilities"). In practice, tor advertises them as a contiguous range because it implements the features in order. The consensus also lists *required* subprotocols: a tor version that lacks support for them knows it is too old and refuses to operate rather than put its users at risk.

Summary

- The network consists of clients, relays (guards, middles, exits, HSDirs, and directory caches), directory authorities, bridges, and onion services.
- The protocol is a stack: TLS channels, cells, circuits, layered encryption, and streams.
- A guard knows the user but not the destination; an exit knows the destination but not the user.
- tor (C) and Arti (Rust) coexist. Features are negotiated as subprotocols advertised in the consensus's `pr` line.

Exercises

Exercise 6.1 ⋆ For each role (guard, middle, exit, HSDir, and directory authority), state what it knows about a user who visits a website through Tor.

Exercise 6.2 ⋆ In the `pr` line shown in this chapter, which features discussed in Chapters 9–11 does the relay support? Could you use it as an exit with CGO?

Exercise 6.3 ⋆⋆ A 100 KB web page is downloaded through Tor. Estimate, layer by layer, how many bytes cross the link between the client and the guard, accounting for cells, relay message headers, and SENDME.

Further Reading

- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04]: the original architecture, much of which still applies.
- Tor Metrics, `https://metrics.torproject.org/`: up-to-date figures on relays, users, and versions.

Chapter 7

Channels and the Link Protocol

Last verified: 23 September 2026 — torspec commit `8e1f245`; tor 0.4.9.13; Arti 2.6.0; link protocol 4–5

Chapter objectives

By the end of this chapter, you will be able to:

- explain what a channel is and why Tor builds it over TLS;
- describe the keys that make up a relay's identity and how they are bound together;
- follow the channel handshake cell by cell: VERSIONS, CERTS, AUTH_CHALLENGE, AUTHENTICATE, and NETINFO;
- read and verify a Tor Ed25519 certificate;
- understand why a client does not authenticate and what this means for anonymity;
- open a verified channel to a relay on the laboratory network (Laboratory 1).

The circuit, Tor's central construct, is a chain of hops. Each hop, however, rests on something simpler: a direct, encrypted connection between two adjacent nodes. The specification calls this connection a **channel**. In this chapter we will see how a channel is established and, above all, how each side verifies the identity of the peer it intended to reach. The format of the cells exchanged over the channel is the subject of chapter 8, to which we will refer frequently.

7.1 What Is a Channel?

A channel links two relays, or a client and a relay, through a direct encrypted connection [tor-spec § Channels]. Technically, it is a TLS session over TCP that carries cells. Three properties characterize it.

Asymmetric roles Anyone may *open* a channel (the *initiator*), but only a relay may *receive* one (the *responder*). A client does not accept connections: it has no port listening for connections from the Tor network.

Verified identity The responder always proves cryptographically that it possesses one or more relay identities. The initiator usually already knows which identity to expect: it reads it from the consensus, from the list of authorities compiled into the source code, or, when the channel is opened to extend a circuit, from the `EXTEND2` message itself. If the identity presented is not the expected one, the channel must be closed.

Sharing A channel carries many circuits, distinguished by their CircIDs (section 8.4). Nodes prefer to reuse an existing channel rather than open a second one to the same relay. A channel with no circuits is closed after a randomly selected idle period whose range

depends on how the channel is used (`channelpadding_get_channel_idle_timeout` in `src/core/or/channelpadding.c`): 3–4.5 minutes for client channels, and 45–75 minutes for channels between two relays, where the central value is the consensus parameter `nf_conntimeout_relays` (one hour). While a channel is in use, `KeepalivePeriod` instead causes a padding cell to be sent every five minutes to prevent firewalls from closing it.

In depth: Channel or connection?

Older documentation and many research papers call channels *connections* (*OR connections*). The specification moved away from this term because it could be confused with the TCP connections opened by exit relays for outgoing streams. Both terms survive in the tor source code: `or_connection_t` is the TLS connection, while `channel_t` is the abstraction that uses it.

7.2 A Relay's Identity

Before examining the handshake, we need to know *what* must be proved. A relay is identified by long-term keys. The specification gives them systematic names: the `KP_` prefix denotes a public key and `KS_` the corresponding secret key [tor-spec § Relay keys and identities]. The keys relevant to the channel are:

`KP_relayid_ed` the **Ed25519 identity**, or "master identity key". It never expires and has just one purpose: to sign the signing key. It can be kept offline.

`KP_relayid_rsa` the 1,024-bit **legacy RSA identity**. It is still the basis of the 40-hexadecimal-digit fingerprints shown on Tor Metrics, but protocol security no longer depends on it.

`KP_relaysign_ed` the medium-term **signing key**, certified by the identity. This is the key the relay uses in everyday operation, online.

`KP_link_ed` a short-term key used only when a relay initiates a channel and must authenticate (section 7.6).

Figure 7.1 shows how certificates bind these keys together. The idea is a *chain of trust* that starts at the identity and extends to the TLS certificate used for the connection.

In the laboratory, these keys are files in each relay's `keys/` directory: `ed25519_master_id_public_key` and its corresponding secret identity key, `ed25519_signing_cert` and `ed25519_signing_secret_key` for the signing key, and `secret_id_key` for the RSA identity. The `fingerprint-ed25519` file in the node's directory contains its base64-encoded Ed25519 identity.

Historical note: From RSA to Ed25519

Until 2015, a relay's only identity was its 1,024-bit RSA key, a size now considered inadequate. Proposal 220 introduced Ed25519 identities and the certificate format described in this chapter; support began with tor 0.2.7.2-alpha. The RSA key was retained for compatibility and bound to the Ed25519 key by a cross-certificate.

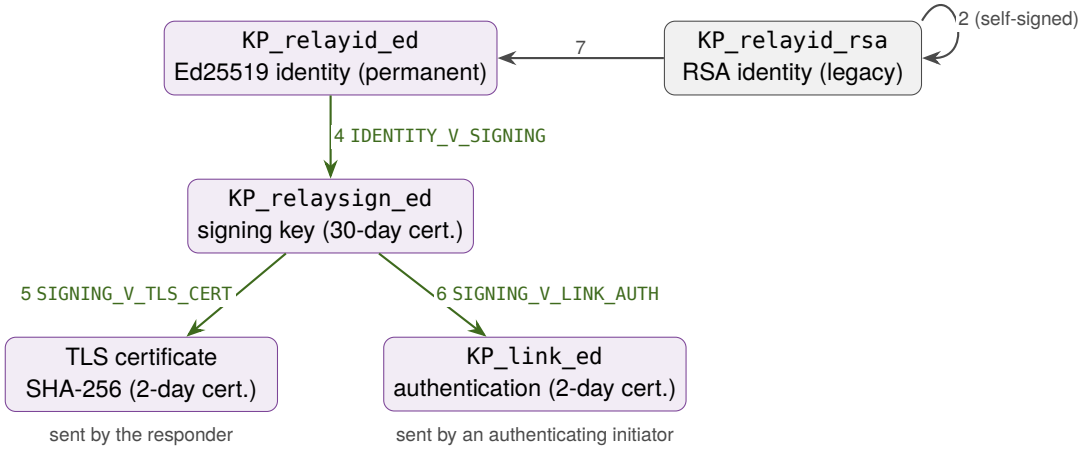


Figure 7.1: The certificate chain binding a relay's Ed25519 identity to the connection's TLS certificate. Each arrow represents a certificate: the key at its origin signs the key (or fingerprint) at its destination. The number identifies the CertType used in the CERTS cell. On the right is the legacy RSA chain, linking the old RSA identity to the Ed25519 identity.

7.3 The Handshake at a Glance

Figure 7.2 summarizes how a channel is established [tor-spec § Negotiating and initializing channels].

1. The initiator opens a TLS connection.
2. Both parties send a **`VERSIONS`** cell and agree on a link-protocol version (section 8.3).
3. The responder sends **`CERTS`** (certificates proving its identity), **`AUTH_CHALLENGE`** (a challenge the initiator will sign if it wishes to authenticate), and **`NETINFO`** (time and addresses).
4. The initiator verifies the certificates. If it is a public relay, it replies with **`CERTS`** and **`AUTHENTICATE`**; in every case, it finishes with **`NETINFO`**.

`NETINFO` marks the end of the handshake: once a party has sent it, that party sends no further handshake cells. This is how the responder knows whether the initiator has chosen not to authenticate. Any number of **`VPADDING`** cells may appear in between, but no other cell type may do so.

7.4 The TLS Layer

Tor uses TLS in an unusual way. On a website, the server's certificate is signed by a certificate authority and contains a domain name; the browser checks both. A relay, by contrast, presents a **self-signed** certificate with no meaningful name, and the client does not verify it according to the usual TLS rules. In the code for Laboratory 1, this becomes two lines that would be a serious security error in almost any other context:

```
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE
```

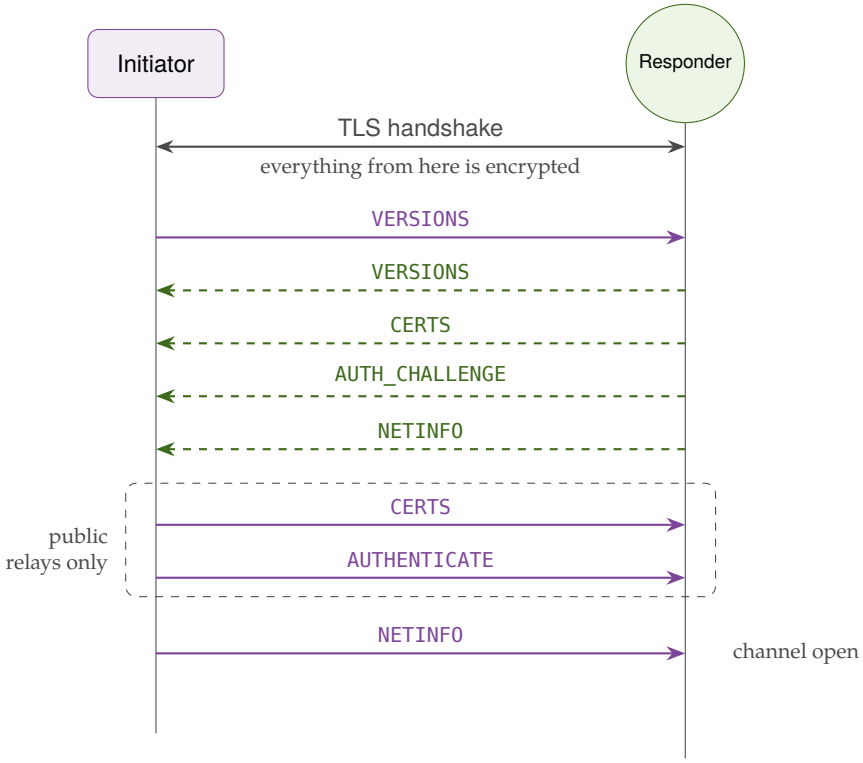


Figure 7.2: The channel handshake. After TLS is established, everything travels encrypted. The responder always sends CERTS, AUTH_CHALLENGE, and NETINFO, in that order. The dashed box applies only when the initiator is a public relay that must establish its own identity; a client or bridge sends only NETINFO.

They are not an error here because authentication takes place *afterward*, inside the channel, using the **CERTS** cell. TLS provides channel encryption and integrity and proves possession of the TLS certificate's key; the cells establish everything else.

The specification adds several rules [tor-spec § The TLS handshake]:

- implementations should support TLS 1.3, and relays should also support TLS 1.2, with modern suites (AES-GCM, ChaCha20-Poly1305, and key exchange using X25519 or P-256);
- *session resumption* should not be permitted: it weakens forward secrecy and has enabled attacks such as the *Triple Handshake* (2013);
- TLS compression should not be used.

On the laboratory network, a Python client using OpenSSL 3.5 negotiates TLS 1.3 with tor using the `TLS_AES_256_GCM_SHA384` suite.

Historical note: Why Such an Unusual Handshake?

The earliest link-protocol versions put Tor certificates directly into the TLS handshake (version 1), then hid them using TLS renegotiation (version 2), signalling the selected variant through the list of offered cipher suites. The reason was censorship: before pluggable transports (chapter 16), Tor tried to resemble an ordinary HTTPS connection between a browser and a web server. With TLS 1.2, certificates travelled in the clear and were easy to recognize. In version 3 the certificates moved inside the encrypted channel, into **CERTS** cells. Today, resistance to censorship is entrusted to pluggable transports, and the older variants are obsolete; the specification notes that, as of March 2025, tor still accepted some of them, but they were scheduled for removal.

7.5 CERTS: Proving Identity

7.5.1 The Cell

CERTS is a variable-length cell (command 129) whose body contains a list of certificates:

Field	Bytes	Contents
N	1	number of certificates
repeated N times:		
CertType	1	certificate type
CertLen	2	certificate length
Certificate	CertLen	the certificate

A **CERTS** cell cannot contain two certificates of the same type. The defined types are listed in table 7.1 [cert-spec § Certificate types (CERT_TYPE field)].

7.5.2 Tor's Ed25519 Certificate

Tor does not use X.509 for Ed25519 certificates. Instead it uses its own much simpler format [cert-spec § Tor Ed25519 Certificates], shown in fig. 7.3.

VERSION has the value 1.

Type	Name	Format	What it certifies
1	`TLS_LINK_X509`	X.509	TLS key, signed by the RSA identity (obsolete)
2	`RSA_ID_X509`	X.509	self-signed RSA identity
3	`LINK_AUTH_X509`	X.509	authentication RSA key (obsolete)
4	`IDENTITY_V_SIGNING`	Ed	signing key, signed by the Ed25519 identity
5	`SIGNING_V_TLS_CERT`	Ed	TLS certificate fingerprint, signed by the signing key
6	`SIGNING_V_LINK_AUTH`	Ed	`KP_link_ed` key, signed by the signing key
7	`RSA_ID_V_IDENTITY`	RSA→Ed	Ed25519 identity, signed by the RSA identity
8–12	onion service (8, 9, 11), ntor-key cross-certificate (10), families (12)		

Table 7.1: Certificate types. "Ed" denotes Tor's Ed25519 certificate format.

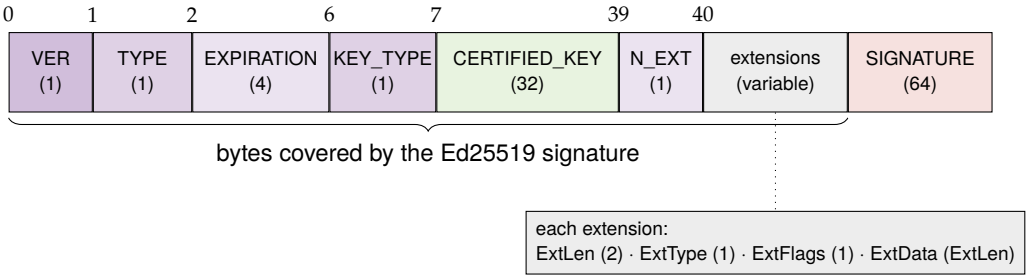


Figure 7.3: Tor's Ed25519 certificate. Every field preceding the signature is covered by that signature. Expiration is expressed in *hours* since the Unix epoch: with an unsigned four-byte field, it will not overflow for hundreds of thousands of years.

CERT_TYPE has the same value as CertType in table 7.1.

EXPIRATION_DATE gives expiration in hours since January 1, 1970.

CERT_KEY_TYPE indicates what the next field contains: 1 means an Ed25519 key, and 3 means the SHA-256 fingerprint of an X.509 certificate. tor versions through 0.4.5.1-alpha always wrote 1, and the specification requires implementations to tolerate this.

CERTIFIED_KEY contains the 32 certified bytes.

Extensions each contain a length, type, flags, and data. The only extension defined today is type 4, *signed-with-ed25519-key*, which includes the key that signed the certificate. Flag 1 (`AFFECTS_VALIDATION`) determines how an unknown extension must be handled: when set, the certificate must be rejected; otherwise the extension is ignored.

SIGNATURE is the Ed25519 signature over all preceding fields.

7.5.3 Verification

The responder must send exactly one certificate of type 4 and one of type 5. The initiator verifies them as follows [tor-spec § Authenticating the responder from its CERTS]:

1. The type-4 certificate contains an Ed25519 key in extension 4: the claimed identity `KP_relayid_ed`. The certificate must be signed by that very key (it is *self-signed*); the certified key is `KP_relaysign_ed`.

2. The type-5 certificate must be signed with `KP_relaysign_ed`, and its certified key must be the SHA-256 fingerprint of the TLS certificate received during the TLS handshake.

3. Neither certificate may have expired.

4. Finally, the resulting identity must be compared with the expected identity.

Why does this chain prove anything? Step 2 binds the signing key to the TLS certificate of *this* connection. The TLS handshake has already proved that the responder holds the private key corresponding to that certificate. Step 1 binds the signing key to the identity. An impostor could copy a genuine relay's **CERTS** cells, since they contain no secrets, but could not use them: the impostor would have to present that genuine relay's TLS certificate during the TLS handshake and therefore possess its private key.

Warning: Check the Expected Identity

A perfectly valid chain proves only that the responder possesses *some* identity. If the initiator omits step 4, anyone with any identity could impersonate the intended relay. Laboratory 1 has the `expected_identity` parameter precisely for this reason, and one test verifies that a relay presenting a different identity is rejected.

7.5.4 The Legacy RSA Chain

The initiator may optionally verify the RSA identity too. The type-2 certificate contains the 1,024-bit RSA key (self-signed, exponent 65537), and the type-7 certificate, in the "cross-certificate" format, binds that key to the already verified Ed25519 identity. The cross-certificate's RSA signature is calculated over the SHA-256 fingerprint of the string `"Tor TLS RSA/Ed25519 cross-certificate"` followed by the certificate fields. The prefix prevents the same signature from being reused in a different context.

7.5.5 What a Relay Actually Sends

On the laboratory network, relay `test003r` sends a **CERTS** cell containing five certificates:

```
type 1  584 bytes
type 2  442 bytes
type 4  140 bytes  expires 2026-10-24 16:00  key_type=1  ext=[4]
type 5  104 bytes  expires 2026-09-29 11:00  key_type=3  ext=[]
type 7  165 bytes
```

In addition to the Ed25519 chain (types 4 and 5) and the RSA chain (types 2 and 7), the relay still sends the old type-1 certificate for older clients. This certificate does not merely have the same length as the TLS certificate: it has the same fingerprint too. Comparing their bytes shows that they are *exactly* the same certificate. The length is not a number worth memorizing, and it is useful to understand why. The certificate's common name is a fictitious domain built by tor as `www.`, followed by 4 to 25 random characters, followed by `.com` (`crypto_random_hostname` in `src/lib/tls/tortls_openssl.c`). That alone makes the length vary by about twenty bytes between relays. It also changes *over time* for the same relay, because tor renews the TLS context, and with it the certificate, every two hours (`MAX_SSL_KEY_LIFETIME_INTERNAL` in `src/core/or/or.h`). Repeating the measurement forty minutes later, we obtained a **CERTS** body of 1,456 bytes rather than 1,451. If your measurements differ, that is expected: what must agree is the *structure*, and the sum of the certificate lengths plus one count byte and three header bytes per certificate must equal the cell-body length. The two expiration dates show the different lifetimes: the signing key is certified for 30 days, while the binding to the TLS certificate lasts only 2 days. These are tor's defaults (options `SigningKeyLifetime` and `TestingLinkCertLifetime`, the latter configurable only in a test network), so they also

apply to the public network. The identity key stays constant while the keys representing it online are renewed frequently, limiting the damage if a relay is compromised.

7.6 AUTH_CHALLENGE and AUTHENTICATE

7.6.1 Who Authenticates?

Initiator authentication is optional, and the rule governing who should perform it is one of the protocol's most important design decisions [tor-spec § Channels]:

- A **public relay** opening a channel to another relay *should* authenticate: the responder then knows it is talking to a network relay rather than a client.
- A **client** or a **bridge** *should not* authenticate.

The reason for the second rule is anonymity. A client has no identity, and creating one would be harmful: a guard could recognize the same client when it reconnects from a different address. A bridge, for its part, must not reveal to its users that it is a network relay. The trade-off is that a relay may infer that it is a circuit's first hop if the channel is unauthenticated (chapter 8, section 8.6). This is one reason relays refuse to serve as exits for single-hop circuits.

7.6.2 The Challenge

The responder sends an **`AUTH_CHALLENGE`** cell in any case. It contains 32 random bytes and a list of accepted authentication methods:

Field	Bytes	Contents
Challenge	32	random value, distinct for each channel
N_Methods	2	number of methods
Methods	2× N_Methods	list of methods

The defined methods are 1 (`RSA-SHA256-TLSSecret`, obsolete), 2 (never implemented), and 3 (`Ed25519-SHA256-RFC5705`). The cell received from laboratory-network relays is 36 bytes long: $32 + 2 + 2$, corresponding to a single method, number 3.

7.6.3 The Response

A relay that wishes to authenticate replies with its own **`CERTS`** cell (using types 4 and 6 rather than 4 and 5, since an initiator has not necessarily presented a TLS certificate), followed by **`AUTHENTICATE`**. For method 3, the body of **`AUTHENTICATE`** is a 352-byte document signed with the `KP_link_ed` key:

Field	Bytes	Contents
TYPE	8	the string `AUTH0003`
CID, SID	32 + 32	SHA-256 fingerprints of the initiator's and responder's RSA identities
CID_ED, SID_ED	32 + 32	initiator's and responder's Ed25519 identities
SLOG	32	SHA-256 of all bytes *received* on the channel through AUTH_CHALLENGE, inclusive
CLOG	32	SHA-256 of all bytes *sent* on the channel before AUTHENTICATE
SCERT	32	SHA-256 of the responder's TLS certificate
TLSSECRETS	32	value derived from the TLS session secrets (*keying material exporter*)
RAND	24	random bytes
SIG	64	Ed25519 signature over all preceding fields

Every field binds the signature to a particular aspect of *this* connection: the identities involved (CID, SID, and so on), the conversation so far (SLOG and CLOG, which include the random challenge), the TLS certificate (SCERT), and the TLS session itself (TLSSECRETS). Such a signature cannot be reused on another connection, nor by an intermediary relaying messages between two different TLS sessions (a *man-in-the-middle* attack), because TLSSECRETS and SCERT would differ on the two sides.

In depth: An Error Preserved in the Specification

TLSSECRETS is calculated with the TLS *exporter* (RFC 5705 and, for TLS 1.3, RFC 8446, §7.5), using the label `"EXPORTER FOR TOR TLS CLIENT BINDING AUTH0003"` and a context value. The intention was to use the initiator's Ed25519 identity as the context, but the implementation mistakenly used CID, the RSA identity fingerprint. Because this does not weaken security, the specification chose to document the actual behavior instead of changing it. This is a useful example of how, for a protocol with thousands of already deployed implementations, actual code behavior can become the standard.

7.7 NETINFO: Time and Addresses

The final handshake cell, sent by both parties, is **`NETINFO`**. It is a fixed-length cell (command 8), with the following body:

Field	Bytes	Contents
TIME	4	seconds since the Unix epoch, big-endian
OTHERADDR	2 + ALEN	address of the other party as seen by the sender
NMYADDR	1	number of the sender's own addresses
MYADDR	NMYADDR × (2 + ALEN)	sender's own addresses

Each address is encoded as a type (one byte: `0x04` for IPv4, `0x06` for IPv6), a length (4 or 16), and a value. These fields serve two purposes:

- **The time** makes it possible to detect an incorrect clock: tor logs a warning if the time received from a relay differs from its own by more than an hour (the `NETINFO_NOTICE_SKEW` constant in `src/core/or/channeltls.c`).
- **The addresses** let a relay discover the public address at which it is visible, and let the initiator check whether it reached the relay at one of its *canonical* addresses, meaning an address published in its descriptor.

Warning: A Client Does Not Disclose Its Clock

Relays send their own time and addresses. A client, however, should send `00 00 00 00` as the time and should not send any of its own addresses. The reason is *fingerprinting*: a computer's clock almost always has a small characteristic offset. If a client disclosed its time to every guard, that offset could help recognize the client over time. This illustrates a general rule we will encounter repeatedly: a Tor client must not disclose anything that is not strictly necessary.

The unsigned four-byte TIME field overflows in 2106; Proposal 338 discusses extending it to eight bytes. A recipient of a **`NETINFO`** cell should not blindly trust its values: the other party may lie about its time and addresses.

Here is the **NETINFO** cell we receive from relay `test003r`, decoded by Laboratory 1:

```
Netinfo(timestamp=1790511791, peer_address='127.0.0.1',
        my_addresses=['127.0.0.1'])
```

The time corresponds to September 27, 2026, at 12:23:11 UTC. Both addresses are loopback addresses, because the entire network runs on the same computer. Our reply, constructed by `client_netinfo_cell`, has the body `00000000 04 04 7f000001 00`: zero time, the relay's address, and no address of its own.

7.8 After the Handshake

At this point, the channel is *open* and can carry circuits. A simple experiment reveals an interesting detail. If we listen after opening the channel without creating any circuits, the relay sends *nothing*: in the laboratory, not one cell arrives over fifteen seconds.

We might expect **PADDING** cells: since link protocol 5, clients and guards exchange padding cells on idle channels at random intervals between 1.5 and 9.5 seconds, so that a network observer (for example, a router recording NetFlow statistics) cannot distinguish periods of activity from periods of silence [padding-spec § Connection-level padding]. The tor source code explains why that does not happen here. The function deciding whether to add padding (`channelpadding_decide_to_pad_channel` in `src/core/or/channelpadding.c`) activates it only on channels *already used* for complete circuits or user traffic. On the relay side, a channel is considered to have been used this way only when a multi-hop circuit passes through it: the first **RELAY_EARLY** cell (the extension toward the second hop) or a **RELAY** cell on a circuit that continues beyond the relay (`src/core/or/relay.c`). A new channel remains silent, as does a channel carrying only single-hop circuits, as we will discover in chapter 9. Padding will appear in chapter 10, when our client extends a circuit.

This small episode offers a methodological lesson: *the absence of an observation is not proof*. Before concluding that padding does not work, we need to check whether the conditions under which it should appear have actually been met.

7.9 Laboratory: Opening a Verified Channel

Laboratory 1: Open a Channel to a Relay and Verify Its Identity

Objective. Write `labs/en/code/torlab/channel.py`, which opens a channel like a Tor client: TLS, **VERSIONS**, verification of **CERTS**, reading **AUTH_CHALLENGE** and **NETINFO**, and sending its own **NETINFO**.
Prerequisites. Laboratory 2 (`cells.py`, chapter 8): cell handling is the foundation of a channel, so it makes sense to write it first, even though we present it in the next chapter. The top-down explanation (channel first, then cells) and the bottom-up implementation (cells first, then channel) proceed in opposite orders here, and that is normal. You also need a running laboratory network (chapter 19) and the `cryptography` library for Ed25519.
Estimated time. 2–3 hours.

Step 1: Decode an Ed25519 Certificate. Following fig. 7.3, we read the fixed-length header, then the extensions, and retain the bytes covered by the signature:

```
def decode_ed_certificate(data):
    if len(data) < 40 + 64 or data[0] != 1:
```

```
        raise ChannelError("malformed certificate")
    _, kind, expiration, key_type = struct.unpack(">BBIB", data[:7])
    key = data[7:39]
    extension_count, pos, extensions = data[39], 40, {}
    for _ in range(extension_count):
        length, extension_type, flag = struct.unpack(">HBB", data[pos:pos+4])
        body = data[pos+4:pos+4+length]
        if extension_type != 4 and flag & 1:     # AFFECTS_VALIDATION
            raise ChannelError("unknown extension affecting validation")
        extensions[extension_type] = (flag, body)
        pos += 4 + length
    return EdCertificate(kind, expiration, key_type, key,
                         extensions, data[:pos], data[pos:])
```

Step 2: Verify the Chain. This function applies the rules in section 7.5.3 and returns the identity that has been proved:

```
def verify_responder_identity(certificates, cert_tls_der):
    id_v_sign = decode_ed_certificate(certificates[4])
    sign_v_tls = decode_ed_certificate(certificates[5])
    identity = id_v_sign.signing_key               # extension 4
    id_v_sign.verify_signature(identity)             # self-signed
    sign_v_tls.verify_signature(id_v_sign.certified_key)
    if sign_v_tls.certified_key != sha256(cert_tls_der).digest():
        raise ChannelError("TLS certificate does not match")
    for c in (id_v_sign, sign_v_tls):
        if c.expired():
            raise ChannelError("expired certificate")
    return identity
```

We obtain the TLS certificate in DER format from the socket using `sock.getpeercert(binary_form=True)`. Python returns it even though, in this case, we have not verified it.

Step 3: The Handshake. The `open_channel` function sends **`VERSIONS`**, offering versions 4 and 5 (like Arti, we omit 3). It then reads cells until **`NETINFO`** arrives, handling each type: it ignores **`VPADDING`** and any **`VERSIONS`** after the first, and rejects any other unexpected cell. Finally, it verifies the certificates, compares the identity with the expected identity, and sends its own **`NETINFO`**. The complete code is in the repository; its central loop is:

```
while netinfo is None:
    cell, buffer = _read(sock, buffer, version)
    if cell.command == CellCommand.VERSIONS:
        if version:
            continue
        version = negotiate_version(SUPPORTED_VERSIONS, cell.body)
    elif cell.command == CellCommand.CERTS:
        certificates = decode_certs(cell.body)
    elif cell.command == CellCommand.AUTH_CHALLENGE:
        ...                                      # accepted methods
    elif cell.command == CellCommand.NETINFO:
        netinfo = decode_netinfo(cell.body)
    elif cell.command != CellCommand.VPADDING:
        raise ChannelError("unexpected cell during the handshake")
```

Step 4: Tests Without a Network. In `labs/en/code/torlab/test/test_channel.py`, we generate two Ed25519 keys, build a valid certificate chain, and verify that the code accepts it. Then we break it in every way described by the specification (altered signature, uncertified signing key, different TLS certificate, expired certificate, duplicate type) and verify that each defect is detected. A test that checks only the valid case proves nothing: a verifier that accepts everything would pass it.

Step 5: Run Against the Laboratory Network. With the network running:

```
from torlab.channel import open_channel, identity_from_file

expected = identity_from_file("labs/en/lab/net/nodes/003r/fingerprint-ed25519")
c = open_channel("127.0.0.1", 5103, expected_identity=expected)
print(c.version, c.ed_identity.hex(), c.netinfo, c.auth_methods)
print(c.registry)
```

The abbreviated output is:

```
5 785d3199f21d20ad...  Netinfo(time=1790511793, ...)  [3]
['TLS TLSv1.3 TLS_AES_256_GCM_SHA384', '<- VERSIONS (6 byte)',
 '<- CERTS (1451 byte)', '<- AUTH_CHALLENGE (36 byte)',
 '<- NETINFO (509 byte)', '-> NETINFO: channel open']
```

Instead, connecting to port 5104 (relay `test004r`) while expecting another relay's identity produces the correct error, `the relay has a different identity from the expected one`.

Summary

- A channel is a TLS session between two adjacent nodes. Only relays accept channels, and one channel carries many circuits.
- A relay's identity is its Ed25519 key `KP_relayid_ed`, which certifies a signing key `KP_relaysign_ed`; the RSA identity remains for compatibility.
- A relay's TLS certificate is self-signed. Authentication takes place inside the channel through the **`CERTS`** cell, which binds the identity, signing key, and TLS-certificate fingerprint.
- The initiator must verify signatures, expiration dates, the TLS binding, and the **expected identity**.
- Relays initiating a channel authenticate with **`AUTHENTICATE`**, a signature bound to the TLS session and the conversation. Clients and bridges do not authenticate, so as not to create recognizable identities.
- **`NETINFO`** ends the handshake. A client sends a zero timestamp and none of its own addresses.
- Link padding requires the channel to have already carried a *complete* circuit or user traffic. A new channel, or one that carries only single-hop circuits, stays silent. Once that threshold has been crossed, padding also starts before the first application traffic (`nf_pad_before_usage`, which defaults to 1).

Exercises

Exercise 7.1 ⋆ Why does a Tor client not need a network-facing listening port? In which cases does a relay initiate channels?

Exercise 7.2 ⋆ An Ed25519 certificate has EXPIRATION_DATE equal to `0x000772F7`. When does it expire? Why did the specification choose hours rather than seconds?

Exercise 7.3 ⋆ An **`AUTH_CHALLENGE`** cell has a 38-byte body. How many methods does it offer? Is a 36-byte cell with N_Methods equal to 2 valid?

Exercise 7.4 ⋆⋆ An adversary records a genuine relay's **`CERTS`** cell and replays it unchanged on their own server, presenting their own TLS certificate. Which verification step fails? What if the adversary also presented the genuine relay's TLS certificate?

Exercise 7.5 ⋆⋆ Write in hexadecimal the body of the **`NETINFO`** cell sent by a relay at `203.0.113.5`, at 12:00:00 UTC on January 1, 2026, to a client that contacted it from `198.51.100.7`.

Exercise 7.6 ⋆⋆ Suppose Tor clients authenticated to guards with a key generated once and stored on disk. Describe two concrete ways in which this would harm user anonymity.

Exercise 7.7 ⋆⋆ Extend `labs/en/code/torlab/channel.py` with optional verification of the RSA identity (types 2 and 7), following section 7.5. Compare the resulting RSA key's SHA-1 fingerprint with the relay's `fingerprint` file. *Hint:* the fingerprint is the SHA-1 of the public key's DER encoding.

Exercise 7.8 ⋆⋆⋆ With the laboratory network running, open a channel with `labs/en/code/torlab/channel.py` and keep it open while client `test008c` downloads a page through the network. Will you receive **`PADDING`** cells on *your* channel? Form a hypothesis, test it, and explain your result in light of `channelpadding_decide_to_pad_channel`.

Exercise 7.9 ⋆⋆⋆ Explain why the TLSSECRETS field in **`AUTHENTICATE`** prevents an intermediary from relaying relay A's authentication to relay B over a different TLS session, even if the intermediary forwards every cell faithfully. What would happen if TLSSECRETS did not exist?

Further Reading

- *Tor Protocol Specification*, the "Channels", "Negotiating and initializing channels", and "Relay keys" chapters, and *Tor certificate formats* (cert-spec) [Thee].
- Proposal 220, *Migrate server identity keys to Ed25519*: the reasons for moving from RSA.
- *Tor channel padding*, in the padding specification: the NetFlow-based adversary model and the choice of intervals.
- RFC 5705 and RFC 8446 (§7.5): TLS *keying material exporter* functions.

Chapter 8

Cells

Last verified: 23 September 2026 — torspec commit `8e1f245`; tor 0.4.9.13; Arti 2.6.0; link protocol 3–5

Chapter objectives

By the end of this chapter, you will be able to:

- describe the fixed-length and variable-length cell formats byte by byte;
- explain why Tor uses fixed-size units, what that design choice protects, and what it does not;
- interpret CircIDs and the rules used to choose them on each channel;
- identify every cell command and the principal relay commands;
- read a relay message header and explain the purposes of `Recognized` and `Digest`;
- write a Python cell encoder/decoder (Laboratory 2).

In the previous chapter we saw how two Tor nodes open a *channel*: a TLS connection over which they agree on a protocol version and authenticate each other. Here we examine *what travels* inside that channel. The answer is simple: **cells**. Everything Tor carries, from the construction of a circuit to downloaded web pages, is divided into cells. Understanding cells gives us the alphabet in which the entire protocol is written.

8.1 Why Cells?

A low-latency anonymity system cannot conceal *that* two nodes communicate, but it can try to conceal *what* they exchange and *how much* intermediate nodes can learn about it. If each node forwarded data exactly as it arrived, in packets of varying sizes, an observer could trace a 1,337-byte message through the network simply by looking for a message of that length on each link.

The original onion-routing design, followed by Tor [DMS04], therefore adopted a **fixed-size** transmission unit: the cell. When all cells have the same size:

- the length of an individual message reveals neither its contents nor the identity of the flow;
- relays can manage queues and buffers simply and efficiently;
- a minimal header allows a relay to forward a cell after reading very few bytes.

Warning: What cells do *not* conceal

Cells standardize the *size* of individual units, not the *number* of units or their *timing*. An adversary observing both ends of a circuit can still correlate traffic volumes and timing. This is a fundamental limitation of Tor, which we will address in chapter 26. Partial defenses—channel- and circuit-level padding—are discussed in chapters 7 and 26 and Part V.

Historical note: 512 bytes

The 2004 paper introducing Tor describes 512-byte cells: a 2-byte circuit identifier, a 1-byte command, and 509 bytes of data [DMS04]. The 509-byte body has survived to the present day. The circuit identifier, however, grew to 4 bytes with link protocol 4, increasing the cell size to 514 bytes.

8.2 From the TLS Stream to Cells

Once the TLS handshake is complete, the two parties exchange cells *in sequence*, one after another, as a continuous byte stream [tor-spec § Cells (messages on channels)]. TLS record boundaries and cell boundaries are independent: one cell may be split across several records, and one record may contain several cells or cell fragments. The specification nevertheless requires that "the framing of TLS records MUST NOT leak information about the type or contents of the cells".

This has a practical consequence for implementers: the receiver cannot expect to read "one cell at a time" from the socket. It must accumulate bytes in a buffer and extract a cell only when the entire cell is available, leaving any remaining bytes in the buffer. Our Laboratory 2 decoder will work exactly this way.

8.3 Cell Formats

There are two cell formats: **fixed-length** and **variable-length**. Both begin with the same two fields, `CircID` and `Command`.

8.3.1 Constants

The specification defines the following lengths [tor-spec § Message lengths], where v is the link protocol version negotiated on the channel:

Name	Bytes	Meaning
`CELL_BODY_LEN`	509	fixed-length cell body
`CIRCID_LEN`(v), $v < 4$	2	CircID length
`CIRCID_LEN`(v), $v \geq 4$	4	
`CELL_LEN`(v), $v < 4$	512	fixed-length cell size
`CELL_LEN`(v), $v \geq 4$	514	

For every v, `CELL_LEN`$(v) = 1 +$ `CIRCID_LEN`$(v) +$ `CELL_BODY_LEN`.

In depth: An older name

Older code and documentation call `CELL_BODY_LEN` by the name `PAYLOAD_LEN`. Both denote the same value: 509.

8.3.2 Fixed-Length Cells

Figure 8.1 shows a fixed-length cell with the current link protocol ($v \geq 4$).

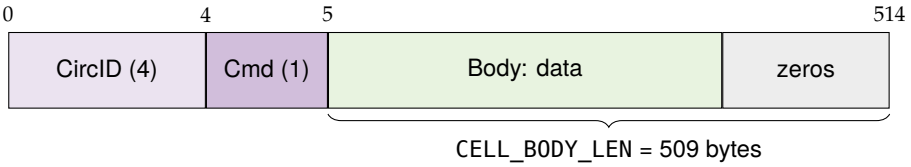


Figure 8.1: Fixed-length cell (link protocol ≥ 4): a 4-byte CircID, a 1-byte command, and a 509-byte body, totaling 514 bytes. If the useful content is shorter than 509 bytes, the rest of the body is filled with zeros. The numbers indicate byte offsets from the beginning of the cell.

When the information to be transmitted occupies fewer than 509 bytes, the sender **must** fill the remaining body bytes with zeros, and the receiver **must** ignore them [tor-spec § Padding fixed-length cell bodies].

8.3.3 Variable-Length Cells

Some cells, particularly those used for the channel handshake, need more than 509 bytes (to carry a certificate, for example) or far fewer. The variable-length format handles these cases (fig. 8.2) by adding a 2-byte *big-endian* integer field, `Length`, and omitting padding.

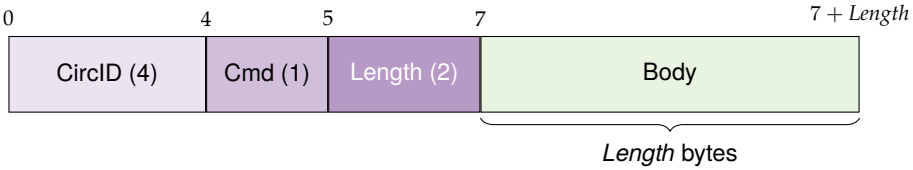


Figure 8.2: Variable-length cell (link protocol ≥ 4): after CircID and the command, a 2-byte, big-endian Length field specifies how many body bytes follow. The body can therefore contain between 0 and 65 535 bytes.

8.3.4 Distinguishing the Two Formats

The receiver needs to know how many bytes to read *before* it has read the entire cell. The rule depends entirely on the command [tor-spec § Cells (messages on channels)]:

- command 7 (**`VERSIONS`**) identifies a variable-length cell;
- every other command below 128 identifies a fixed-length cell;
- commands numbered 128 and above identify variable-length cells.

In other words, the most significant bit of the command byte serves as the format indicator, with the single historical exception of **`VERSIONS`**.

Historical note: The VERSIONS exception

In link protocol 1, all cells had fixed length. Link protocol 2 introduced version negotiation, and **`VERSIONS`**, assigned value 7 after the existing commands (0–6), became the only variable-length cell. Link protocol 3 reserved the range ≥ 128 for variable-length commands. Versions 1 and 2 are now obsolete and implementations should not support them.

8.3.5 The Special Case of VERSIONS

There is a chicken-and-egg problem: the CircID length depends on the negotiated version, but that version is itself negotiated through the **`VERSIONS`** cell. The solution is to **always** send **`VERSIONS`** as though $v = 0$, with a 2-byte CircID. Its body is a sequence of 2-byte, big-endian integers, one for each supported version. A client supporting versions 3, 4, and 5 therefore sends:

Bytes	Field	Meaning
`00 00`	CircID	0, 2-byte CircID
`07`	Command	**`VERSIONS`**
`00 06`	Length	6-byte body
`00 03 00 04 00 05`	Body	versions 3, 4, 5

Both parties choose the **highest version present in both `VERSIONS`** cells. If they share no version, or if the body has an odd length, they must close the connection [tor-spec § Negotiating versions with VERSIONS cells].

Version	Description
1	(obsolete) initial certificate-based handshake
2	(obsolete) TLS renegotiation handshake; introduces variable-length cells
3	(obsolete) current-style "in-protocol" handshake (chapter 7)
4	4-byte CircID
5	support for link padding and its negotiation

In current implementations, tor offers versions 3, 4, and 5 (the `or_protocol_versions` array in `src/core/or/connection_or.c`), while Arti offers only versions 4 and 5. Two up-to-date instances therefore always negotiate version 5.

8.4 CircID

The `CircID` field identifies the circuit to which the cell belongs. Its value is 0 for cells that are not tied to a circuit (for example, **`VERSIONS`**, **`NETINFO`**, and **`PADDING`**).

The crucial point is that **CircIDs are local to a channel**. A three-hop circuit crosses three channels (client–guard, guard–middle, and middle–exit) and has a *different* CircID on each. Each relay maintains a forwarding table that maps an incoming (channel, CircID) pair to an outgoing (channel, CircID) pair, as shown in fig. 8.3.

This choice has two advantages: nodes do not need to coordinate their identifier choices, and the CircID does not become a global label that can be used to connect different segments of the same circuit.

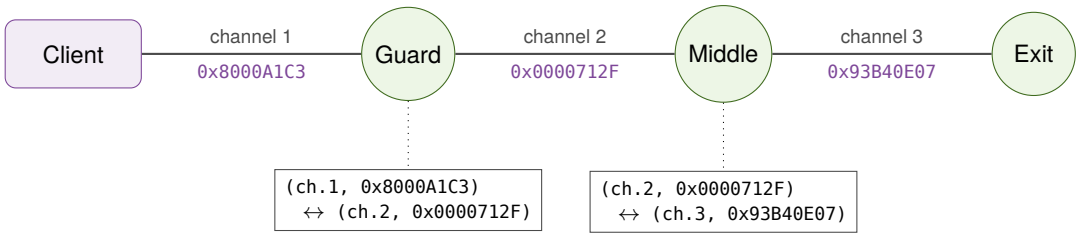


Figure 8.3: The same circuit has a different CircID on every channel. The guard and middle relay translate the identifier using a forwarding table. An observer on one link therefore cannot identify the same circuit on another link by comparing CircIDs. The values are illustrative but follow the most-significant-bit rule described below: on channel 2 the bit is zero, meaning the middle relay opened that connection and the guard is reusing it.

8.4.1 Who Chooses the CircID?

The CircID is chosen by the node sending the circuit-creation cell (**`CREATE2`**, chapter 9). Because either side of a channel can create circuits, the two sides must avoid choosing the same value at the same time. The rule depends on the version [tor-spec § Choosing circuit IDs in create cells]:

- **link protocol** ≥ 4: the node that *opened* the connection uses CircIDs whose most significant bit (MSB) is 1; the other node uses CircIDs whose MSB is 0;
- **link protocol** ≤ 3: the two halves of the space were assigned by comparing the two relays' identity public keys (the relay with the smaller key used an MSB of 0).

The value 0 is reserved; no other value is. Existing implementations choose CircIDs **at random** from the available values, and the specification recommends that new implementations do the same to avoid being distinguishable. Tor gives up trying to create new circuits on a channel after 64 consecutive attempts to find an unused value.

In depth: Why 4 bytes?

With 2 bytes and only half the space available, a channel can carry just over 32 000 circuits per side, and random selection produces collisions well before the space is exhausted. Proposal 214 (2012) states the problem plainly: "relays are running out of circuit IDs". The proposal introduced link protocol 4 with 4-byte CircIDs, implemented in tor 0.2.4.11-alpha.

8.5 Cell Commands

The `Command` byte determines how the body is interpreted. The following tables list every defined value [tor-spec § Interpreting the fields: Command]. In column C, "yes" means the command requires a nonzero CircID and "no" means it requires a zero CircID. Column P gives the first link protocol version supporting the command (a blank means at least version 3).

Unknown commands. No other values are permitted. A receiver of an unknown command *may* silently discard the cell and *may* close the channel. This strictness follows from the fact that each new command is introduced with a new link protocol version: a correct implementation will never send a command that the other side did not declare it supports. Before the link

Value	Command	C	P	Purpose
0	**PADDING**	no		link padding
1	**CREATE**	yes		create circuit (TAP, deprecated)
2	**CREATED**	yes		response to CREATE (deprecated)
3	**RELAY**	yes		end-to-end data
4	**DESTROY**	yes		destroy a circuit
5	**CREATE_FAST**	yes		create circuit without public-key cryptography
6	**CREATED_FAST**	yes		response to CREATE_FAST
8	**NETINFO**	no		time and addresses
9	**RELAY_EARLY**	yes		like RELAY, in limited numbers
10	**CREATE2**	yes		create circuit (extensible handshake)
11	**CREATED2**	yes		response to CREATE2
12	**PADDING_NEGOTIATE**	yes	5	padding negotiation

Table 8.1: Fixed-length cell commands.

Value	Command	C	Purpose
7	**VERSIONS**	no	link protocol negotiation
128	**VPADDING**	no	variable-length padding
129	**CERTS**	no	certificates
130	**AUTH_CHALLENGE**	no	authentication challenge
131	**AUTHENTICATE**	no	initiator authentication
132	**AUTHORIZE**	no	reserved

Table 8.2: Variable-length cell commands.

protocol was versioned, by contrast, unknown commands were ignored on the assumption that they came from a newer version of Tor.

Commands can be grouped by purpose:

Channel management **VERSIONS**, **CERTS**, **AUTH_CHALLENGE**, **AUTHENTICATE**, **NETINFO**: the handshake from chapter 7.

Padding **PADDING**, **VPADDING**, **PADDING_NEGOTIATE**: dummy traffic intended to hinder traffic analysis on a link.

Circuit lifecycle **CREATE2**/**CREATED2** (and their historical variants), **DESTROY**: circuit creation and destruction (chapter 9).

Transport **RELAY** and **RELAY_EARLY**: everything that travels *inside* a circuit.

8.5.1 DESTROY

A **DESTROY** cell's body contains just one byte specifying the *reason* for closing the circuit [tor-spec § Tearing down circuits]. A node receiving **DESTROY** releases the circuit's resources and, unless it is an endpoint, propagates a **DESTROY** cell to the next node.

0	NONE	7	OR_IDENTITY
1	PROTOCOL	8	CHANNEL_CLOSED
2	INTERNAL	9	FINISHED
3	REQUESTED	10	TIMEOUT
4	HIBERNATING	11	DESTROYED
5	RESOURCELIMIT	12	NOSUCHSERVICE
6	CONNECTFAILED		

Warning: The reason code as a side channel

The specification recommends **always** sending reason NONE and not propagating a received reason code. If a relay forwarded the original reason in both directions, two colluding relays on the same circuit could use it as a signal to detect that they were on that circuit: a small example of how even one byte can become a *side channel*.

8.5.2 RELAY_EARLY

RELAY_EARLY has the same format as **RELAY**, but its purpose is to **limit circuit length** [tor-spec § Handling RELAY_EARLY cells]. The rules are:

- a client must send circuit-extension messages (`EXTEND2`) only inside **RELAY_EARLY** cells; since version 0.2.3.11-alpha, relays reject extensions received otherwise;
- a relay receiving more than 8 forward **RELAY_EARLY** cells on a circuit should close it; if it receives even one *backward* cell, toward the client, it must close the circuit immediately;
- to avoid revealing circuit length, clients should also send approximately the first 8 cells not intended for the first hop as **RELAY_EARLY**.

Without this limit, an adversary could build extremely long circuits that traverse the same relays repeatedly, thereby amplifying the adversary's own traffic. This is the long-path congestion attack described by Evans, Dingledine, and Grothoff [EDG09].

Historical note: An unexpected signal

The rule forbidding backward **RELAY_EARLY** cells arose from a 2014 incident: some relays sent **RELAY_EARLY** cells toward clients as a covert channel to mark onion-service users' circuits. We will examine this incident in detail in chapter 27.

8.6 Inside RELAY Cells

RELAY and **RELAY_EARLY** cells carry **relay messages**: the commands exchanged between the client and the nodes of a circuit, from circuit extension to TCP stream data. Their 509-byte bodies are encrypted in layers, one for each hop. This is the "onion" of chapter 10. From the channel's perspective, a RELAY cell body therefore appears completely filled: no zero padding is visible.

After every encryption layer has been removed, the plaintext body follows the traditional format shown in fig. 8.4 [tor-spec § Relay cells].

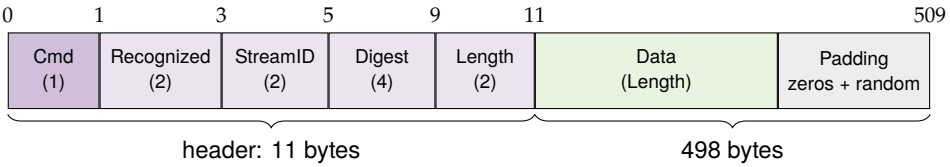


Figure 8.4: *Decrypted* traditional RELAY cell body: an 11-byte header, up to 498 data bytes, and optional padding up to 509 bytes. The specification recommends up to four initial zero bytes followed by random bytes; if fewer than four bytes remain, they are all zero, and with 498 data bytes no padding remains.

Relay command (1 byte) The message type (table 8.3).

Recognized (2 bytes) Zero in a plaintext message. This field lets a relay quickly determine whether the cell is addressed to it: if, after removing its own layer, the field is nonzero, the cell is certainly still encrypted and must be forwarded.

StreamID (2 bytes) The stream to which the message refers, chosen by the client; zero for messages concerning the circuit as a whole.

Digest (4 bytes) The first 4 bytes of a *running* digest over all messages exchanged with that hop (SHA-1 for ordinary circuits, SHA3-256 for onion-service circuits). It confirms that the cell is intended for this relay and has not been modified.

Length (2 bytes) The number of significant bytes in the Data field.

Data The message contents.

Padding The remainder, bringing the body to 509 bytes.

An 11-byte header leaves at most $509 - 11 = 498$ data bytes per cell.

8.6.1 Recognized and Digest: A Two-Stage Filter

Why two fields? Computing the digest for every cell in transit would be expensive. Recognized is a cheap filter: after decryption of a cell that is still encrypted, its 2 bytes are pseudorandom and equal zero only with probability 2^{-16}. Only when Recognized is zero does the relay compute the digest. If the digest matches, the cell is "recognized" and processed; otherwise it is forwarded. We will examine the calculation, and its weaknesses, in chapter 10.

8.6.2 Relay Message Padding

Relay message padding is different from the zero padding of fixed-length cells. The specification recommends 4 zero bytes followed by **random bytes** [tor-spec § Relay cells]. Random bytes make cell contents unpredictable. This is necessary for the authenticated SENDME messages of Proposal 289, which prove that data was actually received by including the digest of received cells (chapter 11). Receivers must not depend on the contents of the padding.

8.6.3 Relay Commands

Table 8.3 lists the defined relay commands. The Dir. column indicates direction: F (*forward*, from the client into the circuit only), B (*backward*, toward the client only), or F/B (either direction). C indicates a *control* message, which must have a zero StreamID.

If a cell is recognized but its relay command is unknown, it must be discarded; its contents nevertheless contribute to the digest and the flow-control windows.

8.7 Toward a New Format: Proposal 340 and CGO

The traditional format has two known limitations. The first concerns efficiency: a large message always occupies at least one whole cell, while several small messages cannot share a cell. The second concerns security: the 4-byte digest and unauthenticated AES-CTR encryption leave room for *tagging* attacks, which we will examine in chapter 10.

Proposal 340 (*Packed and fragmented relay messages*) separates the concept of a *cell* from that of a *message*: one message can be fragmented across several cells, and multiple messages can

Value	Command	Dir.	Purpose
Core protocol			
1	BEGIN	F	open a stream
2	DATA	F/B	stream data
3	END	F/B	close a stream
4	CONNECTED	B	stream successfully opened
5	SENDME	F/B, C?	receipt acknowledgment (flow control)
6–7	EXTEND/EXTENDED	F/B, C	extension using TAP (obsolete)
8–9	TRUNCATE/TRUNCATED	F/B, C	circuit truncation (unused)
10	DROP	F/B, C	end-to-end padding
11–12	RESOLVE/RESOLVED	F/B	remote DNS resolution
13	BEGIN_DIR	F	stream to a directory cache
14–15	EXTEND2/EXTENDED2	F/B, C	circuit extension
16–18	*reserved*		UDP over Tor (Proposal 339)
Conflux			
19–22	CONFLUX_...	F/B, C	LINK, LINKED, LINKED_ACK, SWITCH: multiple circuits (Proposal 329)
Onion services			
32	ESTABLISH_INTRO	F, C	establish an introduction point
33	ESTABLISH_RENDEZVOUS	F, C	establish a rendezvous point
34–35	INTRODUCE1/INTRODUCE2	F/B, C	introduction request
36–37	RENDEZVOUS1/RENDEZVOUS2	F/B, C	join the rendezvous
38–40	..._ESTABLISHED, INTRODUCE_ACK	B, C	acknowledgments
Circuit padding and flow control			
41–42	PADDING_NEGOTIATE/...D	F/B, C	circuit padding
43–44	XOFF/XON	F/B	stream flow control (Proposal 324)

Table 8.3: Relay commands. SENDME is a control message only when it concerns the circuit rather than an individual stream (chapter 11).

be packed into a single cell. Proposal 359 (*Counter Galois Onion, Updated*) introduces a new relay encryption scheme, CGO, and adopts the cell format of Proposal 340. A CGO-decrypted cell has the following structure:

Field	Bytes
authentication tag	16
body	$509 - 16 = 493$
within the body: command	1
length	2
StreamID (for certain commands only)	2
data	*length*
padding (4 zeros, then random bytes)	remainder

StreamID appears only for commands that require it (BEGIN, DATA, END, CONNECTED, RESOLVE, RESOLVED, BEGIN_DIR, XON, XOFF). Recognized and Digest disappear, replaced by the tag. A client uses CGO with a relay only when both support it (subprotocol `Relay=6`) and negotiate it during circuit creation.

In depth: How much data per CGO cell?

The format shows that a DATA message has $493 - 1 - 2 - 2 = 488$ data bytes available, 10 fewer than in the traditional format: this is the cost of the 16-byte authentication tag.

CGO is no longer merely a proposal: tor has supported it since the 0.4.9 series (introduced in 0.4.9.3-alpha, September 2025). In Arti, support is always enabled; the `counter-galois-onion` build option remains only for compatibility. The specifications repository still lists both proposals as "Open": proposal status follows the standardization process and does not always match the state of the code.

8.8 Anatomy of a Session

We can now put the pieces together and follow, cell by cell, what a client sends its guard to build a three-hop circuit and download a web page. Figure 8.5 summarizes the sequence; subsequent chapters examine each step in detail.

1. **Channel handshake**: **`VERSIONS`**, followed by **`CERTS`**, **`AUTH_CHALLENGE`**, and **`NETINFO`** from the guard; the client replies with **`NETINFO`**. CircID is always 0.

2. **First hop**: **`CREATE2`** with a new CircID (MSB 1, because the client opened the connection); the guard replies with **`CREATED2`**.

3. **Second and third hops**: two **`RELAY_EARLY`** cells containing `EXTEND2`; the guard, followed by the middle relay, transforms them into **`CREATE2`** cells sent toward the next node. The replies return as **`RELAY`** cells containing `EXTENDED2`.

4. **Stream**: a cell with `BEGIN` for the exit, which replies with `CONNECTED`; then `DATA` in both directions, interspersed with `SENDME`.

5. **Closure**: `END` for the stream, followed at the end of the circuit's lifetime by **`DESTROY`**.

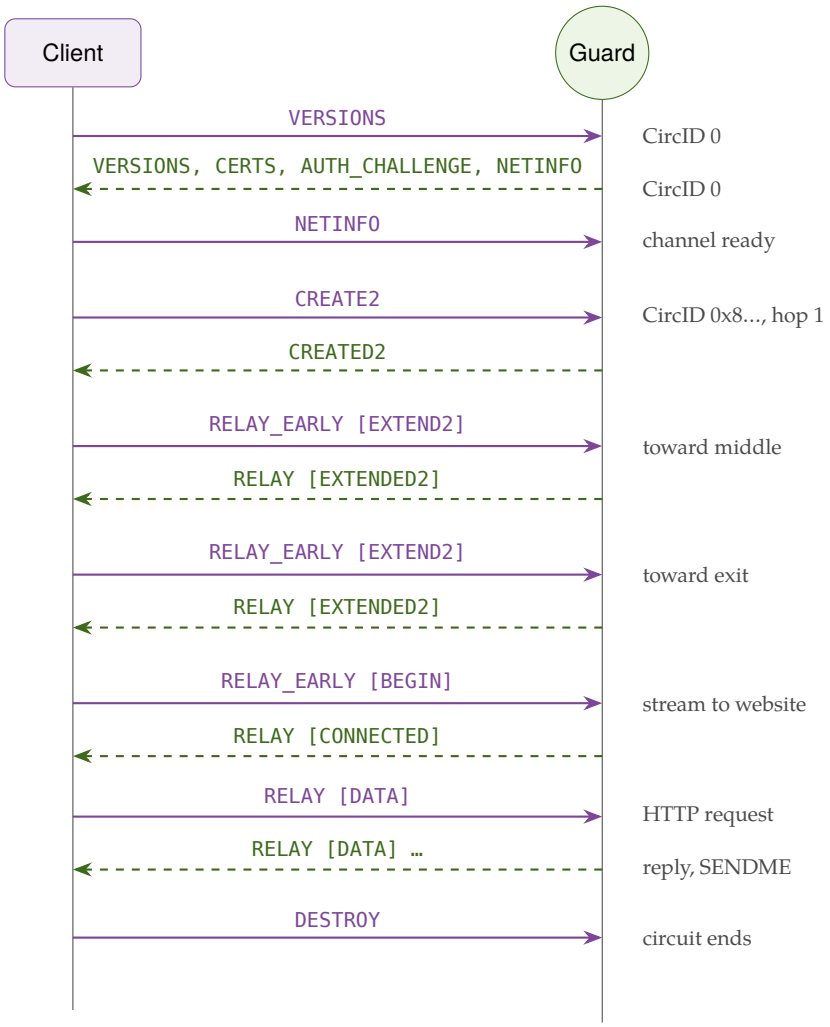


Figure 8.5: Cells exchanged between a client and guard over the TLS channel to create a three-hop circuit and open a stream. The guard sees only cells on its own channel: it cannot read the contents of RELAY cells intended for the middle and exit relays, because they are encrypted with keys it does not possess.

8.9 Laboratory: A Cell Encoder

Laboratory 2: Encoding and Decoding Cells

Objective. Write the `labs/en/code/torlab/cells.py` module, which you will use in all subsequent laboratories.
Prerequisites. Python 3.10 or later, `pytest`. No network is needed: this laboratory operates on manually constructed bytes.
Estimated time. 1–2 hours.

Step 1: Constants and commands. We turn the specification's tables into Python enumerations so that each numerical value appears in just one place:

```
CELL_BODY_LEN = 509
RELAY_HEADER_LEN = 11
RELAY_DATA_MAX = CELL_BODY_LEN - RELAY_HEADER_LEN   # 498

class CellCommand(IntEnum):
    PADDING = 0
    CREATE = 1
    # ... all values from Tables 8.1 and 8.2
    AUTHORIZE = 132

def circid_len(version):
    return 2 if version < 4 else 4

def is_variable_length(command):
    return command == CellCommand.VERSIONS or command >= 128
```

Step 2: Encoding. This function takes a CircID, a command, a body, and a link protocol version, and produces the bytes to send. Note the special handling of **`VERSIONS`** and zero padding:

```
def encode(cell, version):
    if cell.command == CellCommand.VERSIONS:
        version = 0                  # CircID is always 2 bytes
    n = circid_len(version)
    header = cell.circ_id.to_bytes(n, "big") + bytes([cell.command])
    if is_variable_length(cell.command):
        return (header + struct.pack(">H", len(cell.body))
                + cell.body)
    return header + cell.body.ljust(CELL_BODY_LEN, b"\x00")
```

Step 3: Decoding a Stream. Because bytes arrive from the TLS socket in fragments, the decoder returns `None` until a complete cell is available. Otherwise, it returns the cell and any unused bytes:

```
def decode(buffer, version):
    n = circid_len(version)
    if len(buffer) < n + 1:
        return None
    circ_id = int.from_bytes(buffer[:n], "big")
    command = buffer[n]
    if is_variable_length(command):
```

```
        if len(buffer) < n + 3:
            return None
        (length,) = struct.unpack(">H", buffer[n+1:n+3])
        start, end = n + 3, n + 3 + length
    else:
        start, end = n + 1, n + 1 + CELL_BODY_LEN
    if len(buffer) < end:
        return None
    return Cell(circ_id, command, buffer[start:end]), buffer[end:]
```

Step 4: VERSIONS and Version Negotiation.

```
def negotiate_version(ours, received_body):
    if len(received_body) % 2:
        raise CellError("VERSIONS has an odd number of bytes")
    theirs = {v for (v,) in struct.iter_unpack(">H", received_body)}
    common = set(ours) & theirs
    if not common:
        raise CellError("no common link-protocol version")
    return max(common)
```

The diagnostic messages shown in this example are in English, as in `labs/en/code/torlab/cells.py`; identifiers and protocol constants remain unchanged.

Step 5: The Relay Message. Finally, we construct the 11-byte header. The `">BHH4sH"` format string for `struct` matches the fields in fig. 8.4 exactly:

```
def encode_relay(msg, random_bytes=os.urandom):
    header = struct.pack(">BHH4sH", msg.command, msg.recognized,
                                msg.stream_id, msg.digest, len(msg.data))
    space = CELL_BODY_LEN - RELAY_HEADER_LEN - len(msg.data)
    padding = b"\x00" * min(4, space) + random_bytes(max(0, space - 4))
    return header + msg.data + padding
```

The `random_bytes` parameter allows tests to replace random bytes with predictable values.

Verification. The repository includes tests in `labs/en/code/torlab/test/test_cells.py`. Run them from the repository root with `labs/en/code` on `PYTHONPATH`:

```
$ PYTHONPATH=labs/en/code python3 -m pytest -q labs/en/code/torlab/test/test_cells.py
........                                                 [100%]
8 passed
```

(Without the filename, `pytest` runs the tests for all laboratories included in the repository.) For example, one test checks that the **`VERSIONS`** cell for versions 3, 4, and 5 matches the example in section 8.3 byte for byte: `0000 07 0006 0003 0004 0005`.

Extensions. In Laboratory 1 (chapter 7), this module will be used to communicate with a real relay on the private network. In Laboratory 4, we will add digest calculation and layered encryption.

Summary

- Cells are Tor's transmission units on a channel. Since link protocol 4, fixed-length cells contain 514 bytes (CircID 4 + command 1 + body 509); earlier versions used 512 bytes.

- Variable-length cells add a 2-byte Length field. **VERSIONS** (7) and all commands ≥ 128 are variable-length.
- **VERSIONS** always uses a 2-byte CircID; both sides choose the highest common version.
- CircIDs are local to each channel. The side that opened a connection chooses random values with the MSB set to 1.
- **DESTROY** should always carry reason NONE to avoid creating side channels; **RELAY_EARLY** limits circuit length.
- A decrypted RELAY cell body has an 11-byte header (command, Recognized, StreamID, Digest, Length) and at most 498 data bytes. Its padding comprises 4 zero bytes followed by random bytes.
- Proposals 340 and 359 (CGO) introduce a new format with a 16-byte authentication tag and an optional StreamID.

Exercises

Exercise 8.1 ⋆ How many bytes does a **CREATE2** cell occupy on a channel that negotiated link protocol 3? What about version 5? How many bytes does a **VERSIONS** cell with body `00 04 00 05` occupy?

Exercise 8.2 ⋆ For each of the command values 0, 7, 9, 12, 129, and 200, state whether the cell is fixed-length or variable-length. Which of these values is not permitted?

Exercise 8.3 ⋆ A client opens a channel to a guard using link protocol 5. Which of the following CircIDs can it use for a new circuit: `0x00000000`, `0x12345678`, or `0x92345678`? Why?

Exercise 8.4 ⋆⋆ Manually decode the following byte sequence received on a channel using link protocol 4. Give the CircID, command, and body length of each cell:
`00 00 00 00 81 00 03 aa bb cc 80 00 00 07 04 00 …`
(The second cell continues with the rest of its body, which is not shown.)

Exercise 8.5 ⋆⋆ Write the first 11 bytes, in hexadecimal, of the plaintext body of a relay DATA message on stream 5, containing 300 data bytes, before digest calculation. How many padding bytes follow, and how many of them are guaranteed to be zero?

Exercise 8.6 ⋆⋆ An intermediate relay processes $1\,000\,000$ cells that are not addressed to it. On average, how many of them will require digest calculation? Why is the Recognized field alone insufficient to determine whether a cell is intended for this relay?

Exercise 8.7 ⋆⋆ Describe a concrete scenario in which two colluding relays could exploit the reason code in a **DESTROY** cell to learn that they are on the same circuit. Why does using reason NONE prevent this scenario?

Exercise 8.8 ⋆⋆ A client downloads a 1 MiB file ($1\,048\,576$ bytes) through a circuit. How many DATA cells are needed with the traditional format? How many with CGO? In each case, what percentage of the useful data is added as cell-header overhead (link protocol 4, excluding TLS)?

Exercise 8.9 ⋆⋆⋆ Extend Laboratory 2 with a *property-based* test (for example, using the Hypothesis library) that generates random fixed-length and variable-length cells, encodes and concatenates them, and splits the result into fragments of random lengths. Verify that decoding returns precisely the original cells.

Exercise 8.10 ⋆⋆⋆ Find the functions that encode and decode channel cells in the Arti source code (the `tor-cell` crate). Compare their handling of an "incomplete cell" with the `decode` function in Laboratory 2.

Further Reading

- *Tor Protocol Specification*, the sections "Cells", "Relay cells", and "Negotiating channels" [Thee]: the primary source for this chapter.
- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04], Section 4: the original cell format and its motivation.
- N. Evans, R. Dingledine, C. Grothoff, *A Practical Congestion Attack on Tor Using Long Paths* [EDG09]: the attack that motivated **`RELAY_EARLY`**.
- Proposals 340 and 359 in torspec: the future cell format.

Chapter 9

Circuit Construction: TAP, CREATE_FAST, ntor, and ntor-v3

Last verified: 23 September 2026 — torspec commit `8e1f245`; tor 0.4.9.13; proposals 216, 332, 350

Chapter objectives

By the end of this chapter, you will be able to:

- describe how a client builds a circuit one hop at a time with CREATE2 and EXTEND2;
- explain what a circuit handshake must guarantee: relay authentication, secrecy, and forward secrecy;
- follow the CREATE_FAST, ntor, and ntor-v3 handshakes step by step, and understand why TAP was abandoned;
- derive the four keys for one hop (Df, Db, Kf, Kb) from the negotiated key material;
- verify an implementation against the official test vectors;
- build real circuits to a relay on the laboratory network (Laboratory 3).

With a channel open (chapter 7) and the alphabet of cells at our disposal (chapter 8), we have everything we need to construct Tor's central structure: the **circuit**. In this chapter, we will see how the client establishes a secret shared with each relay along the path, known to no other party, and how it turns that secret into the keys that encrypt the traffic. The use of those keys in layered encryption is the subject of chapter 10.

9.1 One Hop at a Time

The client constructs the circuit *incrementally*, or "telescopically" [DMS04]:

1. It chooses a path, for example guard R_1, middle R_2, and exit R_3 (chapter 13).

2. Over its channel to R_1, it sends a **`CREATE2`** cell containing the first half of a cryptographic handshake. R_1 responds with **`CREATED2`**, after which the client and R_1 share key material.

3. It sends an `EXTEND2` message, encrypted for R_1, asking R_1 to extend the circuit to R_2. That message contains the first half of a new handshake, this time intended for R_2.

4. R_1 opens (or reuses) a channel to R_2 and sends it a **`CREATE2`** cell carrying that handshake data. The **`CREATED2`** response returns to R_1, which forwards it to the client in an `EXTENDED2` message.

5. The client completes the handshake with R_2 and repeats the process for R_3, this time through both R_1 and R_2.

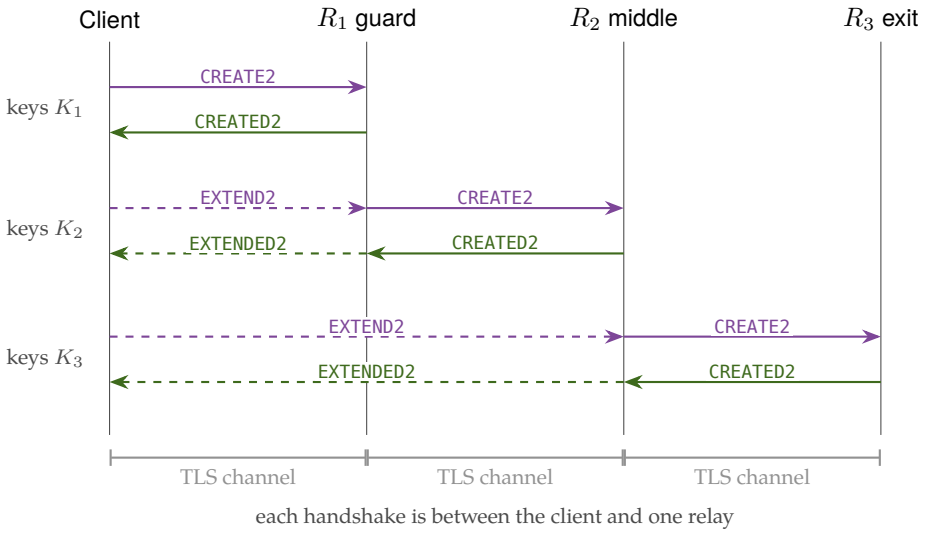


Figure 9.1: Telescopic construction of a three-hop circuit. Each handshake takes place between the client and just one relay; intermediate relays merely carry the messages. Dashed arrows indicate messages traveling within the circuit, encrypted using keys from hops that have already been established.

This arrangement has a fundamental consequence. The handshake with R_3 passes through R_1 and R_2, but it is encrypted for R_3's public key: neither R_1 nor R_2 can derive its shared secret. And because each hop uses a different secret, no relay can read what the client exchanges with the others. Guard R_1 knows who the client is but not where the traffic goes; exit R_3 knows where the traffic goes but not who sent it.

Historical note: First-Generation Onion Routing

In the original Naval Research Laboratory system, the client built a single "onion": a message encrypted in layers using the public keys of every relay. Each relay peeled off one layer to discover the next hop and its own session key (chapter 3). A relay compromised in the future could have decrypted recorded onions. Tor's telescopic circuit, with an ephemeral Diffie-Hellman exchange for every hop, introduced *forward secrecy*: once session keys are erased, even possession of long-term keys does not make it possible to decrypt past traffic [DMS04].

9.2 Cells and Messages

9.2.1 CREATE2 and CREATED2

The **`CREATE2`** format is deliberately generic [tor-spec § Create and Created cells]: a handshake type (HTYPE, 2 bytes), a length (HLEN, 2 bytes), and the data (HDATA). **`CREATED2`** contains only HLEN and HDATA, since the type is already known. Everything fits into a fixed-length cell, so handshake data cannot exceed $509 - 4 = 505$ bytes in **`CREATE2`**.

HTYPE	Handshake	Status
`0x0000`	TAP	obsolete; removed in tor 0.4.9.1-alpha
`0x0001`	—	reserved
`0x0002`	ntor	in use
`0x0003`	ntor-v3	in use; preferred

If the handshake succeeds, the relay responds with **`CREATED2`**; if it fails, it responds with **`DESTROY`**. The new cell's CircID follows the rules in section 8.4: in link protocol 4 and later, the party that opened the channel uses values with the most significant bit set to 1.

9.2.2 EXTEND2 and EXTENDED2

The `EXTEND2` relay message (relay command 14) contains, in addition to the **`CREATE2`** fields, a list of *link specifiers* telling the relay how to reach the next hop and identify it [tor-spec § Extend and Extended messages]:

Type	Contents	Length
`00`	IPv4 address and ORPort	6
`01`	IPv6 address and ORPort	18
`02`	legacy RSA identity (SHA-1, 20 bytes)	20
`03`	Ed25519 identity	32

The extending relay must verify *every* indicated identity, and may extend the circuit only if the relay it reaches proves possession of them during the channel handshake described in chapter 7. This is how the client can be sure that its handshake reaches the intended relay even though it never talks directly to R_2. The relay also rejects requests to extend the circuit to itself or to the relay from which the request originated.

Two details show how carefully the protocol considers *indistinguishability*. First, the specification recommends always sending specifiers in the same order (`00`, `02`, `03`, `01`), so the order does not reveal which implementation produced them. Second, `EXTEND2` always travels inside a **`RELAY_EARLY`** cell (section 8.5); the limited number of these cells prevents the construction of arbitrarily long circuits.

`EXTENDED2` (relay command 15) has exactly the same body as **`CREATED2`**.

9.3 What a Handshake Must Achieve

Before examining the handshakes individually, we should establish what we expect from them. At the end, the client and relay must share a secret with the following properties:

Relay authentication The client must know that it shares the secret with *that specific* relay and no one else. The relay, by contrast, must learn nothing about the client: the handshake is authenticated *in only one direction*; otherwise every relay would know the client's identity.

Secrecy A network observer, or an earlier relay in the circuit that forwards the messages, must not be able to derive the secret.

Forward secrecy Even someone who obtains the relay's long-term keys in the future must not be able to reconstruct past session secrets.

Efficiency A heavily used relay performs thousands of handshakes per second, so computational cost matters.

Each handshake we will examine makes different trade-offs between these requirements.

9.4 TAP: The First Handshake

Tor's original handshake, TAP (*Tor Authentication Protocol*), combined conventional Diffie-Hellman key exchange over a 1,024-bit modular group with 1,024-bit RSA public-key encryption. The client encrypted its Diffie-Hellman half using the relay's RSA "onion" key; only the intended relay could decrypt it and reply. The keys were then derived using KDF-TOR, a SHA-1-based function (section 9.8).

TAP met the requirements, but with steadily shrinking security margins: 1,024 bits are inadequate for both RSA and modular Diffie-Hellman, the hybrid-encryption construction was regarded as fragile, and the handshakes were computationally expensive. Starting in 2013, TAP was supplemented by ntor, and with tor 0.4.9.1-alpha relays stopped supporting TAP altogether (Proposal 350). One trace remains: for compatibility, relays still publish the old RSA "onion" key in their descriptors. On the laboratory network we can see it in microdescriptors under `onion-key`, alongside the `ntor-onion-key` that is actually used.

9.5 CREATE_FAST: When a Public Key Is Unnecessary

There is one case in which a public-key handshake is unnecessary: the *first* hop. The client has just opened a TLS channel to that relay and verified its identity, so TLS already gives it an encrypted, authenticated channel. **`CREATE_FAST`** takes advantage of this [tor-spec § CREATE_FAST/CREATED_FAST cells]:

1. The client sends 20 random bytes X.
2. The relay responds with 20 random bytes Y and 20 bytes KH.
3. Both derive key material with KDF-TOR from $K_0 = X \| Y$. The first 20 bytes must match KH, proving that the relay received X.

The security of **`CREATE_FAST`** depends entirely on the TLS channel: anyone able to read the TLS traffic could also read X and Y. It should therefore be used only for single-hop circuits. A typical use case is a client starting up that needs to download directory information from a relay whose onion key it does not yet know. The tor source states this explicitly: `should_use_create_fast_for_circuit` (in `src/core/or/circuitbuild.c`) selects **`CREATE_FAST`** only if the client has no usable onion key for the first hop.

Warning: CREATE_FAST Reveals the First Hop

A relay receiving **`CREATE_FAST`** knows with certainty that it is the circuit's first hop and therefore that the party at the other end of the channel is the client. The specification suggests that relays use precisely this clue to reject single-hop circuits that try to exit to the Internet.

9.6 ntor

The ntor handshake, proposed by Goldberg, Stebila, and Ustaoglu [GSU13] and introduced into Tor with Proposal 216 (tor 0.2.4.8-alpha, 2013), replaces RSA and modular Diffie-Hellman with **Curve25519**, a fast elliptic-curve key exchange using keys of only 32 bytes (chapter 5).

9.6.1 The Ingredients

The relay has a medium-term Curve25519 onion key: secret b and public $B = b\,G$, published in its descriptor as `ntor-onion-key`. The client knows B and the relay's identity (here, its 20-byte RSA fingerprint, `ID`). We use $\text{EXP}(P, k)$ to denote a Curve25519 Diffie-Hellman exchange between point P and scalar k, and $H(x, t)$ for the HMAC-SHA256 of message x under key t. The t keys are constant strings derived from the protocol identifier `"ntor-curve25519-sha256-1"`.

9.6.2 The Protocol

1. The client generates an ephemeral key pair (x, X) and sends the following in **`CREATE2`**: `ID` (20 bytes), B (32 bytes, identifying which onion key it used), and X (32 bytes). The total is 84 bytes.

2. The relay checks `ID` and B, generates an ephemeral pair (y, Y), and computes

$$\textit{secret_input} = \text{EXP}(X, y) \,\|\, \text{EXP}(X, b) \,\|\, \texttt{ID} \,\|\, B \,\|\, X \,\|\, Y \,\|\, \texttt{PROTOID}.$$

   From this it derives $\textit{KEY_SEED} = H(\textit{secret_input}, t_{\textit{key}})$ and a confirmation value $AUTH$. It responds with Y (32 bytes) and $AUTH$ (32 bytes).

3. The client computes the same *secret_input* using its own secrets, $\text{EXP}(Y, x) \,\|\, \text{EXP}(B, x) \,\|\, \ldots$, and checks that $AUTH$ matches.

The handshake's core consists of two Diffie-Hellman exchanges:

- $\text{EXP}(X, y) = \text{EXP}(Y, x)$ combines two **ephemeral** keys, providing forward secrecy because x and y are erased immediately after the handshake.

- $\text{EXP}(X, b) = \text{EXP}(B, x)$ involves the relay's onion key and provides **authentication**: only a party that knows b can compute it and therefore produce the correct $AUTH$.

Neither exchange is sufficient alone. Without the first, someone stealing b in the future could reconstruct everything; without the second, an intermediary could impersonate the relay. The client, meanwhile, uses no long-term key: the relay knows only that it shares the secret with "whoever sent X".

In depth: Check for the Point at Infinity

The specification requires both parties to verify that no Diffie-Hellman exchange produces the "point at infinity", represented here by an all-zero result. An adversary could send a small-order point as Y, forcing $\text{EXP}(Y, x)$ to a predictable value. For Curve25519, this simple check is a sufficient alternative to fully checking that Y belongs to the correct group. In Laboratory 3 it corresponds to three lines in the `x25519_dh` function.

9.7 ntor-v3

ntor has one limitation: it carries nothing beyond the keys. Over time, it became necessary to *negotiate* options when establishing a hop, such as the use of congestion control (chapter 11) or CGO (chapter 10). ntor-v3 (Proposal 332) extends ntor with **encrypted messages** in both directions, while also changing several cryptographic choices.

9.7.1 What Changes

- The relay identity is now its **Ed25519** identity (32 bytes), rather than its RSA fingerprint.
- The hash functions come from the SHA-3 family: SHA3-256 for hashing and MACs, and SHAKE-256 for variable-length key derivation. Every input is prefixed with a length-encoded domain label (the ENCAP function), so a value calculated for one purpose cannot be mistaken for a value intended for another.
- A verification string `VER`, known to both parties, participates in every calculation: a handshake performed in one context cannot be reused in another. For ordinary circuits, tor uses `"circuit extend"` (the `NTOR3_CIRC_VERIFICATION` constant in `src/core/crypto/onion_crypto.c`).
- The client can send a message CM and the relay a message SM, both encrypted.

9.7.2 The Protocol

1. The client generates (x, X) and immediately calculates $B_x = \text{EXP}(B, x)$. From B_x and the public parameters, it derives two "phase 1" keys: one to encrypt CM using AES-256-CTR and one to authenticate it with a MAC. It sends `ID` (32), B (32), X (32), encrypted CM, and a MAC (32).
2. The relay calculates $\text{EXP}(X, b) = B_x$, derives the same phase-1 keys, verifies the MAC, and decrypts CM. It then generates (y, Y) and combines both exchanges, $\text{EXP}(X, y)$ and $\text{EXP}(X, b)$, into a *key_seed*. Using SHAKE-256, it derives a key to encrypt its response SM and the circuit key stream. It replies with Y (32), $AUTH$ (32), and encrypted SM.
3. The client verifies $AUTH$, decrypts SM, and derives the same key stream.

Warning: The Client Message Lacks Forward Secrecy

CM is encrypted with keys that depend only on B_x, which is derived using the relay's onion key. Someone who records the handshake and later obtains b could decrypt CM. The specification states this explicitly: only the relay's data benefit from forward secrecy. Consequently, CM must contain only negotiation parameters, never user data.

9.7.3 Extensions

The messages CM and SM share a common format [tor-spec § Sending extensions in the circuit extension handshake]: one byte giving the extension count, followed for each extension by its type (1 byte), length (1 byte), and data. The extensions defined for circuit creation are:

Type	Direction	Meaning
1	client → relay	`CC_FIELD_REQUEST`: client requests congestion control from Proposal 324; empty body
2	relay → client	`CC_FIELD_RESPONSE`: relay accepts; 1 byte, `sendme_inc`
3	client → relay	subprotocol request, such as `[02 06]` for CGO (`Relay=6`)

9.7.4 Which Handshake Does tor Use Today?

In tor 0.4.9.13, the choice is made separately for each hop (`circuit_pick_create_handshake`). ntor-v3 is used when the relay supports congestion control and that feature is enabled, as is normally the case on the current network, or when CGO negotiation is desired. Otherwise, tor falls back to ntor. In practice, as the specification observes, modern clients almost always use ntor-v3, with ntor retained as a compatibility alternative.

9.8 From Negotiated Secrets to Circuit Keys

Each handshake produces a stream of pseudorandom bytes from which the keys used to encrypt relay cells under the traditional scheme (called "tor1") are extracted [tor-spec § Setting circuit keys]:

Key	Bytes	Purpose	Direction
Df	20	running digest seed (SHA-1)	client → relay
Db	20	running digest seed (SHA-1)	relay → client
Kf	16	AES-128-CTR key	client → relay
Kb	16	AES-128-CTR key	relay → client

The total is 72 bytes (the `CPATH_KEY_MATERIAL_LEN` constant in tor's code). How the stream is produced depends on the handshake:

KDF-TOR (CREATE_FAST, and formerly TAP) $K = \mathrm{SHA1}(K_0 \| \texttt{00}) \,\|\, \mathrm{SHA1}(K_0 \| \texttt{01}) \,\|\, \ldots$. The first 20 bytes form KH, followed by Df, Db, Kf, and Kb. The single-byte counter limits output to 5,120 bytes, far more than needed.

KDF-RFC5869 (ntor) uses HKDF with SHA-256 and `KEY_SEED` as the key; Df, Db, Kf, and Kb occupy the first 72 bytes.

SHAKE-256 (ntor-v3) uses the first 32 bytes to encrypt SM; Df, Db, Kf, and Kb follow.

Separate keys for the two directions are not a luxury. If the client and relay used the same AES-CTR key in both directions, the encrypted streams would reuse the same keystream, and XORing two cells would reveal the XOR of the two plaintexts.

CGO changes the key arrangement; we will examine it in chapter 10.

9.9 Laboratory: Building Real Circuits

Laboratory 3: Circuit Handshakes

Objective. Write `labs/en/code/torlab/ntor.py` with CREATE_FAST, ntor, and ntor-v3; verify it against the official test vectors; build single-hop circuits to a relay on the laboratory network.
Prerequisites. Laboratories 1 and 2; the running laboratory network; the `cryptography` library (X25519, AES), and the standard `hashlib` module (SHA-3, SHAKE).
Estimated time. 3–4 hours.

Step 1: ntor-v3 in a Few Lines. The implementation follows the specification almost literally. First, the helper functions:

```
NTOR3_PROTOID = b"ntor3-curve25519-sha3_256-1"
def _t(name):  return NTOR3_PROTOID + b":" + name.encode()
def _encap(s): return struct.pack(">Q", len(s)) + s
def _h3(s, t): return hashlib.sha3_256(_encap(t) + s).digest()
def _mac3(k, msg, t):
    return hashlib.sha3_256(_encap(t) + _encap(k) + msg).digest()
def _kdf3(s, t, n): return hashlib.shake_256(_encap(t) + s).digest(n)
```

Then the client side of the handshake:

```
def ntor3_client_start(ID, B, message, ver=b"circuit extend", x=None):
    sk, X = x25519_generate(x)
    Bx = x25519_dh(sk, B)
    phase1 = _kdf3(Bx + ID + X + B + NTOR3_PROTOID + _encap(ver),
                   _t("kdf_phase1"), 64)
    enc_k1, mac_k1 = phase1[:32], phase1[32:]
    encrypted = aes_ctr(enc_k1, message)
    mac = _mac3(mac_k1, ID + B + X + encrypted, _t("msg_mac"))
    state = Ntor3State(sk, X, B, ID, Bx, mac, ver)
    return state, ID + B + X + encrypted + mac

def ntor3_client_finish(st, response, length=72):
    Y, auth, msg = response[:32], response[32:64], response[64:]
    secret_input = (x25519_dh(st.x, Y) + st.Bx + st.ID + st.B + st.X + Y
                    + NTOR3_PROTOID + _encap(st.ver))
    key_seed = _h3(secret_input, _t("key_seed"))
    verify = _h3(secret_input, _t("verify"))
    auth_input = (verify + st.ID + st.B + Y + st.X + st.mac
                  + _encap(msg) + NTOR3_PROTOID + b"Server")
    if not hmac.compare_digest(_h3(auth_input, _t("auth_final")), auth):
        raise HandshakeError("invalid ntor-v3 AUTH")
    stream = _kdf3(key_seed, _t("kdf_final"), 32 + length)
    return aes_ctr(stream[:32], msg), stream[32:]
```

The *AUTH* comparison uses `hmac.compare_digest`, which takes the same amount of time regardless of where the two values differ. An ordinary comparison might reveal how many initial bytes are correct through timing measurements.

Step 2: Test Vectors. We do not verify a cryptographic handshake merely by "making it work": two implementations with the same mistake would work perfectly well together. Instead, we use the specification's **test vectors**: fixed inputs (including the supposedly "random" keys) and expected outputs, byte for byte. Proposal 332 provides a complete vector for ntor-v3, and Proposal 216 provides several for ntor's HKDF function. The `labs/en/code/torlab/test/test_ntor.py` file reproduces them; for example:

```
def test_ntor3_vectors_proposal_332():
    st, hdata = ntor3_client_start(h(V["ID"]), h(V["B"]), h(V["CM"]),
                                   ver=h(V["VER"]), x=h(V["x"]))
    assert hdata.hex() == V["client"]
    sm, stream = ntor3_client_finish(st, h(V["server"]), length=256)
    assert sm == h(V["SM"])
    assert stream[:32].hex() == V["keystream_start"]
```

Other tests verify that an altered *AUTH* or a different `VER` string causes the handshake to fail.

Step 3: The Relay's Keys. To perform the handshake, we need the relay's Ed25519 identity and `ntor-onion-key`. A real client obtains them from the consensus and microdescriptors (chapter 12); we take them from the same sources, using the laboratory client's `cached-microdescs.new` file, where each relay has an entry like this:

```
onion-key
-----BEGIN RSA PUBLIC KEY-----
MIGJAoGBAJet1Oxp+cvg/vgryEmJuVv2s++WxQt5Id1BQ9UNwCOVFcYvEN9KVklM
...
-----END RSA PUBLIC KEY-----
ntor-onion-key gsi2OisnQE96UkjOVvBKtWDLfwRdYG9d4DScg2wz4Qs
id ed25519 Kz1dj/u6/g7WvhHrt9Ds0jVZjKC+KHo26hWViXAJ1Pg
```

Notice the old TAP key, still published but unused, and the two base64 lines we actually need.

Step 4: Against a Real Relay. We open a channel to `test003r` using Laboratory 1, and send a **`CREATE2`** for each handshake type:

```
c = open_channel("127.0.0.1", 5103, expected_identity=ed)
st, hd = ntor3_client_start(ed, B, encode_extensions([]))
c.send(Cell(0x80000001, CellCommand.CREATE2,
              create2_body(HTYPE_NTOR_V3, hd)))
r = c.receive()
sm, stream = ntor3_client_finish(st, hdata_created2(r.body))
keys = CircuitKeys.from_stream(stream)
```

Repeating the test with the variants gives:

```
<- CircID=0x80000001 CREATED2 body=509 bytes
   ntor-v3 ok, relay message: []
<- CircID=0x80000002 CREATED2 body=509 bytes
   ntor-v3 with CC_FIELD_REQUEST ok, response: [(2, b'\x1f')]
<- CircID=0x80000003 CREATED2 body=509 bytes
   ntor ok
<- CircID=0x80000004 CREATED_FAST body=509 bytes
   CREATE_FAST ok
<- CircID=0x80000005 DESTROY body=509 bytes     (modified MAC)
   reason: 1
```

All of the handshakes succeed when unaltered. In the second case, the relay accepts congestion control and responds with extension 2, with `sendme_inc` equal to 31 (chapter 11). In the last case we deliberately changed one bit of the MAC: the relay rejects the handshake with a **`DESTROY`** whose reason is 1, `PROTOCOL`. The actual reason may be disclosed here because this relay originates the error; it is not forwarding another node's **`DESTROY`** (section 8.5).

An Unexpected Observation. In chapter 7, we saw that a new channel receives no **`PADDING`** cells. Now this channel has four open circuits, yet not one cell arrives during twenty seconds of waiting. The reason lies in tor's code (`src/core/or/relay.c`): a relay considers a channel "in use", and therefore eligible for padding, only when it carries a circuit that extends beyond that relay. Our single-hop circuits do not count. In chapter 10, when we extend a circuit, padding will appear.

Summary

- A circuit is built one hop at a time: **`CREATE2`**/**`CREATED2`** with the first relay, then `EXTEND2`/`EXTENDED2` through the circuit for subsequent relays. Each handshake involves

the client and just one relay.

- `EXTEND2` contains link specifiers (address, RSA and Ed25519 identities); the extending relay must verify those identities.
- A circuit handshake authenticates the relay but not the client, and must provide secrecy and forward secrecy.
- TAP (1,024-bit RSA and DH) was removed in tor 0.4.9.1-alpha. **`CREATE_FAST`** relies on TLS and should be used only for single-hop circuits.
- ntor combines two Curve25519 exchanges: ephemeral–ephemeral for forward secrecy, and ephemeral–onion-key for authentication.
- ntor-v3 uses the Ed25519 identity and the SHA-3 family, adding encrypted messages to negotiate options such as congestion control and CGO; the client message lacks forward secrecy.
- The key stream supplies Df and Db (20 bytes each) and Kf and Kb (16 bytes each): a key and a digest for each direction.

Exercises

Exercise 9.1 $\star$ Calculate the length of HDATA in **`CREATE2`** for ntor and for ntor-v3 with a three-byte message. What is the maximum size of the ntor-v3 CM message in a **`CREATE2`** cell?

Exercise 9.2 $\star$ Why must **`CREATE_FAST`** not be used for the second hop of a circuit? What could the first relay do?

Exercise 9.3 $\star$ Why do the client and relay use different AES keys in the two directions (Kf and Kb)? Describe what would happen if they used just one key.

Exercise 9.4 $\star\star$ Write in hexadecimal the body header of an `EXTEND2` message (specifier count and IPv4 specifier) to extend a circuit to laboratory relay `test004r`. How many bytes does the entire message occupy with ntor-v3 and a single-byte CM?

Exercise 9.5 $\star\star$ In the ntor handshake, suppose the client uses only $\mathrm{EXP}(Y, x)$ and omits $\mathrm{EXP}(B, x)$. Describe a concrete attack by an intermediate relay forwarding the handshake to the next hop. Now suppose instead that the client omits $\mathrm{EXP}(Y, x)$: which property is lost?

Exercise 9.6 $\star\star$ What is the purpose of ntor-v3's `VER` string? Imagine a protocol other than Tor using the same handshake with the same keys: what might happen without `VER`?

Exercise 9.7 $\star\star$ Repeat laboratory Step 4 with subprotocol request `[02 06]` (CGO) in addition to `CC_FIELD_REQUEST`. Does the relay accept it? Using the specification for Proposal 359, explain why CGO also requires congestion control.

Exercise 9.8 $\star\star\star$ The tor repository includes a Python reference implementation of ntor (`src/test/ntor_ref.py`). Use it to generate complete ntor handshake test vectors (client and relay sides), and add them to `test_ntor.py`.

Exercise 9.9 $\star\star\star$ Implement the *relay side* of ntor-v3 (functions `server_part1` and `server_part2`) and verify it using the Proposal 332 vectors. Then run a complete handshake between your client and relay, in memory.

Further Reading

- *Tor Protocol Specification*, the "Creating and extending circuits" and "Setting circuit keys" chapters [Thee].
- I. Goldberg, D. Stebila, B. Ustaoglu, *Anonymity and one-way authentication in key exchange protocols* [GSU13]: the paper that defines ntor and establishes its security.
- Proposal 216 (ntor), Proposal 332 (ntor-v3), and Proposal 350 (removal of TAP).

Chapter 10

The Onion: Relay-Cell Encryption and CGO

Last verified: 23 September 2026 — torspec commit `8e1f245`; tor 0.4.9.13; proposal 359

Chapter objectives

By the end of this chapter, you will be able to:

- describe how a client wraps a message in one layer of encryption for each hop and how the relays remove those layers;
- explain the roles of Recognized and the running digest, and why both are needed;
- identify weaknesses of the traditional scheme: malleability and *tagging* attacks;
- describe the basic operation of Counter Galois Onion (CGO), the scheme replacing it;
- extend a real circuit to three hops using your code (Laboratory 4).

In chapter 9, the client agreed on four keys with each relay in the circuit: Kf and Kb for encryption, and Df and Db for integrity checking. In this chapter, we put them to work. This is the mechanism that gives onion routing its name: a message is wrapped in as many layers as there are relays, and each relay removes one layer, like peeling an onion.

10.1 The Traditional Scheme

The scheme used since Tor's beginnings, called *tor1* in the more recent code, combines two tools for each hop and each direction:

- a stream cipher, **AES-128 in counter mode** (CTR), using key Kf or Kb;
- a **running SHA-1 digest**, initialized using Df or Db.

Both maintain *state* for the lifetime of the circuit. The AES-CTR counter does not restart at zero for each cell: the first cell uses the first 509 bytes of the keystream, the second uses the next 509, and so on. Likewise, the digest is not calculated from a single cell, but from *all* cells exchanged with that hop so far. Consequently, the client and relay must process cells in exactly the same order: a lost or duplicate cell would desynchronize both states. TLS, which guarantees ordering and integrity for the channel, makes this possible.

10.1.1 Forward: From the Client into the Circuit

To send a message to hop d of a circuit, the client [tor-spec § Routing relay cells]:

1. prepares the plaintext relay-message body (section 8.6), with Recognized and Digest set to zero;
2. updates hop d's forward digest using that body, and writes the first four bytes of the result into the Digest field;
3. encrypts the body using hop d's Kf, then hop $d-1$'s Kf, and so on back to hop 1: the outermost layer belongs to the first relay.

Every relay receiving the cell decrypts it using its own Kf, removing one layer, and checks whether the cell is intended for it. If not, it forwards the cell to the next hop, which removes another layer. Figure 10.1 shows the layers.

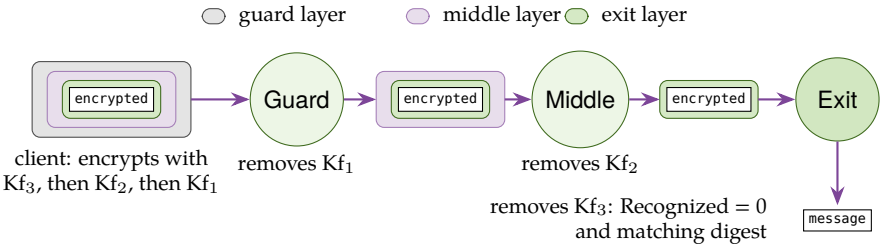


Figure 10.1: Traditional tor1 RELAY-cell encryption: the client applies Kf_3, Kf_2, Kf_1 and each relay removes its layer. The exit recognizes the relay message with Recognized $= 0$ and a matching running digest; application data may still be protected by HTTPS. The inner boxes schematically depict still-encrypted data.

10.1.2 Backward: From the Circuit to the Client

In the opposite direction, the roles are reversed. A relay originating a message signs it with its own backward digest and encrypts it using Kb. Each subsequent relay toward the client *adds* another layer using its own Kb. The client, which knows all the keys, removes the layers one at a time, starting at the first hop, and after each layer checks whether that hop originated the cell.

10.1.3 The Circuit as a "Leaky Pipe"

The client can send a message to *any* hop in the circuit, not just the last one, simply by stopping the encryption at that hop. This is how an `EXTEND2` message for the third hop is delivered to the second relay. The specification calls this the *leaky pipe topology*: the circuit is a pipe from which messages can leave partway along it. Tor's original paper [DMS04] also proposes using this property defensively: sending some streams out through an intermediate relay would make it harder for an exit observer to determine which circuit they belong to. In practice, however, tor sends streams out only through the final hop.

10.2 Recognizing a Cell

How does a relay know, after removing its own layer, whether the cell is meant for it? It cannot simply "check whether the contents make sense": to a program, both a still-encrypted

cell and a plaintext cell are sequences of bytes. The traditional scheme uses two checks in sequence [tor-spec § Relay cells]:

1. **Recognized**: these two bytes must be zero. If the cell is still encrypted, those bytes are pseudorandom and equal zero only once in $65\,536$. This check is almost free and immediately filters out the vast majority of cells in transit.

2. **Digest**: if Recognized is zero, the relay calculates the running digest *as though* the cell belonged to it, with the Digest field zeroed, and compares the first four bytes against the field's contents. If they match, the cell has been recognized: the relay updates its digest state and processes the cell.

An easily missed detail: if Recognized is zero while the digest differs, this is a false positive, and the digest state **must not** be updated. In Laboratory 4, we achieve this by working on a copy of the state (`self.digest.copy()`) and keeping it only on success.

The digest covers the entire body, including padding bytes, which are therefore random (section 8.6). For onion-service circuits, in which the client and service share the keys of a "virtual hop" (chapter 14), SHA3-256 is used instead of SHA-1.

What happens if no one recognizes the cell? If the final relay receives a forward cell it does not recognize, an error has occurred: it destroys the circuit using **`DESTROY`**. If none of the hops recognizes an incoming cell, the client does the same.

10.3 Weaknesses of tor1

10.3.1 Malleability

Counter mode has an inconvenient property: the ciphertext is simply $C = P \oplus S$, where S is the keystream. Changing one bit of C changes that same bit of P, without requiring the key. If an adversary knows, or can guess, part of the plaintext, they can replace it with chosen text: they need only XOR the ciphertext with the difference between the two texts. Laboratory 4 includes a test that does exactly this, changing "pay 10 euros" to "pay 99 euros".

In tor1, integrity is checked only *at the destination*: the intended relay checks the digest, but intermediate relays do not. An altered cell can therefore travel undisturbed to its destination, where the digest finally fails to match.

10.3.2 The Tagging Attack

Together, these properties permit a specific attack called *tagging*. Imagine an adversary controlling both the guard and the exit of a circuit:

1. The guard changes some bits in a cell in transit according to a recognizable pattern (the "mark", or *tag*).

2. The cell passes through the middle, which cannot detect the change.

3. Before checking the digest, the exit checks whether the mark is present in the cell. If it is, the exit knows that this circuit passes through its cooperating guard and can therefore associate the traffic destination with the client's address, which the guard knows.

If the exit is not cooperating, the digest fails, the circuit is destroyed, and the client builds another one. The adversary can try again, and each failed attempt reveals something. The mark can also be applied in the reverse direction, from the exit toward the client.

In depth: How Much Does Tagging Matter?

An adversary controlling both guard and exit can already correlate traffic timing and volumes without any mark (chapter 26). For this reason, Tor's designers long considered tagging a genuine but lower-priority problem [DMS04]. Tagging, however, is more reliable than statistical correlation and works even with very little traffic. Proposals aimed at solving it appeared over more than a decade: proposals 202, 261 (based on the AEZ cipher), 295, and 308, followed by 359, which introduced CGO.

10.3.3 Other Limitations

The digest is only four bytes long, and its underlying algorithm, SHA-1, is no longer considered secure for new applications. Moreover, the scheme does not offer forward secrecy *within* a circuit: someone who obtains a hop's keys while the circuit is active could also decrypt every earlier cell they recorded, because the keystream is determined entirely by the initial key.

10.4 Counter Galois Onion

CGO (*Counter Galois Onion*) is a relay-cell encryption scheme designed by Degabriele, Melloni, Münch, and Stam and adopted in Proposal 359. We give a functional overview here; the complete cryptographic construction is described in the proposal and the authors' papers.

10.4.1 Properties

According to the proposal, CGO offers three improvements:

- It is **non-malleable**: modifying a cell in any way makes that cell, and every subsequent cell in the circuit, undecipherable. A mark can neither be read nor removed: it can only destroy the circuit.
- It provides **cell-level forward secrecy**: after processing a cell, a party no longer holds the material needed to decrypt it again.
- Authentication is 128 bits strong, rather than relying on the traditional scheme's much shorter check.

Since CGO no longer uses SHA-1, it should also be faster overall.

10.4.2 How It Works, at a High Level

The basic ingredients are AES-128 and POLYVAL, a universal hash function from the same family used in AES-GCM-SIV mode. With these, the proposal constructs a *wide-block*, *tweakable* cipher: it operates on all 509 bytes of the cell as though they were one block, so changing even a single bit anywhere unpredictably changes the entire result. The decrypted cell consists of a *tag* (16 bytes) and a body (493 bytes), as we saw in section 8.7.

For each hop and each direction, the client and relay share:

- a 64-byte key K and a 16-byte value N, derived from the circuit handshake;
- the *tag* T' of the last processed cell, initially zero.

Three ideas make the scheme resistant to tagging:

Tag-based recognition Once a relay has removed its layer, the cell belongs to it if the resulting tag matches N. This is a 16-byte check instead of the two Recognized bytes plus the four-byte digest.

Chaining The previous cell's tag enters the next cell's processing as a cipher parameter. An altered cell produces an incorrect tag, which in turn makes every following cell impossible to decrypt: the circuit can no longer operate.

Key updates Whenever a hop recognizes or originates a cell, it replaces K and N with new values derived from the preceding ones. The old values are erased, providing cell-level forward secrecy.

The cell's command byte (`RELAY` or `RELAY_EARLY`) is included in the calculations as *associated data*: a relay cannot turn one into the other without making the cell impossible to decrypt.

Warning: Directions Are Not Interchangeable

The construction underlying CGO is non-malleable in one direction and malleable in the other. The proposal carefully chooses which direction to use in the client and which in relays, and explicitly warns implementers not to reverse them. This illustrates how a choice that appears to be an implementation detail can undermine the security of an entire cryptographic scheme.

10.4.3 CGO in Practice

- **Keys**: from the stream produced by the handshake, each direction receives a 64-byte K and a 16-byte N, rather than Df, Db, Kf, and Kb.
- **SENDME**: authenticated flow control from Proposal 289 used the running digest to prove receipt of data. CGO no longer has that digest, so a SENDME instead carries the tag of the last acknowledged cell (chapter 11).
- **Negotiation**: a client uses CGO with a relay only if the relay advertises subprotocol `Relay=6` and the client requests it in the ntor-v3 handshake (section 9.7). In chapter 9, we saw that the relay rejects the request unless it is accompanied by a congestion-control request.
- **Deployment**: tor supports CGO beginning with the 0.4.9 series, and Arti has always enabled it. A circuit can contain hops using CGO alongside hops still using tor1: the choice is made independently for each hop.

10.5 Laboratory: A Three-Hop Circuit

Laboratory 4: Layered Encryption and Circuit Extension

Objective. Write `labs/en/code/torlab/encryption.py`, implementing the client side of tor1, and use it to extend a real circuit all the way to an exit.
Prerequisites. Laboratories 1–3; the running laboratory network.
Estimated time. 3 hours.

Step 1: One Direction of One Hop. At the heart of the laboratory is a small class combining an AES-CTR cipher and the SHA-1 digest for one direction:

```
class Direction:
    @classmethod
    def new(cls, key, digest_seed):
        d = hashlib.sha1(); d.update(digest_seed)
        return cls(_aes_ctr(key), d)

    def apply(self, data):             # encrypt or decrypt: CTR uses the same
    operation
        return self.cipher.update(data)

    def sign(self, body):                 # write the Digest field
        self.digest.update(_without_digest(body))
        return body[:5] + self.digest.digest()[:4] + body[9:]

    def recognizes(self, body):
        if body[1:3] != b"\x00\x00":
            return False
        trial = self.digest.copy()
        trial.update(_without_digest(body))
        if trial.digest()[:4] != body[5:9]:
            return False                # false positive: state unchanged
        self.digest = trial
        return True
```

The cipher object from `cryptography` preserves the counter across calls: that is exactly the behavior required here.

Step 2: The Client. With one `Direction` object per direction and per hop, encryption and decryption each become a loop:

```
def encrypt(self, msg, destination):
    body = self.hops[destination].forward.sign(encode_relay(msg))
    for i in range(destination, -1, -1):
        body = self.hops[i].forward.apply(body)
    return body

def decrypt(self, body):
    for i, hop in enumerate(self.hops):
        body = hop.backward.apply(body)
        if hop.backward.recognizes(body):
            return i, decode_relay(body)
    raise CellError("cell not recognized")
```

Step 3: In-Memory Tests and Their Limitations. The `labs/en/code/torlab/test/test_encryption.py` file builds a simulated circuit of three relays. It verifies that each hop recognizes only its own cells, that intermediate relays see only pseudorandom bytes, and that a modification is detected at the destination; it also demonstrates the malleability of AES-CTR. However, these tests have an important limitation: the simulated client and relays use *the same code*. If, for example, we used the wrong order to update the digest, both sides would make the same mistake and the tests would still pass. The definitive test is to communicate with an independent implementation.

Step 4: Extend a Real Circuit. We establish the first hop using ntor-v3 to `test003r`, as in Laboratory 3, then extend to `test004r` and exit `test006e`. The `labs/en/code/torlab/laboratory.py` module reads the relays' identities and ntor keys from the laboratory network's files:

```
for r, est in ((middle, []), (exit_, [(1, b"")])):
    st, hd = ntor3_client_start(r.id_ed, r.ntor, encode_extensions(est))
    body = extend2_body("127.0.0.1", r.port, r.id_rsa, r.id_ed,
                        HTYPE_NTOR_V3, hd)
    msg = RelayMessage(RelayCommand.EXTEND2, 0, body)
    c.send(Cell(CIRC, CellCommand.RELAY_EARLY, circ.crypto.encrypt(msg)))
    i, response = circ.crypto.decrypt(c.receive().body)
    sm, k = ntor3_client_finish(st, hdata_created2(response.data))
    circ.hops.append(Hop.from_keys(CircuitKeys.from_stream(k)))
```

The `EXTEND2` message has StreamID zero (it is a control message) and travels in a **`RELAY_EARLY`** cell. We also request congestion control for the exit, just as tor does. The run produces:

```
hop 1: 003r
  <- CircID=0x80000100 RELAY body=509 bytes
  from hop 1 EXTENDED2 67 bytes
hop 2: 004r  relay extensions: []
  <- CircID=0x80000100 RELAY body=509 bytes
  from hop 2 EXTENDED2 70 bytes
hop 3: 006e  relay extensions: [(2, b'\x1f')]
```

The circuit now has three hops. Every "from hop" line proves that our code encrypted correctly for the intended relay and that the relay encrypted correctly for us. A single incorrect key, digest byte, or layer order would have caused the relay to destroy the circuit. The `EXTENDED2` lengths agree with section 9.7: two bytes for HLEN, 32 for Y, 32 for $AUTH$, and the SM message, which is one byte (no extensions) for the middle and four bytes (one single-byte extension) for the exit.

Step 5: Padding at Last. We continue listening on the channel after extending the circuit:

```
  4.1 s  <- CircID=0x00000000 PADDING body=509 bytes
  7.1 s  <- CircID=0x00000000 PADDING body=509 bytes
 12.8 s  <- CircID=0x00000000 PADDING body=509 bytes
 21.5 s  <- CircID=0x00000000 PADDING body=509 bytes
 28.3 s  <- CircID=0x00000000 PADDING body=509 bytes
```

This is what we predicted in chapters 7 and 9: now that a multi-hop circuit passes through the channel, the guard considers it "in use" and sends **`PADDING`** cells at random intervals whenever the channel is idle. The intervals observed, between three and 8.7 seconds, fall within the specification's 1.5–9.5-second padding window.

Summary

- In tor1, each hop has an AES-128-CTR stream and a running SHA-1 digest for each direction, both maintaining state for the lifetime of the circuit.
- In the forward direction, the client applies layers from the last hop to the first and each relay removes one. In the backward direction, each relay adds a layer and the client removes them all.
- The client can address any hop (*leaky pipe*).

- A cell is recognized when Recognized is zero and the running digest matches; a Recognized false positive must not change the digest state.
- AES-CTR is malleable and integrity is checked only at the destination: this enables tagging.
- CGO makes cells non-malleable, chains their tags, updates keys after every message, and authenticates with 128 bits; it is negotiated separately with each hop using ntor-v3.

Exercises

Exercise 10.1 ⋆ A client with a three-hop circuit wants to send a message to the second hop. Which keys does it use to encrypt, and in what order? What does the third hop see?

Exercise 10.2 ⋆ After the client has sent 1,000 cells along the circuit, where is the first hop in its AES-CTR keystream? Why must the same key not be reused for a second circuit?

Exercise 10.3 ⋆⋆ Explain what would happen if a relay updated its running digest even when Recognized was zero but the digest did not match. After how many cells, on average, would the problem occur?

Exercise 10.4 ⋆⋆ A cooperating guard wants to mark a forward cell so that a cooperating exit can recognize it. Which cell bytes would be suitable to modify, given the structure of a plaintext `DATA` message? What would an honest exit see?

Exercise 10.5 ⋆⋆ Repeat the preceding exercise's scenario using CGO. What does the cooperating exit see? What happens to subsequent cells in the circuit?

Exercise 10.6 ⋆⋆ In Laboratory Step 5, calculate the intervals between the observed **`PADDING`** cells. According to the specification, the delay is drawn from the distribution of the maximum of two uniform random variables. What is the expected delay over the interval from 1.5 to 9.5 seconds? Is it consistent with the observations?

Exercise 10.7 ⋆⋆⋆ Extend the laboratory by sending a `DROP` message (relay command 10, long-range padding) to the second hop. Does the relay reply? How can you verify that it recognized the message and did not destroy the circuit?

Exercise 10.8 ⋆⋆⋆ In the laboratory, deliberately flip one bit of an encrypted **`RELAY_EARLY`** cell containing `EXTEND2`. Which relay notices, and how does it react? Predict the result, then verify it.

Further Reading

- *Tor Protocol Specification*, the "Relay cells" and "Routing relay cells" chapters [Thee].
- Proposal 359, *Counter Galois Onion, Updated*, with references to the papers by Degabriele, Melloni, Münch, and Stam.
- Proposals 202, 261, 295, and 308: the long history of approaches to preventing tagging.
- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04], Section 4 (circuits, leaky pipes, and integrity checks).

Chapter 11

Streams, SENDME, and Congestion Control

Last verified: 23 September 2026 — torspec commit `8e1f245`; tor 0.4.9.13; proposals 289, 324

Chapter objectives

By the end of this chapter, you will be able to:

- open, use, and close a TCP stream through a circuit using BEGIN, CONNECTED, DATA, and END;
- explain why name resolution takes place at the exit and how RESOLVE works;
- describe Tor's original flow control, including circuit and stream windows and SENDME messages;
- understand why SENDME messages are authenticated and what they prove;
- describe the congestion control introduced by proposal 324 and the roles of XON and XOFF;
- download a file through a three-hop circuit you have built yourself and measure the windows of a real exit (Laboratory 5).

The circuit in chapter 10 is an encrypted conduit to the exit, but it does not yet carry anything useful. In this chapter, we put it to work: carrying TCP connections to the Internet, known as **streams**. We then examine a problem every network must solve, and one that takes a particular form in Tor: preventing a sender that transmits faster than the network or recipient can absorb data from filling relay queues.

11.1 Streams

A stream is a client's TCP connection, for example to a web server, carried through a circuit. One circuit can carry many streams simultaneously. They are distinguished by the **StreamID** in relay messages (section 8.6), which the client chooses and which must be nonzero. The lifecycle of a stream is shown in fig. 11.1.

11.1.1 Opening a stream: BEGIN and CONNECTED

To open a stream, the client sends the exit a `BEGIN` message (relay command 1), using a new StreamID. The message body specifies the destination [tor-spec § Opening streams and transmitting data]:

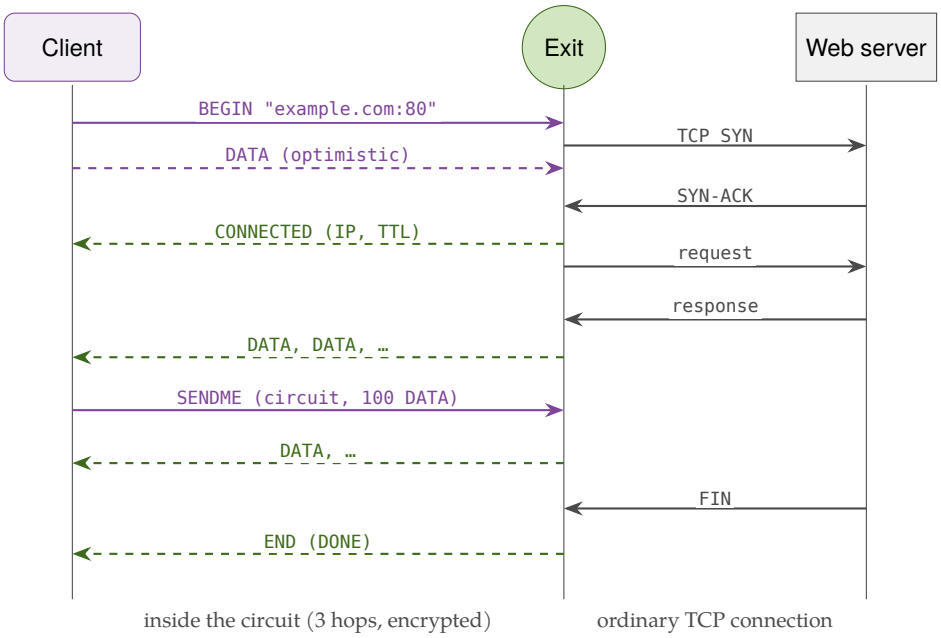


Figure 11.1: Lifecycle of a stream to a web server. Messages between the client and the exit travel through the circuit under layered encryption; between the exit and the server there is an ordinary TCP connection, unencrypted if the application does not use TLS. The client may send data before receiving CONNECTED (*optimistic data*).

Field	Length	Contents
ADDRPORT	variable	`address:port`, terminated by a zero byte
FLAGS	4 (optional)	IPv4 and IPv6 preferences

The address can be a DNS name, an IPv4 address, or an IPv6 address enclosed in square brackets. Two small rules reveal the protocol's design priorities: the string must be sent **in lowercase**, so that differences in capitalization cannot help distinguish one client from another, and FLAGS must be **omitted** when it is zero. The FLAGS bits specify whether the client accepts IPv6 (bit 1), refuses IPv4 (bit 2), and prefers IPv6 (bit 3).

After receiving `BEGIN`, the exit:

1. checks that it is not the circuit's first hop: the channel on which the circuit arrived must have been opened by an authenticated relay listed in the consensus. Tor must not function as a single-hop proxy, which would make exits more attractive targets; such an attempt is rejected with **`DESTROY`**;

2. applies its *exit policy*, the list of addresses and ports to which it accepts connections (chapter 22);

3. resolves the name, if necessary, and opens the TCP connection.

If all goes well, it replies with `CONNECTED` (relay command 4). The body usually contains the reached IPv4 address (4 bytes) and the number of seconds for which the client may cache it (TTL, 4 bytes). The IPv6 format is slightly different, and an empty body is also permitted. The client *may* ignore the address; if it caches the address, it must not reuse it on other circuits. A dishonest exit could lie or return a unique address to recognize the client later.

11.1.2 Transferring data: DATA

Application data travels in `DATA` messages (relay command 2), each carrying up to 498 bytes in the traditional format. The exit copies their contents onto the TCP connection and vice versa. The client can send `DATA` immediately after `BEGIN` without waiting for `CONNECTED`: the exit queues these messages and forwards them as soon as the connection opens. Such *optimistic data* saves an entire round trip through the circuit, which in Tor can easily cost a few hundred milliseconds.

11.1.3 Closing a stream: END

The side closing the connection, or encountering an error, sends `END` (relay command 3) with a one-byte reason [tor-spec § Closing streams]. The most common reasons are `DONE` (6, normal closure), `CONNECTREFUSED` (3), `RESOLVEFAILED` (2), `TIMEOUT` (7), and `EXITPOLICY` (4); the last is followed by the rejected address and a TTL. For streams it originated, the client should send only `MISC` (1): as with **`DESTROY`**, there is no reason to give the other end more information than necessary.

11.1.4 DNS at the exit

In `BEGIN`, the client can specify a *name* instead of an address: resolution takes place at the exit. This is not merely convenient; it is a security decision. If the client resolved names itself, it would ask its own DNS resolver, outside Tor, for the address of every site it was about to visit. Anyone observing those requests, starting with its connectivity provider, would know where it was going. This is the familiar "DNS leak", demonstrated in exercise 19.3 and observed from tor's point of view in Laboratory 18 (section 28.3).

When the client needs only an address, without opening a connection, it uses `RESOLVE` (relay command 11), with a zero-terminated name. The exit replies with `RESOLVED` (12), containing one or more answers in type, length, value, and TTL format: type `0x04` for IPv4, `0x06` for IPv6, `0x00` for a name (in reverse lookups), and `0xF0` and `0xF1` for temporary and permanent errors [tor-spec § Remote hostname lookup]. This mechanism underlies the DNS port a tor client can offer local applications (the `DNSPort` option).

11.1.5 Directory streams: BEGIN_DIR

`BEGIN_DIR` (relay command 13) opens a stream not to the Internet but to the relay's own directory service. This is how clients download consensuses and descriptors (chapter 12) without opening unencrypted connections. The body is empty, and the exit policy does not apply because the stream never leaves the tor process.

11.2 Original Flow Control

11.2.1 The problem

Imagine an exit with a fast Internet connection and a slow client. If the exit read from the server and sent everything it received into the circuit, data would accumulate in intermediate relay queues, consuming memory and slowing other circuits. TCP addresses this problem on each individual link, but a circuit is a chain of different TCP links: TCP backpressure between the middle and the client does not reach the exit. An *end-to-end* mechanism is needed.

11.2.2 Windows and SENDME

Tor's original mechanism uses **windows**, counted in `DATA` cells [tor-spec § Flow control]:

- At the **circuit** level, a sender has a *package window* starting at 1000 cells (or the `circwindow` consensus parameter). Each `DATA` cell sent decrements it; when it reaches zero, sending stops. The receiver maintains a symmetrical *deliver window* and, for every 100 cells received, sends a `SENDME` message (relay command 5) with StreamID zero, restoring 100 cells to the sender's window.
- At the **stream** level, windows start at 500 cells and grow by 50 for each `SENDME` carrying that stream's StreamID.

Because the client can exchange data with any hop (section 10.1), it maintains a pair of windows for every hop. These windows count only cells originated and consumed, not cells forwarded.

Consequently, at most 1000 cells can be in flight per circuit and 500 per stream at any given moment: roughly 500 KB and 250 KB. This bounds relay memory use but also limits throughput. A circuit with a half-second round-trip time cannot exceed 1000 cells every half second, or about 1 MB/s, regardless of available bandwidth. If the network is congested, even 1000 cells in flight are too many: queues grow, and so does latency.

11.2.3 Authenticated SENDME messages

A malicious client could send `SENDME` messages without having received anything. The exit would keep sending data, which would accumulate in the network, allowing the client to exhaust relay memory or amplify traffic with little effort. Starting with version 0.4.1.1-alpha, circuit `SENDME` messages are **authenticated** (Proposal 289). Their bodies carry a version (1), a length (20), and the **running digest** of the `DATA` cell that triggered the `SENDME`. The recipient of the `SENDME` knows that digest because it computed it when sending the cell and can compare the values. A mismatch closes the circuit.

The digest depends on the contents of all received cells, but a client could try to predict it when it knows the cell contents in advance, for example when downloading a known file. This is why at least one cell in each group of 100 must contain random bytes: one reason for the random padding in relay messages (section 8.6).

In depth: Which digest, exactly?

The specification refers to the "digest of the cell preceding the `SENDME`". In tor's code, it is the complete 20-byte SHA-1 digest of the backward stream after the last recognized cell, saved in the `sendme_digest` field by `tor1_relay_digest_matches_v0` (`src/core/crypto/relay_crypto_tor1.c`). With CGO, the tag of the last cell replaces it (section 10.4).

11.3 Congestion Control

Fixed windows have two opposite shortcomings: they are too small for fast circuits and too large for congested ones. Proposal 324, by Mike Perry, introduced genuine **congestion control** in Tor, analogous to TCP's, adopted starting with tor 0.4.7.7 (April 2022).

11.3.1 The signal: round-trip time

How does the exit recognize congestion in a circuit? By using `SENDME` messages as a clock. The exit records when it sends the `DATA` cell that will trigger a `SENDME`; when that `SENDME`

arrives, it calculates the circuit's **round-trip time** (RTT). If RTT increases relative to the observed minimum, queues along the circuit are filling, indicating that the sender should slow down. The sender adjusts a *congestion window* (*cwnd*), attempting to keep it near the circuit's bandwidth-delay product: the amount of data the circuit can carry without creating queues.

11.3.2 The algorithms

The proposal describes three algorithms inspired by TCP: Tor Westwood, Tor Vegas, and Tor NOLA. Following evaluations in Shadow simulation (chapter 19) and on the live network, tor's default is **Vegas** (the `CC_ALG_DFLT` constant in `src/core/or/congestion_control_common.c`). Like TCP Vegas, it estimates how many cells are "queued" by comparing the current window against the window needed at the lowest observed RTT, then increases or decreases the window to keep that estimate within a threshold.

11.3.3 The parameters: why 31?

Congestion control is negotiated hop by hop using ntor-v3 (section 9.7). The client requests it with the `CC_FIELD_REQUEST` extension, and the relay answers with `CC_FIELD_RESPONSE`, containing `sendme_inc`: the number of `DATA` cells after which it requests a `SENDME`. In chapter 9, the laboratory network's exit returned 31. The value is not arbitrary: tor's code defines it as how many cells a maximum-size TLS record can hold,

$$\texttt{TLS_RECORD_MAX_CELLS} = \left\lfloor \frac{16384}{514} \right\rfloor = 31,$$

and the initial window is $4 \times 31 = 124$ cells. More frequent SENDME messages than in the past—one every 31 cells rather than 100—provide more frequent RTT measurements and faster reactions.

11.3.4 No more stream windows: XON and XOFF

With congestion control, stream-level `SENDME` messages disappear. Stream windows created a dilemma: small windows limit each download's speed; large ones let a malicious client cause the exit to buffer an entire window of data for every one of the many streams it opens. The proposal replaces them with two messages:

XOFF (relay command 43): "stop sending on this stream", sent when the queue toward the application exceeds a threshold;

XON (relay command 44): "resume", with a 4-byte field specifying the rate at which the queue is draining, so the other end can adapt. The field is named `kbps_ewma`, and Proposal 324 does not specify its unit. tor's code calculates and interprets it in units of **1000 bytes per second**, not 1000 bits (`src/core/or/congestion_control_flow.c`): the name is misleading, and interpreting it literally produces an eightfold error.

Circuit `SENDME` messages, in contrast, are sent as soon as cells are decrypted and recognized, before delivery to the application. They measure the network, not the application.

Warning: Congestion control and anonymity

Any mechanism that reacts to traffic can become a side channel. An adversary observing a window's behavior might infer information about the circuit. Proposal 324 devotes

a section to these risks and their mitigations, including limiting the frequency of XON and XOFF messages. This is the same type of precaution that, in chapter 8, leads to always sending reason NONE in a **DESTROY** cell: any observable variation that depends on internal state gives information away. Attacks exploiting clues derived from traffic, which this precaution aims not to facilitate, are the subject of chapter 26.

11.4 Laboratory: Downloading a File Through Tor

Laboratory 5: Streams and flow control

Objective. Write `labs/en/code/torlab/circuit.py`, which brings the previous laboratories together in a `Circuit` class, and `labs/en/code/torlab/stream.py`, which opens streams and transfers data. Download a file from a local web server over a three-hop circuit and measure the exit's windows.
Prerequisites. Laboratories 1–4; the laboratory network running; a local web server (`python3 -m http.server 8080`).
Estimated time. 3 hours.

Step 1: a circuit in one line. The `build` function opens the channel to the guard, creates the first hop, and extends the circuit, requesting congestion control only at the final hop, as tor does:

```
circ = build([relay("003r"), relay("004r"), relay("006e")])
print(circ.exit_extensions)             # [(2, b'\x1f')]: sendme_inc = 31
```

Step 2: opening the stream.

```
def begin_body(address, port, flag=0):
    body = f"{address.lower()}:{port}".encode() + b"\x00"
    return body + (struct.pack(">I", flag) if flag else b"")

s = Stream(circ, 1)
s.open("127.0.0.1", 8080)       # BEGIN, then wait for CONNECTED or END
```

The exit replies with `CONNECTED` and an 8-byte body, `7f000001 00000708`: the address 127.0.0.1 and a TTL of `0x708` = 1800 seconds.

Step 3: SENDME messages. The circuit counts each `DATA` cell received from the exit and sends an authenticated `SENDME` every `sendme_inc` cells:

```
def count_received_data(self):
    est = dict(self.exit_extensions)
    sendme_inc = est[2][0] if 2 in est else 100
    self.data_to_ack += 1
    if self.data_to_ack == sendme_inc:
        self.data_to_ack = 0
        digest = self.crypto.hops[-1].backward.digest.digest()
        body = struct.pack(">BH", 1, 20) + digest
        self.send(RelayMessage(RelayCommand.SENDME, 0, body))
```

Warning: An instructive mistake

In the first version of this laboratory, the counter was an attribute of the *stream*. The first stream, a two-cell page, worked; the second, a 1 MB file, was interrupted by the exit with a **DESTROY**. The reason: circuit SENDME counts DATA cells from *all* streams on the circuit. The exit had already counted the first stream's two cells and expected the SENDME after the second stream's 29th cell. We sent it after the 31st cell, with a digest different from the expected one, and the exit did exactly what Proposal 289 requires: it closed the circuit. Moving the counter to the `Circuit` class fixed the problem.

Step 4: the download. We send an HTTP request and read until END:

```
s.send(b"GET /grande.bin HTTP/1.0\r\nHost: 127.0.0.1\r\n\r\n")
response = s.receive_all()
```

With a random file of $1\,048\,576$ bytes, one of our runs gave:

```
stream 1: 209 byte in 0.05 s, DATA cells 2, circuit SENDMEs so far 0,
          other ['END DONE']
stream 2: 1048781 byte in 0.18 s, DATA cells 2120, circuit SENDMEs so far 68, other ['
    END DONE']
```

The received file's SHA-256 fingerprint matches the original. The 68 SENDME messages are exactly $\lfloor (2 + 2120)/31 \rfloor$: one counter per circuit, as we learned. There are 2120 cells rather than the minimum $\lceil 1048781/498 \rceil = 2106$: the exit does not wait until it has 498 bytes before sending a cell, but packages data as it reads it from the server.

Step 5: observing the windows. We now run an experiment that no ordinary client can perform: download the same file *without ever sending* SENDME, and count how many cells the exit sends before stopping. We repeat the experiment with and without congestion control:

```
circ = build(path, cc=cc)
circ.count_received_data = lambda: None      # no SENDME
```

```
congestion control=True:  the exit stops after 124 DATA cells
congestion control=False: the exit stops after 500 DATA cells
```

With congestion control, the exit stops at 124 cells: its initial congestion window, 4×31. Without it, the exit stops at 500 cells, not 1000: the *stream* window (500) is exhausted before the circuit window (1000), because under original flow control a client must also send stream-level SENDME messages. In either case, the exit simply waits: the data remain in its own buffer, not in the network.

Summary

- A stream opens with BEGIN (`address:port`, lowercase), is confirmed with CONNECTED, carries DATA, and closes with END; the client can send data before CONNECTED.
- Name resolution takes place at the exit to avoid revealing destinations to the local resolver; RESOLVE and RESOLVED allow resolution without a connection.
- Original flow control uses windows of 1000 cells per circuit and 500 per stream, with SENDME every 100 and 50 cells respectively.

- Circuit `SENDME` messages are authenticated using the running digest of the triggering cell (Proposal 289).
- With congestion control (Proposal 324, default algorithm Vegas), the window adapts to the RTT measured through `SENDME`; `sendme_inc` is 31, the number of cells fitting in one TLS record, and the initial window is 124.
- XON and XOFF replace stream windows.

Exercises

Exercise 11.1 ⋆ Write the body of a `BEGIN` message to `Example.COM` on port 443 in hexadecimal, for a client that also accepts IPv6.

Exercise 11.2 ⋆ Interpret the `CONNECTED` body `c0000201 0000012c`. What should the client do with this information?

Exercise 11.3 ⋆ Estimate the time saved by *optimistic data* for a circuit with a 600 ms RTT and a page requiring just one HTTP request. Why must the exit discard queued data if the connection fails?

Exercise 11.4 ⋆⋆ Under original flow control, what is the maximum speed of a single stream over a circuit with a 400 ms RTT? What about a circuit with multiple streams? Compare these figures with the initial congestion-control window.

Exercise 11.5 ⋆⋆ Describe a memory-exhaustion attack against an exit based on unauthenticated `SENDME` messages. Why is random padding in `DATA` cells necessary for authentication to work?

Exercise 11.6 ⋆⋆ Extend `stream.py` with a `resolve(name)` function that uses `RESOLVE` and interprets `RESOLVED`. Test it in the laboratory network with `localhost`. How does the exit respond to a nonexistent name?

Exercise 11.7 ⋆⋆⋆ Try opening a stream to a port that the laboratory network's exit does not accept after changing its exit policy (`torrc` for `test006e`, then `chutney hup`). Which reason does `END` report? Interpret the additional fields.

Exercise 11.8 ⋆⋆⋆ Modify the laboratory to measure, at the client, the time between sending each `SENDME` and receiving the next `DATA` cell. What are you measuring? Compare the values with and without a second download running in parallel over the same circuit.

Further Reading

- *Tor Protocol Specification*, sections "Opening streams and transmitting data", "Closing streams", "Remote hostname lookup", and "Flow control" [Thee].
- Proposal 289, *Authenticating sendme cells to mitigate bandwidth attacks*.
- Proposal 324, *RTT-based Congestion Control for Tor*: motivation, algorithms, parameters, and security analysis.

Chapter 12

The Directory System

Last verified: 23 September 2026 — torspec commit `8e1f245`; tor 0.4.9.13; consensus method 36

Chapter objectives

By the end of this chapter, you will be able to:

- explain why Tor needs a directory system and which risks that system must avoid;
- distinguish its documents: descriptors, microdescriptors, votes, consensuses, and authority certificates;
- describe how directory authorities vote each hour and combine their votes;
- understand the criteria used to assign flags;
- read a consensus line by line and verify its signatures;
- download and verify the laboratory network's consensus using your own code (Laboratory 6).

In the previous laboratories, we always knew in advance which relays to connect to, which keys they used, and which ports they listened on: we read that information from the laboratory network's files. A real client does not have that luxury. It must discover which relays exist, where they are, which keys they use, and how trustworthy they are. That is the job of the **directory system**, the subject of this chapter.

12.1 The Problem

A directory system for Tor must satisfy several partly competing requirements:

Correctness Information about relays (addresses and keys) must be authentic. An adversary who convinced a client that a particular relay had a different key could intercept its traffic.

Common view All clients must see *the same* list of relays. If an adversary could give each client a different list, perhaps containing only a few adversary-controlled relays, clients could be recognized by the relays they use. This is a *partitioning*, or *epistemic*, attack.

Resistance to abuse Anyone can add relays to the network. The system must limit the influence of someone adding many relays (a *Sybil* attack) and be able to exclude relays that misbehave.

Scalability Millions of clients download directory information several times a day. Distribution must be economical.

Tor's solution, developed over three versions of the directory protocol, is a small set of trusted servers, the **directory authorities**. Every hour, they vote and jointly sign a single document: the **consensus** [dir-spec § Outline].

Historical note: Three generations

Under the first version of the protocol, each authority published its own relay list, so a dishonest authority could mislead clients. Under the second, clients downloaded every authority's list and combined the views themselves, multiplying bandwidth costs. The third version, in use for over fifteen years, delegates that combination to the authorities: they produce one document, signed by all, that clients need only verify. At that point the authorities also stopped signing with the same key they used as relays, which they had to keep in plaintext in memory.

12.2 The Participants

Directory authority Servers operated by trusted members of the Tor community. The public network has nine, plus a separate authority for bridges (chapter 16). Their list is compiled into tor (`src/app/config/auth_dirs.inc`) so that every client starts with the same set. Each authority has a long-term *identity* key kept offline, which certifies a *signing* key valid for 3 to 12 months. The latter signs votes and consensuses.

Directory cache Relays that fetch consensuses and descriptors from authorities and redistribute them to clients, reducing the authorities' load. They have the `V2Dir` flag and serve `BEGIN_DIR` streams (chapter 11).

Fallback directory A list of approximately two hundred stable relays, also compiled into tor (`fallback_dirs.inc`, 200 entries in tor 0.4.9.13). A newly installed client asks one of them for its first consensus instead of overwhelming the authorities.

Relay Relays periodically publish their own descriptors to the authorities.

Client Clients download the consensus and microdescriptors, verify them, and use them to select paths (chapter 13).

Figure 12.1 shows how these documents circulate.

12.3 The Documents

All directory documents are text in the format the specification calls *netdoc*: a sequence of lines (*items*), each containing a keyword and arguments, sometimes followed by a base64-encoded object between `-----BEGIN ...------` and `-----END ...------` lines. This format is straightforward for people to read and programs to parse, as we will see in the laboratory.

Relay descriptor (*server descriptor*) Written and signed by the relay: addresses and ports, keys (identity, ntor onion key, Ed25519 certificates), advertised bandwidth, full exit policy, family, platform, and version. It is republished at least every 18 hours (`FORCE_REGENERATE_DESCRIPTOR_INTERVAL` in tor's code), or sooner if something important changes.

Extra-info document Statistics not needed for protocol operation (such as bridge traffic broken down by country), published separately to avoid bloating descriptors. They feed Tor Metrics (chapter 24).

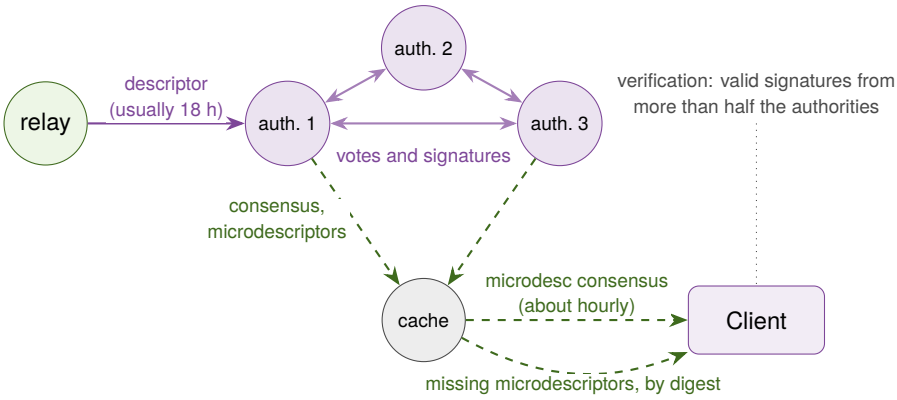


Figure 12.1: The flow of directory documents. Relays publish descriptors to the authorities; authorities exchange votes, compute consensus from the available votes, and exchange signatures; directory caches download consensuses and microdescriptors and serve them to clients.

Microdescriptor The subset of a descriptor that a client actually needs and that changes infrequently: the ntor onion key, family, an exit-policy summary, and Ed25519 identity. Authorities calculate it; each microdescriptor is identified by the SHA-256 digest of its text.

Vote An authority's assessment of the network: the flags, bandwidth, and version for each relay.

Consensus The result of combining votes, signed by the authorities. It has two *flavors*: `ns`, containing the full descriptor digest for each relay, and `microdesc`, containing the microdescriptor digest. Modern clients use only the more compact second flavor.

Authority key certificate Binds an authority's signing key to its identity.

In depth: Why microdescriptors?

A full descriptor is several kilobytes long and changes frequently, for example whenever advertised bandwidth changes. Downloading all descriptors for thousands of relays several times a day would cost clients considerable bandwidth, especially on mobile connections. A microdescriptor contains only information that rarely changes. Frequently changing information, such as bandwidth and flags, belongs in the consensus, which the client downloads anyway. Because a client requests microdescriptors by digest rather than relay identity, a directory cache cannot give one client a microdescriptor different from the one everyone else receives.

12.4 Voting

12.4.1 The schedule

Each consensus has three timestamps [dir-spec § Voting timeline]:

`valid-after` when it becomes valid;

`fresh-until` when it ceases to be the freshest consensus; normally this matches the next consensus's `valid-after`;

`valid-until` when the client can no longer use it.

On the public network, the voting interval is one hour and a consensus remains valid for three intervals (the `V3AuthVotingInterval` and `V3AuthNIntervalsValid` options). Thus several valid consensuses coexist at any given time and a delay at the authorities need not block clients. Before each interval begins, authorities have five minutes to exchange votes and five minutes to exchange signatures (`V3AuthVoteDelay` and `V3AuthDistDelay`). In the laboratory network, everything is accelerated: a vote every 20 seconds, with 4 seconds for each phase (section 19.5).

12.4.2 Combining the votes

Each authority collects the others' votes and computes the consensus with a deterministic algorithm, so that all authorities obtain the same document and can sign it [dir-spec § Computing a consensus from a set of votes]. The principal rules are:

- a relay appears in the consensus only if enough authorities know it and, under current consensus methods (24 and later; the laboratory uses 36), only if it has both `Running` and `Valid`;
- a relay receives a flag if more than half the authorities voting on that flag assign it;
- bandwidth (`w`) is the lower median of the voted bandwidths, except that if at least three authorities have *measured* its bandwidth, the median of those measurements is used;
- network parameters (`params`) are included if a majority of all authorities, or at least three authorities, vote for them (`i > total_authorities/2 || i >= 3` in the code); the chosen value is the lower median;
- timestamps are the medians of the proposed times.

Because authorities may run different tor versions, the document declares the *consensus method* in use: a number identifying the exact set of rules, chosen as the highest method supported by enough authorities. The laboratory network's consensus uses method 36.

12.4.3 Flags

Flags summarize each authority's assessment of a relay [dir-spec § Assigning flags in a vote]. The specification warns that the exact criteria may change and clients must not depend on them. Table 12.1 summarizes the current criteria.

`BadExit` and `MiddleOnly` let authorities respond to suspicious relays without removing them altogether. The `Sybil` flag limits the effect of someone running many relays from one address. In the laboratory network, where every relay runs on 127.0.0.1, the limit is disabled by `AuthDirMaxServersPerAddr=0`; the laboratory will show this in the `params` line.

12.4.4 Bandwidth

How fast is a relay? The value advertised in its descriptor is not trustworthy: a malicious relay could claim enormous bandwidth to be chosen more often. Some authorities, called *bandwidth authorities*, therefore measure relay bandwidth by passing traffic through them with a dedicated tool (currently *sbws*, chapter 24) and include it in their votes as `Measured`. If at least three measurements are available, the consensus uses their median. Otherwise, the `w` line reports `Unmeasured=1` and a limited value. Consensus bandwidth is not strictly a speed but a *weight*: it determines the probability that a relay will be selected (chapter 13).

Flag	Criterion (summary)
`Running`	the authority has reached the relay on all its ORPorts within the past 45 minutes
`Valid`	tor version not known to be defective; relay not excluded by the authorities
`Fast`	bandwidth among the fastest seven-eighths, or at least 100 KB/s
`Stable`	time-weighted mean time between failures at least the median, or at least 7 days
`Guard`	`Fast`, `Stable`, *familiar* (on the network for several weeks), good availability, sufficient bandwidth, `V2Dir`
`Exit`	permits exit traffic to at least one /8 block on both ports 80 and 443
`HSDir`	`Stable` and `Fast`, running for at least 96 hours: may store onion service descriptors
`V2Dir`	serves directory information (directory cache)
`Authority`	is a directory authority
`BadExit`	exit that should not be used as such (for example, because it modifies traffic)
`MiddleOnly`	should be used only as a middle relay (introduced in 0.4.7.2-alpha)
`StaleDesc`	descriptor published more than 18 hours ago
`Sybil`	exceeds the maximum number of relays per IP address (normally 2)

Table 12.1: Consensus flags and a summary of how authorities assign them.

12.5 Anatomy of a Consensus

Here is an abbreviated microdesc consensus from the laboratory network, downloaded in Laboratory 6. The excerpts in this section come from a copy preserved in the repository (`labs/en/code/torlab/test/data/consensus-microdesc.txt`), used by the laboratory tests. You can therefore inspect them line by line without starting the network. The copy was captured on 23 September 2026, when the laboratory relays were still running tor 0.4.9.12. That is the version in the relay entry's `v` line, not the book's reference version, which the network runs today.

Header.

```
network-status-version 3 microdesc
vote-status consensus
consensus-method 36
valid-after 2026-09-23 15:37:20
fresh-until 2026-09-23 15:37:40
valid-until 2026-09-23 15:38:00
voting-delay 4 4
known-flags Authority Exit Fast Guard HSDir ... Running Stable ... Valid
params AuthDirMaxServersPerAddr=0 cc_alg=2
```

The `params` line contains the *consensus parameters*, which authorities use to adjust the entire network's behavior without requiring software updates. Here, besides the per-address limit, it specifies the congestion-control algorithm (`cc_alg=2`, Vegas, section 11.3). The public network has dozens of parameters.

A relay entry.

```
r test006e ml1aPg9UFy4VCx85YcpsRs/9shc 2038-01-01 00:00:00 127.0.0.1 5106 0
m FfrgO+s4VYS6WKBq3pjfMDs+uZ2uVsvgG88JilbhIOo
s Exit Fast Guard HSDir Running Stable V2Dir Valid
v Tor 0.4.9.12
pr Conflux=1 Cons=1-2 Desc=1-4 ... Link=3-5 ... Relay=2-6
w Bandwidth=4096
```

`r` nickname, RSA identity digest (base64), a date field no longer meaningful in the consensus, IPv4 address, ORPort, and DirPort;

`m` digest of the microdescriptor to download;

`s` flags;

`v`, `pr` tor version and supported *subprotocols*: this is where a client learns, for example, that the relay supports CGO (`Relay=6`) or link protocols 3–5;

`w` bandwidth weight in kilobytes per second.

In the `ns` flavor, each entry also has a `p` line summarizing the exit policy by port. In the microdesc flavor, this summary is in the microdescriptor and is not repeated in the consensus.

Footer and signatures.

```
directory-footer
bandwidth-weights Wbd=821 Wbe=0 Wbg=5000 Wbm=10000 Wdb=10000 ...
directory-signature sha256 3776EF73A9214587AEFB3D732FA4A443CFF491BA
                           79E102D226590BC5D1D1EBBAD8A16122EC0F9801
-----BEGIN SIGNATURE-----
...
-----END SIGNATURE-----
```

The `bandwidth-weights` values are coefficients clients use to balance relay usage across circuit positions (chapter 13). Signatures follow, one per authority: the hash algorithm, authority identity digest, signing-key digest, and signature.

12.6 Verifying a Consensus

A client accepts a consensus only if it bears valid signatures from **more than half** the authorities it knows. In tor's code, the threshold is `n_v3_authorities/2 + 1` (`src/feature/nodelist/networkstatus.c`): 5 signatures out of 9 on the public network, or 2 out of 3 in the laboratory. To verify a signature, the client needs:

1. the authority's **certificate**, which the client downloads (`/tor/keys/all`) and verifies against the authority identity known in advance;
2. the **signed text**: the entire document from its beginning up to and including the space following the first `directory-signature` keyword;
3. the signature scheme: RSA with PKCS#1 v1.5 padding applied directly to the digest (SHA-1 or SHA-256), *without* the algorithm identifier normally expected by PKCS#1 [dir-spec § Signing documents].

In depth: Up to the space or up to the newline?

The specification's general section on signed documents says the signature covers the text through the *newline* following the signature line. The consensus-specific section instead specifies the *space* following `directory-signature`, ensuring that all authorities sign exactly the same bytes even though their signature lines differ. In Laboratory 6, we tested both readings against a real consensus from the laboratory network. All three signatures were valid using the space rule; none was valid using the newline rule. When in doubt, an experimental check against an independent implementation remains the

final arbiter.

Warning: The authority list is a trust anchor

The entire system rests on the list of authorities compiled into the software. A user can change it, but the specification advises against doing so: a client using different authorities would see a different consensus from everyone else and would be recognizable. The authorities themselves are targets. If a majority were compromised, they could sign a consensus containing malicious relays. This is why they are operated by different people in different countries, with identity keys kept offline.

12.7 How a Client Stays Up to Date

A client does not download a new consensus at the stroke of every hour: if all clients did that together, directory caches would be overwhelmed. Instead, it chooses a random time between three-quarters of the first interval after the consensus ceases to be fresh and seven-eighths of the remaining time before it expires [dir-spec § Client operation]. The specification gives an example: for a consensus valid from 1:00, fresh until 2:00, and valid until 4:00, the client will seek a new one at a random time between 2:45 and 3:50.

To save bandwidth, a client can request only the *differences* from the consensus it already holds (a *consensus diff*), identified by its SHA3-256 digest. It then downloads only missing microdescriptors, requesting them by digest (`/tor/micro/d/` followed by digests separated with `-`).

12.8 Laboratory: Downloading and Verifying a Consensus

Laboratory 6: The laboratory network's consensus

Objective. Write `labs/en/code/torlab/directory.py` to download a consensus, certificates, and microdescriptors from an authority through a circuit, parse them, and verify the signatures.
Prerequisites. Laboratories 1–5; the laboratory network running.
Estimated time. 3 hours.

Step 1: downloading with BEGIN_DIR. Like tor, we use a one-hop circuit to a directory, without congestion control, and open `BEGIN_DIR` streams. We speak HTTP/1.0 over the stream:

```
def download(circ, path, stream_id):
    s = Stream(circ, stream_id)
    circ.send(RelayMessage(RelayCommand.BEGIN_DIR, stream_id, b""))
    _, msg = s._next()                      # CONNECTED, empty body
    s.send(f"GET {path} HTTP/1.0\r\n\r\n".encode())
    header, _, body = s.receive_all().partition(b"\r\n\r\n")
    return body

circ = build([relay("000a")], cc=False)
text = download(circ, "/tor/status-vote/current/consensus-microdesc", 1)
```

Step 2: the netdoc format. A roughly twenty-line function, `items`, splits a document into items and collects `-----BEGIN`/`-----END` objects. Two parsers are built on it: `parse_consensus`, which produces an entry for each relay with its flags, bandwidth, policy, and microdescriptor digest, and `parse_certificates`.

Step 3: verifying signatures.

```
def signed_text(text):
    start = text.index("network-status-version ")
    pos = text.index("\ndirectory-signature ") + len("\ndirectory-signature ")
    return text[start:pos].encode()

def verify_signature(consensus, signature, cert):
    h = hashlib.sha256 if signature.algorithm == "sha256" else hashlib.sha1
    expected = h(signed_text(consensus.text)).digest()
    recovered = cert.signing_key.recover_data_from_signature(
        signature.signature, padding.PKCS1v15(), None)
    return recovered == expected
```

The `recover_data_from_signature` method, with algorithm `None`, returns the digest embedded in the signature without expecting an algorithm identifier: exactly Tor's scheme.

Step 4: the results. In one of our runs on the laboratory network (dates, names, and counts depend on the network instance):

```
consensus: 6035 bytes; 2026-09-23 15:37:20 -> 2026-09-23 15:38:00
method 36; signatures 3; certificates 3
params: {'AuthDirMaxServersPerAddr': 0, 'cc_alg': 2}
  test006e 127.0.0.1:5106 w=4096 flag=['Exit', 'Fast', 'Guard', ...]
  test000a 127.0.0.1:5100 w=20   flag=['Authority', 'Fast', 'Guard', ...]
  ...
valid signatures: 3 (required: 2)
microdescriptors: 8; match consensus m lines: True
```

The consensus covers 40 seconds (two 20-second intervals), is signed by all three authorities, and the SHA-256 digest of every downloaded microdescriptor matches its `m` line in the consensus. The chain of trust runs from the authorities' keys to each relay's ntor key.

Step 5: negative controls. A verification that never fails verifies nothing. Change a single digit in the consensus (`Bandwidth=4096` to `4097`) and the number of valid signatures drops to zero; with only one known authority, it drops to one. The `test/test_directory.py` files repeat these checks against a saved copy of the consensus, so they also run without the network.

Step 6: the real consensus. The same code works on the public network. Download a consensus and matching authority certificates from CollecTor for the same period, then verify them:

```
c = parse_consensus(open("consensus-microdesc-2026-09-23-15").read())
cert = parse_certificates(open("certs").read())
valid, required, _ = verify_consensus(c, cert, 9)
print(valid, required)          # 9 5
```

All nine signatures are valid, well above the required five. One detail causes trouble on the first attempt: CollecTor files begin with an annotation line, `@type network-status-microdesc-consensus-3 1.0`, which is not part of the signed document. Including it changes the digest, making *all* signatures invalid. This looks like a cryptographic error but is actually a formatting error. The `signed_text` function therefore starts at `network-status-version`, and a test checks that behavior.

Summary

- The directory system must give all clients the same authentic view of the network, resist Sybil attacks, and scale to millions of users.
- Nine directory authorities (plus one for bridges), listed in the code, vote every hour and sign a single consensus, valid for three hours.
- Relays publish descriptors; clients use the microdesc consensus and microdescriptors, downloaded by digest.
- A flag is assigned if more than half the authorities responsible for it vote for it; bandwidth measured by at least three authorities takes precedence over advertised bandwidth.
- A client accepts a consensus bearing valid signatures from more than half the authorities; the signature covers text up through the space following `directory-signature`.
- Consensus parameters adjust the entire network's behavior without requiring software updates.

Exercises

Exercise 12.1 ⋆ A consensus has `valid-after` at 10:00, `fresh-until` at 11:00, and `valid-until` at 13:00. During which interval will the client seek its replacement? What happens if it has not found a new consensus by 13:00?

Exercise 12.2 ⋆ A relay has 500 KB/s bandwidth, has been on the network for two days, and has exit policy `accept *:443, reject *:*`. Which flags can it receive? Why not `Exit`?

Exercise 12.3 ⋆⋆ Describe a partitioning attack in which a malicious directory cache sends one particular client an old but still valid consensus. What limits the attack? Why does requesting microdescriptors by digest help?

Exercise 12.4 ⋆⋆ Three authorities measure a relay's bandwidth: 5000, 5200, and 90000. Which value appears in the consensus? What if the third authority, colluding with the relay, were the only one to measure it?

Exercise 12.5 ⋆⋆ Find the `v3-status-votes` file in one authority's directory on the laboratory network. Compare that authority's vote with the consensus: which lines differ, and why?

Exercise 12.6 ⋆⋆ Choose a relay from the consensus downloaded in the laboratory. Calculate its microdescriptor digest manually (using `sha256sum`) and compare it with the `m` line. At which line does the hashed text start, and where does it end?

Exercise 12.7 ⋆⋆⋆ Stop one of the laboratory network's three authorities and download the consensus again after one minute. How many signatures does it contain? Does your verifier accept it? Repeat with two authorities stopped, and explain what you observe when the consensus expires.

Exercise 12.8 ⋆ ⋆ ⋆ Also download the `ns` consensus (`/tor/status-vote/current/consensus`) and verify its signatures. Which hash algorithms does it use? What additional information does the `r` line contain compared with the microdesc flavor?

Further Reading

- *Tor directory protocol, version 3* (dir-spec) [Thee]: formats, voting, flag assignment, and client/cache operation.
- The shared-randomness protocol document (srv-spec), which authorities use to produce a common random value each day; we will need it for onion services (chapter 14).
- Tor Metrics and CollecTor, `https://metrics.torproject.org/`: historical archives of consensuses, votes, and descriptors.

Chapter 13

Path Selection, Guards, and Vanguards

Last verified: 23 September 2026 — torspec commit `8e1f245`; tor 0.4.9.13; public consensus of 23/09/2026 at 15:00 UTC

Chapter objectives

By the end of this chapter, you will be able to:

- describe the order and constraints tor uses to select relays for a circuit;
- explain how consensus bandwidth and *bandwidth-weights* balance the load among circuit positions;
- explain why guards are used and trace their history from three guards to one and then two;
- describe the guard algorithm: sampled, filtered, confirmed, and primary guards;
- explain what vanguards are and why onion services need them;
- simulate path selection using a real public-network consensus (Laboratory 7).

The directory system (chapter 12) gives the client a list of relays. What remains is the question that, more than any other, determines how well a user is protected: *which* relays should it use? If an adversary controls both the first and the last relay on a circuit, it can correlate incoming and outgoing traffic and discover who is communicating with whom (chapter 26). Path selection is therefore largely the art of making that event as rare as possible without sacrificing performance.

13.1 Order and Constraints

For a three-hop circuit, tor selects the **exit first**, then the guard, and finally the middle [path-spec § Path selection and constraints]. The exit comes first because it is the most constrained relay: its exit policy must allow the requested destination. If the client knows only the destination's name rather than its address, it selects an exit that *might* accept it: one whose policy does not reject the requested port before accepting it.

With a few exceptions, all paths satisfy these constraints:

- no relay without the `Fast` flag;
- a relay cannot appear twice;
- at most one relay from each IPv4 /16 network (or IPv6 /32 network): two relays in the same data center may be under the same operator's control;

- at most one relay per **family**: operators with multiple relays should declare them as a family, and tor avoids placing two in the same circuit;
- the first relay is a guard (section 13.3);
- no exit with the `BadExit` flag.

Connections expected to remain open for a long time (tor classifies some ports this way, such as SSH's 22 and IRC's 6667) also require `Stable` relays. Onion services add a constraint independent of the port: service-side introduction circuits and rendezvous paths are built as `Stable` circuits, and their endpoints must not be `MiddleOnly` [path-spec § Path selection and constraints]. The "few exceptions" chiefly concern onion-service circuits built with vanguards (section 13.4), which deliberately relax some constraints (for example, allowing a guard at a middle position). Disallowing those combinations would tell an adversary which relays had *not* been selected, revealing information about guard selection.

In depth: Two ways to declare a family

Traditionally, an operator declares a family by listing all its other relays in each relay's descriptor. Two relays belong to the same family only if they list each other. With many relays, these lists become enormous, and a single configuration mistake can break them. A more recent mechanism, *happy families*, gives each family a key. Each relay proves membership with a certificate signed by that key (`family-cert` in the descriptor, `family-ids` in the microdescriptor). The `use-family-lists` and `use-family-ids` consensus parameters determine which mechanism clients apply.

13.2 Weighting by Bandwidth

13.2.1 Why weighting matters

If clients selected relays uniformly at random, a 1 MB/s relay would receive as much traffic as a 1 GB/s relay. The first would be overwhelmed and the second underused. Instead, Tor selects each relay with probability **proportional to its bandwidth** in the consensus (the `w` line, section 12.5), distributing load according to capacity.

13.2.2 Circuit positions are not equivalent

There is a complication. Relays with the `Exit` flag are relatively scarce, and their outbound bandwidth is the network's scarcest resource. Using an exit as a guard or middle would consume capacity needed for exiting. The same principle applies, to a lesser degree, to guards. That is why authorities calculate **bandwidth-weights** for each consensus: coefficients multiplying relay bandwidth according to its *type* (guard only, exit only, guard and exit, or neither) and the *position* for which it is selected [path-spec § Weighting node selection].

Each weight is named `Wpt`: p is the position (`g` for guard, `m` for middle, `e` for exit, `b` for directory requests), and t is the type (`g`, `e`, `d` for guard and exit, `m` for neither). Values are expressed in ten-thousandths: `Wmg=3805` means that, when a guard is selected as a middle, only 38.05 % of its bandwidth counts.

13.2.3 Weights on the real network

The following weights come from the public consensus of 23 September 2026 at 15:00 UTC, downloaded from CollecTor in Laboratory 7:

Position \ type	guard (`g`)	middle (`m`)	exit (`e`)	guard+exit (`d`)
guard (`Wg`)	6195	6195	—	0
middle (`Wm`)	3805	10000	0	0
exit (`We`)	10000	10000	10000	10000

The implications are immediate. Exits, and relays that are both guards and exits, have **zero** weight in guard and middle positions. On this network, exit bandwidth (33 % of total bandwidth) is so valuable that it is reserved for the exit position. Guards, which account for 70 % of bandwidth, are plentiful: they are weighted at 62 % as guards and 38 % as middles. As the laboratory will show, the middle in 60 % of circuits is itself a relay carrying the `Guard` flag.

The formulas by which authorities calculate these weights are in the directory specification. They distinguish several cases depending on which type of bandwidth is scarce and seek, wherever possible, to equalize available capacity across the three positions.

13.3 Guards

13.3.1 The problem they solve

Suppose an adversary controls a fraction c of the bandwidth of possible first relays and possible exits. If the client chose a fresh guard and exit at random for every circuit, the probability that both belonged to the adversary would be approximately c^2 per circuit. After C circuits, the probability that at least one had been compromised would be $1 - (1 - c^2)^C$, approaching 1 [guard-spec § Introduction and motivation]. A user who runs Tor every day builds thousands of circuits; sooner or later, almost certainly, the adversary would see some of that user's traffic.

The idea behind **guards** (called *helper nodes* in the research literature) is to select the first relay *once* and keep using it for a long time. If that guard is honest, no circuit is compromised in this particular way; if it belongs to the adversary, many are. The probability of compromise *at least once* drops from nearly 1 to approximately c.

Circuits	Without guards	With a fixed guard
1	0.0025	0.0025
100	0.221	0.050
1000	0.918	0.050
10 000	1.000	0.050

Calculated in Laboratory 7 for $c = 5\,\%$, the table illustrates the trade-off. Guards do not make compromise impossible; instead, they turn an event that would become nearly certain over time into a rare event affecting a small proportion of users, rather than eventually affecting almost everyone. This model is deliberately simplified: for example, it ignores the possibility that the selected guard changes over time.

13.3.2 Three, one, two

Guards were introduced in 2006, after Øverlier and Syverson demonstrated how to locate an onion service by repeatedly inducing it to connect to adversary-controlled relays [ØS06]. For years, clients used three guards, replacing them after two or three months. Subsequent studies showed that three guards and frequent rotation increase exposure. Proposal 236 (2014) proposed using just one guard, retained for nine or ten months. A single guard,

however, makes connectivity more fragile and makes it easier to force the client to replace the guard. Proposal 291 (2018) moved to **two** guards. tor's built-in default is still one (`DFLT_N_PRIMARY_GUARDS_TO_USE` in `src/feature/client/entrynodes.h`), but the public consensus carries `guard-n-primary-guards-to-use=2`, which overrides it for all clients. This is a concrete example of how authorities regulate the network through the consensus.

13.3.3 The algorithm

The current algorithm, specified in Proposal 271 and the guard specification, manages several sets [guard-spec § The algorithm]:

Sampled guards A set of bandwidth-weighted, randomly selected guards persisted on disk. It grows only when necessary, to a maximum of 60 relays or 20 % of the number of guards in the consensus. Limiting the sample prevents an adversary from making a client try every guard on the network, one after another, until it reaches an adversarial one.

Filtered guards Sampled guards still present in the consensus and usable with the current configuration (for example, behind a firewall allowing only certain ports).

Confirmed guards Guards actually used for user traffic, in preference order.

Primary guards Normally three guards drawn from the filtered list: confirmed and filtered guards first, in preference order, then other filtered guards if needed. The client uses the first reachable primary guard (or the first two under the consensus parameter discussed above). If none is reachable, it tries other guards, but returns to primary guards as soon as possible.

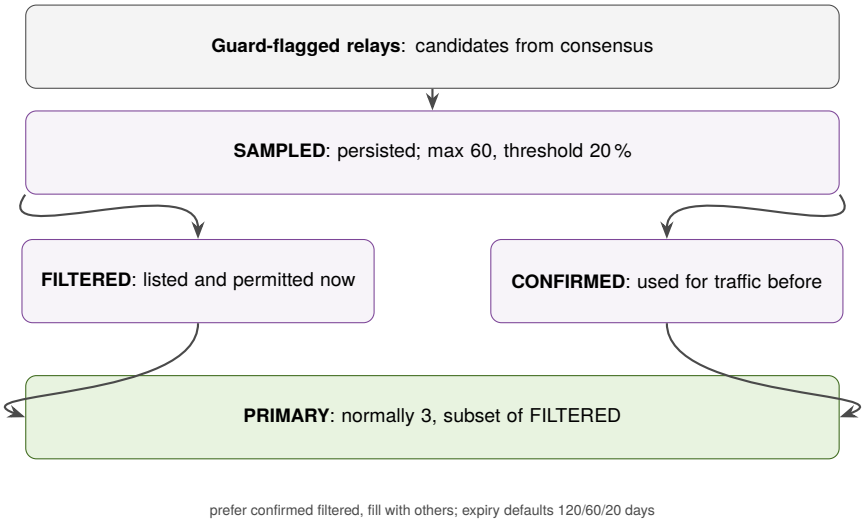


Figure 13.1: Relations among Tor guard lists: the persistent sample is drawn from consensus guards, while FILTERED and CONFIRMED are distinct and potentially overlapping subsets of that sample. PRIMARY guards are selected only from FILTERED, giving priority to confirmed guards and adding other filtered guards as needed. Sampling limits reduce exposure to new guards.

How long a guard stays in the sample depends on its state. A guard **never confirmed** (`CONFIRMED_ON_DATE` unset) leaves after `GUARD_LIFETIME`, or 120 days from sampling. A **confirmed** guard, which the client actually uses, instead leaves `GUARD_CONFIRMED_MIN_LIFETIME`

= 60 days after confirmation. It also leaves if it is absent from the consensus for 20 days. Saved timestamps (`ADDED_ON_DATE` and `CONFIRMED_ON_DATE`) are randomly backdated by as much as `GUARD_LIFETIME`/10 = 12 days. The state file therefore does not reveal precisely when the client began using Tor, and the effective lifetime of a confirmed guard becomes 48–60 days. Thus it is the *confirmed* guard that rotates roughly every two months, not every four. This detail matters for the simulation in chapter 29.

Warning: A new guard is a clue

Tor carefully distinguishes an *unreachable* guard from a *failed* circuit. If the client changed guards whenever a circuit failed, an adversary could intentionally make circuits through honest guards fail until the client selected an adversarial guard. This is precisely what tor's security release 0.4.9.13, published on 23 September 2026, addressed: the client now attributes failure to a guard only when it cannot establish the connection, not when circuits, streams, or directory requests fail.

13.4 Vanguards

Guards protect the *first* hop. Onion services (chapter 14) face an additional problem: a malicious client can induce a service to build as many circuits as it wants to rendezvous points of the attacker's choosing. If the middle on each circuit were selected afresh at random, sooner or later it would belong to the adversary, which would then observe the service's guard. The next step could be to attack that guard or obtain its data. **Vanguards** extend the guard idea to later hops, keeping relays at the second level (and, in some configurations, the third level) fixed as well.

Vanguards-lite (Proposal 333, tor 0.4.7.1-alpha) Enabled by default in clients and onion services. It uses four second-layer relays, not saved to disk, each with a randomized lifetime. Here the code and the specification differ. tor 0.4.9.13 uses a range of 1 to 12 days (about one week on average), defined by `MIN_SECOND_GUARD_LIFETIME` and `MAX_SECOND_GUARD_LIFETIME` in `src/feature/client/entrynodes.c`. The `guard-hs-l2-lifetime-min` and `-max` consensus parameters can override these values, but neither appeared in the consensus of 23 September 2026. The vanguards specification, by contrast, still gives 1–22 days (two weeks on average). This mode is designed for services and users active for no more than a few weeks.

Full vanguards Relays fixed at two levels (the second and third hops), with longer lifetimes, intended for long-running, high-risk onion services. In tor, they are implemented through the *vanguards* add-on, which sets `HSLayer2Nodes` and `HSLayer3Nodes`. Arti implements them directly, with a `Full` mode alongside `Lite`.

On the laboratory network (section 19.7.3), we saw circuits with purpose `HS_VANGUARDS`: circuits pre-built with vanguards, ready to be extended toward an introduction or rendezvous point.

13.5 Laboratory: Examining the Public Network

Laboratory 7: Bandwidth-weighted path selection

Objective. Write `labs/en/code/torlab/path_selection.py` and use it to analyze a real public-network consensus.
Prerequisites. Laboratory 6 (`directory.py`). A public consensus downloaded from CollecTor, the Tor Project's public archive (`https://collector.torproject.org/`). These data are published specifically for research, and downloading them involves no network users.
Estimated time. 2 hours.

Step 1: a relay's weight.

```
def _relay_type(r):  # g = Guard only, e = Exit only, d = both, m = neither
    g, e = "Guard" in r.flag, "Exit" in r.flag and "BadExit" not in r.flag
    return {(1, 1): "d", (1, 0): "g", (0, 1): "e", (0, 0): "m"}[(g, e)]

def weight(r, position, weights):
    key = f"W{position[0]}{_relay_type(r)}"          # e.g. "Wmg"
    return (r.bandwidth or 0) * weights.get(key, 10000) / 10000
```

Step 2: selecting a path. The `Selector` class computes weights once per position. Constraints are applied *by rejection*: draw a relay and, if it violates a constraint (same relay or same /16), draw another.

```
def path(self, rng, guard=None):
    e = self.choose("exit", [], rng)
    g = guard or self.choose("guard", [e], rng)
    m = self.choose("middle", [e, g], rng)
    return g, m, e
```

A first version that filtered over nine thousand relays and recalculated weights and networks on every selection took 310 seconds to construct 2000 paths. This version builds 20 000 in 7 seconds. In a statistical simulation, performance is not a minor detail.

Step 3: the public network in numbers. For the consensus of 23 September 2026 at 15:00 UTC:

```
relays: 9358
bandwidth with Guard: 70.1%    bandwidth with Exit: 33.1%
guard : eligible  5306; top 10 = 1.7%; top 100 = 12.6%
middle: eligible  8961; top 10 = 2.8%; top 100 = 13.7%
exit  : eligible  3126; top 10 = 2.3%; top 100 = 13.7%
20000 paths: gge 30.9%  ggd 29.4%  gme 20.4%  gmd 19.3%
```

The number of *eligible* relays is smaller than the number carrying the relevant flag (chapter 6 reports 3177 `Exit` relays, versus 3126 here). The `eligible` function also requires `Running`, `Valid`, and `Fast`, while excluding `BadExit` and `MiddleOnly`. The difference comprises 48 exits without `Fast` and 3 marked `BadExit`. The last line is a sample obtained with random seed 7. Without a fixed seed, the values change by a few tenths of a percentage point on every run and two experiments cannot be compared. The letter codes identify the guard, middle, and exit types on each path. For instance, `gme` means a path with a guard, a middle with

neither flag, and a pure exit. As the weights predict, no path uses an exit in the first or second position; in 60 % of cases, the middle carries the `Guard` flag. Concentration is low: the ten most frequently selected exits account for 2.3 % of selections together, and the top hundred for 13.7 %.

Step 4: guards and adversaries. The `Adversary` class calculates the table in section 13.3. Try changing the adversary's share: is the effect of guards still evident when $c = 1\,\%$ or $c = 20\,\%$?

Summary

- tor selects the exit first, followed by the guard and middle, avoiding duplicate relays and relays from the same /16 or family.
- Each relay is selected with probability proportional to its bandwidth multiplied by a weight depending on relay type and circuit position; on the network considered here, exits are used only as exits.
- Guards turn compromise, which becomes nearly certain over time with randomly selected first relays, into a rare event; clients currently use two.
- The guard algorithm maintains a limited sample, confirmed guards, and primary guards, and cautiously attributes failures to guards.
- Vanguards also keep the second layer fixed for onion services; vanguards-lite is enabled by default.
- Consensus parameters can override software defaults.

Exercises

Exercise 13.1 ⋆ Why does tor select the exit first? What might go wrong if it selected the guard first?

Exercise 13.2 ⋆ A relay has the `Guard` and `Exit` flags and a consensus bandwidth of 20000. Under the real-network weights reported in this chapter, what is its weight in each of the three circuit positions?

Exercise 13.3 ⋆⋆ Repeat the guard-table calculation for $c = 1\,\%$ and $c = 20\,\%$. After how many circuits does compromise probability without guards exceed 50 % in each case?

Exercise 13.4 ⋆⋆ An adversary wants to increase the probability that clients select its relays as exits. What resources must it invest, and which mechanisms (bandwidth measurements, families, per-address limits, and weights) make that harder?

Exercise 13.5 ⋆⋆ Why is the guard sample limited to 60 relays or 20 % of all guards? Describe the attack this limit prevents.

Exercise 13.6 ⋆⋆⋆ Extend `Selector` to enforce the family constraint using `family` and `family-ids` lines in microdescriptors. Download the microdescriptors corresponding to the consensus from CollecTor and measure how many draws are rejected because of families.

Exercise 13.7 ⋆ ⋆ ⋆ Group public-consensus relays by declared family and calculate the exit-selection probability share of the ten largest families. Compare this with the share of the ten individually most-selected exits. What can you conclude about risks associated with operators running many relays?

Further Reading

- *Tor Path Specification* and *Tor Guard Specification* [Thee].
- L. Øverlier and P. Syverson, *Locating Hidden Servers* [ØS06]: the attack that motivated guards.
- Proposals 236, 271, 291, and 333: from one guard to two, the current algorithm, and vanguards-lite.
- The vanguards specification (vanguards-spec), including tables of Sybil-attack timelines.

Chapter 14

Onion Services v3

Last verified: 23 September 2026 — torspec commit `8e1f245`; tor 0.4.9.13; rend-spec v3

Chapter objectives

By the end of this chapter, you will be able to:

- explain what an onion service provides and how it differs from a site reached through an exit;
- derive a service's public key from its `.onion` address and vice versa;
- describe blinded keys, time periods, and how a client locates the responsible HSDirs;
- read the layered structure of a descriptor;
- follow the introduction and rendezvous protocol step by step;
- download, verify, and decrypt the descriptor of an onion service on the laboratory network (Laboratory 8).

Until now, we have used Tor to conceal *who* connects to a server. The server itself, however, has been public: its IP address was known, and the exit connected directly to it over the public network, learning the destination. The content was plaintext only if the application did not use end-to-end encryption such as HTTPS. **Onion services** reverse that arrangement. They let someone offer a service, such as a website, without revealing its network location. The client and service meet halfway, each protected by its own circuit, and neither learns the other's address.

14.1 What an Onion Service Provides

Service anonymity The server's IP address is never exposed: the service communicates with the network only through Tor circuits that it initiates.

Self-authenticating addresses The `.onion` address *encodes* the service's public key together with a checksum and version byte. Anyone connecting to that address knows cryptographically that they are communicating with the holder of the corresponding private key, without requiring certificate authorities.

End-to-end encryption Traffic between the client and service is encrypted with keys only those two parties know; there is no exit that can observe plaintext traffic.

Reachability The service does not need to accept inbound connections. It works behind a NAT or firewall that permits only outbound connections.

The cost is path length. In the basic model, the client reaches a meeting point through a three-hop circuit, and the service reaches it through another three-hop circuit: six relays in all, compared with three when connecting to a conventional website. With vanguards-lite enabled by default (section 13.4), the path is no longer symmetrical. The service inserts a fixed second-layer relay, making its rendezvous-side circuit four hops ($\texttt{S} \to \texttt{G} \to \texttt{L2} \to \texttt{M} \to \texttt{Rend}$) compared with the client's three.

Historical note: From v2 to v3

Hidden services have existed since 2004. The first widely deployed version, v2, used 16-character addresses derived from a 1024-bit RSA key by truncating a SHA-1 digest to 80 bits. It also allowed HSDirs to collect the addresses of services whose descriptors they hosted. Version 3 (Proposal 224), available since tor 0.3.2.1-alpha (2017), introduced Ed25519 keys, 56-character addresses containing the entire public key, encrypted descriptors, and blinded keys. V2 services were fully disabled in October 2021 with tor 0.4.5.11 and 0.4.6.8. The name has changed too: today the term is *onion service*.

14.2 The Address

A v3 address base32-encodes the service's identity public key, a short checksum, and a version number [rend-spec § Encoding onion addresses]:

$$\texttt{address} = \text{base32}(\mathit{PUBKEY} \,\|\, \mathit{CHECKSUM} \,\|\, \mathit{VERSION}) \,\|\, \texttt{".onion"}$$

Here *PUBKEY* is the 32-byte Ed25519 key, *VERSION* is the byte `0x03`, and *CHECKSUM* consists of the first 2 bytes of SHA3-256(`".onion checksum"` $\|$ *PUBKEY* $\|$ *VERSION*). The resulting 35 bytes encode to exactly 56 base32 characters. The checksum does not protect against an adversary, who can calculate it like anyone else, but it detects typing errors.

In one of our laboratory runs, the onion service had the following address. This is the address of the example preserved in `labs/en/code/torlab/test/data/`, which the laboratory tests use. All outputs in this section refer to that example and can therefore be reproduced without the network. When readers start their own network, it generates fresh keys and therefore a different address (section 19.6).

```
wdjtp7qktdjkkgjjbbx2axvg2gri5p4tuhlc4bq23n7o3uwu4rveppqd.onion
```

The `key_from_address` function in Laboratory 8 extracts the public key from this address and verifies its checksum.

Warning: You cannot recognize an address by memory

A random 56-character address is neither memorable nor easy to recognize at a glance. Users copy it from a source they consider trustworthy, and an address differing by only a few characters may go unnoticed. This is the main *phishing* vector against onion services. *Vanity addresses*, generated by trying millions of keys until their initial characters form a word, help only partially: an adversary can also generate an address with the same prefix. We return to this problem in chapter 21.

14.3 Blinded Keys

14.3.1 The problem

The service must publish information about how to reach it somewhere: the *descriptor*. If it published the descriptor under its identity key, anyone operating an HSDir would see the addresses of all the services it hosted. They could enumerate those services or link one service's descriptors over time. That was precisely the situation with v2.

14.3.2 The solution

In v3, the service signs descriptors using a **blinded key**, different for each *time period* [rend-spec § Appendix A: Signature scheme with key blinding]. Given the identity key $A = aB$ (where B is the Ed25519 base point), it computes a factor

$$h = \text{SHA3-256}(\texttt{"Derive temporary signing key\0"} \,\|\, A \,\|\, B \,\|\, N),$$
$$N = \texttt{"key-blind"} \,\|\, \textit{period} \,\|\, \textit{period_length}$$

(with certain bits forced, as for Ed25519 keys), and the blinded key is

$$A' = h \cdot A.$$

This construction has valuable properties:

- anyone who knows the address, and therefore A, can calculate A' for each period and knows where to look for the descriptor;
- someone who sees only A', such as an HSDir, cannot recover A or determine that two blinded keys from different periods belong to the same service;
- the service can sign under A' using private key $a' = h \cdot a$, and the resulting signature verifies under A' like an ordinary Ed25519 signature.

An unusual detail illustrates how closely the specification follows the code: the constant string includes a trailing zero byte because tor calculates its length using `sizeof` rather than `strlen`. A source comment (`src/feature/hs/hs_common.c`) acknowledges that documenting the "surprise" in the specification was preferable to changing the code.

14.3.3 Credentials and offline keys

The identity key yields a *credential*, SHA3-256(`"credential"` $\|\, A$). For each period, this in turn yields a *subcredential*, SHA3-256(`"subcredential"` $\|$ *credential* $\|\, A'$). The subcredential is used in descriptor encryption and the introduction handshake: it demonstrates knowledge of the address, which an HSDir does not possess.

The key hierarchy is also designed to allow the identity key to remain *offline*. Each period's blinded key certifies a short-term descriptor-signing key (a type-8 Ed25519 certificate, table 7.1). In principle, the server can receive advance signing keys and certificates for future periods without ever possessing the key corresponding to its address. Not every implementation supports this mode.

14.4 Locating the Descriptor

14.4.1 Time periods

Time is divided into periods, one day long on the public network (the `hsdir-interval` consensus parameter) [rend-spec § Dividing time into periods]. Periods do not begin at midnight but at noon UTC. They are offset by 12 hours from the authorities' shared-randomness protocol, which publishes a new **shared random value** (SRV) in the consensus every day at midnight. The period number is calculated from the minutes since 1 January 1970, minus the offset, divided by the interval. The specification gives an example: at 11:15:01 UTC on 13 April 2016, the time period is 16903, which began at noon the previous day. Laboratory 8 tests exactly this example.

In test networks, with a vote every 20 seconds, waiting a day would be inconvenient. tor therefore reduces the period to one full SRV protocol round, or 24 votes (the `get_time_period_length` function). In the laboratory, a period lasts 8 minutes and is offset by 4 minutes.

14.4.2 The HSDir ring

Which HSDir holds the descriptor? Each relay with the `HSDir` flag occupies a position on a 256-bit *ring* [rend-spec § Where to publish a hidden service descriptor]:

$$\textit{relay_index} = \text{SHA3-256}(\texttt{"node-idx"} \parallel \textit{Ed25519 identity} \parallel \textit{SRV} \parallel \textit{period} \parallel \textit{period_length})$$

For each of two *replicas*, the service calculates an index:

$$\textit{service_index}(r) = \text{SHA3-256}(\texttt{"store-at-idx"} \parallel A' \parallel r \parallel \textit{period_length} \parallel \textit{period}).$$

The service uploads its descriptor to the first four HSDirs following each index around the ring (parameter `hsdir_spread_store`); the client randomly selects one of the first three (`hsdir_spread_fetch`). Including the SRV prevents an adversary from predicting far in advance which relays will be responsible for a service and placing malicious relays at the right positions to censor it or observe descriptor requests. The service publishes two descriptors at a time, for two consecutive periods, so it remains reachable to clients whose consensuses are slightly ahead or behind. Which two periods are used depends on the point in the cycle. Before or after SRV rotation, they are either the previous and current periods or the current and next periods [rend-spec § Generating and publishing hidden service descriptors].

In depth: Which SRV should be used?

Because periods and SRVs are offset by half a day, a client must select the SRV carefully. In the half-day between the beginning of a period and the next SRV, it uses the current SRV; in the other half-day, it uses the previous one. If the consensus lacks the required SRV—for example, immediately after a network starts—tor calculates a fallback, the *disaster SRV*: SHA3-256(`"shared-random-disaster"` ‖ *period_length* ‖ *period*). The selection is implemented in `hs_get_current_srv` and `hs_get_previous_srv` (`src/feature/hs/hs_common.c`). If the value is absent from the consensus, tor quietly falls back to the disaster SRV. This occurs on a newly created laboratory network and does not last long: in our network's authority log, the first SRV appears 115 seconds after startup; until then the consensus has no `shared-rand-current-value` line. To observe the fallback, inspect the first few minutes after creating the network. Once the network has been running for a while, as in section 14.9, the real SRVs are used.

14.4.3 Anonymous requests only

A client fetches a descriptor with an HTTP request to `/tor/hs/3/`, followed by the blinded key in base64 without trailing = characters, over a `BEGIN_DIR` stream to the HSDir. There is one condition: the request must arrive over an **anonymous** circuit. An HSDir rejects a request arriving on a circuit opened directly by a client—that is, a single-hop circuit. We discovered this in the laboratory:

```
HTTP/1.0 503 Rejecting single hop HS v3 descriptor request
```

The check is in `connection_dir_is_anonymous` (`src/feature/dircommon/directory.c`). It prevents an HSDir from associating a descriptor request with the requesting client's IP address.

14.5 The Descriptor

The descriptor has three layers [rend-spec § Generating and publishing hidden service descriptors], shown in fig. 14.1.

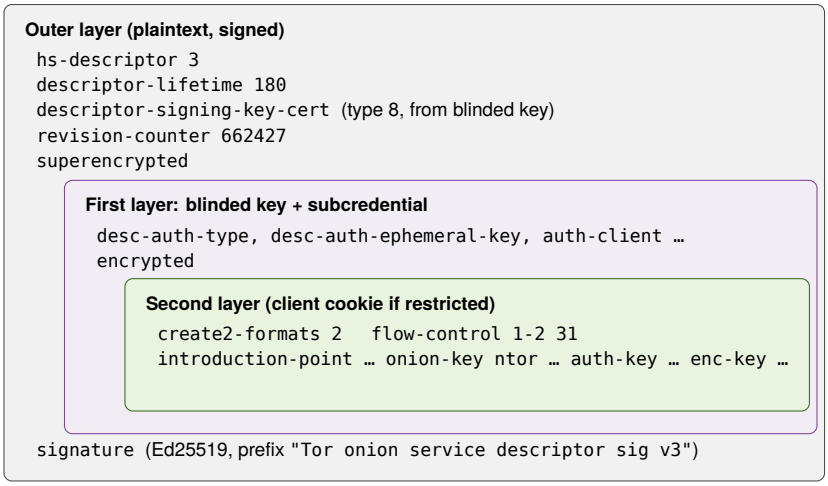


Figure 14.1: The v3 onion descriptor structure. HSDirs can read the signed outer wrapper; the first layer can be decrypted by clients knowing the onion identity. With restricted discovery, an authorized client's X25519 key is used to recover the descriptor cookie needed to decrypt the second layer.

The outer layer is plaintext and signed. It contains the version, validity period (`descriptor-lifetime`, 180 minutes in the laboratory), signing-key certificate issued by the blinded key, a revision counter, the encrypted `superencrypted` block, and a signature. The signature is an Ed25519 signature, made with the descriptor-signing key over text prefixed with `"Tor onion service descriptor sig v3"`. The prefix prevents a signature generated for a descriptor from being valid for something else. The revision counter must increase with every new version, so an HSDir does not accept descriptors older than the ones it already has.

The first encrypted layer is encrypted under a key derived from the blinded key and subcredential. An HSDir that does not know the address cannot read it. It contains the *restricted discovery* fields (section 14.7) and the second layer. To avoid revealing whether restricted discovery is enabled, the fields are always present, filled with random data when they are not needed.

The second layer contains the information needed to connect: supported handshake formats, flow-control parameters, any anti-DoS parameters, and, for each **introduction point**, the link specifiers needed to reach it (in base64), its ntor key, a certified authentication key, and an encryption key. By default, a service uses three introduction points.

Both layers use the same scheme. A random 16-byte salt, secret data, and a layer-specific constant are fed into SHAKE-256 to derive an AES-256 key, initialization vector, and authentication key. The text is encrypted in counter mode and authenticated with SHA3-256. The first layer's plaintext is padded to a multiple of 10 000 bytes, and `auth-client` lines to a multiple of 16, so that descriptor size reveals as little as possible about the number of introduction points and authorized clients.

14.6 Introduction and Rendezvous

Armed with the descriptor, the client can connect. The protocol, shown in fig. 14.2, involves three "special" relays: an introduction point chosen by the service, a rendezvous point chosen by the client, and the HSDirs.

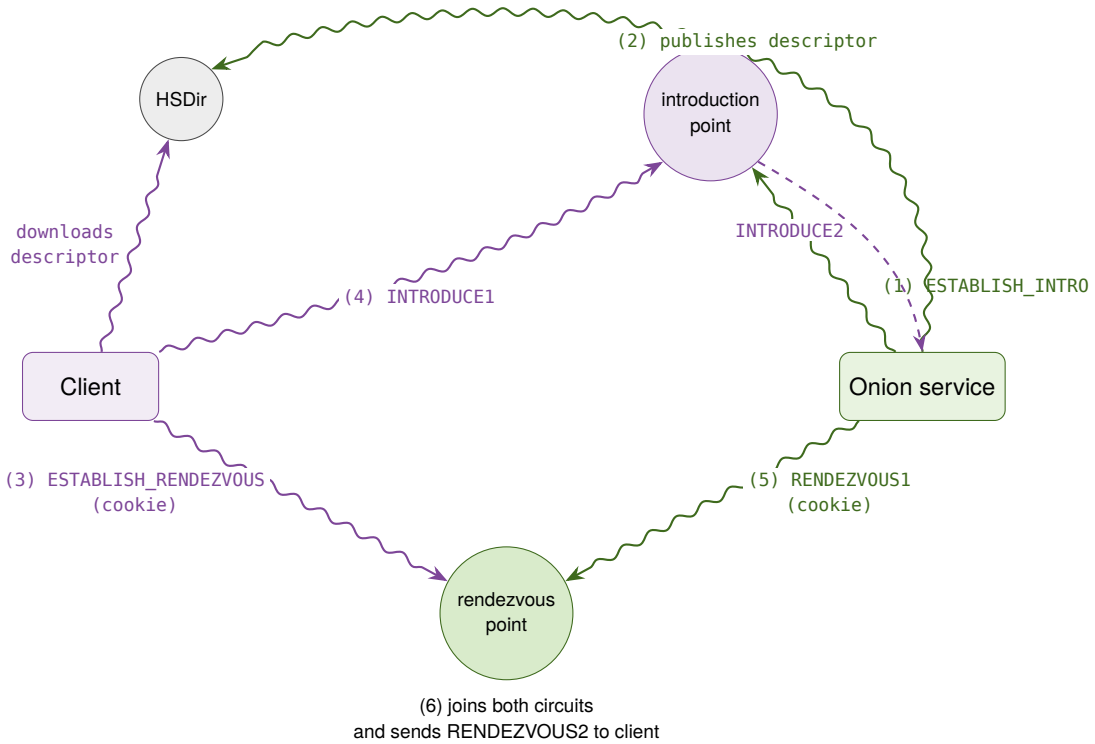


Figure 14.2: The rendezvous protocol. Each arrow represents a multi-hop circuit; no direct connection joins the client, service, and intermediate points. Numbers indicate the order of the steps described in the text.

1. **The service prepares introduction points.** It selects several relays, builds a circuit to each, and sends `ESTABLISH_INTRO` (relay command 32) containing a different authentication key and a signature for each point. The relay replies with `INTRO_ESTABLISHED` (38); from then on, it forwards requests bearing that key to the service.

2. **The service publishes its descriptor** to the responsible HSDirs, as described above.

3. **The client prepares the rendezvous.** It randomly selects a relay as its rendezvous point, builds a circuit to it, and sends `ESTABLISH_RENDEZVOUS` (33) with a random 20-byte *cookie*. The relay replies with `RENDEZVOUS_ESTABLISHED` (39).

4. **The client introduces itself.** It builds a circuit to an introduction point and sends `INTRODUCE1` (34). The plaintext portion specifies the introduction point's authentication key. The encrypted portion, readable only by the service, contains the cookie, the rendezvous point's link specifiers and ntor key, and the first half of a handshake. The introduction point sends `INTRODUCE_ACK` (40) to the client and forwards the message to the service as `INTRODUCE2` (35).

5. **The service reaches the rendezvous point.** It decrypts `INTRODUCE2`, checks that the message does not duplicate an already-seen request (the *replay cache*), builds a circuit to the specified rendezvous point, and sends `RENDEZVOUS1` (36) with the cookie and the second half of the handshake.

6. **The rendezvous point joins the circuits.** It finds the client's circuit using the same cookie, forwards the handshake's second half in a `RENDEZVOUS2` (37), and from then on copies cells between the two circuits.

The client-service handshake, called *hs-ntor*, is an ntor variant (section 9.6) that includes the subcredential and the introduction point's authentication key. Only someone who knows the address can produce an `INTRODUCE1` that the service accepts. The derived keys add another layer of encryption, like a *virtual hop* added to the client's circuit, and use SHA3-256 for the running digest rather than SHA-1 (section 10.2). Neither the introduction nor the rendezvous point can read the traffic.

Over the joined circuit, the client opens streams with `BEGIN`, just as it would with an exit (chapter 11). The service forwards these streams to local ports configured with `HiddenServicePort`.

In depth: Why so many relays?

The introduction point knows the service only as "the authentication key of a circuit"; it does not know which `.onion` address it serves. The rendezvous point sees a random cookie and two circuits; it does not know which service it is connecting to which client. Each role sees only a fragment, and none knows the client's or service's IP address, which remain behind their respective guards.

14.7 Restricted Discovery

A service may want to be reachable only by certain clients, for example as an administration panel. Under **restricted discovery** (formerly *client authorization*), each authorized client has an X25519 key pair, and the service knows its public component. The service separately encrypts the descriptor's second-layer key for each client in the first layer's `auth-client` lines. Someone who knows the address but lacks an authorized key cannot read the introduction-point list and therefore cannot even attempt to connect. In tor, this is configured by adding client public keys to the service's `authorized_clients` directory.

14.8 Defenses

Onion services are natural attack targets: a service cannot simply block attacking clients' IP addresses because it cannot see them. The principal defenses, discussed in more detail

elsewhere, are:

- **vanguards**, protecting the service's guard against attacks based on circuits built on demand (section 13.4);
- the **replay cache**, preventing an attacker from making the service repeat the same work;
- anti-DoS parameters that the service can set at introduction points through an `ESTABLISH_INTRO` extension, limiting the number of `INTRODUCE2` messages per second;
- **proof of work**, under which an attacked service asks clients to solve a small computational problem before serving them (chapter 15).

14.9 Laboratory: Finding and Reading a Descriptor

Laboratory 8: The laboratory onion service's descriptor

Objective. Write `labs/en/code/torlab/onion.py`: address handling, blinded key, HSDir ring, descriptor download, verification, and decryption.
Prerequisites. Laboratories 1–6; the laboratory network running.
Estimated time. 4 hours.

Step 1: the blinded key. Common cryptography libraries provide Ed25519 signatures, but not the point-by-scalar multiplication required to compute $A' = h \cdot A$. The module therefore includes a small implementation of Ed25519 arithmetic, about thirty lines derived from the formulas in RFC 8032, sufficient for this purpose but unsuitable for general use: it is slow and provides no protection against timing attacks.

```
def blinding_factor(A, period, duration_minutes):
    n = b"key-blind" + struct.pack(">QQ", period, duration_minutes)
    h = bytearray(hashlib.sha3_256(
        b"Derive temporary signing key\x00" + A + BASE_STR + n).digest())
    h[0] &= 248; h[31] &= 63; h[31] |= 64
    return int.from_bytes(h, "little")

def blinded_key(A, period, duration_minutes):
    return _encode_point(_multiply(blinding_factor(A, period, duration_minutes),
                                   _decode_point(A)))
```

Step 2: the period and SRV. The `time_period_info` function reproduces tor's calculation from the consensus's `valid-after` timestamp and voting interval, determining whether the current or previous SRV is needed. If the consensus lacks the required value, the disaster SRV is used.

Step 3: tracing an error. Our first run found the descriptor on none of the network's eight HSDirs. Because we had tried all of them, choosing the wrong HSDir could not explain the failure. That left the blinded key or the request. To separate those possibilities, we instructed the network's real client to download the descriptor through the control port (`HSFETCH`, followed by `GETINFO hs/client/desc/id/...`). We read the blinded key actually used by the service from the resulting certificate: it matched ours. The request was therefore at fault, and tor's source code revealed why: HSDirs reject single-hop requests. With a three-hop circuit to the HSDir, we obtained the descriptor. This is a broadly useful method: when an

experiment fails, look for an intermediate point where your result can be compared with a reference implementation.

Step 4: verifying and decrypting. In this example, `path` is a list of three `RelayInfo` objects obtained from the laboratory network, with the HSDir last; `A` is the identity key and `Ap` its blinded key. Constructing those objects is not shown.

```
d = download_descriptor(Ap, path)
desc = parse_descriptor(d, A, Ap)
```

`parse_descriptor` verifies that the signing-key certificate was issued by the expected blinded key, verifies the descriptor signature, and decrypts both layers. An abbreviated result from an earlier laboratory session:

```
lifetime 180 revision 662427 introduction points 3
create2-formats 2
flow-control 1-2 31
introduction-point AwAGfwAAARPyAhQXE5GoH+wp3MoxW9M1QV6XSxb0fgM...
onion-key ntor yRURxDb2eeEmF/p5M/bmHA+giwVyxjApKYyNY1KkCDo=
auth-key
-----BEGIN ED25519 CERT-----
...
```

The `flow-control 1-2 31` line tells clients that the service supports congestion control, with the same `sendme_inc` value of 31 encountered in chapter 11.

Step 5: a consistency check. In the laboratory network there are eight HSDirs and the service uploads the descriptor to $2 \times 4 = 8$ of them—all of them. Finding the descriptor therefore does not prove that the ring calculation is correct. The client's log (`info.log`), however, records which HSDir it contacted: `test002a`, at 16:07. It is one of the three returned by our code for that replica at that moment. A single observation is not proof, but it is consistent. Exercise 14.6 proposes a stronger test.

Summary

- An onion service offers a service without revealing its IP address; its `.onion` address contains and authenticates the service's public key.
- Descriptors are signed with period-specific blinded keys that only someone knowing the address can link to the service.
- Descriptor positions on the HSDir ring depend on the blinded key, period, and shared random value; clients download descriptors through anonymous requests.
- The descriptor has a signed outer layer and two encrypted layers; the second lists introduction points.
- Client and service meet at a rendezvous point selected by the client, following a request through an introduction point. The hs-ntor handshake adds an end-to-end encryption layer.
- Restricted discovery, vanguards, replay caches, rate limits, and proof of work protect services.

Exercises

Exercise 14.1 ⋆ How many bytes does a v3 address encode, and why does it produce exactly 56 base32 characters? What is the probability that a random typing error will pass the checksum check?

Exercise 14.2 ⋆ Calculate the public network's time-period number at 09:00 UTC on 1 January 2026. When did that period begin?

Exercise 14.3 ⋆⋆ An adversary wants to block access to an onion service by making sure it controls the responsible HSDirs. What must it predict, and how far in advance can it do so? What role does the SRV play?

Exercise 14.4 ⋆⋆ List what the introduction point, rendezvous point, and HSDir know about the client and service. Which of these roles would be most interesting to an adversary trying to estimate a service's popularity?

Exercise 14.5 ⋆⋆ Why does an HSDir reject descriptor requests arriving over single-hop circuits? What could it learn otherwise?

Exercise 14.6 ⋆⋆⋆ Test the ring calculation more rigorously. Add relays to the laboratory network (exercise 19.7) until it has more than eight HSDirs, then check that the descriptor is available *only* on the HSDirs selected by your code.

Exercise 14.7 ⋆⋆⋆ Decode the link specifiers of an introduction point from the laboratory descriptor (the base64 value in an `introduction-point` line). Which laboratory relay is it? What does the certificate on the `auth-key` line contain?

Further Reading

- *Tor Rendezvous Specification - Version 3* (rend-spec) [Thee]: addresses, blinded keys, descriptors, introduction and rendezvous, including hs-ntor test vectors.
- Proposal 224: the design of v3 services.
- The shared-randomness specification (srv-spec): how authorities produce the SRV.

Chapter 15

DoS Defenses and Resource Management

Last verified: 23 September 2026 — torspec commit `8e1f245`; tor 0.4.9.13; dos-spec, hspow-spec

Chapter objectives

By the end of this chapter, you will be able to:

- explain why a denial-of-service attack against Tor is also an attack on anonymity;
- describe how tor protects relay memory, CPU capacity, and bandwidth;
- understand per-address limits on connections and circuits and the role of consensus parameters;
- explain how onion services defend against floods of introduction requests;
- describe the v1 proof-of-work mechanism (Equi-X and Blake2b) and how effort is controlled;
- trigger and observe a relay's and an onion service's defenses on the laboratory network (Laboratory 9).

An open, anonymous, volunteer-operated network is a natural target for anyone who wants to make it unusable. In this chapter we examine Tor's defenses against *denial-of-service* (DoS) attacks: attempts to exhaust a relay's or onion service's resources so that others cannot use it. Concrete incidents from recent years, including the wave of attacks against onion services in 2022–2023, are discussed in chapter 27. Here, we focus on the mechanisms.

15.1 Why DoS Is an Anonymity Problem

In an ordinary network, a DoS attack is "only" an availability problem. In Tor, it can also be a weapon against anonymity [dos-spec § Overview]:

- An adversary able to disable a relay on demand can force clients using it as a guard to select another, hoping they select an adversarial guard. Jansen, Tschorsch, Johnson, and Scheuermann demonstrated how an adversary with few resources could exhaust a relay's memory and use that capability to deanonymize onion services: the *Sniper Attack* [Jan+14].

- Making honest relays slow or unreachable pushes traffic toward adversarial relays, increasing the share of traffic they can observe.

- An attack that selectively causes circuit failures can serve as a signal, for example to determine which circuits pass through a particular relay.

Defenses therefore aim to do more than simply "keep relays running": they must avoid introducing observable behavior that an adversary could exploit.

15.2 Physical Resources

15.2.1 Memory

Memory is the most sensitive resource, and the one protected by the most mature mechanisms [dos-spec § Memory exhaustion]. On Linux, a process that requests too much memory usually does not immediately receive an error. The system promises the memory and may later terminate the process when it actually tries to use it. Waiting for allocation to fail is therefore ineffective. tor imposes its own limit, `MaxMemInQueues`, set by the operator or calculated as a fraction of system memory. It accounts for everything that consumes memory: cells queued on circuits, connection buffers, compression state, cached onion service descriptors, and cached DNS lookups.

When usage exceeds the limit, tor frees memory in two stages. First it evicts the oldest entries from caches consuming more than 20 % of the total, until those caches fall below 10 %. If usage still exceeds 90 % of the limit, it closes circuits and connections in order of the age of their oldest queued data, starting with those whose data have remained queued the longest. The goal is to target participants accumulating data without reading them, which is characteristic of this kind of attack.

15.2.2 CPU

CPU capacity is a weak point of the C implementation, which processes networking on a single main thread. Its most expensive operations, circuit handshakes (chapter 9), are delegated to a pool of worker threads and queued. If too many handshakes arrive, however, the queue grows and legitimate circuits time out. Tor does not measure CPU usage directly. It protects the CPU indirectly by limiting operations that consume it, including circuit creation and onion service introductions.

15.2.3 Bandwidth

Operators can cap bandwidth through options such as `RelayBandwidthRate`, but these limits primarily prevent them from exceeding their connectivity plans; they are not defenses in themselves. Against abnormal bandwidth consumption, Tor relies on flow and congestion control (chapter 11) and fair scheduling of cells among circuits.

15.3 Per-Address Limits

The most effective defense against unsophisticated floods is straightforward. A guard sees the connecting client's IP address and can limit how much each address may request. This is the only place in the network where such limits can be applied, because later hops deliberately do not know the traffic's origin. Tor applies three families of limits to clients (not to authenticated relays), configurable with `DoS…` options:

Connections (`DoSConnection…`) A maximum number of concurrent TLS connections and a maximum rate of new connections per address. Connections exceeding these thresholds are closed.

Circuits (`DoSCircuitCreation…`) A per-address *token bucket*: every new circuit consumes one token. Tokens refill at a fixed rate up to a maximum (the *burst*). An address that has exhausted its bucket *and* holds at least a specified number of connections is marked; its subsequent circuit requests are refused for a period.

Single-hop rendezvous (`DoSRefuseSingleHopClientRendezvous`) Refuses rendezvous requests arriving on single-hop circuits, such as those used by Tor2web gateways to access onion services without client anonymity. These offer a cheap way to generate network load.

15.3.1 The code and the consensus

In tor's code these defenses are **disabled** by default (`DOS_CC_ENABLED_DEFAULT 0` in `src/core/or/dos.h`). Network-wide settings come from consensus parameters. The public consensus of 23 September 2026 contains:

Parameter	Consensus	Code default
`DoSCircuitCreationEnabled`	1	0
`DoSCircuitCreationMinConnections`	2	3
`DoSCircuitCreationRate`	2	3
`DoSCircuitCreationBurst`	60	90
`DoSConnectionEnabled`	1	0
`DoSConnectionMaxConcurrentCount`	50	100
`DoSRefuseSingleHopClientRendezvous`	1	—

The network's values are stricter than the code defaults: they reflect years of attacks and subsequent adjustments. Being able to change them through the consensus, within hours and without updating thousands of relays, is one of the Tor Project's most valuable tools during an attack.

Warning: Limits of per-address limits

An adversary with many IP addresses, such as a botnet, can evade per-address limits. Conversely, overly strict limits affect legitimate users sharing one address, for example behind a mobile operator's or university's NAT. The values are always a compromise.

15.4 Onion Services Under Attack

Onion services are particularly vulnerable. Each introduction request is inexpensive for the client: one `INTRODUCE1` (section 14.6). For the service, the same request is costly: it must decrypt the request, perform a handshake, and build a multi-hop circuit to the specified rendezvous point. This creates **amplification**: with relatively few resources, an adversary can force both the service and all the relays on its circuits to perform considerably more work. The service cannot block attackers' IP addresses because it never sees them.

15.4.1 Limits at introduction points

One initial defense is to have introduction points rate-limit requests. With `HiddenServiceEnableIntroDoSDefense` and the related `…RatePerSec` and `…BurstPerSec` options, a service uses a `DOS_PARAMS` extension in `ESTABLISH_INTRO` to tell each introduction point how many `INTRODUCE2` messages per second it is willing to receive. This protects the service but does not distinguish an attacker from a legitimate user. Under attack, both may be rejected.

15.4.2 Proof of work

The newer defense, proposed in Proposal 327 and available since tor 0.4.8.4 (August 2023), is **proof of work** (PoW) [hspow-spec § Motivation]. Clients must demonstrate that they have performed some computation before being served; requests demonstrating greater effort receive priority. A few seconds of work may be acceptable to a legitimate user. For an attacker sending thousands of requests per second, multiplying that cost across all requests can be prohibitive.

The mechanism has four components [hspow-spec § Common protocol]:

1. **Descriptor parameters.** In the descriptor's second layer (section 14.5), the service publishes a `pow-params` line containing the scheme (`v1`), a random 32-byte *seed*, *suggested effort*, and seed expiration. The seed is renewed every 105–120 minutes.

2. **The client solves the problem.** Having chosen effort E, it searches for a solution to the Equi-X problem for the challenge

$$\texttt{"Tor hs intro v1\0"} \,\|\, A' \,\|\, \mathit{seed} \,\|\, \mathit{nonce} \,\|\, E,$$

   where A' is the service's blinded key. An Equi-X solution alone is not sufficient. The client also calculates a 32-bit Blake2b hash, R, of the challenge and solution; the solution is acceptable only if $R \cdot E \leq 2^{32} - 1$. Otherwise, it changes the nonce and tries again.

3. **Proof in the introduction message.** The client includes the nonce, effort, first 4 bytes of the seed, and solution in an extension of the encrypted portion of `INTRODUCE1`, which only the service can read.

4. **The priority queue.** The service verifies the proof—far faster than finding one—and places the request in a queue ordered by effort. It serves the highest-effort requests first; those without proof have effort zero. If the queue fills, it immediately discards the lowest-effort half.

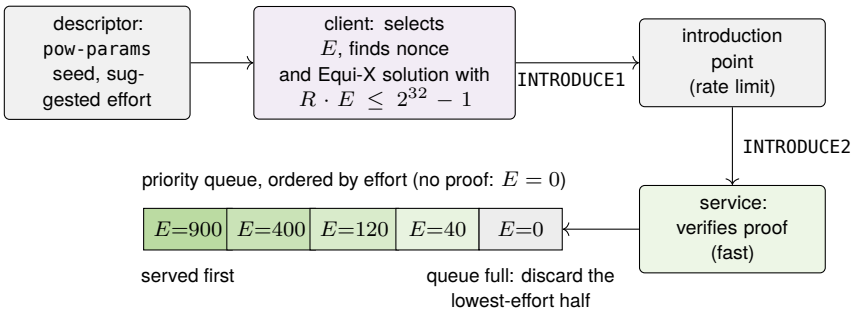


Figure 15.1: Onion service proof of work. The client reads the seed and suggested effort from the descriptor, finds a solution passing the test, and sends it in the encrypted portion of `INTRODUCE1`. The service verifies it and orders requests by effort.

In depth: Why Equi-X?

Equi-X, designed by tevador for this purpose, is an *asymmetric* problem: finding a solution takes memory and substantial computation, but checking it takes very little. It builds on HashX, a family of hash functions whose implementation changes with each seed and which exploits the parallel execution capabilities of modern CPUs. This makes dedicated-

hardware acceleration difficult, limiting an adversary's advantage over a user with an ordinary computer. There are two implementations, one in C included in tor and another in Rust written for Arti, kept compatible by automated tests. The C code is licensed under the LGPL: to include it, tor must be compiled with `--enable-gpl`. A build without that option has no PoW support: `tor --list-modules` then reports `pow: no` rather than the `pow: yes` we will encounter in chapter 17. This is not the usual case: both the laboratory image and Debian's `tor` package are compiled with the option, and the module is present in both.

15.4.3 Linear effort

Why add the Blake2b check? The cost of finding an individual Equi-X solution is roughly fixed. A random solution satisfies $R \cdot E \leq 2^{32} - 1$ with probability about $1/E$, so the client must try an average of E solutions. Effort thus increases *linearly* with E, making it straightforward to adjust. Many proof-of-work systems instead require some number of leading zero bits in a hash, doubling the cost with each additional bit: too coarse a scale for fine-grained control. Effort is also declared *before* searching for a solution and forms part of the challenge. A client cannot hope for a lucky result and then claim to have expended greater effort than it actually did.

15.4.4 Controlling effort

Suggested effort is not fixed permanently. The service begins with a suggested effort of zero, meaning the defense is dormant. Every five minutes it updates the suggestion based on queue conditions: it increases the value if the queue has remained full or if it has had to discard requests with the suggested effort, and decreases it otherwise. The control logic is inspired by TCP congestion control (additive increase, multiplicative decrease). Clients in turn raise their effort after a connection attempt times out. The result is a two-loop feedback system in which more determined users can offer greater effort to move ahead in the queue.

15.5 Laboratory: Triggering the Defenses

Laboratory 9: Anti-DoS limits and proof of work on the laboratory network

Objective. Enable a relay's circuit-creation limits and exceed them; enable onion service proof of work and read its parameters; study the Blake2b effort test using `labs/en/code/torlab/pow.py`.
Prerequisites. Laboratories 1–8; the laboratory network running with an image compiled with `--enable-gpl`.
Estimated time. 2 hours.

Warning: Only on your own network

This laboratory deliberately generates abnormal traffic. Run it exclusively on the book's private network. Doing the same against real relays or services is an attack that harms other people and is illegal in many jurisdictions (chapter 25).

Step 1: enable limits on a relay. We add very low limits to the `torrc` of relay `test003r` so that they trigger quickly, and tell it to reload its configuration with a `HUP` signal:

```
DoSCircuitCreationEnabled 1
DoSCircuitCreationMinConnections 2
DoSCircuitCreationRate 1
DoSCircuitCreationBurst 10
DoSCircuitCreationDefenseType 2
DoSCircuitCreationDefenseTimePeriod 60 seconds
```

Defense type 2 means “reject new circuits”.

Step 2: exceed the limits. We open one or two channels to the relay and request 30 circuits in rapid succession using **`CREATE_FAST`**. To obtain separate counters for each run, we originate connections from different loopback addresses (127.0.0.2, 127.0.0.3), using the new `source_address` parameter of `open_channel`:

```
source 127.0.0.2, 2 channels: 9 ok, 21 DESTROY(5); first rejection: 10
source 127.0.0.3, 1 channel: 30 ok
source 127.0.0.1, 2 channels: 9 ok, 21 DESTROY(5); first rejection: 10
```

Three observations follow:

- The relay rejects requests with a **`DESTROY`** cell carrying reason 5, `RESOURCELIMIT`.
- With a *burst* of 10, nine circuits succeed rather than ten. The code (`cc_has_exhausted_circuits` in `src/core/or/dos.c`) marks the client as soon as the bucket reaches zero, so the tenth request is already rejected.
- With only one channel, no restriction triggers: the defense requires at least `MinConnections` concurrent connections. This avoids penalizing an ordinary client, which usually keeps just one connection open to its guard.

We had predicted that connections from 127.0.0.1, the laboratory relays’ address, would be exempt. The experiment disproved that prediction, and `dos.c` confirms that no such exemption exists for circuits. At the end, we restore the original `torrc`.

Step 3: onion service proof of work. We add the following after the `HiddenService…` lines in the `torrc` of `test009h`:

```
HiddenServicePoWDefensesEnabled 1
```

After the `HUP` signal, the service publishes a new descriptor. Downloading and decrypting it with Laboratory 8 reveals the following line in the second layer:

```
pow-params v1 tx2Mtmzw5GHgWjcDoivTvWzwQPNZONwMwSMddOc+ZVQ= 0 2026-09-23T18:20:53
```

It records scheme `v1`, the base64 seed, suggested effort **zero** (the service has not yet experienced a queue), and seed expiration, one hour and fifty minutes after publication, within the specified 105–120-minute range.

Step 4: effort is linear. The `pow.py` module implements the Blake2b check and simulates Equi-X solutions with random bytes, which have the same distribution for the purposes of this test:

```
def check_value(service_id, seed, nonce, effort, solution):
    challenge = P_STRING + service_id + seed + nonce + struct.pack(">I", effort)
    h = hashlib.blake2b(challenge + solution, digest_size=4).digest()
    return int.from_bytes(h, "big")
```

```
5
6 def valid_effort(r, effort):
7     return r * effort <= UINT32_MAX
```

Measuring the average number of solutions required over 400 trials for each effort value gives:

```
effort     1: 1.0 attempts
effort    10: 9.7
effort   100: 102.3
effort  1000: 1068.8
```

The number of attempts grows in proportion to effort, as predicted. The expected value equals the effort because each attempt succeeds with probability $1/effort$ and the number of attempts follows a geometric distribution. Its standard deviation is as large as its mean. Thus the average over 400 trials has a typical relative deviation of $1/\sqrt{400} = 5\,\%$. Two runs may differ by ten percent even when nothing has changed; on repeating the measurement, we indeed obtained 969.9 where the listing shows 1068.8. This is a useful reminder that a sample mean must be considered alongside its uncertainty. Notice another detail: `blake2b` with `digest_size=4` does not equal the first four bytes of a 64-byte Blake2b result because output length affects the function's initial state. The specification emphasizes this subtlety, which is easy to overlook.

Summary

- In Tor, DoS attacks are also attacks on anonymity: disabling relays on demand or causing them to fail can redirect traffic toward an adversary.
- Memory is protected by a self-imposed limit, `MaxMemInQueues`, beyond which tor evicts caches and closes circuits accumulating data.
- Guards impose per-address connection and circuit limits on clients using token buckets. Disabled in the code defaults, these defenses are enabled and adjusted through the consensus.
- Onion services face amplification: initiating a request is cheap, responding to it is expensive.
- V1 proof of work combines Equi-X with a Blake2b test that makes effort linear. The service prioritizes higher-effort requests and adjusts suggested effort according to queue conditions.

Exercises

Exercise 15.1 ⋆ With `DoSCircuitCreationRate` 2 and `Burst` 60, as in the public consensus, how many circuits can a client create in one minute starting with a full bucket? And in one hour? Assume the defense has already been triggered, meaning the address has reached `DoSCircuitCreationMinConnections` concurrent connections. Below that threshold there is no limit.

Exercise 15.2 ⋆ Why do per-address limits apply only to clients and only at the first hop? What would happen if a middle tried to limit circuits "per address"?

Exercise 15.3 ⋆⋆ Estimate the ratio between the client's work and the network's work for a single introduction request: how many circuits, handshakes, and cells does the service's response generate?

Exercise 15.4 ⋆⋆ Compare Tor's proof of work with a scheme requiring k leading zero bits in a hash. Which effort values can each scheme express? Why is a linear scale preferable for feedback control?

Exercise 15.5 ⋆⋆ A client declares effort $E = 50$, but by luck it immediately finds a solution with very small R that would also satisfy $E = 5000$. Can it submit the solution while claiming $E = 5000$? Why?

Exercise 15.6 ⋆⋆⋆ On the laboratory network, set `MaxMemInQueues` to its minimum value on a middle relay and build a circuit through it, downloading a large file *without reading* incoming data (and without sending SENDME). What happens to accumulated cells? Look for messages from the memory manager in that relay's logs.

Exercise 15.7 ⋆⋆⋆ Write a small simulator of an onion service's priority queue. Requests arrive with random effort (many at zero, a few at higher values); the service handles a certain number per second and discards the lowest-effort half whenever the queue exceeds a threshold. Measure average waiting time as a function of effort.

Further Reading

- *Tor denial-of-service mitigations* (dos-spec) and *Onion service proof-of-work* (hspow-spec) [Thee].
- R. Jansen, F. Tschorsch, A. Johnson, B. Scheuermann, *The Sniper Attack: Anonymously Deanonymizing and Disabling the Tor Network* [Jan+14].
- tevador's Equi-X repository and development journal, cited in the specification.

Chapter 16

Censorship, Bridges, and Pluggable Transports

Last verified: 23 September 2026 — tor 0.4.9.13; pt-spec v1; obfs4proxy 0.0.14 (Debian); Tor Project documentation

Chapter objectives

By the end of this chapter, you will be able to:

- describe the ways a censor can block Tor;
- explain what bridges are and why distributing them is difficult;
- describe the protocol through which tor communicates with a pluggable transport;
- distinguish the strategies used by obfs4, meek, Snowflake, WebTunnel, and Conjure;
- operate an obfs4 bridge on the laboratory network and observe what a censor can see (Laboratory 10).

Everything we have covered so far assumes that the client can reach the network. In many countries it cannot: connections to Tor are blocked, sometimes only during periods of political tension and sometimes permanently. This chapter concludes Part II with **censorship circumvention**, one of the Tor Project's most active areas in recent years.

16.1 How Tor Is Blocked

A censor, usually a government controlling a country's connectivity providers, has several tools:

Address blocking The relay list is public: it is the consensus (chapter 12). A censor can simply download it and block every relay IP address, or just the directory authorities and relays from which clients obtain consensuses.

Website blocking Blocking `torproject.org` prevents people from downloading Tor Browser, even if the Tor network itself remains reachable.

Protocol recognition *Deep packet inspection* (DPI) may recognize a Tor connection from properties of its TLS handshake, packet sizes, or timing, even when the destination address is not yet known.

Active probing When DPI suggests that a server may be an undisclosed relay, the censor can connect to it and try speaking the Tor protocol. If the server responds, it can be blocked.

Allowlisting In extreme cases, a censor permits only explicitly authorized protocols or destinations and blocks everything else.

Each technique has a corresponding countermeasure, and the contest between censors and developers continues. Projects such as OONI measure where and how Tor is blocked (chapter 24).

16.2 Bridges

The response to address blocking is the **bridge**: a relay absent from the public consensus. A bridge acts as a guard (section 13.3). The client uses it as the first hop, and the circuit then continues normally through the network. Because the bridge address is not public, a censor cannot block it merely by reading a list. Bridges publish descriptors to a **bridge authority**, which checks their reachability and maintains a list without making it public. Published bridge statistics use a digest *of a digest* of the identity to avoid exposing it.

A client configures a bridge using a *bridge line*. On the laboratory network, client `test011bc` has the following in its `torrc`:

```
UseBridges 1
Bridge obfs4 127.0.0.1:9909 2557828B596B33AB02075FE7EAD86F2DC3CEAFD5 cert=4sKDQih...
    iat-mode=0
ClientTransportPlugin obfs4 exec /usr/bin/obfs4proxy
```

The line specifies the transport type, address, bridge identity fingerprint (used to verify the peer, chapter 7), and transport parameters. The value `cert=4sKDQih...` is *abbreviated* for print and must not be copied as a valid configuration. Readers must use the complete certificate and fingerprint generated by their own bridge instance.

16.2.1 The distribution problem

A bridge is useful for as long as the censor does not know its address. But to be useful, its address must be given to users, and a censor can pretend to be a user. There is no perfect solution: a distributor must give enough bridges to many legitimate users while revealing as few as possible to the censor. The Tor Project distributes bridges through several channels, each with its own defenses against automated harvesting [Theg]:

- the website `https://bridges.torproject.org/`;
- email, by writing to `bridges@torproject.org`;
- a Telegram bot (`@GetBridgesBot`);
- directly through Tor Browser, which detects blocking and requests suitable bridges for the user's country (*Connection Assist*, based on the channel called *moat*);
- private channels run by organizations that know their users personally.

For years, the system assigning bridges to distribution channels was BridgeDB. It was replaced by **rdsys**, with migration completed in October 2024 [The24b]. Bridge operators can select a channel with the `BridgeDistribution` option.

16.2.2 Lox

Lox [TG23] approaches distribution with a *reputation* system. A new user receives a bridge and an anonymous credential. If that bridge remains unblocked over time, the credential gains levels, up to a maximum of 4; starting at level 2, it also permits the user to invite others. If the bridge is blocked, the user can obtain another but loses reputation. The premise is that a censor that blocks every bridge it receives cannot gain levels, whereas honest users can. The credentials employ cryptography that prevents even the system itself from learning who invited whom, protecting users' *social graph*. Lox is being tested in alpha versions of Tor Browser.

16.3 Pluggable Transports

Hiding the address is not enough to defeat protocol recognition: the traffic itself must look different. Rather than modifying tor whenever a censor learns to recognize a new feature, the Tor Project has defined an interface for external programs that transform traffic: **pluggable transports** (PTs) [pt-spec § Architecture Overview].

16.3.1 The architecture

A PT is a separate process launched by tor (hence *managed*). On the client side, it offers tor a local SOCKS proxy. tor hands the connection to the bridge to the PT, which transforms the traffic before sending it. On the bridge side, a PT listens on the public port, reverses the transformation, and forwards traffic to tor's local ORPort. It may use a small additional protocol, the *Extended ORPort*, to tell tor the client's actual address for statistics.

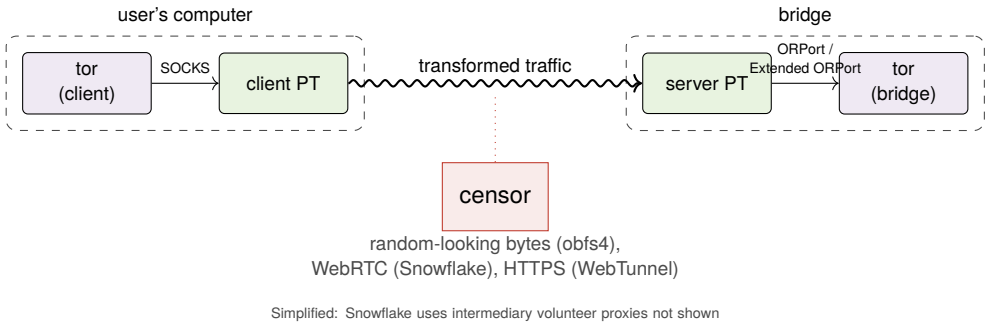


Figure 16.1: Schematic pluggable transport architecture. On the client, tor communicates with the PT via a local SOCKS proxy; at the bridge, the PT forwards restored traffic to tor via ORPort or, when configured, Extended ORPort. Snowflake also involves volunteer intermediary proxies, omitted here. A censor sees transformed traffic on the monitored link, but it may still be detectable.

16.3.2 The protocol between tor and the PT

tor configures the PT through environment variables, and the PT responds by writing lines to its standard output [pt-spec § Specification]. We can test this by manually starting `obfs4proxy` in the laboratory container, just as tor would. As a client:

```
$ env TOR_PT_MANAGED_TRANSPORT_VER=1 TOR_PT_STATE_LOCATION=/tmp/pt \
      TOR_PT_EXIT_ON_STDIN_CLOSE=1 TOR_PT_CLIENT_TRANSPORTS=obfs4 obfs4proxy
VERSION 1
```

```
CMETHOD obfs4 socks5 127.0.0.1:40057
CMETHODS DONE
```

The PT declares the protocol version and announces that it provides the `obfs4` transport as a SOCKS5 proxy on a local port of its own choosing. As a server, with variables specifying the public listening address and the ORPort destination:

```
$ env ... TOR_PT_SERVER_TRANSPORTS=obfs4 \
      TOR_PT_SERVER_BINDADDR=obfs4-127.0.0.1:19999 \
      TOR_PT_ORPORT=127.0.0.1:5104 obfs4proxy
VERSION 1
SMETHOD obfs4 127.0.0.1:19999 ARGS:cert=5
    W6vcbWbFGGZgabwm4C3WipezwnAfadoBhqfNVpY72xnkjB/Szczbxg8fLWJJ1uRlyjPSg,iat-mode=0
SMETHODS DONE
```

The parameters following `ARGS:` are what the client will need; they appear in the bridge line. The PT also writes them to `obfs4_bridgeline.txt` in its state directory. The `TOR_PT_EXIT_ON_STDIN_CLOSE` variable tells the PT to terminate when tor closes its input, so the PT cannot outlive the process that launched it. (Through an oversight, the specification's example says the PT writes its messages to "stdin"; in practice, as we have just seen, it writes them to standard output.)

The server command's `env ...` is *schematic*: the ellipsis stands for other required environment variables and must not be typed literally. Likewise, the SOCKS5 port in the client output comes from a reference session and is not fixed.

16.3.3 Lyrebird

The transport program used by Tor Browser is **lyrebird**, derived from the earlier `obfs4proxy`. It implements obfs4, meek, Snowflake, and WebTunnel [Theg]. The laboratory uses `obfs4proxy` 0.0.14, available in Debian packages and still compatible with obfs4.

16.4 Strategies

Transports currently in use follow two opposing strategies: *look like nothing recognizable* or *look like something permitted*.

16.4.1 obfs4: looking like nothing recognizable

obfs4, designed by Yawning Angel, transforms traffic into a sequence of bytes indistinguishable from random data: no recognizable header, no TLS handshake, and variable packet sizes. Three design choices make it effective:

- Its key exchange is a variant of ntor (section 9.6) in which Curve25519 public keys are encoded with Elligator2, a technique that makes a curve point indistinguishable from a random string.
- The client must know the bridge's `cert` parameter, which contains its identity and public key. Without that parameter, an observer cannot construct a valid handshake and the bridge does not respond. This defeats **active probing**: a curious censor connecting to the bridge receives no recognizable answer.
- The `iat-mode` (*inter-arrival time*) parameter can, at a performance cost, add randomized delays and fragmentation to obscure timing as well.

The limitation of obfs4 is also its strength: a censor employing an allowlist, or blocking "anything that looks random", can stop it. This happens in some countries.

16.4.2 meek: behind a large service

meek [Fif+15] uses *domain fronting*. The client opens an HTTPS connection to a large content-distribution service, using an innocuous domain name in the visible part of the connection, and specifies the true destination (a server acting as a gateway to Tor) only within encrypted data. Blocking meek would require the censor to block the entire service, with extensive collateral damage. However, this technique depends on providers permitting it, and many have made it impossible in recent years. meek is also slow and expensive because all traffic passes through the provider's servers.

16.4.3 Snowflake: many ephemeral proxies

Snowflake [Boc+24] uses thousands of volunteers running small proxies, for example a browser extension or a program on a server. The client contacts a *broker* through a hard-to-block channel such as domain fronting to obtain a proxy, then connects to it using WebRTC, the protocol used for video calls in browsers. The proxy forwards traffic to a Snowflake bridge run by the Tor Project. There are many proxies, and they change constantly. Blocking all their addresses is impractical, while blocking WebRTC means blocking video calls.

16.4.4 WebTunnel: looking like a website

WebTunnel, announced by the Tor Project in March 2024, wraps traffic in an HTTPS connection with behavior similar to WebSocket [The24a]. The bridge can sit behind an ordinary website, sharing its domain, address, and port. A reverse proxy forwards only requests for a secret path to the bridge and serves the normal website to everyone else. WebTunnel targets censors using allowlists, against which obfs4 does not work: it has the "shape" of a permitted protocol.

16.4.5 Conjure: hiding in the address space

Conjure [Fro+19] is a *refraction networking* system. Instead of a bridge at a known address, it uses stations installed at cooperating connectivity providers. These stations observe traffic destined for ranges of unused addresses and divert it toward Tor. The client connects to a "phantom" address chosen from millions of possibilities. Blocking Conjure would require the censor to block entire portions of the Internet. According to Tor Project documentation, Conjure has been tested successfully but still needs stabilization work and is available only in alpha versions of Tor Browser and some applications [Theg].

Warning: No transport works everywhere

Every transport has weaknesses, and effectiveness changes by country and from month to month. In the Tor Project forum, for example, users reported in April 2025 that obfs4, meek, and Snowflake were unusable in China and that WebTunnel connected but was quickly blocked [Tor25]. Tor Browser therefore offers several transports and a feature that suggests one suited to the user's country.

16.5 Laboratory: An obfs4 Bridge Through a Censor's Eyes

Laboratory 10: An obfs4 bridge on the laboratory network

Objective. Start a network containing an obfs4 bridge and a client using it; capture and compare traffic from an ordinary Tor connection and an obfs4 connection.
Prerequisites. The laboratory network (chapter 19). The image includes `obfs4proxy` and `snowflake-client` from Debian packages.
Estimated time. 1–2 hours.

Step 1: the bridge network. The `labs/en/lab/chutney/networks/book-bridge` file adds to the book's network a bridge authority (`test003ba`), an obfs4 bridge (`test009br`), and a client using it (`test011bc`):

```
$ make -C labs/en/lab clean
$ make -C labs/en/lab start NETWORK=book-bridge
```

The client's log from an earlier reference session includes:

```
[notice] new bridge descriptor 'test009br' (fresh): $2557828B...~test009br
[warn] Only one bridge (transport: 'obfs4') is configured. You should
       have at least two for conflux, for any transport that is not 'snowflake'.
[notice] Bootstrapped 100% (done): Done
```

The client obtained the bridge descriptor through the bridge itself and finished bootstrapping. The warning reminds us that Conflux (chapter 31) uses two parallel circuits and cannot do so with just one bridge. A web request through this client's SOCKS port (9011) works as it does with the ordinary client.

Step 2: what the censor sees. Inside the container, we capture traffic with `tcpdump` on the bridge's obfs4 port (9909) and a normal ORPort (5104), while the client uses the bridge and we open a channel to `test004r` with Laboratory 1. We then analyze the capture with `tshark`. Here is the result from one of our sessions:

```
== ORPort 5104 (TLS), first bytes sent by the client ==
1533  16030105f8010005f403033e1234a0401520d1c52b...
== obfs4 9909, first bytes sent by the client ==
1957  b566475349099523c53005e1004a392160f15d473e...
== packets recognized as TLS ==
port 5104: 83
port 9909: 0
== obfs4 segment sizes (client -> bridge) ==
1957 1677 1888 1891 1716 1716 3436 1395 1716 1748 1500 1194 ...
```

On the ORPort, the first bytes are unmistakable: `16 03 01` starts a TLS handshake record, and Wireshark recognizes the entire conversation as TLS. The censor sees a TLS connection to inspect for Tor-specific features; the relay's address is also listed in the consensus. Toward the obfs4 bridge, the bytes have no recognizable structure, no packet is identified as TLS, and segment sizes do not mirror 514-byte cells. The censor sees only a stream of apparently random data toward an unknown address.

Step 3: probing the bridge. We try connecting to the bridge's port 9909 using Laboratory 1, as a censor suspecting a relay might:

```
TimeoutError: The handshake operation timed out
```

The bridge does not respond *at all*: no error message and no bytes, until our client gives up after 15 seconds. To obfs4, our TLS ClientHello is an invalid handshake, and the response to an invalid handshake is silence. A censor probing the address learns nothing, not even that a service speaking some other protocol is listening there. Without the `cert` parameter, it cannot construct a valid obfs4 handshake anyway.

Summary

- A censor can block Tor by address, recognize its protocol, probe suspect servers, or permit only explicitly authorized traffic.
- Bridges are unpublished relays. Their distribution, currently handled by rdsys and being explored with Lox's reputation mechanism, balances access for users against discovery by censors.
- Pluggable transports are external processes that transform traffic; tor configures them through environment variables and reads their responses (`CMETHOD`, `SMETHOD`).
- obfs4 makes traffic look random and resists active probing; meek, Snowflake, WebTunnel, and Conjure conceal it behind services, protocols, or addresses that are costly for a censor to block.
- No transport works everywhere or forever.

Exercises

Exercise 16.1 ⋆ How does a bridge differ from a guard? Why does a client using bridges not also use a public guard?

Exercise 16.2 ⋆ Classify obfs4, meek, Snowflake, WebTunnel, and Conjure by strategy ("look like nothing recognizable" or "look like something permitted") and by what a censor would have to block to stop each one.

Exercise 16.3 ⋆⋆ Describe how a censor might discover an obfs4 bridge by observing only traffic, without actively probing it. Which statistical features could it use, and what does `iat-mode` do to counter them?

Exercise 16.4 ⋆⋆ In Lox, why can a censor not advance through reputation levels? What could it do to evade the mechanism, and what makes that costly?

Exercise 16.5 ⋆⋆ Calculate the Shannon entropy (in bits per byte) of the first 10 KB captured from an obfs4 connection and an ordinary TLS connection in the laboratory. What do you expect, and why might the difference be small? What, then, makes TLS recognizable?

Exercise 16.6 ⋆⋆⋆ Try running Laboratory 1 against the obfs4 port of your bridge. What error occurs? Then use `obfs4proxy` as a client with an incorrect `cert` parameter: what do you observe in the bridge logs?

Further Reading

- *Pluggable Transport Specification (Version 1)* (pt-spec) [Thee].
- L. Tulloch and I. Goldberg, *Lox: Protecting the Social Graph in Bridge Distribution* [TG23].

- D. Fifield et al., *Blocking-resistant communication through domain fronting* [Fif+15].
- C. Bocovich et al., *Snowflake, a censorship circumvention system using temporary WebRTC proxies* [Boc+24].
- S. Frolov et al., *Conjure: Summoning Proxies from Unused Address Space* [Fro+19].
- The Tor Project blog post announcing WebTunnel [The24a] and the bridge support pages [Theg].

Part III

Tor in Practice

Chapter 17

Installation and torrc

Last verified: 23 September 2026 — tor 0.4.9.13; the `tor(1)` manual for the same version

Chapter objectives

By the end of this chapter, you will be able to:

- choose the appropriate way to install Tor for your purpose;
- read and write a `torrc` file and know where to find documentation for every option;
- configure tor as a client, with SOCKS ports, stream isolation, and a protected control port;
- validate a configuration before using it and diagnose problems from logs.

This chapter begins the practical part of the book. Previous chapters explained how the protocol works; now we learn to use and configure the program that implements it. Every option mentioned here is documented in the tor manual (`man tor`), which is the authoritative reference: defaults can change between versions.

17.1 Which Tor Should You Install?

Tor Browser For web browsing, this is almost always the appropriate choice. It includes a preconfigured tor and a browser modified to avoid disclosing information that would undermine the network's protections (chapter 23). Using tor with an ordinary browser, by contrast, leaves users open to many forms of identification.

The `tor` package For using Tor with other applications or operating a relay or onion service. Linux distributions include it, although often not the latest version. The Tor Project maintains its own package repository for Debian and Ubuntu and distributes the *Tor Expert Bundle*, containing tor and pluggable transports, for Windows and macOS.

From source As in the book's laboratory (section 19.3), for studying tor or obtaining an exact version.

Arti The Rust client (chapter 30), increasingly mature but configured differently.

Whichever method you choose, download the software only from the official site and verify its signature. A version of tor modified by an adversary would nullify its protections.

17.2 The torrc File

tor reads its configuration from a text file, usually `/etc/tor/torrc` when it runs as a system service. The format is simple: one option per line, followed by its values. Lines beginning with # are comments. Some options may appear more than once, such as multiple SOCKS ports. Options can also be supplied on the command line with two leading hyphens:

```
$ tor --SocksPort 9150 --DataDirectory /tmp/tor-test
```

The manual documents hundreds of options; `tor --list-torrc-options` prints them all. Distributions may specify a second file of defaults using `--defaults-torrc`. The file specified with `-f` takes precedence.

17.2.1 Check Before Use

Three commands can prevent many surprises:

```
$ tor --verify-config -f /etc/tor/torrc     # is the configuration valid?
Configuration was valid
$ tor --dump-config short -f /etc/tor/torrc # what differs from the defaults?
$ tor --list-modules                        # which modules are compiled?
relay: yes
dirauth: yes
dircache: yes
pow: yes
```

As we saw in chapter 15, the last command reveals, for instance, whether onion service proof of work is available.

17.3 A Client Configuration

A client needs very little configuration. Indeed, the minimal configuration file is empty: by default, tor opens a SOCKS port at 9050 on localhost. Here is a more explicit, annotated example for a system tor service whose account can access `/var/lib/tor` and `/var/log/tor` (not a generic unprivileged invocation):

```
# Where tor stores state, keys, and cache
DataDirectory /var/lib/tor

# SOCKS port for applications (localhost only!)
SocksPort 127.0.0.1:9050

# Control port, protected by the cookie (see Chapter 18)
ControlPort 127.0.0.1:9051
CookieAuthentication 1

# Log at notice level
Log notice file /var/log/tor/notice.log
```

Warning: Do Not Expose the SOCKS Port

The manual is explicit: the SOCKS protocol is neither encrypted nor authenticated. A `SocksPort` listening on a network-accessible address turns the computer into an open proxy and exposes applications' requests in plaintext on the local network.

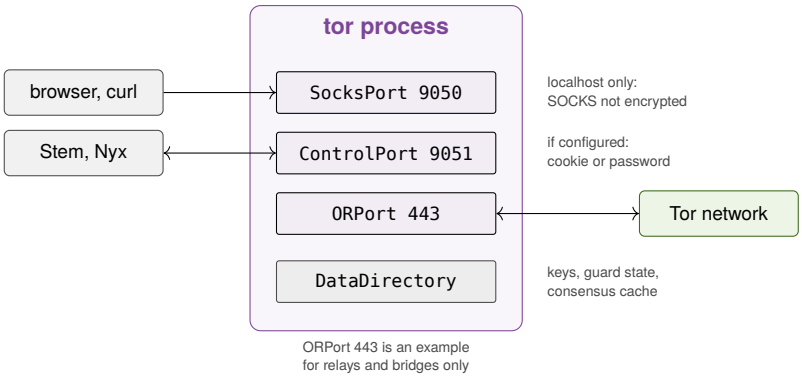


Figure 17.1: Example ports and directory of a tor process. SOCKS on 9050 serves local applications; ControlPort 9051 is optional and must be authenticated. Neither listener should be exposed to the network. DataDirectory is a filesystem directory, not a port. ORPort 443 is one possible choice for a relay or bridge, not a required port number.

17.3.1 Stream Isolation

Multiple streams can share the same circuit. This is efficient, but it allows an exit to link different activities performed by the same user. If personal email and a pseudonymous forum share a circuit, the exit can tell that they come from the same person. tor therefore provides **stream isolation**: rules determining which streams may share a circuit. The rules are expressed as `SocksPort` options:

`IsolateClientAddr` Streams from different client addresses cannot share circuits (enabled by default).

`IsolateSOCKSAuth` Streams with different SOCKS credentials cannot share circuits (enabled by default).

`IsolateDestAddr`, `IsolateDestPort` Isolation by destination.

`IsolateClientProtocol` Isolation by protocol (SOCKS4, SOCKS5, DNS...).

Different SOCKS ports are also isolated from each other by default: each listener is assigned a separate *session group*. The `SessionGroup` option can override this behavior and permit ports to share circuits. The most widely used mechanism is SOCKS authentication isolation: Tor Browser sends different SOCKS credentials for each visited website, obtaining a separate circuit for each site. We will observe this in chapter 18.

A circuit is not used forever. Once `MaxCircuitDirtiness` has elapsed—10 minutes by default, counted from first use—the circuit accepts no new streams, and tor builds another.

17.3.2 Choosing Relays: Exercise Caution

The `EntryNodes`, `ExitNodes`, and `ExcludeNodes` options can force or exclude relays by fingerprint, nickname, address, or country code. `StrictNodes` makes `ExcludeNodes` mandatory even when doing so breaks functionality. These tools are useful for experiments and specific circumstances. Using them merely to "feel safer", however, usually makes matters worse: narrowing relay selection reduces path diversity, makes the user stand out, and defeats the protections provided by weighted path selection and guards (chapter 13).

17.3.3 Other Ports

`DNSPort` tor answers DNS requests, resolving names through the network (section 11.1.4).

`TransPort` Accepts *transparent* TCP connections redirected by the system firewall. Systems such as Whonix use this mechanism to route application traffic through Tor (chapter 23).

`AutomapHostsOnResolve` Assigns virtual addresses to `.onion` names requested through DNS, allowing applications that do not support SOCKS to reach onion services through `TransPort`.

17.4 The Control Port

`ControlPort` allows other programs to control tor while it is running: read its status, request new circuits, or create onion services dynamically. It is extremely powerful and must therefore be protected. Two authentication methods are available:

Cookie With `CookieAuthentication 1`, tor writes a file containing 32 random bytes in its data directory. Anyone who can read that file can authenticate. Security therefore depends on file permissions.

Password With `HashedControlPassword`, tor stores a password hash generated by `tor --hash-password`:

```
$ tor --hash-password test
16:458528FE0C0AF36260B5071446823FB789212185D7EE7F615BC120D55D
```

The hash shown is an illustrative, verified hash for the sample password `test`, using a fixed salt for the printed example. A real invocation chooses a random salt, so its output normally differs even for the same password. Never reuse the sample password or hash as a credential.

Like the SOCKS port, the control port must never be exposed to the network. The control protocol is the subject of chapter 18.

17.5 Logs

tor records its activity at several levels of detail: `err`, `warn`, `notice`, `info`, and `debug`. The `notice` level is enough for everyday operation. `info` and `debug` are valuable when investigating the protocol, as we did in the laboratories, but produce voluminous output. The `SafeLogging` option, enabled by default, replaces addresses and destinations in logs with `[scrubbed]`. A forgotten log file must not become a record of the user's activities. The laboratory network deliberately disables this protection (`SafeLogging 0`) so that we can inspect all the details.

The most useful messages during startup are the *bootstrap* messages. They show the stages of connecting to the network, ending with `Bootstrapped 100% (done): Done`. If startup stalls at a particular percentage, the stage suggests where to look. For example, a stall while connecting to the first relay usually indicates censorship or a firewall and suggests using bridges (chapter 16).

Summary

- Use Tor Browser for web browsing; the `tor` package serves other applications, relays, and onion services.

- A `torrc` contains one option per line. The manual is the authoritative reference, and `--verify-config` validates the configuration.
- Keep SOCKS and control ports on localhost. Protect the control port with a cookie or password.
- Stream isolation prevents unrelated activities from sharing a circuit. Tor Browser achieves this with different SOCKS credentials for each site.
- Forcing or excluding relays usually reduces security; `SafeLogging` protects logs.

Exercises

Exercise 17.1 ⋆ In the laboratory container, create a `torrc` with a typo in an option name and check it using `--verify-config`. What message do you get? Then run `--dump-config short` on the laboratory network client's `torrc`.

Exercise 17.2 ⋆⋆ A user connects a personal email client under their real identity and a pseudonymous forum through the same `SocksPort`. What could an exit infer? Suggest two configurations that avoid this problem.

Exercise 17.3 ⋆⋆ Why would `ExitNodes {it}`, meaning Italian exits only, reduce the anonymity of an Italian user visiting an Italian website? Reason in terms of the threat model in chapter 4.

Further Reading

- The tor manual, `man tor`, and `doc/man/tor.1.txt` in the source tree.
- Tor Project support pages on installation and package repositories, `https://support.torproject.org/`.

Chapter 18

Control Port and Stem

Last verified: 23 September 2026 — tor 0.4.9.13; control-spec; Stem 1.8.2

Chapter objectives

By the end of this chapter, you will be able to:

- communicate with tor through the control port, including manually;
- use the main commands: `GETINFO`, `SETCONF`, `SIGNAL`, `ADD_ONION`, and event subscriptions;
- write Python programs using the Stem library;
- observe stream isolation and the effect of a new identity on a running system (Laboratory 11).

Tor Browser displays the circuit used by the current page; tools such as Nyx show a relay's bandwidth and connections; network measurement systems build circuits on demand. All these applications use the same interface: tor's **control protocol**, accessible through `ControlPort` (section 17.4).

18.1 The Control Protocol

The protocol is text-based, similar to email protocols [control-spec § Message format]. The client sends one command per line, and tor replies with lines beginning with a three-digit code. Code `250` means success. A hyphen after the code (`250-`) indicates that the response continues, while a space (`250 `) indicates its last line; a `+` introduces a multiline block terminated by a line containing only a period. Codes `5xx` indicate errors, while `650` marks asynchronous events.

We can try it on the laboratory network's client through a simple TCP connection to port 8008:

```
> PROTOCOLINFO 1
250-PROTOCOLINFO 1
250-AUTH METHODS=COOKIE,SAFECOOKIE COOKIEFILE="/lab/net/.../008c/control_auth_cookie"
250-VERSION Tor="0.4.9.13"
250 OK
> AUTHENTICATE 52977550adebb73908610d7fa3355aba3fbdbdaef01d5ef86523d3232b87e5fa
250 OK
> GETINFO status/bootstrap-phase
250-status/bootstrap-phase=NOTICE BOOTSTRAP PROGRESS=100 TAG=done SUMMARY="Done"
250 OK
```

This is a transcript from a previous illustrative run: paths, versions, and authentication cookies depend on the instance. **Never publish a real control cookie**: possession of it grants control-port authentication. `PROTOCOLINFO` is among the few commands allowed before authentication (the others are `AUTHCHALLENGE`, `AUTHENTICATE`, and `QUIT`). It reports supported authentication methods and the path to the cookie file. With `COOKIE` authentication, the client sends the file's contents as hexadecimal. `SAFECOOKIE` avoids transmitting the cookie itself: the client and tor exchange random values and prove knowledge of the cookie through an HMAC, preventing a program impersonating tor from stealing it.

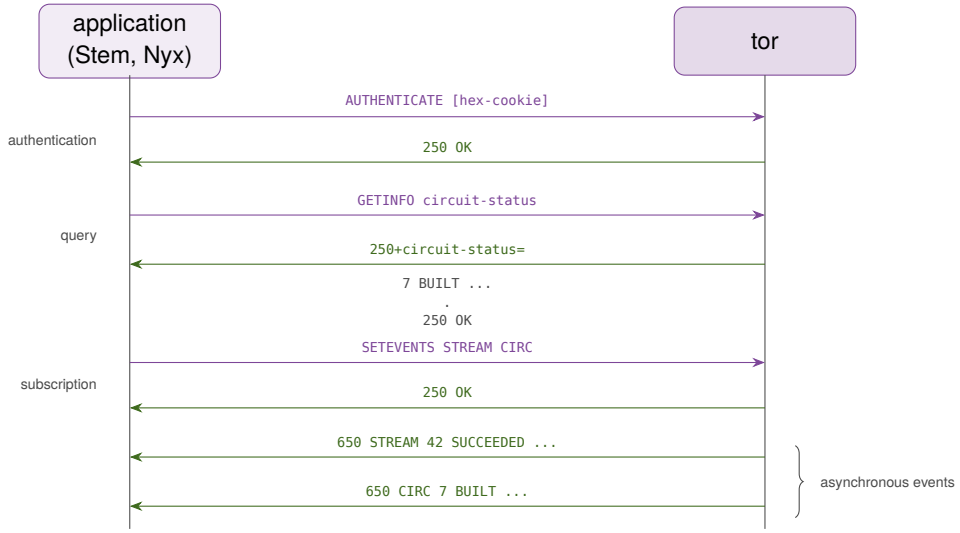


Figure 18.1: Simplified control-protocol session: after COOKIE authentication (shown with a placeholder), GETINFO circuit-status can return multiline data terminated by a separate dot and 250 OK. SETEVENTS is acknowledged by 250 OK; asynchronous 650 events may interleave with other replies. IDs and data are illustrative.

18.1.1 Circuits as Seen by tor

`GETINFO circuit-status` lists the client's circuits. An abbreviated entry:

```
3 BUILT $50160F8F...~test004r,$12415FEB...~test005r,$9449F36E...~test007e
  BUILD_FLAGS=NEED_CAPACITY,NEED_UPTIME PURPOSE=CONFLUX_LINKED
  CONFLUX_ID=DE224696F58F72950D70DDA2D7981726 CONFLUX_RTT=42538
```

Each relay is represented by its RSA identity fingerprint and nickname. Conflux circuits appear in pairs with the same `CONFLUX_ID`, and the two "legs" start from different guards. tor measures their round-trip times (`CONFLUX_RTT`, in microseconds) to decide which leg should carry data (chapter 31). One detail in tor's code (`src/feature/control/control_fmt.c`) deserves attention: `CONFLUX_ID` exposes only *half* of the pair's secret identifier. Someone who knew the entire identifier could add legs to the pair from an exit and inject data.

18.1.2 The Main Commands

Command	Purpose
`GETINFO`	reads information: version, bootstrap status, circuits, streams, consensus, descriptors…
`GETCONF`, `SETCONF`	reads and changes the running configuration
`SIGNAL`	sends signals: `NEWNYM` (new identity), `RELOAD`, `SHUTDOWN`…
`SETEVENTS`	subscribes to events: `CIRC`, `STREAM`, `BW`, `HS_DESC`…
`EXTENDCIRCUIT`	builds a circuit along a path chosen by the controller
`ATTACHSTREAM`	assigns a stream to a circuit (with `__LeaveStreamsUnattached`)
`ADD_ONION`, `DEL_ONION`	creates and removes onion services without editing `torrc`
`HSFETCH`	downloads an onion service descriptor (used in section 14.9)

Warning: Control the Port, Control tor

The control port can instruct tor which relays to use, make it build single-hop circuits, and reveal every stream destination. A malicious program with access to the control port can defeat the user's anonymity. Systems such as Tails and Whonix therefore put a filter (onion-grater) between applications and the control port, permitting only strictly necessary commands.

18.2 Stem

Stem is the Tor Project's Python library for the control protocol. It hides the details of the wire format, parses responses and events, and provides convenient objects. Its most recent version, 1.8.2, dates from June 2023. The library works with current tor versions and is used by many tools, but it has not had a new release for some time. Newer functionality sometimes requires raw commands (`Controller.msg`). Arti has a different interface, Arti RPC (chapter 30).

Here is a minimal example, which we have already used in section 19.7.3:

```
from stem.control import Controller

with Controller.from_port(port=8008) as c:
    c.authenticate()                  # locate the cookie or prompt for a password
    print(c.get_version())
    for circ in c.get_circuits():
        print(circ.id, circ.purpose, [name for _, name in circ.path])
```

Register a function to receive events. The following is a *self-contained program*: it registers its listener while the controller remains open, then waits ten seconds so callbacks can arrive. The duration is only illustrative.

```
import time
from stem.control import Controller, EventType

def on_circuit(event):
    print(event.id, event.status, event.purpose)

with Controller.from_port(port=8008) as c:
    c.authenticate()
    c.add_event_listener(on_circuit, EventType.CIRC)
    time.sleep(10)
```

Stem invokes this function from its event thread. There is only one such thread, and it calls all registered listeners sequentially. A slow callback therefore delays subsequent events. If processing is computationally expensive, delegate it to a separate thread.

18.3 Laboratory: Live Stream Isolation

Laboratory 11: Stream isolation and new identity

Objective. Use Stem on the laboratory network's client to check which requests share a circuit depending on their SOCKS credentials, and what changes after `SIGNAL NEWNYM`.
Prerequisites. The laboratory network running and a local web server on port 8080. Run the `labs/en/lab/isolation.py` script inside the container, which includes Stem and `curl`.
Estimated time. 1 hour.

Step 1: record streams. We subscribe to `STREAM` events and, whenever a stream succeeds, record which circuit it was attached to. Filtering by destination is essential: tor also opens streams for its own purposes, such as downloading directory documents.

```
assigned = []
def on_stream(ev):
    if ev.status == StreamStatus.SUCCEEDED and ev.target == DESTINATION:
        assigned.append((ev.id, ev.circ_id, ev.target))

c.add_event_listener(on_stream, EventType.STREAM)
```

This snippet is taken from `labs/en/lab/isolation.py`: the full script also defines `DESTINATION`, imports `StreamStatus`, and keeps controller `c` open. Run the full script rather than this excerpt in isolation.

Step 2: requests with different credentials. We issue four requests with `curl`: two with SOCKS credentials `site-a.example:x1`, one with `site-b.example:x2`, and one without credentials. Tor Browser uses the main site's domain as the SOCKS username for each tab:

```
$ curl --proxy-user site-a.example:x1 --socks5-hostname 127.0.0.1:9008 http
    ://127.0.0.1:8080/
```

Step 3: results. This sample output comes from an earlier reference run; Conflux IDs, paths, and NEWNYM delays are not fixed values.

```
credentials site-a.example -> conflux AADCE71E  ['test003r', 'test004r', 'test007e']
credentials site-a.example -> conflux AADCE71E  ['test003r', 'test004r', 'test007e']
credentials site-b.example -> conflux 31B1E3E0  ['test003r', 'test004r', 'test007e']
credentials -              -> conflux 35FD16D0  ['test003r', 'test004r', 'test007e']
(NEWNYM delayed by 6 s)
after NEWNYM, site-a.example -> conflux F7011ABE
```

The two requests for "site A" use the same tunnel despite being made at different times. The request for "site B" and the unauthenticated request use different tunnels. The printed path is one of the tunnel's two legs; these happen to coincide because the laboratory network has few relays. The following discussion explains why we should examine the tunnel rather than an individual circuit.

In depth: A circuit is not enough: Conflux tunnels

If we repeat the laboratory many times, the two requests using identical credentials occasionally appear on circuits with different numbers. This is not an isolation failure. Querying tor's full circuit list reveals that almost all client circuits have purpose `CONFLUX_LINKED` and appear *in pairs* with the same `CONFLUX_ID`. These are the two "legs" of one tunnel (section 31.4), and a `STREAM` event can report either leg. Isolation therefore separates *tunnels*, not individual legs. When available, the book's script groups them by `CONFLUX_ID`.
Two precautions remain, both verifiable in the logs:

- `SIGNAL NEWNYM` is honored at most once every ten seconds (`MAX_SIGNEWNYM_RATE` in `src/core/mainloop/mainloop.c`). An earlier request is delayed, and tor logs "Rate limiting NEWNYM request: delaying by N second(s)". Without waiting, a previous experiment's NEWNYM may take effect in the middle of a new experiment.
- `STREAM` events also report streams tor opens for itself, such as directory-document downloads. Filter them by destination.

Even with these precautions, on a small network under heavy load tor may choose a freshly built tunnel for the second request. That is why the book's automated checker repeats the experiment.

Warning: A Mistake in the Laboratory

In our first run, all four requests used the same circuit, seemingly showing that isolation did not work. The problem was in our script: we had placed `--proxy-user` immediately after `curl`'s `-o` option, which interpreted it as an output filename. No credentials were ever sent. Before concluding that a system behaves differently from its specification, check that the experiment is actually doing what you think it is doing.

Summary

- The control protocol is text-based: one-line commands, three-digit response codes, and asynchronous events with code 650.
- Authenticate with a cookie (preferably `SAFECOOKIE`) or password. Anyone controlling the port controls tor.
- Stem provides a Python interface to commands and events. It is stable but has had no new release since 2023.
- SOCKS-credential isolation gives each site a separate tunnel, which under Conflux consists of two circuits. `NEWNYM` switches to new tunnels at most once every ten seconds.

Exercises

Exercise 18.1 ⋆ Interpret the response `552 Unrecognized key "pippo"` to `GETINFO pippo`. What distinguishes a `5xx` code from a `650` event?

Exercise 18.2 ⋆⋆ Using Stem, create a temporary onion service with `create_ephemeral_hidden_service` on the laboratory network's client, forwarding port 80 to the local server on port 8080. Reach it with `curl`. Where are its keys stored?

Exercise 18.3 ⋆⋆ Write a program that displays the client's incoming and outgoing bandwidth every second using `BW` events while downloading a large file through the laboratory network.

Exercise 18.4 ⋆⋆⋆ Using `__LeaveStreamsUnattached` 1, `EXTENDCIRCUIT`, and `ATTACHSTREAM`, direct a request through a path you select (for example, `test004r`, `test005r`, `test006e`). Why is this ability dangerous to anonymity in real-world use?

Further Reading

- *Tor's control protocol* (control-spec) [Thee]: all commands and events.
- Stem documentation, `https://stem.torproject.org/`.

Chapter 19

A Private Tor Network: Chutney and Shadow

Last verified: 23 September 2026 — tor 0.4.9.13; Chutney commit `6cc1588`; Shadow 3.3.0

Chapter objectives

By the end of this chapter, you will be able to:

- explain why Tor experiments should run on a private network rather than the public network;
- build and start, with two separate commands, the laboratory network used throughout the book;
- identify the configuration options that distinguish a test network from the real network;
- navigate a node's state files, including keys, consensus documents, and logs;
- inspect client circuits with Stem and send web and onion-service traffic across the network;
- understand when to use a simulator such as Shadow instead of an emulated network such as Chutney.

This chapter appears in Part III, but it is worth reading early: every laboratory in the book, beginning with Laboratory 1 in chapter 7, runs on the network we build here. Readers working through Part II can read sections 19.1 and 19.3 first and return to the remaining material later.

19.1 Why a Private Network?

The public Tor network consists of thousands of volunteer-operated relays and is used daily by millions of people, many of whom rely on it for their safety. Experimenting on it raises two problems: one ethical and one practical.

The ethical problem. Any experiment that measures, changes, or even merely observes real-network traffic involves users who have not given consent and may be harmed. Even a seemingly harmless experiment, such as a relay recording more information than necessary, can create data that could deanonymize someone if disclosed. For this reason, the Tor Project maintains a *Tor Research Safety Board* to review researchers' experimental plans and recommends a general principle: if a research question can be answered without involving real users, it should be. We revisit these principles in chapter 29.

The practical problem. We have little control over the public network: we cannot predict which relays will be selected, circuits change, the consensus updates hourly, and an experiment cannot be repeated under identical conditions. On a private network, by contrast, we can choose the number of nodes, read detailed logs, stop and restart the whole network in minutes, and repeat experiments until we understand the results.

Warning: Laboratory Rule

Every practical exercise in this book that sends hand-built cells, modifies node behavior, or captures traffic must run **exclusively** on the private network. The educational `torlab` code does not protect anonymity and must never be used on the public network.

19.2 Emulation or Simulation?

There are two ways to build a laboratory Tor network, both used by the Tor Project.

Emulation Many real `tor` processes run on the same computer and communicate through the loopback interface. Time is real, the code is real, and networking is provided by the operating system. This is **Chutney**'s approach. Chutney is a Tor Project tool created for integration testing of tor and Arti.

Simulation Real programs run inside a discrete-event simulator that intercepts system calls and provides a virtual network and virtual clock. This is **Shadow**'s approach [JH12], developed to study Tor at scale.

	Chutney (emulation)	Shadow (simulation)
Typical scale	A few to a few dozen nodes	Hundreds to thousands of nodes
Time	Real	Virtual and deterministic
Network	Loopback, without latency or loss	Topology with configurable latency and bandwidth
Reproducibility	Limited by operating-system scheduling	Complete with the same random seed
Typical use	Development, testing, education	Research on performance and attacks
In this book	All laboratories	section 19.9 and Part V

Table 19.1: Chutney and Shadow compared.

Chutney is the natural choice for learning the protocol. It is lightweight, starts within minutes, and lets us interact with running nodes. This chapter therefore focuses on Chutney and introduces Shadow at the end.

19.3 The Book's Laboratory

To avoid differences between operating systems, the whole laboratory is packaged in a Docker image described by the repository's `labs/en/lab/Dockerfile`. The image:

- compiles tor **from official source** at the book's reference version, 0.4.9.13, using `--enable-gpl` to include the onion-service proof-of-work module (chapter 15);
- installs Chutney at a pinned revision, along with Stem and the traffic-capture tools (`tcpdump`, `tshark`) used in chapter 20.

Pinning versions matters: a book describing protocol messages byte by byte must ensure that readers observe the same bytes.

In depth: When a New Version of tor Is Released

The outputs reported in this chapter and in Part II come from the laboratory network running tor 0.4.9.13, the book's reference version. Some were recorded with its predecessor, 0.4.9.12, released only days earlier, and then reproduced: the script `scripts/verify_labs.py`, which reruns all the laboratories live and checks their results, gives the same outcomes with both versions (currently 53 checks, plus 14 for the bridge laboratory). The script runs the laboratories of the original Italian edition, in `labs/it`, from which this edition's laboratory code is translated; in the listings, names and messages follow this edition's code. Only execution-specific elements change: keys, fingerprints, `.onion` addresses, timestamps, and certificate sizes, which vary even from one relay to another. The same script lets you check, with the version you use, that what the book describes still holds.

19.3.1 Requirements

You need Linux with Docker and approximately 1 GB of free disk space. Docker Desktop works on macOS and Windows, but the host networking mode used here behaves differently; run the experiments inside the container in that case (section 19.7.4). The network occupies ports beginning at 5100, 7100, 8000, and 9000 on the host. Check that these ports are free.

19.3.2 Building the Image and Starting the Network

From the **repository root** (not from within `labs/en/lab`):

```
$ make -C labs/en/lab image        # build tor and prepare Chutney (once)
$ make -C labs/en/lab start           # configure and start the network; wait for
    bootstrap
```

Building the image takes a few minutes. On the machine used to write this chapter, starting the network took around two and a half minutes and ended with the following abbreviated summary:

```
  test006e:
    bootstrap progress: 100 (SUCCESS), done, Done
    desc publishing: 100 (SUCCESS), all nodes, DESC MD MD_CONS NS_CONS
  test008c:
    bootstrap progress: 100 (SUCCESS), done, Done
  test009h:
    bootstrap progress: 100 (SUCCESS), done, Done
    oniondesc publishing: 200 (ONIONDESC_PUBLISHED), hidden service v3
```

Each node has completed bootstrap, each relay sees its descriptor published in the consensus, and the onion service has published its descriptor. The `make -C labs/en/lab verify` command then runs Chutney's test, transferring data through both an exit and an onion service and checking that it arrives intact:

```
  not-done:0 successes:4 failures:0
Transmission: Success
```

`make -C labs/en/lab status` shows node status. `make -C labs/en/lab shell` opens a shell inside the container, `make -C labs/en/lab stop` stops the network, and `make -C labs/en/lab clean` also deletes generated data.

19.4 Network Topology

The book's network is described by a short Python file, `labs/en/lab/chutney/networks/book`, written in Chutney's configuration language:

```
Authority = Node(tag="a", authority=1, relay=1)
Middle    = Node(tag="r", relay=1)
Exit      = Node(tag="e", relay=1, exit=1)
Client    = Node(tag="c", client=1, launch_phase=2)
Onion     = Node(tag="h", hs=1, launch_phase=2)

NODES = (Authority.getN(3) + Middle.getN(3) + Exit.getN(2)
         + Client.getN(1) + Onion.getN(1))
ConfigureNodes(NODES)
```

Each `Node` is a template, and `getN` creates multiple instances. The client and onion service start in a second phase (`launch_phase=2`), once authorities have produced a consensus. Chutney numbers nodes in order and names each one by combining its number and tag: `test000a`, ..., `test009h`. Node numbers also determine port assignments, as shown in fig. 19.1.

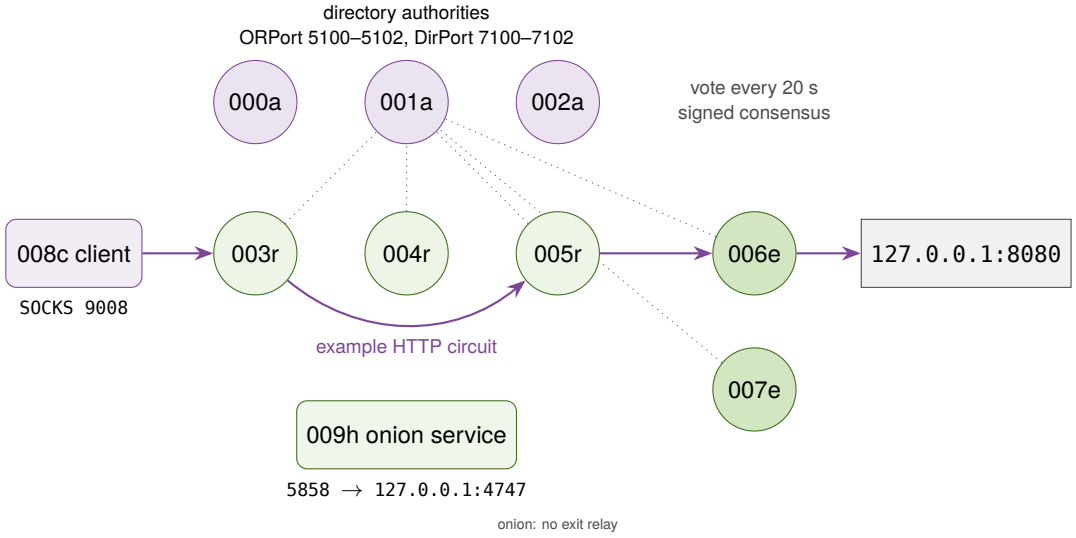


Figure 19.1: Chutney lab topology bound to `127.0.0.1`: three directory authorities/relays, three middle relays, two exits, one client, and one onion service. ORPort $5100 + n$ applies only to authorities and other relays, including exits; all nodes have ControlPort $8000 + n$. Authorities use DirPort $7100 + n$, the client SocksPort 9008, and the onion service virtual port 5858 mapped to `127.0.0.1:4747`. The example HTTP circuit through an exit is separate from onion-service traffic, which does not use an exit relay. Loopback is not a separate container network.

Node	Role	ORPort	ControlPort	Other
`test000a–002a`	directory authority	5100–5102	8000–8002	DirPort 7100–7102
`test003r–005r`	middle relay	5103–5105	8003–8005	
`test006e–007e`	exit	5106–5107	8006–8007	
`test008c`	client	—	8008	SocksPort 9008
`test009h`	onion service	—	8009	virtual port 5858

In depth: Why Three Authorities?

A consensus must be signed by a majority of directory authorities (chapter 12). With only one, we could not observe voting; with three, we can observe the process and, in an exercise, see what happens if one authority stops. The public network has nine, plus a separate bridge authority. Their list is compiled into tor itself (`src/app/config/auth_dirs.inc`).

19.5 A Test Network Is Not the Real Network

Chutney generates a `torrc` for each node. These files reveal exactly what must change to make a miniature Tor network work, and therefore which assumptions about the public network do not hold. Here are the most important options from an authority's torrc (`net/nodes/000a/torrc`):

```
TestingTorNetwork 1
AuthoritativeDirectory 1
V3AuthoritativeDirectory 1
AssumeReachable 1
V3AuthVotingInterval 20 seconds
V3AuthVoteDelay 4
V3AuthDistDelay 4
TestingDirAuthVoteGuard *
TestingDirAuthVoteHSDir *
```

`TestingTorNetwork` Enables test mode. It can only be used alongside an explicit directory-authority list (`DirAuthority`), preventing test nodes from accidentally joining the public network. It also changes many defaults to speed up operation.

`V3AuthVotingInterval` On the real network, authorities vote on a new consensus hourly. Here, they do so every 20 seconds, as the consensus itself shows:

```
valid-after 2026-09-23 14:37:40
fresh-until 2026-09-23 14:38:00
valid-until 2026-09-23 14:38:20
```

`AssumeReachable` A relay normally verifies that it is externally reachable before advertising itself. That test is meaningless on loopback and is skipped.

`TestingDirAuthVoteGuard *` On the real network, relays earn the Guard flag through weeks of stability and adequate bandwidth. Here, authorities assign it to all relays. Therefore **every relay in our consensus is a potential guard**, unlike on the public network.

The exits' torrc files also contain:

```
ExitPolicyRejectPrivate 0
ExitPolicy accept private:*
```

By default, exits reject connections to private and loopback addresses to avoid exposing their local networks. In this laboratory, `127.0.0.1` is the only available "Internet", so that policy must be reversed.

Warning: Do Not Copy These Options Elsewhere

`Testing…` options, `AssumeReachable`, and especially `ExitPolicy accept private:*` are only for test networks. On a public relay, these settings would be incorrect or dangerous to its host network.

19.6 Inside a Node's Directory

Each node has its own data directory under `labs/en/lab/net/nodes/NNNx/`. Among other files, an authority's directory contains:

File or directory	Contents
`torrc`	Chutney-generated configuration
`keys/`	node identity and onion keys (chapters 5 and 6)
`fingerprint`, `fingerprint-ed25519`	RSA and Ed25519 identity fingerprints
`cached-consensus`, `cached-microdesc-consensus`	both consensus flavors (chapter 12)
`cached-microdescs.new`	downloaded microdescriptors
`v3-status-votes`	authority votes (authorities only)
`sr-state`	shared-randomness protocol state (authorities only)
`notice.log`, `info.log`	logs at *notice* and *info* levels
`control_auth_cookie`	control-port authentication cookie

The onion service's directory additionally holds `hidden_service/`, with its keys and a `hostname` file containing the `.onion` address, for example:

```
rpns2r36zn5u46wctlgnvtwjotxaf7iflsc3k5eas3lqoxeesv4zwyad.onion
```

The address changes with each fresh configuration because a new key is generated (chapter 14).

A consensus is a human-readable text file. Extracting the `r` lines (relay identities and addresses) and `s` lines (flags) gives these two example relays:

```
r test000a H+9DDHx4rEo6I6Jdn8gIyKyi3jk ... 127.0.0.1 5100 7100
s Authority Fast Guard HSDir Running Stable V2Dir Valid
r test006e dEPHoFFHamQyy6pRzlsLB9/ilzU ... 127.0.0.1 5106 0
s Exit Fast Guard HSDir Running Stable V2Dir Valid
```

Notice that all relays carry the Guard flag, as expected, while only the two exits carry Exit. The fields are explained in chapter 12.

19.7 Using the Network

19.7.1 Browsing Through the Private Network

Run both HTTP examples *inside the laboratory container*, in `/lab`, after opening it with `make -C labs/en/lab shell` from a separate host terminal. Using `--noproxy` with an empty string forces curl to use the SOCKS proxy even when the destination is `127.0.0.1` and `NO_PROXY` is set. Port 4747 serves the `index.html` created in the first example.

Client `test008c` exposes SOCKS on port 9008. Any SOCKS5-capable program can use it. Start a small local web server and access it through Tor:

```
$ echo "Hello from the private network" > index.html
$ python3 -m http.server 8080 --bind 127.0.0.1 &
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 http://127.0.0.1:8080/index.html
Hello from the private network
```

The request traveled through a three-relay circuit before one of the exits connected to the server. Using `--socks5-hostname` rather than `--socks5` makes the exit resolve destination names, instead of our own system. On the real network, this is essential to avoid leaking visited destinations to the local DNS resolver (chapter 11).

19.7.2 Reaching the Onion Service

Chutney configures the onion service with:

```
HiddenServicePort 5858 127.0.0.1:4747
```

This maps virtual port 5858 at the `.onion` address to local port 4747. All we need is a server listening there:

```
$ python3 -m http.server 4747 --bind 127.0.0.1 &
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 \
    http://$(cat net/nodes/009h/hidden_service/hostname):5858/index.html
Hello from the private network
```

This time, no exit is involved. The client and service meet at a rendezvous point within the network (chapter 14).

19.7.3 Observing Circuits with Stem

The control port lets us inspect a running node. With Stem (chapter 18), included in the image, we list the client's circuits:

```
from stem.control import Controller

with Controller.from_port(port=8008) as c:
    c.authenticate()                    # authenticate using the configured cookie
    print(c.get_version())
    for circ in c.get_circuits():
        print(circ.id, circ.status, circ.purpose,
              [name for _, name in circ.path])
```

An excerpt from the output after `make -C labs/en/lab verify`:

```
0.4.9.13 (git-3c575400909efe65)
10 BUILT GENERAL ['test004r', 'test005r', 'test007e']
124 BUILT CONFLUX_LINKED ['test006e', 'test004r', 'test007e']
114 BUILT CONFLUX_UNLINKED ['test007e', 'test004r', 'test006e']
128 BUILT CIRCUIT_PADDING ['test004r', 'test001a', 'test007e', 'test006e']
163 BUILT HS_CLIENT_REND ['test004r', 'test001a', 'test006e']
166 BUILT HS_CLIENT_HSDIR ['test004r', 'test006e', 'test007e', 'test003r']
181 BUILT HS_VANGUARDS ['test004r', 'test007e', 'test006e']
```

Even on a network this small, the list previews much of this book:

- Nearly all circuits start with `test004r`: the client's **guard**, selected once and reused (chapter 13). The exceptions are the legs of a Conflux tunnel starting with the second guard (chapter 18).

- Three-hop `GENERAL` circuits carry ordinary traffic; single-hop circuits, omitted here, fetch directory information.
- `CONFLUX_LINKED` circuits come in pairs used together for the same traffic: the Conflux feature (Proposal 329, chapter 31).
- `HS_VANGUARDS`, `HS_CLIENT_HSDIR`, and `HS_CLIENT_REND` identify circuits involved in reaching an onion service (chapter 14).
- Under the control-protocol specification, `CIRCUIT_PADDING` identifies a circuit retained to mask when it really stopped being used. Its closure thus does not necessarily reveal when user activity ended.

19.7.4 Working Inside the Container

The container uses host networking, making all node ports directly reachable from the host computer. If this is unavailable, for example with Docker Desktop, `make -C labs/en/lab shell` opens a shell inside the container with tor, Chutney, Stem, Python, and capture tools installed; laboratory files are available under `/lab`.

19.8 First Contact with a Relay

The following Python snippet imports the teaching package `torlab` from `labs/en/code/`; the Chutney container mounts only `labs/en/lab/` at `/lab`, so the package is *not automatically importable inside that container*. On Linux with host networking, save the snippet as a script and run it from the repository root with `PYTHONPATH=labs/en/code python3 script.py`, or explicitly make the code package available in the container. This example is intended only for the private network.

We return to chapter 8. With `labs/en/code/torlab/cells.py` from Laboratory 2, we can open a TLS connection to a laboratory relay's ORPort, send a **`VERSIONS`** cell, and decode the reply. Laboratory 1 in chapter 7 provides the complete code with certificate verification. Here, we only observe the initial exchange:

```
import socket, ssl
from torlab.cells import *

ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE   # private lab only; TLS certificates NOT verified
s = ctx.wrap_socket(socket.create_connection(("127.0.0.1", 5103)))
s.sendall(encode(versions_cell([3, 4, 5]), 0))

buf, version, received = b"", 0, 0
while received < 4:
    chunk = s.recv(4096)
    if not chunk:
        raise ConnectionError("TLS channel closed early")
    buf += chunk
    while (r := decode(buf, version)) is not None:
        cell, buf = r
        received += 1
        print(describe(cell, version))
        if cell.command == CellCommand.VERSIONS:
            version = negotiate_version([3, 4, 5], cell.body)
```

```
22 s.close()
```

Caution: this short demonstration deliberately disables TLS certificate verification with `ssl.CERT_NONE`; it does *not* actually validate the relay identity from the CERTS cell. Do not run it against public relays. Laboratory 1 supplies the complete verification procedure.

The following excerpt comes from a previous reference session; circuit IDs, fingerprints, onion addresses, timestamps, and certificate sizes may change between runs.

Relay `test003r` responds:

```
CircID=0x0000 VERSIONS body=6 bytes
CircID=0x00000000 CERTS body=1451 bytes
CircID=0x00000000 AUTH_CHALLENGE body=36 bytes
CircID=0x00000000 NETINFO body=509 bytes
```

This is exactly the sequence described in section 8.8: first, **`VERSIONS`** with a two-byte CircID; after negotiating version 5, subsequent cells use a four-byte CircID. **`CERTS`** and **`AUTH_CHALLENGE`** have variable lengths and retain their exact message bodies; **`NETINFO`** is fixed-length and carries all 509 bytes. Notice that `version = ...` is assigned immediately after **`VERSIONS`**: decoding the following cells depends on the freshly negotiated version.

19.9 Shadow: A Network Inside a Simulator

Chutney is not enough for questions about the behavior of the network *as a whole*. How would a different congestion-control algorithm affect latency? How much traffic would an adversary need for a particular attack? Answering these questions requires thousands of nodes, realistic latencies across continents, and, above all, reproducible experiments.

Shadow, originally developed by Rob Jansen and Nicholas Hopper [JH12], meets these needs. Current versions run real programs, including tor, as normal Linux processes, while intercepting networking and time-related system calls. A virtual graph supplies user-defined latency and bandwidth, and time advances only when the simulator chooses. One simulated hour may therefore take more or less than an hour of real time, depending on processing power, and repeating an experiment with the same random seed produces the same result.

To build realistic Tor networks in Shadow, researchers use **tornettools**. Based on public Tor Metrics data (chapter 24), it generates a scaled-down network with distributions of bandwidth, flags, and roles matching those of the real network.

We will use Shadow in a guided experiment in chapter 29. For now, it is enough to know when the simulator is the appropriate tool.

Summary

- Run Tor experiments on private networks. On the public network, they could involve users without consent and would not be reproducible.
- Chutney emulates a Tor network with real `tor` processes on loopback. Shadow runs the real software inside a discrete-event simulator suited to large networks and repeatable experiments.
- The book's laboratory (`labs/en/lab/`) builds tor 0.4.9.13 from source. `make -C labs/en/lab start` starts a ten-node network: three authorities, three middles, two exits, a client, and an onion service.
- Node n uses ORPort $5100 + n$, ControlPort $8000 + n$, and SocksPort $9000 + n$.

- Test settings (`TestingTorNetwork`, voting every 20 seconds, assigning Guard to all relays, and permitting private exit addresses) make the miniature network work but distinguish it from the real one. Measurements from it require caution before extrapolation.
- SocksPort 9008 carries application traffic through the network; Stem observes circuits; `torlab` speaks directly with relays.

Exercises

Exercise 19.1 ⋆ Without reading the torrc, give the ORPort and ControlPort of `test006e`. Does `test009h` have an ORPort? Why?

Exercise 19.2 ⋆ Every relay on the private network has the Guard flag. Which option causes this, and why does it not happen on the real network? What effect does it have on laboratory path-selection measurements?

Exercise 19.3 ⋆ Repeat the `curl` request from section 19.7, using `--socks5` instead of `--socks5-hostname` and `localhost` instead of `127.0.0.1`. What changes in what the exit receives? Why does that matter on the public network?

Exercise 19.4 ⋆⋆ Observe the client's consensus `valid-after` field for several minutes (`grep valid-after net/nodes/008c/cached-microdesc-consensus`). How often does it change? Calculate how many consensuses are produced per day on the laboratory network and on the real network.

Exercise 19.5 ⋆⋆ Stop one of the three authorities (`make -C labs/en/lab shell`, then `kill` the corresponding process using its `pid` file). Does the network continue to produce consensuses? What happens if you stop two? Explain the result; the underlying reason appears in chapter 12.

Exercise 19.6 ⋆⋆ Modify the Stem script from section 19.7.3 to print the ORPorts of all relays on each three-hop circuit, obtaining them from the consensus with `get_network_status`.

Exercise 19.7 ⋆⋆ Create `labs/en/lab/chutney/networks/book-large` with five authorities, ten middle relays, and four exits, then start it by setting `NETWORK=book-large` in the `make` command. How does bootstrap time compare with the book's network?

Exercise 19.8 ⋆⋆⋆ Find the messages in the client's `info.log` relating to circuit construction, from node selection to completion. Match each message to a cell in fig. 8.5.

Exercise 19.9 ⋆⋆⋆ Inside the container, capture traffic to port 5103 with `tcpdump -i lo` while running the script from section 19.8. Can you see the cells? Why not? What remains observable: sizes, timing? Chapter 20 shows how to go further.

Further Reading

- Chutney, `https://gitlab.torproject.org/tpo/core/chutney`: its README and `networks/` directory contain many example networks, including examples with bridges and Arti.
- R. Jansen, N. Hopper, *Shadow: Running Tor in a Box for Accurate and Efficient Experimentation* [JH12].
- Shadow and tornettools documentation, `https://shadow.github.io/`.

- Tor Research Safety Board, `https://research.torproject.org/safetyboard/`: guidance for ethical research on Tor.

Chapter 20

Observing Traffic: Wireshark and Logs

Last verified: 23 September 2026 — tor 0.4.9.13; tshark (Debian trixie); laboratory network

Chapter objectives

By the end of this chapter, you will be able to:

- capture Tor traffic with `tcpdump` and analyze it using `tshark` or Wireshark;
- distinguish what an external observer can see from what remains hidden;
- decrypt the TLS on *your own* channel using session keys and inspect its cells (Laboratory 12);
- follow the protocol step by step using tor's logs.

In the preceding chapters, we examined the protocol "from the inside" by writing code to speak it. Here, we observe it "from the outside", as someone watching the network would. This exercise serves two opposite purposes: understanding what an adversary can infer from traffic and obtaining an independent way to verify our own implementation.

Warning: Only Your Own Traffic, and Only in the Laboratory

Capturing other people's traffic without authorization is unlawful in Italy, as in most countries (chapter 25), and conflicts with the ethics of anonymity research (section 19.1). All captures in this chapter involve the book's private network or connections that we open ourselves.

20.1 What an Observer Can See

Someone observing the connection between a client and its guard, such as an Internet provider, can see:

- IP addresses and ports: the client connects to an address that, unless it belongs to a bridge, appears in the public consensus and is therefore recognizable as a Tor relay;
- the TLS handshake: certificates travel encrypted under TLS 1.3, but some characteristics of the client's initial message remain visible;
- the sizes and timing of TLS records.

The observer cannot see cell contents, circuits, or destinations. Yet sizes and timing are precisely what *website fingerprinting* and correlation attacks rely on (chapter 26). Encrypted content does not mean that traffic reveals nothing.

In section 16.5, we compared ordinary Tor traffic, recognizable as TLS, against obfs4 traffic with no recognizable structure. Here, we go one step further and look inside TLS.

20.2 The Tools

`tcpdump` Captures packets from a network interface and saves them to a `pcap` file. For example, inside the laboratory container, capture traffic to `test004r`'s ORPort:

```
$ docker exec -u 0 torbook-network tcpdump -i lo \
    -w /lab/net/tls.pcap "tcp port 5104"
```

Wireshark and `tshark` Analyze packet captures, the former through a graphical interface and the latter from the command line. They recognize TLS, but there is no official Tor-protocol dissector. Even after decryption, Tor cells appear as raw bytes. Our `cells.py` module will serve as the dissector.

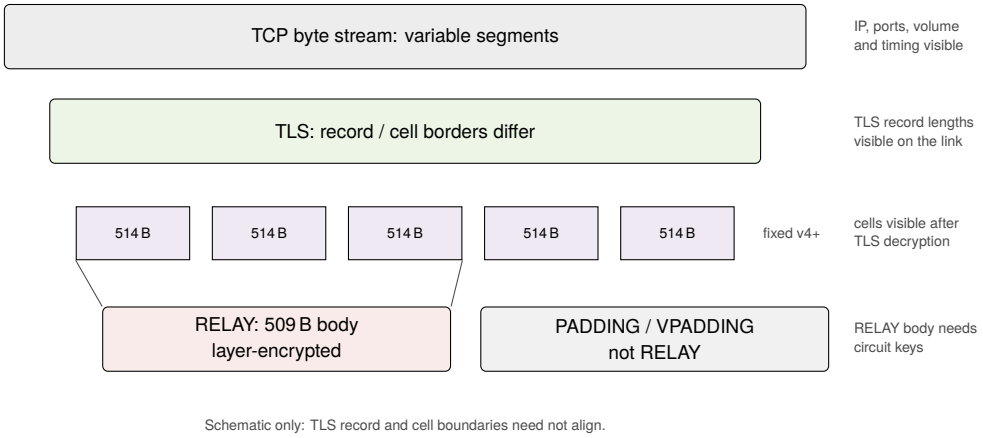


Figure 20.1: Conceptual Tor channel: TCP segments carry TLS records whose boundaries need not match cell boundaries. IP addresses, ports, record lengths and timing are observable; TLS decryption exposes cells. Under link protocol v4+, a fixed-length cell is 514 bytes (5-byte header, 509-byte body), though variable-length cells also exist. Only `RELAY` cell bodies are encrypted in circuit layers; padding cells are not RELAY messages.

20.2.1 Decrypting Your Own TLS

TLS uses session keys that external observers do not possess. A party *participating* in the connection, however, can export those keys. Browsers and applications using Python's `ssl` library can export these keys if they enable key logging. Our `channel.py` explicitly sets `SSLContext.keylog_filename` when `SSLKEYLOGFILE` is defined. Wireshark and `tshark` use that file to decrypt a capture. Laboratory 1's `channel.py` follows this convention: set the variable before running it. The file contains lines such as:

```
# TLS secrets log file, generated by OpenSSL / Python
SERVER_HANDSHAKE_TRAFFIC_SECRET 4878224d3f9fb9aedb96d... 0da89dff6feb...
```

These record the client random value and the secrets from each phase of TLS 1.3 for each session.

In depth: Why This Is Not an Attack on Tor

Decrypting TLS on your own channel reveals nothing that the client did not already know. Even with TLS session keys, **RELAY** cells remain encrypted in layers with keys for individual hops (chapter 10). Reading those would require circuit keys held only by the client and the relays involved. TLS is the onion's outer shell, not its only layer.

20.3 Laboratory: Cells Inside TLS

Laboratory 12: Capturing, decrypting, and reading a channel

Objective. Capture a channel opened from our client to a relay, decrypt it with session keys, and reconstruct the sequence of cells using `labs/en/code/torlab/traffic.py`.
Prerequisites. Laboratories 1–3; the laboratory network running; `tcpdump` and `tshark` installed in the container.
Estimated time. 1–2 hours.

For Laboratory 12, start the private network and open two terminals at the repository root. Run the capture command below on the *host*: it starts `tcpdump` as root *inside the laboratory container*, since the ordinary Makefile shell runs under an unprivileged user. Keep the capture running while the client executes in the second terminal, then stop it with Ctrl+C. The file is written to the shared `labs/en/lab/net/` volume.

Step 1: capture. Start capturing inside the container. Then, from the host, open a channel to `test004r` and build a one-hop circuit while exporting the session keys:

```
$ PYTHONPATH=labs/en/code SSLKEYLOGFILE=labs/en/lab/net/tls-keys.log python3 -c "
from torlab.laboratory import relay
from torlab.circuit import build
build([relay('004r')], cc=False)"
```

Run the following Python command from the *repository root* on Linux with host networking. `PYTHONPATH` makes `torlab` importable from `labs/en/code/`; `labs/en/code/torlab/channel.py` explicitly reads `SSLKEYLOGFILE` and enables TLS secret logging. The key file contains sensitive session secrets: do not publish it, and delete it after the experiment.

Step 2: identify our connection. Other laboratory relays also connect to the same port, but their session keys are different. Ask `tshark` to follow each TCP stream after attempted decryption, retaining the one that decrypts successfully:

```
$ docker exec torbook-network tshark -r /lab/net/tls.pcap \
    -o tls.keylog_file:/lab/net/tls-keys.log \
    -q -z follow,tls,raw,2
```

The TCP stream identifier 2 is only an example from an earlier capture. Discover candidate stream IDs using `tshark` with `-T fields -e tcp.stream`, then follow a decryptable stream. The output below is historical; certificate lengths and circuit IDs can change between runs. The output contains the decrypted data from both directions in hexadecimal; one direction is indented with a tab.

Step 3: inspect cells. The `cells_from_follow` function reconstructs both streams and passes them to Laboratory 2's `decode`. Each direction has its own version negotiation: its first cell is **`VERSIONS`** with a two-byte CircID. Four-byte CircIDs are used only after negotiation. The result:

```
client -> relay  CircID=0x0000 VERSIONS body=4 bytes
relay  -> client CircID=0x0000 VERSIONS body=6 bytes
relay  -> client CircID=0x00000000 CERTS body=1443 bytes
relay  -> client CircID=0x00000000 AUTH_CHALLENGE body=36 bytes
relay  -> client CircID=0x00000000 NETINFO body=509 bytes
client -> relay  CircID=0x00000000 NETINFO body=509 bytes
client -> relay  CircID=0xc244f1f6 CREATE2 body=509 bytes
relay  -> client CircID=0xc244f1f6 CREATED2 body=509 bytes
```

Observed from outside, this is exactly the sequence in fig. 8.5 and chapter 7. The client's **`VERSIONS`** cell has a four-byte body because `channel.py` offers versions 4 and 5 only. Circuit ID `0xc244f1f6` has its most significant bit set to 1 because the client opened the channel (section 8.4).

Warning: Two Errors in Our First Version

Our initial analysis made two mistakes, revealed by comparing the output against the expected sequence. It used one negotiated protocol version for both directions, misinterpreting the relay's **`VERSIONS`** cell. It also assigned blocks to the wrong direction because we guessed which direction `tshark` had indented. The correct rule is simple: the first block is always the client's **`VERSIONS`** cell.

20.4 tor Logs

A packet capture shows *what* travels over the network. tor's logs explain *why*. At `info` level, tor records circuit construction stages, guard choices, and descriptor downloads and publication. At `debug`, its reports extend to individual cells. We have already used logs to verify which HSDir the client queried (section 14.9) and when an onion service rotated its descriptors.

A few practical suggestions:

- `debug` produces enormous amounts of text. Enable it only for a particular domain, for example `Log [circ]debug file circ.log`, to limit detailed messages to circuit events.
- On the real network, `SafeLogging` masks addresses. A client's logs should never contain the destinations it visits.
- Public-relay logs may contain other people's personal data, with the legal consequences discussed in chapter 25; retain only what is strictly necessary.

Summary

- A connection observer sees addresses, the TLS handshake, packet sizes, and timing, but not Tor cells or destinations.
- `SSLKEYLOGFILE` permits decryption of TLS on your own channel and inspection of its cells; **`RELAY`** cells nevertheless remain encrypted in layers.
- Analyzing a packet capture provides an independent way to verify an implementation.

- tor's `info` and `debug` logs explain the program's choices; during real-world use, logging should be minimized and protected.

Exercises

Exercise 20.1 ⋆ In a capture of a Tor download, TLS records are not all multiples of 514 bytes long. Why, if cells have a fixed size?

Exercise 20.2 ⋆⋆ Extend Laboratory 12 by decrypting the **`RELAY`** cells on your own circuit using keys from `circuit.py`. Which relay messages can you see while constructing a three-hop circuit?

Exercise 20.3 ⋆⋆ Capture one minute of a laboratory client's channel without generating application traffic. Count TLS records in each direction and connect your observations to channel padding in chapter 7.

Exercise 20.4 ⋆⋆⋆ Write a small Wireshark dissector in Lua that recognizes Tor cells within decrypted data and displays their CircID and command.

Further Reading

- Wireshark documentation on decrypting TLS using session key log files.
- The tor manual's `Log` entry, explaining logging domains and verbosity levels.

Chapter 21

Publishing an Onion Service

Last verified: 23 September 2026 — tor 0.4.9.13; Onionbalance documentation

Chapter objectives

By the end of this chapter, you will be able to:

- configure an onion service and protect its keys;
- avoid configuration mistakes that reveal the server;
- offer visitors to an ordinary website its `.onion` version through Onion-Location;
- restrict access to authorized clients using restricted discovery (Laboratory 13);
- assess vanity addresses, single-hop services, and load balancing with Onionbalance.

Chapter 14 explained how onion services work. This chapter is its practical counterpart: publishing a service, avoiding simple mistakes that compromise its anonymity, and choosing tools for common operational needs.

21.1 Your First Onion Service

Only two lines in `torrc` are required:

```
HiddenServiceDir /var/lib/tor/my_service/
HiddenServicePort 80 127.0.0.1:8080
```

The `HiddenServiceDir` path must be readable and writable by the tor process; in the laboratory it is a path *inside the Chutney container*. This first example is a *public* onion service and requires no `alice.auth` file. Client authorization, including the `authorized_clients/*.auth` files, is covered later in this chapter. The first specifies the directory in which tor stores the service's keys. The second maps *virtual* port 80 of its `.onion` address to the local server on port 8080. When tor restarts, it creates the following files in that directory:

`hostname` The `.onion` address (section 14.2).

`hs_ed25519_public_key`, `hs_ed25519_secret_key` The service's identity key pair. **Whoever possesses the secret key controls the address**: they can publish descriptors and impersonate the service. Protect it like a certificate's private key, and keep a secure backup. If it is lost, the address is lost forever.

The directory must be accessible only to the user running tor; otherwise, tor refuses to use it. One tor process can host several services, each in its own directory. In the laboratory, we added a second service to `test009h` with two further lines.

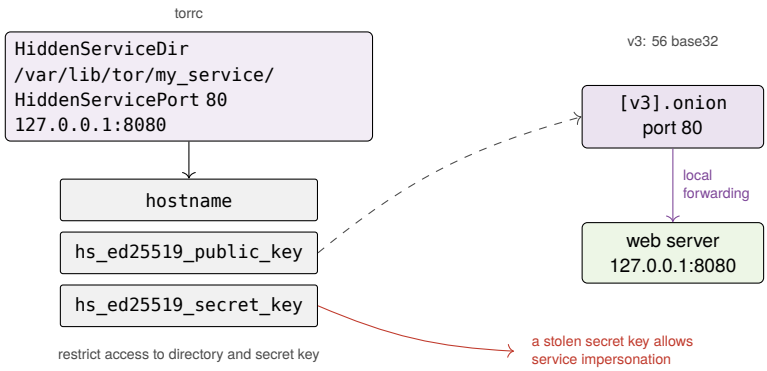


Figure 21.1: Two `HiddenServiceDir` and `HiddenServicePort` directives configure a public v3 onion service. Tor creates `hostname` and Ed25519 identity keys in the directory; the onion address contains 56 base32 characters derived from the public key, a checksum and the version (the figure uses a placeholder). Virtual port 80 forwards to `127.0.0.1:8080`. Theft of the private key permits impersonation; no `alice.auth` file is needed for this public example.

21.1.1 Avoid Revealing the Server

The protocol conceals the server's address, but a careless configuration can disclose it. Common mistakes include:

- **Making the service externally reachable as well.** If the web server listens on all interfaces rather than only `127.0.0.1`, someone scanning the Internet may find it and compare its content with the onion service. Bind the server to localhost or, preferably, use a Unix socket (`HiddenServicePort 80 unix:/path/socket`).
- **Leaking details in responses.** Error pages may expose the server's name, HTTP headers may include internal addresses, TLS certificates may be issued for a public domain, or web-server status pages may be left accessible.
- **External resources.** Pages that load images or scripts from ordinary websites send those requests outside the onion service. In Tor Browser, they still travel through Tor, so the visitor's IP address is not revealed, but the onion service's end-to-end authentication and protection no longer cover them: an exit can observe connection metadata and plaintext HTTP, while HTTPS still protects the application content. Server-side applications contacting external services for updates or analytics may reveal the server's address.
- **Clocks and timing.** A server displaying a local time associated with a particular time zone, or becoming unreachable whenever a known public server undergoes maintenance, leaves potentially correlatable traces.

In chapter 28, we examine real cases in which such mistakes helped locate services.

21.2 Onion-Location

Many public websites, including major news organizations, social networks, and the Tor Project, offer a `.onion` version. An ordinary website can advertise this version to Tor Browser visitors through the HTTP header:

```
Onion-Location: http://<address>.onion/path
```

An equivalent `meta` element can be used in the page. Tor Browser then displays a button in the address bar offering to switch to the onion version. Visitors avoid the exit and obtain a connection authenticated by the address itself; websites receive Tor traffic without routing it through exits, which are sometimes blocked or treated suspiciously.

21.3 Restricted Discovery

A personal service, such as SSH access to a home server or an administration panel, need not be reachable by everyone who knows its address. Restricted discovery (section 14.7) encrypts the descriptor's second layer so that only authorized clients can decrypt it. The configuration uses X25519 key pairs [Thee]:

Service side Place one `.auth` file per client in the service's `authorized_clients/` directory. Each file contains a line consisting of `descriptor:x25519:` followed by the client's base32-encoded public key.

Client side The `ClientOnionAuthDir` option specifies a directory containing one `.auth_private` file per service. Each file has the address without `.onion`, followed by `:descriptor:x25519:` and the client's base32-encoded private key.

Tor Browser allows users to enter the private key when visiting a service that requires it.

Laboratory safety notice: the automated script `labs/en/lab/restricted_discovery.py` deletes any existing `restricted_service/` and `onion_auth/` directories during its initial cleanup. Run it *only* in a dedicated disposable Chutney network, after confirming those directories contain no data to keep. Never run it against a real onion service. For a manual exercise, write the authorization files before reloading only the affected nodes.

Laboratory 13: An Onion Service with Restricted Discovery

Objective. Configure a second onion service on `test009h`, accessible only to an authorized client, and test access with and without the key.
Prerequisites. The laboratory network running; a local web server on port 8080.
Estimated time. 1 hour.

Step 1: the client's key pair. Generate one with the `cryptography` library: The next snippet is a self-contained Python example that prints the public authorization line. Save the private key `priv` separately and securely for the subsequent `.auth_private` file; do not disclose it.

```
import base64
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
from cryptography.hazmat.primitives.serialization import (
    Encoding, PrivateFormat, PublicFormat, NoEncryption
)

sk = X25519PrivateKey.generate()
priv = sk.private_bytes(Encoding.Raw, PrivateFormat.Raw, NoEncryption())
pub = sk.public_key().public_bytes(Encoding.Raw, PublicFormat.Raw)
b32 = lambda b: base64.b32encode(b).decode().rstrip("=")
print("descriptor:x25519:" + b32(pub))
```

Base32 values must omit the trailing = padding characters. This snippet prints only the public authorization line; it does *not save* the private key. If executed as a separate script, `priv` is lost when the process exits. For a manual workflow, save the private key securely with

restrictive permissions before exiting, or use the bundled automated lab, which creates the client authorization file and removes temporary keys afterwards. The manual and automated workflows are alternatives.

Step 2: the service. Create `restricted_service/authorized_clients/alice.auth` containing the public key. Add the following to `test009h`'s `torrc`:

```
HiddenServiceDir /lab/net/nodes/009h/restricted_service
HiddenServicePort 80 127.0.0.1:8080
```

Then send `HUP`. tor generates the address and publishes the descriptors.

The next command reads this instance's actual onion address from the `hostname` file that tor creates with the service: every network has its own. The SOCKS error and log lines below come from an earlier reference session and may vary.

Step 3: without the key. From the network's client:

```
$ ONION_ADDR=$(cat labs/en/lab/net/nodes/009h/restricted_service/hostname)
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 \
    "http://${ONION_ADDR}/index.html"
(curl: SOCKS proxy error 97)
```

The client log reports:

```
[notice] Fail to decrypt descriptor for requested onion address.
         It is likely requiring client authorization.
```

The client found and downloaded the descriptor but cannot decrypt its second layer. It does not even know the introduction points and cannot attempt a connection.

Warning: An Empty Directory Makes the Service Unreachable

If `authorized_clients/` exists but contains no files, the service in the source experiment attempted to publish a descriptor that HSDirs rejected:

```
[warn] Uploading hidden service descriptor: http status 400 ("Invalid HS
       descriptor. Rejected.") response from dirserver 127.0.0.1:5106.
```

The relevant specification requires at least one `auth-client` line in the superencrypted layer [rend-spec § Hidden service descriptors: encryption format]. With restricted discovery activated but no authorized clients, the observed configuration produced none, making the descriptor invalid. This is an operational trap when revoking the final client or creating the directory before adding keys. An interrupted laboratory script can likewise leave a test configuration incomplete.

Step 4: with the key. Write the `.auth_private` file in a directory specified by `ClientOnionAuthDir`, reload the client's configuration, and repeat the request. This time the page is delivered. The script `labs/en/lab/restricted_discovery.py` automates the experiment, cleanup, and restoration of `torrc`; its actual execution was not part of this editorial review.

21.4 Other Requirements

21.4.1 Vanity Addresses

An address beginning with a recognizable word can help users identify it. Generate random keys until an address starts with the desired prefix using a specialized program such as *mkp224o*. Each additional base32 prefix character multiplies the expected number of attempts by 32. A six- or seven-character prefix can be found in a reasonable time on an ordinary computer, but each subsequent character costs dozens of times more. As discussed in section 14.2, an adversary can also generate an address with the same prefix. The prefix aids recognition but does not authenticate the service. Treat keys generated by third-party programs as carefully as any other secret keys.

21.4.2 Single-Hop Services

A public website that does not need to conceal its location, such as a known newspaper's onion edition, can trade away server anonymity for speed. With `HiddenServiceSingleHopMode` and `HiddenServiceNonAnonymousMode`, the service builds single-hop circuits to introduction and rendezvous points. The client remains anonymous as usual. Descriptor publication still uses three-hop circuits: as we found in section 14.9, HSDirs reject direct requests, a point explicitly noted in the tor manual. The "NonAnonymous" part of the option name is intentional: a tor instance configured this way must not also be used as an anonymous client. The choice is also difficult to reverse. Once a service has operated non-anonymously, its IP address may be linked to its `.onion` address. Simply disabling the option does not restore anonymity. Doing so requires a new address, with new keys and a new `HiddenServiceDir`, plus a clean instance.

21.4.3 Onionbalance

A single tor process is both a performance bottleneck and a single point of failure. Onionbalance [Theb] allows multiple servers to serve one address. Each server operates its own onion service with its own introduction points. Onionbalance holds the main address's key and publishes a descriptor combining introduction points from all the servers. Clients are distributed across those servers transparently. The Tor Project uses Onionbalance for its own websites.

Summary

- An onion service needs two configuration lines. Its secret key *controls* the address and must be protected and backed up.
- The server should listen only on localhost or a Unix socket and must not reveal information about itself through responses or external connections.
- Onion-Location advertises a website's onion version to Tor Browser visitors.
- Restricted discovery uses X25519 keys: without authorization, a client cannot even decrypt the descriptor.
- Vanity addresses, single-hop services, and Onionbalance address recognizability, speed, and scalability.

Exercises

Exercise 21.1 ⋆ How many attempts, on average, does it take to generate a six-character base32 prefix? What about eight characters? If a computer attempts ten million keys per second, how long would each take?

Exercise 21.2 ⋆⋆ An onion service displays `Apache/2.4.62 (Debian) Server at 192.0.2.10 Port 80` on its error pages. What information does this reveal? How can the leak be avoided?

Exercise 21.3 ⋆⋆ In Laboratory 13, how would you revoke Alice's access? What happens to clients that have already downloaded the descriptor?

Exercise 21.4 ⋆⋆⋆ Modify the laboratory service to forward connections to a Unix socket rather than a local TCP port. What additional benefits does this provide besides avoiding an exposed port?

Further Reading

- The tor manual entries for `HiddenServiceDir`, `HiddenServicePort`, and client authorization.
- Tor Project documentation on onion services and Onionbalance [Theb].

Chapter 22

Running a Relay or a Bridge

Last verified: 23 September 2026 — tor 0.4.9.13 (manual); laboratory network

Chapter objectives

By the end of this chapter, you will be able to:

- choose between a middle relay, exit, and bridge according to your resources and the risks you can accommodate;
- write the essential configuration for each, including bandwidth limits and offline key storage;
- read and write an exit policy and test its effects on the laboratory network (Laboratory 14);
- correctly declare a relay family.

The Tor network exists because thousands of people and organizations contribute bandwidth and computers. Earlier chapters examined relays from the client's perspective; here, we switch sides. This is not a system administration manual—the Tor Project maintains an up-to-date operator guide—but an explanation of what the main operational choices mean in protocol terms.

22.1 Which Type of Node?

Middle relay (which may eventually also become a guard): forwards cells between other relays but never opens connections to external Internet destinations. This is the least demanding role. Its address appears in the consensus, but websites do not see traffic coming from it.

Exit Opens connections to destinations on behalf of clients. This role is particularly valuable because exits are scarce relative to demand (section 13.2), but it also carries the greatest exposure: recipients of abusive traffic see the exit's IP address as the apparent source.

Bridge An unlisted relay used as the first hop by people unable to reach the public network (section 16.2). Even a modest residential connection can help when combined with a pluggable transport such as obfs4 or WebTunnel.

Warning: Before You Begin

Running an exit has legal and operational consequences, including abuse reports and requests from authorities, discussed in chapter 25. Consider starting with a middle relay or bridge, inform your connectivity provider, and avoid operating a relay on the same connection used as a Tor client for sensitive activities.

22.2 Essential Configuration

A middle relay needs only a few `torrc` lines:

```
Nickname        MyRelay
ContactInfo     operator@example.org
ORPort          443
ExitRelay       0
SocksPort       0
RelayBandwidthRate  4 MBytes
RelayBandwidthBurst 8 MBytes
```

`ContactInfo` lets others contact you when the relay has problems. The manual reminds operators that descriptors are archived and published, so this address may remain publicly accessible indefinitely. `ExitRelay 0` makes explicit what is already the default in recent versions: with `auto`, tor becomes an exit only when an operator writes an exit policy. `RelayBandwidthRate` and `RelayBandwidthBurst` cap average and peak bandwidth. `AccountingMax`, by contrast, caps total traffic over an accounting interval, such as a month. Once reached, the relay stops accepting new traffic until the next interval.

Operational note. With `ExitRelay` set to `auto`, specifying `ReducedExitPolicy` or `IPv6Exit` can also enable exiting: an explicit `ExitPolicy` is not the only trigger. By contrast, the example's `ExitRelay 0` disallows all exit traffic. Verify the effective configuration before deploying a public relay.

22.2.1 The Life of a New Relay

A freshly launched relay publishes its descriptor to the authorities, which add it to the consensus and start measuring it (section 12.4). It receives little traffic at first: clients select relays in proportion to their bandwidth weights, which remain low until measurements stabilize. Flags are earned over time (section 12.4.3). `Stable` requires extended uninterrupted operation, and `Guard` additionally requires sufficient age and high availability. Once it becomes a guard, a relay may initially see a decline in traffic: clients that have already selected their guards do not select it as a middle at the same rate. Traffic then grows gradually as new clients adopt it (section 13.3).

Two kinds of setbacks are not equivalent. Frequent restarts without changing identity preserve bandwidth measurements but reduce the sustained availability needed for `Stable` and `Guard`; these flags must be earned again. Changing the *identity keys*, however, creates an entirely new relay in the authorities' view, starting from zero even in bandwidth measurements.

22.2.2 Keys

A relay's identity is its primary Ed25519 key (section 7.2). By default tor stores it on the relay's disk. With `OfflineMasterKey 1`, the master key remains on another computer; only temporary signing keys, certified using `tor --keygen`, are transferred to the server. If the

server is compromised, the attacker obtains expiring keys, not the relay's long-term identity. This approach requires discipline: temporary keys must be renewed before expiration or the relay stops operating.

22.3 Exit Policy

An exit policy is an ordered list of `accept` and `reject` rules, each specifying an address or network and a port or range. Tor processes the rules **in order**, applying the first matching rule. Operator-defined rules precede the default rules listed in the manual. By default, ports used for email (25), NNTP (119), NetBIOS and SMB (135–139, 445), and certain file-sharing applications are rejected; remaining traffic is accepted. Separately, tor rejects private destinations by default, including `127.0.0.0/8`, `10.0.0.0/8`, and `192.168.0.0/16`, as well as the relay's own public address. This protects the operator's internal network (`ExitPolicyRejectPrivate`). Our laboratory disables that protection to allow traffic to `127.0.0.1`.

Protection rule ordering. When `ExitPolicyRejectPrivate` is enabled (the default), rules rejecting private and local destinations are *prepended* ahead of operator-defined exit policy rules. The private Chutney network deliberately disables this safety measure; do not copy that setting to a public exit.

An exit restricted to web traffic can use:

```
ExitRelay  1
ExitPolicy accept *:80
ExitPolicy accept *:443
ExitPolicy reject *:*
```

The Tor Project also suggests `ReducedExitPolicy` as a common compromise: it permits a selection of widely used service ports while excluding ports that generate more abuse complaints.

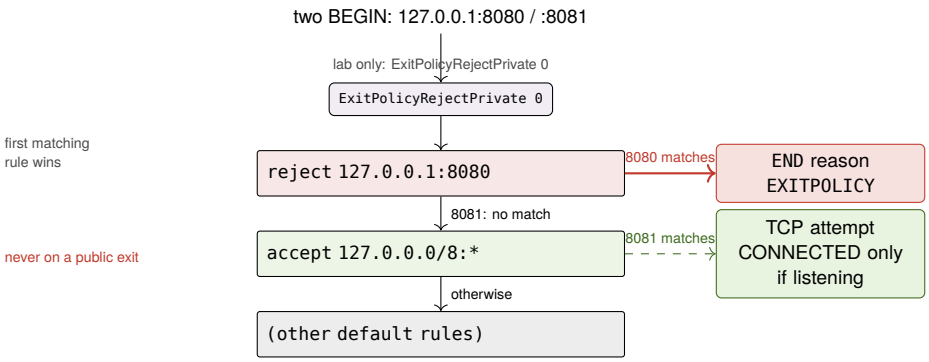


Figure 22.1: Chutney laboratory policy: `ExitPolicyRejectPrivate 0` is used solely to permit private-address destinations in the test network; normally Tor prepends their rejection before operator-defined rules. `BEGIN` to `127.0.0.1:8080` matches `reject` and yields `END EXITPOLICY`; port 8081 skips the rejection and matches `accept`. Acceptance permits a TCP attempt, but does not guarantee `CONNECTED` without a listening server. Reversing the rules makes the specific rejection ineffective.

What clients receive. The relay descriptor includes the complete policy, while the consensus and microdescriptors carry a port-only summary such as `p accept 80,443`. The client uses it to decide whether an exit may handle a request (section 13.1). Since that summary omits

addresses, the client may discover address-specific rejection only by trying the connection, as the laboratory shows.

22.4 Laboratory: Testing an Exit Policy

Laboratory 14: An exit rule and its consequences

Objective. Add a rule to a laboratory exit and use `labs/en/code/torlab` to observe responses to accepted and rejected destinations.
Prerequisites. Laboratories 3 and 4; the laboratory network running.
Estimated time. 1 hour.

Step 1: the rule. In `test006e`'s `torrc`, insert a rule rejecting one port before the existing rules allowing `127.0.0.0/8`:

```
ExitPolicy reject 127.0.0.1:8080
ExitPolicy accept 127.0.0.0/8:*
```

The listing shows the relative order of the two relevant rules. When changing an existing `torrc`, insert only the new `reject` before its first `ExitPolicy` line; do not duplicate an existing `accept`. Order matters. If placed after `accept`, our rejection would have no effect, because the first matching rule wins. That was the mistake recorded in the earlier reference experiment. After restarting the relay, start two small HTTP servers on container ports 8080 and 8081.

Step 2: three requests. Using `build` and `stream.py`, build a circuit terminating at `006e` and send three `BEGIN` messages to ports 8080, 8081, and 9999, where nothing is listening. The responses, in one of our sessions (codes, address, and TTL may vary with the version and environment):

```
127.0.0.1:8080 -> END  reason=EXITPOLICY address=127.0.0.1 TTL=1800
127.0.0.1:8081 -> CONNECTED
127.0.0.1:9999 -> END  reason=MISC
```

Step 3: interpreting the responses. Policy rejection produces the `EXITPOLICY` reason. The `END` cell also contains the resolved address and a TTL. The client can remember that this exit does not serve that destination and select another without waiting for a new descriptor. In contrast, the port with no listening server produces generic `MISC`, not `CONNECTREFUSED`, which the specification lists for refused connections [tor-spec § Closing streams].

In depth: Why `MISC` Rather Than `CONNECTREFUSED`?

The answer appears in the exit's `info` logs. Tracing it is a useful example of investigating a difference between a specification and its implementation:

```
[info] connection_handle_write_impl(): in-progress connect failed.
       Removing. (Connection refused)
[info] errno_to_stream_end_reason(): Didn't recognize errno 115 (Operation
       now in progress); telling the client that we are ending a stream
       for 'misc' reason.
```

The exit *knows* the connection was refused but derives the reason from a different value. Destination connections are nonblocking: `connect()` initially returns `EINPROGRESS` (errno 115), and the outcome becomes available later through a write event. At that point,

`connection.c` retrieves the real error with `getsockopt(SO_ERROR)`, logs "Connection refused", and calls `connection_edge_end_errno()`. That function, however, recalculates the reason using `errno_to_stream_end_reason(tor_socket_errno(…))`. On Unix, `tor_socket_errno` is defined as the macro `(errno)` (`src/lib/net/socket.h`), referring to the global variable that still contains `EINPROGRESS`. Value 115 is absent from the list in `src/core/or/reasons.c`, so the default case returns `MISC`.
The correct information exists but is not used. The two branches in that area of code also differ: the branch for relay-to-relay connections passes along the retrieved error, while the stream branch ignores it. According to the existing report and patch in `docs/it/segnalazioni/`, a previous private-network test of tor 0.4.9.13 using the retrieved error changed the third response to `CONNECTREFUSED`. This is a minor defect without anonymity implications: the client still knows the stream failed, but loses actionable information. `CONNECTREFUSED` would indicate that the exit reached the destination and retrying through another exit would be pointless. Analysis and a patch are available in `docs/it/segnalazioni/` for the Tor Project. This also demonstrates why specifications should be verified against real behavior.

The localized script is `labs/en/lab/exit_policy.py`. On a Linux host with the dedicated Chutney network running, launch it from the repository root with `PYTHONPATH=labs/en/code python3 labs/en/lab/exit_policy.py`. It performs three probes and attempts to restore `006e`'s original `torrc` in a `finally` block. Use it only on a disposable lab network: it temporarily changes the exit configuration and opens local HTTP servers. The `make -C labs/en/lab verify` target, by contrast, checks only the Chutney network and does not run this script. One important detail: the circuit must have three hops. An exit refuses to forward traffic from a single-hop circuit and would return `END` with reason `TORPROTOCOL` (section 15.3).

22.5 Relay Families

Operators running multiple relays must declare a **family**. Clients never place two relays from the same family in a circuit (section 13.1), because a common operator could correlate traffic. The historical mechanism is `MyFamily`: each relay lists the fingerprints of every other relay, and family membership counts only when the declarations are mutual. Maintaining ten relays therefore means maintaining ten matching lists, all of which must change whenever a relay is added.

The mechanism introduced with tor 0.4.9 instead uses a **family key**. The operator creates one with `tor --keygen-family MyFamilyKey`, copies the generated `MyFamilyKey.secret_family_key` file into each relay's key directory (`KeyDirectory`), retaining the `.secret_family_key` suffix, and adds the generated `FamilyId` configuration line to every `torrc`. Keep this secret file protected and backed up; if it leaks, rotate the family key across all member relays. Each relay uses that key to sign a certificate binding itself to the family. Verifiers no longer need to compare lists. During the transition, configuring both mechanisms is advisable.

In depth: Why Declaring a Family Helps

Nothing forces an operator to declare its relays as a family, and an adversary might deliberately avoid doing so. However, authorities and volunteers monitoring the network look for shared characteristics such as common networks, versions, simultaneous startup times, and contact details. An undeclared family can attract scrutiny and may lead to its

relays being excluded from the consensus. Declaration is part of the trust arrangement on which the network depends.

Do not include bridges in a public `MyFamily` list: doing so could compromise their obscurity. Tor's operator guide explicitly advises against setting that option on bridges. Also, `tor --keygen-family MyFamilyKey` may overwrite existing key files using that name. Choose a fresh basename and protect the resulting secret file.

22.6 Running a Bridge

An obfs4 bridge can be configured as in Laboratory 10 (section 16.5):

```
BridgeRelay 1
ORPort 9001
ServerTransportPlugin obfs4 exec /usr/bin/obfs4proxy
ServerTransportListenAddr obfs4 0.0.0.0:443
ExtORPort auto
BridgeDistribution any
```

`BridgeDistribution` tells the distribution system, currently rdsys, how the operator wants the bridge distributed. `any` leaves the choice to the system, while `none` excludes it entirely, which is useful for privately sharing a bridge with trusted people. Available channels, including the website, email, and Telegram, are listed on the bridge information page. A bridge does not appear in the public consensus, but its descriptor still goes to the bridge authority, which checks reachability.

The transport should listen on a port different from the ORPort and, where possible, on a familiar port such as 443. That is the port clients use and the censor sees (section 16.4).

Summary

- Running a middle relay is the least demanding way to contribute; an exit is especially valuable but carries more exposure; a bridge helps people facing censorship.
- Only a few options are essential: identity, contact details, ports, and bandwidth limits. The master identity key can remain offline.
- An exit policy is an ordered list in which the first matching rule wins; private destinations are rejected by default.
- Policy rejection reaches the client as an `END` message with reason `EXITPOLICY`, address, and TTL.
- Operators of multiple relays should declare them as a family using `MyFamily` or the newer family-key mechanism.

Exercises

Exercise 22.1 ⋆ Given the policy `accept *:80`, `reject 192.0.2.0/24:*`, `reject *:*`, can a client connect through this exit to `192.0.2.7:80`? Rewrite the policy so that it cannot.

Exercise 22.2 ⋆ Why is the port-only summary in the consensus insufficient to determine whether an exit will accept a connection? What does the client do after receiving `END` with reason `EXITPOLICY`?

Exercise 22.3 ⋆⋆ An operator manages five relays using `MyFamily` and adds a sixth but updates the configuration only on the new relay. What do clients see? What changes with a family key?

Exercise 22.4 ⋆⋆ Verify the explanation in the technical discussion: enable `info` logs on the exit and repeat the three requests. What would change if `connection_edge_end_errno()` received the error returned by `getsockopt`? Which other stream-closure reasons could then be reported correctly?

Exercise 22.5 ⋆⋆⋆ A relay has `AccountingMax 1 TBytes` per month and 10 MB/s bandwidth. After how many days of full-speed operation does it exhaust its allowance? How does a relay that remains offline for half the month affect the network compared with one running all month at half the bandwidth?

Further Reading

- The Tor Project's relay operator guide, `https://community.torproject.org/relay/`.
- The tor manual entries for `ExitPolicy`, `ReducedExitPolicy`, `FamilyId`, `OfflineMasterKey`, and `BridgeDistribution`.
- The specification for stream-closure reasons, [tor-spec § Closing streams].

Chapter 23

Tor Browser, Tails, and Whonix

Last verified: 23 September 2026 — Tor Browser design document (2018); Tor Browser manual; Tails and Whonix documentation

Chapter objectives

By the end of this chapter, you will be able to:

- explain why configuring an ordinary browser to use Tor is insufficient;
- list the security and privacy requirements behind Tor Browser's design;
- connect Tor Browser's per-site isolation with tor's stream isolation;
- compare what Tor Browser, Tails, and Whonix protect and the costs of each approach.

The 2004 threat model explicitly stated that Tor does not normalize application protocols (section 4.2). If the browser reveals your identity, onion routing cannot save you. This chapter examines tools designed to address precisely that layer: Tor Browser, which makes the browser consistent with the network's protections, and two operating systems, Tails and Whonix, which extend the approach to the whole computer.

23.1 Why a Proxy Is Not Enough

Configuring an ordinary browser to use tor's SocksPort hides the IP address in many circumstances, but not all, and conceals little else:

- Some components may connect outside the proxy: local DNS resolution, WebRTC, extensions, or external programs launched from the browser.
- Cookies and other browser state link today's visits to yesterday's, perhaps made without Tor.
- A site can collect dozens of browser characteristics—window size, installed fonts, language, time zone, rendering details—and combine them into a **fingerprint** distinguishing one user from millions.
- If all tabs use the same circuit, a tracker embedded in several websites may observe requests from the same exit at the same time.

The first issue threatens the address; the others threaten *unlinkability* between activities, one of Tor's central goals.

23.2 Tor Browser

Tor Browser is a modified version of Mozilla's extended-support Firefox branch, distributed by the Tor Project together with tor. Its design is described in a document that, although no longer current in every detail, remains an important statement of its principles [Per+18]. It separates requirements into two groups.

Security requirements. These concern safe use of Tor; violating them may expose a user immediately:

Proxy obedience No content may bypass tor.

State separation No information may be shared with other browsers or sessions outside Tor.

No persistent traces Nothing revealing browsing activity should remain on disk or in memory beyond the session unless the user explicitly chooses to retain it.

Application isolation Anything the browser leaves on the system must be removable by deleting it.

Privacy requirements. These prevent different activities from being linked and are organized around the **address-bar origin**, the site the user is visiting:

Identifiers Data stored by a third party, such as a tracker embedded on several sites, while visiting one site must not become available to that party when it appears on another.

Fingerprints Browser characteristics must not permit linkage across different websites.

Long-term unlinkability A user must be able to clear all state and obtain a new identity through one simple action.

The document also offers a useful design principle: protections should not undermine how users expect a browser to behave. Tor Browser's predecessor, the Torbutton extension, let users toggle Tor within the same browser. Users confused tabs opened in different states. An interface that induces mistakes is itself a security weakness.

23.2.1 One Circuit per Site

Origin isolation extends to the network. Tor Browser identifies the address-bar site in the SOCKS credentials it sends to tor. By default, tor isolates streams carrying different credentials (`IsolateSOCKSAuth`) onto different circuits. We verified this mechanism in Laboratory 11 (section 18.3). From the user's perspective, as described in the manual [Thed], two websites embedding the same tracker use distinct circuits, while tabs showing the same site share one.

Two commands act on this arrangement:

New Circuit for This Site Reloads the current site over a new circuit, for example when an exit cannot reach it. It does not clear browser state or change other sites' circuits.

New Identity Closes all tabs, clears cookies, history, and other state, and obtains new circuits for all activity, implementing the long-term unlinkability requirement.

Neither action changes the guard, which remains stable for months for the reasons explained in section 13.3.

Per-site isolation is a *separation between sites*, not a promise that each site has exactly one circuit throughout a session: circuits age out and can be replaced. "New Circuit for This Site"

reloads the active tab; other tabs at that site will adopt the replacement circuit when reloaded, while existing connections are not necessarily interrupted. "New Identity" also stops ongoing activity and downloads. Neither option is intended to rotate the entry guard; tor manages that separately.

23.2.2 Resisting Fingerprints

Tor Browser's strategy against fingerprinting is not to make every user different, but to make users look *alike*. If all Tor Browser users expose the same values, a fingerprint says little more than "this is a Tor Browser user". Several visible design choices follow. Windows open at rounded sizes, and resizing adds margins around page content to keep the reported dimensions in standardized increments (*letterboxing*). The reported time zone is UTC. Many interfaces exposing operating-system details are disabled or standardized. This also explains why installing extensions or changing settings is discouraged: every difference shrinks the crowd in which a user can hide, the same anonymity-set principle discussed in chapter 1.

The objective is to *minimize observable differences*, not to make every fingerprint literally identical. Platform, language, security level and other attributes can still place users in distinguishable groups. The result is a reduction in uniqueness, not a guarantee of perfect uniformity.

23.2.3 Security Levels

Many browser vulnerabilities exploit JavaScript and complex features. Tor Browser provides three levels [Thed]:

Standard All features are enabled; this is the default.

Safer JavaScript is disabled on non-HTTPS sites; some fonts and mathematical symbols are disabled; audio and video start only after a click.

Safest JavaScript is disabled everywhere, and images and other content may also be restricted. Static websites remain usable.

Choosing a level balances security and usability. The choice can itself affect fingerprinting, since a website may infer the selected level.

In depth: Reproducible Builds

People downloading Tor Browser need confidence that the distributed binary corresponds to the published code. The Tor Project produces *reproducible builds*: using the same source code and build environment should yield bit-for-bit identical files. The design document describes the work required to eliminate sources of differences, including dates, file paths, and archive member ordering [Per+18]. An independent rebuild and comparison of the outputs can detect malicious changes inserted into a distribution build environment.

23.3 Beyond the Browser

Tor Browser protects its own traffic, not that of unrelated applications on the same computer. Those applications can still connect directly, and a mistake or exploited browser vulnerability may reveal the real address. Two operating systems extend protection to the whole machine through distinct architectures (fig. 23.1).

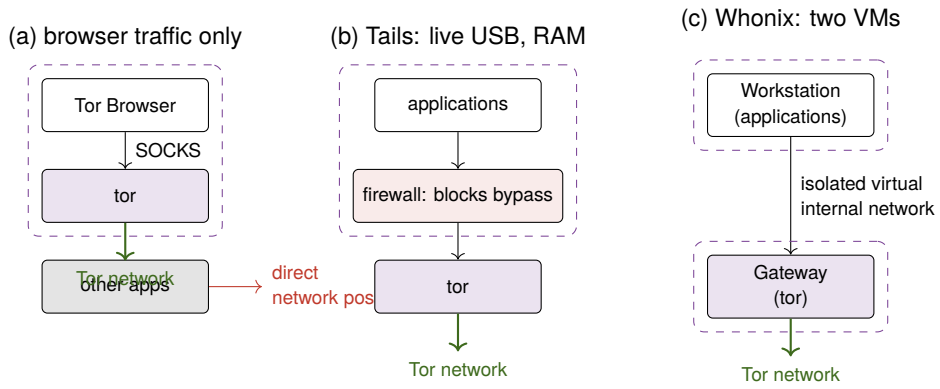


Figure 23.1: Three routing and isolation models, not absolute anonymity guarantees. (a) Tor Browser routes its own traffic over tor, but unrelated applications on the host may connect directly. (b) Tails normally enforces Tor routing with firewall rules that block application bypass; its *Unsafe Browser* for captive portals is an explicit non-anonymous exception. Tails runs as a live USB system, mainly in RAM, with optional persistent storage. (c) Whonix separates application Workstation and tor Gateway into two VMs on an isolated virtual network; compromise of the host or hypervisor can still defeat that isolation.

23.3.1 Tails

Tails is a Debian-based operating system booted from a USB drive [Taia]. It has two defining properties:

Everything through Tor Applications attempting to reach the Internet without tor are blocked.

Amnesia The system runs in memory, does not write to the computer's disk, and forgets its state at shutdown. Data the user chooses to retain is stored in optional encrypted *persistent storage* on the USB drive.

Tails includes Tor Browser and other cautiously configured applications. The project joined the Tor Project in September 2024 [The24c]. Its documentation is explicit about limits: Tails cannot protect against tampered hardware, such as a physical keylogger, or against a USB drive prepared on an already infected computer.

These are properties of Tails under its normal operating assumptions, *not an absolute guarantee* that no storage device can ever hold traces. Users may explicitly access disks or create persistent files, while malware, compromised hardware and activity outside Tails can also undermine this threat model.

23.3.2 Whonix

Whonix separates its functions into two virtual machines. The **Gateway** runs tor and is the only virtual machine connected to the external network. The **Workstation** runs applications and connects only to the Gateway through an internal virtual network. All Workstation traffic must therefore pass through tor by architecture rather than merely by application configuration. Compared with Tails, this provides stronger separation: even an attacker gaining administrative control over the Workstation cannot directly learn the real address, because that machine does not know it. The trade-offs are dependence on the virtualization software's security and the absence of amnesia unless Whonix runs inside a system that provides it.

Separating the Gateway and Workstation is a valuable boundary, *not an absolute guarantee* against a compromised Workstation. An attacker may still reveal the user's identity through application data; compromise of the host, hypervisor, or Gateway could also defeat the network boundary. Firewall integrity and deployment configuration remain essential.

A filtered control port. Some Workstation applications, including Tor Browser's New Identity function, need to communicate with tor's ControlPort on the Gateway. Yet the control protocol (section 18.1) also allows requests for tor's public address using `GETINFO address`, precisely what the Workstation should not learn. Whonix therefore inserts *onion-grater*, a proxy allowing only explicitly permitted commands [Who]. A feature harmless in one context can become a leak in another.

Warning: Tools Cannot Replace Good Habits

Tor Browser, Tails, and Whonix reduce possible technical mistakes but not behavioral ones: signing into a personal account, opening downloaded documents outside a protected environment, or disclosing identifying details. Chapter 28 addresses that layer.

Summary

- An ordinary browser behind Tor may leak traffic outside its proxy and, more importantly, link activities through shared state, fingerprints, or circuits.
- Tor Browser follows explicit design requirements: proxy obedience, separation of state, limited persistence, unlinkability between sites, and an accessible New Identity function.
- Per-site isolation relies on SOCKS credentials and tor's stream isolation.
- Tor Browser counters fingerprinting by standardizing users' observable characteristics; security levels reduce the attack surface.
- Tails forces all traffic through Tor and avoids persistent traces; Whonix confines applications inside a machine that does not know the real network address.

Exercises

Exercise 23.1 ⋆ In terms of circuits and browser state, what is the difference between "New Circuit for This Site" and "New Identity"? Why does neither change the guard?

Exercise 23.2 ⋆ A user installs an advertisement-blocking extension in Tor Browser, believing it will improve privacy. What might the user gain and lose?

Methodological note. Laboratory 11 simulates distinct SOCKS credentials; it does not create three real Tor Browser tabs. Matching credentials may reuse a circuit but are not guaranteed to do so at every moment, because circuits age out and other isolation rules can apply.

Exercise 23.3 ⋆⋆ Using the laboratory network and Laboratory 11's script, simulate three tabs: two visiting the same site and one visiting another, with the site name as the SOCKS credential. Check through the ControlPort which streams share a circuit.

Exercise 23.4 ⋆⋆ For each scenario, identify which tool among Tor Browser, Tails, and Whonix fits the stated needs, and explain your reasoning: (a) someone browsing censored websites

from their home computer; (b) a journalist processing confidential documents on other people's computers; (c) a researcher running several long-lived programs that must all use Tor.

Exercise 23.5 ⋆⋆⋆ Choose five browser properties that websites can read through JavaScript and estimate how many bits of information each exposes in an ordinary browser. How does Tor Browser standardize them, and which differences remain among users?

Further Reading

- M. Perry, E. Clark, S. Murdoch, G. Koppen, *The Design and Implementation of the Tor Browser* [Per+18].
- The Tor Browser manual [Thed].
- Tails documentation [Taia] and Whonix documentation, especially the onion-grater page [Who].

Chapter 24

Measuring the Network: Tor Metrics and OONI

Last verified: 23 September 2026 — Tor Metrics data through 20 September 2026; consensus of 23 September 2026 at 15:00; OONI specifications ts-023 and ts-030

Chapter objectives

By the end of this chapter, you will be able to:

- describe the data the Tor Project publishes and where they come from;
- explain how to estimate user numbers on a network designed not to know its users, and how uncertain such estimates are;
- extract network statistics from a real consensus (Laboratory 15);
- explain how OONI measures Tor blocking from users' perspectives.

An anonymity network poses a curious problem for its operators: improving it requires measurement, but measurement means gathering information about precisely what it is supposed to protect. The Tor Project's approach is to measure sparingly, aggregate and publish results, and document every step so anyone can reproduce the calculations [Thec]. This chapter follows the data from their origins to the charts, then looks at the network from the outside through OONI's measurements.

24.1 Data Sources

Almost everything publicly known about the Tor network comes from documents we have already encountered:

Consensus and descriptors (chapter 12): the relay list, flags, bandwidth weights, versions, and exit policies.

Extra-info descriptors Published by each relay alongside its descriptor, these contain historical byte-transfer counts and aggregate statistics, including directory-request counts broken down by country.

Active measurements OnionPerf periodically downloads files through Tor from selected locations worldwide, recording transfer times and failures.

CollecTor archives these documents, going back to the network's early years, and makes them downloadable. **Tor Metrics** derives charts and tables from them, accompanied by

downloadable CSV data and documented methods. **Relay Search** supports queries about individual relays. The consensus used in this book's laboratories (`fonti/collector`) came from CollecTor.

24.2 Counting Anonymous Users

Tor does not register users, and no relay can tell whether two connections belong to one person. How, then, can the network estimate that it has roughly three million users? The method is indirect (fig. 24.1). Every client periodically downloads a consensus, and directory requests can be counted without identifying requesters. Directory-serving relays count successful requests each day, grouped by country inferred from the source address. They publish aggregate totals in extra-info descriptors, rounding country counts upward to multiples of eight so that small, precise counts disclose less information [dir-spec § Extra-info document format].

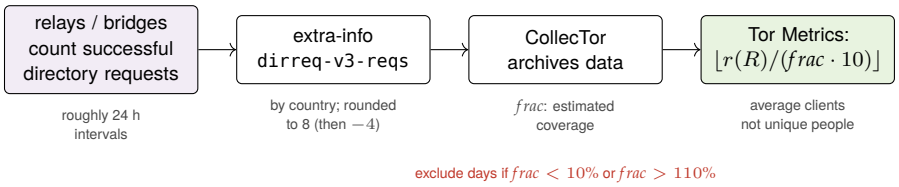


Figure 24.1: From successful directory requests to an indirect estimate of *average connected clients*, not distinct people. Directory-serving relays and bridges provide separate aggregate series in extra-info descriptors, including country counts in `dirreq-v3-reqs`, rounded upward to multiples of 8. CollecTor archives these data; Tor Metrics corrects binning by subtracting 4 from valid counts, estimates the reporting coverage $frac$ and applies $\lfloor r(R)/(frac \cdot 10) \rfloor$. The divisor 10 is an assumption about average requests per client per day. Dates with $frac < 10\%$ or $frac > 110\%$ are excluded.

Tor Metrics collects these figures and applies three steps [Thec]:

1. Account for rounding by subtracting half the rounding interval from each value.
2. Estimate the **fraction**, $frac$, of network-wide directory requests covered by reporting relays, accounting for how long each relay remained operational and how much directory traffic it served.
3. Divide reported requests by that fraction to estimate the network total, then divide by 10:

$$r(N) = \left\lfloor \frac{r(R)}{frac \cdot 10} \right\rfloor .$$

The divisor 10 is an assumption. A continuously active client makes about 15 requests daily, but many clients are online for only part of the day, and the Tor Project chose 10 as the average. Estimates with a reporting fraction below 10 % or above 110 % are discarded.

Warning: What "Users" Means

The result estimates the number of *connected clients on average*, not individual people. A client online for two hours daily generally generates fewer requests than one online around the clock, but the ratio is not fixed; multiple programs running Tor on one machine count as multiple clients. Changes in software behavior, such as fetching consensuses

less frequently, alter the estimate even when the number of people remains unchanged. Charts are most useful for interpreting *changes*, not absolute numbers.

The numbers at the book's reference date. From 1 August through 20 September 2026, estimated daily directly connected users fluctuated between 2.8 and 3.5 million, with reporting fractions between 57 % and 73 %. The Italian average was approximately 60 000, but daily results carry wide expected intervals: on 20 September, for example, about 34 000 to 111 000. Changes within ranges like these do not, by themselves, demonstrate an event. On that same day, estimated bridge users numbered about 98 000 for obfs4, 60 000 for Snowflake, and 35 000 for WebTunnel.

24.2.1 Detecting Censorship

Metrics uses the same time series to flag potential censorship events. For countries with substantial user counts, it compares each country's week-on-week change with changes typical of other countries and highlights dates when that country falls outside the expected interval [Thec]. A sudden drop in direct users accompanied by a rise in bridge users is a characteristic sign of blocking. It is a clue, not proof. Confirmation requires measurements from inside the country, which is OONI's role.

24.3 Performance

OnionPerf measures, from fixed locations, the time required to download files of various sizes over Tor, failures, and circuit latency: the interval between sending an HTTP request and receiving the response headers [Thec]. On 20 September 2026, median latency to a public server was about 110–140 ms from the German measurement point, 280 ms from the US point, and more than 500 ms from Hong Kong. Geographic distance from the network's main concentration in Europe and North America matters. Some measurement points use Conflux, the multipath system described in chapter 31, allowing comparisons under similar conditions.

24.4 Laboratory: Reading a Real Consensus

Laboratory 15: Deriving network statistics from a consensus

Objective. Extract key network statistics from a real CollecTor consensus using a new module, `labs/en/code/torlab/metrics.py`.
Prerequisites. Laboratory 6 (`directory.py`) and Laboratory 7 (`path_selection.py`); no running network required.
Estimated time. 1 hour.

The module adds a `summary` function reusing Laboratory 6's consensus parser, along with the two user-estimation functions. Counts are also sorted by name when values tie. Without this precaution, their order would change across runs according to Python's set iteration order. For the historical consensus of 23 September 2026 at 15:00, first prepare the snapshot from the repository root with `make -C labs/en data` (this preparation needs Internet access). Applied to that consensus (output abbreviated):

```
$ PYTHONPATH=labs/en/code python3 -m torlab.metrics labs/en/data/collector/consensus-
    microdesc-2026-09-23-15
relay: 9358
flag: {'Running': 9358, 'Valid': 9358, 'Fast': 8961, 'Stable': 8199,
       'V2Dir': 7674, 'Guard': 5306, 'HSDir': 4050, 'Exit': 3177,
       'BadExit': 32, 'MiddleOnly': 28, 'StaleDesc': 21, 'Authority': 10}
total_bandwidth: 198927900
exit_bandwidth_share: 0.3308947161257923
guard_bandwidth_share: 0.7010271560701139
versions: [('Tor 0.4.9.12', 6027), ('Tor 0.4.9.11', 2520), ...]
networks16: [('64.65', 771), ('192.42', 636), ('204.8', 598), ...]
```

A few observations:

- The 9,358 relays are consistent with the Tor Metrics chart, which reports 9,416 relays and 2,333 bridges on 20 September.
- Roughly one-third of relays have the `Exit` flag and account for roughly one-third of measured bandwidth; guards account for 70 %. These fractions overlap because a relay can hold both flags. The imbalance is addressed by the bandwidth weights discussed in section 13.2.
- `BadExit` (32 relays) and `MiddleOnly` (28) reflect authority decisions about suspect or misconfigured relays.
- Ten relays carry the `Authority` flag, but only nine authorities vote on the consensus (section 6.1). The tenth is `Serge`, the bridge authority, which has the flag but does not vote.
- Most relays run recent 0.4.9 versions.
- Several /16 networks host hundreds of relays, typically large exit operators. The one-relay-per-/16 rule for circuit selection (section 13.1) limits reliance on any single network.

The sum of consensus bandwidth weights represents bandwidth units measured by the authorities, not actual transferred traffic. To estimate traffic, Metrics uses byte histories in extra-info descriptors.

24.5 OONI

The *Open Observatory of Network Interference* (OONI) measures Internet censorship from users' perspectives. Volunteers worldwide install OONI Probe, which periodically runs tests and publishes open results. Some tests specifically concern Tor [OON]:

`tor` Checks reachability of selected directory authorities and bridges. Tests go beyond opening a TCP connection and perform protocol-specific actions, such as an obfs4 handshake.

`torsf` Attempts to start tor using Snowflake, recording whether bootstrap succeeds and how long it takes.

The test specifications contain safeguards to prevent publication of private bridge addresses: including them in results would hand them to censors. Metrics and OONI complement each other. Metrics may show a decline in users from a country; OONI can investigate whether relays or bridges were actually unreachable from within that country and identify the blocking technique.

In depth: Measure Without Harming

Every statistic a relay publishes is a trade-off. Greater detail makes data more useful but increases the risk of disclosing information. This is why counts are rounded and aggregated by day and country. Network research follows the same principles (section 19.1): collect the minimum necessary, aggregate before publication, and do not retain unnecessary data.

Summary

- Network data originate from consensuses, descriptors, and extra-info documents archived by CollecTor and from OnionPerf active measurements. Tor Metrics analyzes them using documented, reproducible methods.
- Users are estimated from directory requests: reported requests divided by the covered fraction and then by 10. This estimates average connected clients and is especially useful for studying changes.
- The same data can suggest possible censorship events that require confirmation through local measurements.
- A real consensus provides relay counts, flags, bandwidth shares, versions, and network-level concentration.
- OONI measures authority and bridge reachability and Snowflake operation from within countries.

Exercises

Exercise 24.1 ⋆ If all clients started downloading consensuses half as often, what would happen to the user estimate, assuming the number of people stayed constant? How might someone interpreting the charts notice the change?

Exercise 24.2 ⋆ On one day, reporting relays recorded twenty million requests, and the estimated reporting fraction was 60 %. How many users would Metrics estimate? What if the fraction were 8 %?

Exercise 24.3 ⋆⋆ Extend `summary` to compute the bandwidth share of relays carrying both `Guard` and `Exit`, and the overall weighted probability that a circuit's exit belongs to one of the five most represented /16 networks.

Exercise 24.4 ⋆⋆ Download direct-user and bridge-user time series from Metrics for a country with a documented Tor-blocking event. Describe what happens to the two series on the dates of the block.

Exercise 24.5 ⋆⋆⋆ Country-level counts are rounded upward to multiples of eight. Describe an attack made more difficult by this rounding, and explain why it does not protect against an adversary controlling many relays.

Further Reading

- The Tor Project's *Reproducible Metrics* page [Thec], explaining the methods behind individual charts.

- OONI test specifications [OON] and OONI Explorer, `https://explorer.ooni.org/`.
- The extra-info descriptor format, [dir-spec § Extra-info document format].

Chapter 25

Legal and Ethical Aspects in Italy and the EU

Last verified: 23 September 2026 — Regulation (EU) 2022/2065; Legislative Decree 196/2003 (consolidated text published by the Italian Data Protection Authority); Tor Research Safety Board guidelines

Chapter objectives

By the end of this chapter, you will be able to:

- distinguish the legality of a tool from the legality of how it is used;
- interpret the European "mere conduit" rule in light of how a Tor exit operates;
- identify unresolved questions facing relay operators in Italy, including liability, data retention, and seizures;
- apply the Tor Research Safety Board's safety principles to research on Tor.

Warning: This Chapter Is Not Legal Advice

Law changes, depends on the facts of each case, and has produced relatively few decisions specifically concerning Tor. This chapter frames relevant issues and identifies legislation examined in its official text, while making uncertainty explicit. Anyone making an operational decision, such as running an exit, should seek advice from a lawyer specializing in technology law.

25.1 A Tool and Its Uses

In Italy, as elsewhere in the European Union, no law generally prohibits using Tor or other tools that protect the confidentiality of communications. On the contrary, confidentiality of correspondence and other communications (Article 15 of the Italian Constitution) and freedom of expression (Article 21) are fundamental rights, as are respect for private life and protection of personal data under Articles 7 and 8 of the EU Charter of Fundamental Rights.

What people do through Tor remains subject to the same laws that apply without Tor. Unauthorized access to a computer system (Article 615-*ter* of the Italian Criminal Code) does not become lawful because it passes through three relays. The distinction between a tool and its use is central to this chapter and also matters for those who provide the infrastructure: relay operators provide transmission capacity, like connectivity providers, rather than selecting the content being transmitted.

25.2 Relay Operators

25.2.1 Mere Conduit

The European *Digital Services Act* (DSA), applicable from 17 February 2024, replaced the intermediary-liability rules originally established by the 2000 E-Commerce Directive, which Italy implemented through Legislative Decree 70/2003 [22]. Article 4 addresses **mere conduit**: a service transmitting information provided by others is exempt from liability for that information if it does not initiate the transmission, select its recipient, or select or modify the transmitted information. Figure 25.1 compares those conditions with the behavior of a Tor exit at the cell-protocol level.

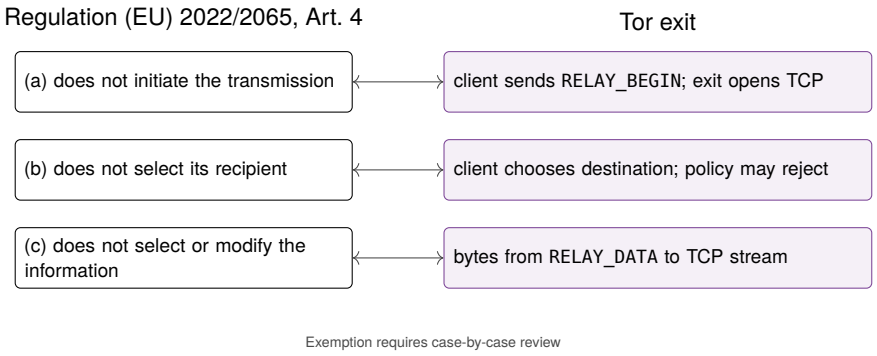


Figure 25.1: Technical comparison, not an automatic legal conclusion, of Article 4's *mere conduit* criteria under Regulation (EU) 2022/2065 and Tor exit behavior. The client specifies the destination using `RELAY_BEGIN`; the exit opens a TCP connection and may reject it under its exit policy. Application bytes carried by `RELAY_DATA` are transferred to the TCP stream, not forwarded as intact Tor cells. Any exemption depends on the actual service and all applicable legal requirements.

Two further provisions matter. Article 8 rules out a *general* obligation to monitor transmitted information or actively seek evidence of unlawful activity: an operator is not generally required to inspect traffic indiscriminately; a middle relay cannot read the encrypted application payload in plaintext. Article 4(3), however, preserves the power of judicial or administrative authorities to order service providers to prevent or terminate infringements.

The open question. The regulation applies to "information society services", which EU law defines as services *normally provided for remuneration*. Does a freely operated volunteer relay fall within that definition? There are arguments for a broad interpretation, and the exemption reflects a principle that a court could consider: someone who merely carries information should not be liable for its content. However, the Italian legal position concerning volunteer Tor relays has not been established here through a directly applicable judicial ruling. This uncertainty helps explain why some exits are run by associations or organizations that already provide network services.

25.2.2 Civil Liability and Criminal Investigations

The exemption concerns liability for transmitted information. It does not prevent a criminal investigation from beginning with an exit IP address. From a victim's perspective, an offense committed through Tor appears to originate from the exit. Investigators may initially search or seize an operator's computers. The Electronic Frontier Foundation's legal FAQ for relay

operators, written under US law but useful as a record of experience, says it knows of no operator convicted solely for running a relay but describes investigations and seizures based on misunderstandings of Tor [Ele]. Several precautions may reduce practical risk:

- Run the exit on a dedicated server in a data center, not at home or on a personal work computer.
- Make clear that the IP address belongs to an exit, for example through descriptive reverse DNS and an information page that tor can serve on its DirPort using `DirPortFrontPage`.
- Inform the connectivity provider in advance and agree on how abuse reports will be handled.
- Explain Tor in responses to reports and retain correspondence.

25.2.3 Data Retention

Article 132 of Italy's Personal Data Protection Code requires covered *providers* to retain telematic traffic data for twelve months for criminal investigation and prosecution [03], versus twenty-four months for telephone traffic. Paragraph 5-*bis* preserves a special rule: Article 24 of Law No. 167 of 20 November 2017 extends the period to **seventy-two months** for investigations concerning offenses identified in Articles 51(3-*quater*) and 407(2)(a) of the Code of Criminal Procedure, including terrorism and other serious crimes [17]. The relevant part of the Data Protection Code applies to electronic communication services *available to the public over public communications networks* (Article 121).

Whether a volunteer relay operator qualifies as such a provider is another unresolved question, also depending on how the service is offered. Operators, especially organizations that already provide communications services, should have a lawyer assess it before running an exit. Technically, under recommended logging settings tor does not ordinarily keep a history of users' destinations. An exit can see the destinations to which it forwards connections, but not the originating client's IP address. The data actually available depend on the relay's role and configuration: an exit should not be assumed capable of identifying individual Tor users.

25.2.4 Logs Are Personal Data

IP addresses are personal data. The General Data Protection Regulation (GDPR) requires processing only data that are necessary and retaining them only as long as necessary (Article 5, the data-minimization principle). For a relay, the security-aligned choice is to log only at `notice` level, keep `SafeLogging` enabled, and avoid packet captures (section 20.4).

25.3 Researchers

Research on Tor has a history of ethically questionable experiments, some discussed in chapter 27. Researchers have operated relays that monitored or manipulated real users' traffic to demonstrate attacks. Beyond ethical concerns, such experiments may engage Italian criminal law, including provisions on unlawful interception of computer communications (Article 617-*quater* of the Criminal Code), and data-protection rules.

The **Tor Research Safety Board** is a group of researchers who advise on experiment safety and have developed guidelines [Tor]:

- Use a test network wherever possible, and attack only your own traffic.

- Collect only the data needed and only data that would be safe to publish.
- Protect collected data and experimental tools.
- Reduce data precision with rounding or noise.
- Consider third-party datasets that could expose information when combined with your own.
- Ask whether users expected the relevant data to remain private.
- Ensure that anticipated benefits justify the risks.

The Board asks researchers to explain what they want to learn, how they intend to do it, what risks the experiment introduces, and why its benefits justify those risks. This book follows the same discipline: laboratories generating active Tor traffic use a private network (section 19.1), while public-document analyses such as those in chapter 24 run offline on publicly available aggregate data.

Summary

- Using Tor is lawful in itself; activity carried out through Tor remains governed by ordinary law.
- Article 4 of the DSA exempts qualifying services that merely transmit others' information if certain conditions are met. The way an exit works is relevant to those conditions, but whether a volunteer relay qualifies for the regulation remains an open legal question.
- The exemption does not prevent investigations or seizures. Dedicated servers, clear exit identification, and arrangements with connectivity providers can reduce practical problems.
- Data-retention requirements apply to covered publicly available communications providers; whether and how they apply to volunteer relays requires case-specific legal assessment.
- Research on Tor should avoid experiments involving real users wherever possible: use test networks, minimize data, publish only safe aggregates, and assess benefits against risks.

Exercises

Exercise 25.1 ⋆ For each of the three conditions in DSA Article 4, describe a change in exit behavior that would breach it.

Exercise 25.2 ⋆ Use an example to explain the distinction between the legality of a tool and the legality of its uses, applying it to both a Tor user and a relay operator.

Exercise 25.3 ⋆⋆ Draft a response to an abuse report received from an exit's connectivity provider after a website complains of repeated access attempts from your IP address. What can you explain, and what practical steps can you take?

Exercise 25.4 ⋆⋆ A researcher proposes counting domain names seen at an exit to estimate how many Tor users visit Italian news websites. Evaluate the proposal using the Tor Research Safety Board guidelines and suggest a safer alternative.

Exercise 25.5 ⋆⋆⋆ Research judicial decisions involving Tor relay operators from courts in Italy and other European countries. Summarize the legal question and outcome for each, citing primary sources.

Further Reading

- Regulation (EU) 2022/2065, particularly Articles 4, 8, and 9 [22].
- Legislative Decree 196/2003, Articles 121 and 132, in the consolidated text published by the Italian Data Protection Authority [03].
- Electronic Frontier Foundation, *Legal FAQ for Tor Relay Operators* [Ele].
- The Tor Research Safety Board website [Tor].

Part IV

Attacks and Defenses

Chapter 26

Traffic Correlation, Fingerprinting, and Sybil Attacks

Last verified: 23 September 2026 — papers cited in this chapter; tor 0.4.9.13; laboratory network

Chapter objectives

By the end of this chapter, you will be able to:

- classify attacks against Tor anonymity by the adversary's vantage point;
- explain end-to-end traffic correlation and measure it on your own traffic in the laboratory network (Laboratory 16);
- describe website fingerprinting and the limitations of its evaluations;
- recognize Sybil attacks and the network's defenses, from guards to bandwidth measurements.

The threat model in chapter 4 made clear that Tor does not withstand an adversary observing both ends of a communication. Twenty years of research have quantified that warning and demonstrated that observation points are more numerous and easier to reach than they might first appear. This chapter looks at attacks from the defender's perspective: what an attack requires, what it achieves, and which of the protocol mechanisms we have studied make it harder.

26.1 The Adversary's Position

It is useful to classify attacks according to what the adversary can observe:

One end The client-to-guard connection, observed by a connectivity provider, local network, or censor; or the exit-to-destination connection. On the client-to-guard link the adversary sees timing, volume, and direction of encrypted traffic. Between an exit and its destination, application-layer encryption depends on the protocol in use. Website fingerprinting can exploit client-side metadata without reading content.

Both ends An adversary observes both entry and exit, either by operating relays at both positions or by controlling networks on the paths between them. This enables correlation.

Inside the network Hostile relays, including middle relays, can measure, delay, mark, or discard cells.

Above the network The underlying Internet, including autonomous systems (ASes) and Internet exchange points (IXPs) traversed by relay connections.

26.2 End-to-End Correlation

Tor deliberately avoids long mix-network-style delays or reordering of cells to remain usable (section 2.2). Consequently, the *shape* of traffic—how many bytes pass during each interval—often remains sufficiently recognizable at both ends despite network delay, buffering, multiplexing, and added traffic (fig. 26.1). Someone observing both ends can compare the shapes and decide whether the flows match without breaking any encryption.

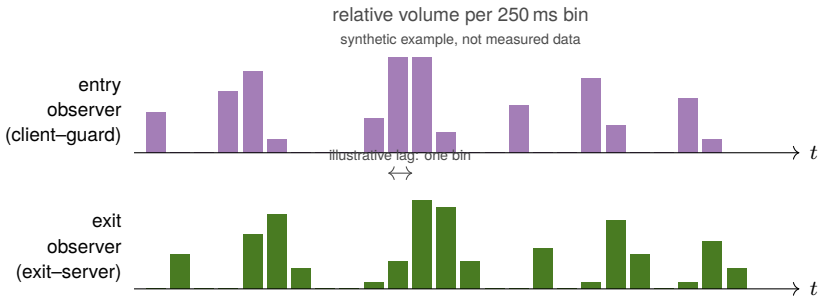


Figure 26.1: Two *synthetic* traces, not actual measurements, of traffic volume in one direction on client–guard and exit–server links. Bursts resemble one another but are not identical; the illustrated one-bin 250 ms lag is not a fixed Tor delay. Encryption conceals content without necessarily concealing timing and volume. Buffering, overhead, multiplexing and competing traffic can change both patterns; statistical similarity alone does not prove a match.

How Easy Is Matching? In 2018, DeepCorr trained a neural network to recognize pairs of Tor flows. With around 900 packets per flow, it achieved 96 % accuracy compared with 4 % for the earlier statistical method under the same conditions [NBH18]. For the adversary, the hard part is usually not comparing flows but occupying the right vantage points.

How likely is the adversary to observe both ends? Johnson and colleagues simulated typical users for months with Tor's actual path-selection algorithm [Joh+13]. With malicious relays of moderate bandwidth, 80 % of users across the user types they modeled were deanonymized at least once within six months. Against a single AS, users in some common locations approached 100 % within three months. These historical simulation results do not mean every connection is exposed; rather, opportunities accumulate over time. That reasoning motivated longer-lived guards and fewer guards per client (section 13.3).

The Internet as adversary. RAPTOR identified three ways an AS can increase its opportunities [Sun+15]: exploiting asymmetric Internet routes, since observing *one* direction at each end can suffice, including TCP acknowledgments; exploiting natural routing changes over time; and hijacking or intercepting prefixes with BGP. These techniques do not modify Tor. Defenses involve routing monitoring and potentially topology-aware path selection.

Active attacks. An adversary controlling one end can deliberately *imprint* a distinctive traffic pattern by delaying or grouping packets and search for it at the other end. The marking attack discussed at the encryption layer in section 10.3 is an extreme form of this idea. As early as 2005, Murdoch and Danezis showed that observing the actual links is not always necessary. A malicious server imposes a rhythm on traffic toward a victim; the adversary measures latency on its own circuits through candidate relays, and those that slow with that rhythm may be on the victim's circuit [MD05].

26.3 Laboratory: Correlating Your Own Traffic

Laboratory 16: End-to-end correlation

Objective. Demonstrate on the laboratory network that a traffic pattern survives a circuit and can identify which exit-side flow corresponds to which client.
Prerequisites. Laboratory 11; the laboratory network running. All flows are generated by the experiment's own script.
Estimated time. 1 hour.

The `labs/en/lab/correlation.py` script launches a server on container port 8095 and six clients. Each client uses different SOCKS credentials: with `IsolateSOCKSAuth` enabled, their streams cannot share a circuit (section 18.3). For 20 seconds, clients randomly alternate bursts of 2 KB blocks with silence. Two "observers" record data:

- The entry observer records when each client writes data. This idealizes what an observer on the link to the guard could see. There, data would be packaged into cells and TLS, but timing patterns needed for correlation survive both layers.
- The exit observer, our server, records when bytes arrive on each connection and receives a four-byte evaluation identifier, excluded from the matching algorithm.

Both aggregate bytes into 250 ms intervals. For each client, we calculate Pearson correlation against all six exit-side flows. The most correlated flow is the proposed match. The script knows the correct answer, supplied by the client in its first four bytes, but uses it only to evaluate matches, never to choose the most-correlated flow.

```
$ docker exec -w /tmp torbook-network python3 /lab/correlation.py 1
client 0: +0.79 -0.11 +0.12 +0.17 +0.11 -0.01  -> exit_flow 0 OK
client 1: -0.11 +0.80 -0.02 +0.07 +0.18 +0.08  -> exit_flow 1 OK
client 4: +0.13 +0.02 +1.00 +0.21 +0.10 +0.21  -> exit_flow 2 OK
client 3: +0.21 +0.09 +0.33 +0.81 -0.07 +0.03  -> exit_flow 3 OK
client 2: +0.17 +0.05 +0.10 -0.09 +0.85 -0.03  -> exit_flow 4 OK
client 5: -0.00 +0.01 +0.23 +0.06 -0.13 +0.92  -> exit_flow 5 OK
correct matches: 6/6
```

Columns identify exit-side flows in connection arrival order, which differs from client order. In this illustrative output, the correct correlation stands out in each row, between 0.79 and 1.00; connection order and coefficients may differ between runs. No decryption keys and no decrypted cells are needed, only data volume and timing. The laboratory network is small and lightly loaded, so delay is minimal. DeepCorr's historical results indicate that substantially more sophisticated classifiers can operate with a few hundred packets in the real network under the conditions studied. This is an excerpt from a reference run, not an exact prediction: some runs yield 5/6 correct matches. The laboratory is part of `make verifica-lab`.

26.4 Website Fingerprinting

An adversary observing only the client-to-guard link cannot perform end-to-end correlation, but may *recognize* a website. Each web page generates a characteristic sequence of cells in both directions, including counts, order, and bursts. The adversary visits target websites through Tor, records their sequences, and trains a classifier. It then compares the victim's traffic with those learned patterns.

Attack research has been a continuing contest between classifiers and defenses:

- In 2011, Panchenko and colleagues used volume, timing, and directional features to improve recognition over a closed set of 775 sites from 3 % to 55 %. They also demonstrated an effective *open-world* attack, where victims may visit sites outside the attacker's training set [Pan+11].

- In 2018, Deep Fingerprinting used a convolutional neural network to exceed 98 % accuracy across 95 undefended websites and 90 % against WTF-PAD, an adaptive-padding defense considered for Tor [Sir+18].

In depth: The Base-Rate Fallacy

Extremely high accuracy figures require careful interpretation. Juarez and colleagues showed that many studies assumed favorable conditions: one page visited at a time, the same Tor Browser version for victim and adversary, and training data gathered shortly before an attack [Jua+14]. More fundamentally, in the open world, monitored websites represent a tiny fraction of all visits. Even a classifier with a low false-positive rate can produce more false alarms than correct identifications over millions of unrelated visits. This is the same statistical trap encountered in screening tests for rare medical conditions; real adversaries must contend with it.

Defenses. Channel padding (chapter 7) makes idle periods harder to identify but is not designed to defeat page fingerprinting. tor also implements the more general *circuit padding* framework: state machines can schedule dummy cells. At the source's reference date, deployed machines help onion-service introduction and rendezvous circuits resemble ordinary circuits (chapter 14, Proposal 302). A broadly effective website-fingerprinting defense with acceptable bandwidth overhead remains an open research problem.

26.5 Sybil: Multiplying Relay Identities

A **Sybil attack** introduces many identities into a network. In Tor, an adversary adds numerous relays in the hope of being selected frequently as guards and exits. Three defenses raise the cost:

Measured bandwidth A relay lying about its available bandwidth might attract too many clients. Bauer and colleagues demonstrated this descriptor-based deception in 2007 [Bau+07]. Today, consensus weights rely on authority measurements rather than mere claims (section 12.4); attracting substantial traffic therefore requires providing real bandwidth.

Path constraints Two relays in the same family or IPv4 /16 network cannot appear on the same circuit (section 13.1). Honest operators declare families; dishonest ones are at least constrained by their addresses.

Network monitoring Authorities can exclude relays or assign `BadExit`. Volunteers and researchers investigate groups with shared characteristics and monitor exit behavior. In 2014, for example, an exit scanner found 25 relays manipulating traffic or misconfigured, including HTTPS interception, DNS alteration, and *sslstrip* [WL14].

Chapter 27 examines malicious relay groups uncovered in later years and shows why monitoring is a continuing task.

Sybil attacks against onion services. Under v2, the positions of HSDirs responsible for a service were predictable. An adversary could generate relay identities selected to occupy those positions. Biryukov and colleagues used this in 2013 to measure service popularity, make services unreachable, or facilitate location attacks [BPW13]. In v3, positions depend on blinded keys and a shared random value changing daily (chapter 14), making targeted precomputation and advance placement substantially harder. Once a period's values are public, responsible HSDirs can still be calculated and malicious HSDirs remain possible.

26.6 Discovering the Guard

A guard protects the client but is also an attractive target: identifying it tells an adversary where to observe. Against an onion service, the adversary can trigger many circuit constructions, for example by requesting many connections, until an adversarial relay is selected as the middle adjacent to the guard. The first attack of this kind, published in 2006, helped motivate guards themselves [ØS06]. Vanguards (section 13.4) add a layer of middle relays kept fixed for a period. This slows attempts to work backward toward the actual guard but does not make guard discovery impossible.

Summary

- Attacks differ by vantage point: one end, both ends, inside the network, or the underlying Internet routes.
- Correlation compares traffic shapes at both ends without breaking encryption. The adversary's challenge is occupying the right positions; opportunities accumulate over time.
- Website fingerprinting identifies pages from encrypted traffic at one end. Evaluations must account for base rates and realistic conditions.
- Sybil defenses include measured weights, families, /16 restrictions, and monitoring of exits and relay groups. Under v3, the selection is harder to manipulate far in advance, though responsible HSDirs can be calculated for the current period.
- Guards and vanguards make discovering the first hop slower and more expensive.

Exercises

Exercise 26.1 ⋆ For each attack in this chapter, identify the minimum adversarial vantage point required and the Tor defense that makes the attack harder.

Exercise 26.2 ⋆ A classifier recognizes a monitored website with probability 0.95 and mistakenly flags an unrelated visit with probability 0.001. If only one visit in 100 000 actually goes to the monitored website, what fraction of alerts are correct?

Exercise 26.3 ⋆⋆ Repeat Laboratory 16 using two-second bins, then with a duration of five seconds, and finally with all clients using the same traffic rhythm. Which modification degrades matching most, and why?

Exercise 26.4 ⋆⋆ Modify the Laboratory 16 clients to send blocks at a constant rate, including padding when they have no data. How much bandwidth does this cost, and what happens to correlation?

Exercise 26.5 ⋆⋆⋆ Using `path_selection.py` and Laboratory 15's real consensus, calculate the probability that an adversary controlling 5 % of guard bandwidth and 5 % of exit bandwidth occupies both ends of at least one client circuit within six months. Compare a fixed guard with changing guards for every circuit. State an assumed number of circuits over six months; only in this simplified model, treat bandwidth-weighted draws as independent, and explain why it cannot predict real-world outcomes.

Further Reading

- A. Johnson et al., *Users Get Routed* [Joh+13].
- Y. Sun et al., *RAPTOR: Routing Attacks on Privacy in Tor* [Sun+15].
- M. Juarez et al., *A Critical Evaluation of Website Fingerprinting Attacks* [Jua+14].
- The *Anonymity Bibliography*, `https://www.freehaven.net/anonbib/`, for later research.

Chapter 27

Real Cases Against the Network

Last verified: 23 September 2026 — Tor Project statements (2014, 2022, 2023); tor 0.4.9.13; laboratory network

Chapter objectives

By the end of this chapter, you will be able to:

- reconstruct the 2014 attack involving **`RELAY_EARLY`** cells and the defense added to tor, and verify that defense in a live laboratory (Laboratory 17);
- describe the KAX17 incident and the relay-monitoring work that followed;
- connect the 2022–2023 denial-of-service attacks with the defenses described in chapter 15;
- draw broader lessons about securing an open network.

The previous chapter treated attacks as research topics: models, experiments, and percentages. Here, we examine three real-network security incidents reconstructed from public Tor Project statements. For KAX17, the operator's intentions and any malicious activity were not established. We focus on what they teach us about the protocol and managing an open network rather than on operational attack details.

27.1 2014: Signaling Through Cell Types

What happened. On 4 July 2014, the Tor Project discovered and removed a group of relays it believed were attempting to deanonymize people publishing onion-service descriptors or requesting information about onion services, then known as *hidden services*. The relays had joined on 30 January: approximately 115 fast, non-exit relays across two /16 networks, collectively representing about 6.4 % of guard capacity. On 30 July, the Tor Project published a security advisory describing the attack and its countermeasures [Din14].

How it worked. The attack combined Sybil placement (section 26.5) with active traffic confirmation (section 26.2). The relays remained stable enough to obtain both `HSDir` and `Guard` flags. When an adversarial HSDir received a descriptor request, it replied with a sequence of cells whose alternating **`RELAY`** and **`RELAY_EARLY`** types encoded the requested service name (fig. 27.1). If another member of the group was the same client's guard, it could read this sequence and associate the client with a request for that service's descriptor, without establishing that the service was actually visited.

Why this field? **`RELAY_EARLY`** cells constrain circuit length: clients must use them for extension requests, and relays accept at most eight per circuit ([tor-spec § Handling RELAY_EARLY

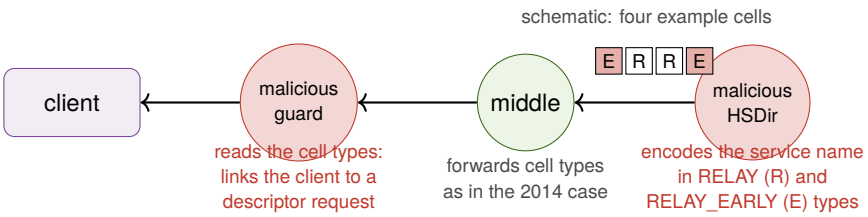


Figure 27.1: The historical 2014 attack: a malicious HSDir encoded the requested service in the outer RELAY and RELAY_EARLY cell commands, visible to relays along the circuit but protected by TLS on the links. A colluding malicious guard could link the client to the descriptor request, not prove that a visit occurred. The four cells are schematic, not a complete onion-address encoding.

cells]). The old checks, however, applied only to client-to-network traffic. No one normally sent such cells in the opposite direction, and no one checked for them there. Unlike layered-encrypted contents (chapter 10), the outer cell command remains visible to every relay. A signal written there survives transit unchanged. The advisory also noted that the signal encoded the service name in readable form, so a relay receiving those cells could interpret it. A third-party observer relying only on recorded network packets would first have to decrypt the link-level TLS traffic to recover the signal retrospectively.

The response. Three steps followed: the malicious relays were excluded; tor 0.2.4.23 and 0.2.5.6-alpha began closing circuits on receipt of client-bound **RELAY_EARLY**; and work proceeded toward reducing each client's guards from three to one, limiting opportunities to observe each user's traffic. The advisory emphasized that *passive* traffic confirmation remained an open problem. The cell-type channel made confirmation easier, not possible for the first time.

27.2 Laboratory: The RELAY_EARLY Limit

Laboratory 17: Eight Are Allowed; Nine Are Not

Objective. Use `labs/en/code/torlab` to verify how a current relay applies the **RELAY_EARLY** cell limit.
Prerequisites. Laboratories 4 and 5; the laboratory network running.
Estimated time. 30 minutes.

Our `circuit.py` already refuses to send more than eight **RELAY_EARLY** cells. The `labs/en/lab/relay_early.py` script bypasses that safeguard. It constructs a one-hop circuit to `test004r`, sends n **RELAY_EARLY** cells containing a `DROP` message that the relay discards without replying, and then requests a directory connection with `BEGIN_DIR` to check whether the circuit remains active:

```
$ PYTHONPATH=labs/en/code python3 labs/en/lab/relay_early.py 0 8 9
0 RELAY_EARLY cells -> CONNECTED
8 RELAY_EARLY cells -> CONNECTED
9 RELAY_EARLY cells -> circuit destroyed, reason 1
```

The ninth cell causes the circuit to close with a **DESTROY** cell carrying reason 1, `PROTOCOL`. The listing comes from a reference run: addresses, circuit identifiers, and logs vary between runs. An example relay log reads:

```
[warn] Received too many RELAY_EARLY cells on circ 4045911407 from
       127.0.0.1:58048.  Closing circuit.
```

The check is in `src/core/or/command.c`, before the cell is decrypted. A few lines above is the check introduced in 2014: a client-bound **`RELAY_EARLY`** results in a warning and circuit closure. The source comment explains that such cells once arose from an old tor bug, but relays affected by it have long disappeared. Today such a cell is considered "unexpected" and handled as an attack. This laboratory is also part of `make verifica-lab`.

27.3 2020–2021: KAX17 and Network Health

A second incident illustrates a less visible part of defense: discovering adversaries in the first place. The Tor Project discussed the incident in April 2022 in an article about its network-health team [The22].

In December 2020, relay graphs showed an unusual pattern. Every month, precisely at midnight on the first day, dozens of relays joined the network and then slowly disappeared. The pattern resembled a bug in the hibernation logic (`AccountingMax`, section 22.2). In September 2021, groups of relays that appeared to share one operator began appearing without family declarations or contact information. Authorities excluded them, but replacements kept appearing. In late October, an anonymous report helped identify a group later called KAX17, which was removed. Both the monthly spikes and relay waves then ceased, leading the Tor Project to consider attribution to the same operator plausible. The operator's identity and objectives remain unknown.

The article is unusually candid about the delayed response: monitoring tools had been neglected for years; trust between volunteers, employees, and authorities was weak; and the organization was not structured for this kind of work. It also explains the governance process. Network monitors propose excluding relays, but directory authorities make the decision by majority rather than Tor Project staff. This is the distributed trust described in chapter 12, applied to network operations.

In depth: Relays Without Family Declarations

The initial warning sign in the KAX17 case was not an observed exploit but a breach of a social rule: numerous relays apparently controlled by the same operator with no declared family (section 22.5). The broader lesson is that an adversary may not need to break a protocol if they can bypass the conventions on which path selection relies.

27.4 2022–2023: Denial of Service

Beginning in summer 2022, the network experienced months of varied denial-of-service attacks. In February 2023, the Tor Project reported that users sometimes could not load pages or reach onion services, that tactics and targets had changed over time, and that the attackers' identities could not be determined with certainty [The23b]. Onion services were especially exposed: they could not block attacking IP addresses because requests arrived over anonymous circuits.

The longer-term response was the proof-of-work system introduced in August 2023 with tor 0.4.8 (section 15.4) [The23a]. Normally dormant, the defense asks clients under attack to solve increasingly difficult computational problems and prioritizes those that have done the most work. The Tor Project described costs of a few milliseconds for a normal client

and potentially about a minute during an intense attack. The principle is the same as in Laboratory 9: make an attack that was cheap for the attacker expensive, without asking users to identify themselves.

27.5 Lessons

- **Every visible field is a potential channel.** The 2014 attack did not break cryptography. It exploited a field that relays could read and forward. An anonymity protocol must ask, for every unencrypted bit, who can write it and who can read it. This is among the motivations for the newer relay encryption design (section 10.4).
- **Sybil attacks may be cheap; monitoring is not.** Adding relays costs relatively little compared with maintaining the tools, expertise, and trust needed to identify coordinated malicious behavior.
- **Defenses must be evaluated over time.** Using one guard, keeping guards longer, and introducing vanguards all reduce the adversary's opportunities over time (section 26.2).
- **Transparency is a defense.** In all three incidents, the Tor Project published both what it knew and what it did not know, enabling users, operators, and researchers to assess risk.

Summary

- In 2014, a Sybil group whose relays served as both guards and HSDirs encoded the names of onion services whose descriptors were requested into cell types. tor now closes circuits receiving client-bound **`RELAY_EARLY`**, while the move from three guards to one, and later to two in 2018, reduced opportunities to select malicious guards (section 13.3).
- The eight-**`RELAY_EARLY`**-cell limit can be verified on the laboratory network: the ninth destroys the circuit.
- KAX17 was identified after months of indirect clues, including monthly peaks and undeclared relay families, and an anonymous report. Directory authorities are responsible for excluding relays.
- The 2022–2023 denial-of-service attacks contributed to the introduction of onion-service proof of work.

Exercises

Exercise 27.1 ⋆ Why could a signal in a cell's encrypted contents, written by an HSDir, not have been read by the guard, while a signal in the outer cell type could?

Exercise 27.2 ⋆ Explain how reducing the number of guards from three to one mitigates an attack like the one in 2014, and what disadvantage this brings.

Exercise 27.3 ⋆⋆ Modify Laboratory 17 to show that a **`RELAY_EARLY`** cell counts toward the limit even if it does not contain a circuit-extension request. Then inspect tor's code to determine what happens when a **`RELAY_EARLY`** cell arrives in the direction of a client.

Exercise 27.4 ⋆⋆ Using CollecTor data, which clues could reveal relays operated by one party without a declared family? Suggest three criteria and discuss false positives.

Exercise 27.5 ⋆⋆⋆ List the cell and relay-message fields visible in plaintext to a middle relay (chapter 8). For each, discuss whether a malicious relay could exploit it as a channel toward another relay on the circuit and identify any existing or proposed defense.

Further Reading

- Tor Project, *Tor security advisory: "relay early" traffic confirmation attack* [Din14].
- Tor Project, *Malicious relays and the health of the Tor network* [The22].
- Tor Project, *Introducing Proof-of-Work Defense for Onion Services* [The23a].

Chapter 28

Operational Security Mistakes and Deanonymization: Lessons

Last verified: 23 September 2026 — Tor Project support; Tails warnings; tor 0.4.9.13; laboratory network

Chapter objectives

By the end of this chapter, you will be able to:

- distinguish the layers at which identity can leak and identify which tools protect each;
- recognize common operational mistakes, from DNS leaks to an anonymity set that is too small;
- use tor itself to check whether an application delegates name resolution to it (Laboratory 18);
- analyze a real incident in terms of a threat model.

Chapter 26 ended with an uncomfortable observation: protocol attacks can be expensive and require privileged observation points. In many documented cases, anonymity has also been lost through mistakes in how tools were used, which a network protocol alone cannot prevent. This chapter examines those mistakes, not to explain how to identify people but to clarify where Tor's protections end and users' own responsibilities begin.

Warning: Scope of This Chapter

We discuss mistakes and defensive practices from the perspective of people seeking to protect themselves. We do not provide procedures for identifying people. The incidents mentioned are public and reconstructed from the cited sources.

28.1 Four Layers

Identity can leak at four layers (fig. 28.1). The first, the network, is the layer tor protects: for an ordinary Tor connection, the destination does not see the originating client's IP address, and an individual circuit relay normally lacks a complete view of both ends. This separation does not eliminate correlation attacks or other risks. The second, the application, is addressed by Tor Browser, Tails, and Whonix (chapter 23). The last two, content and behavior, are not automatically protected by tor. Specialized tools can reduce some risks, but what users write, upload, and do also matters.

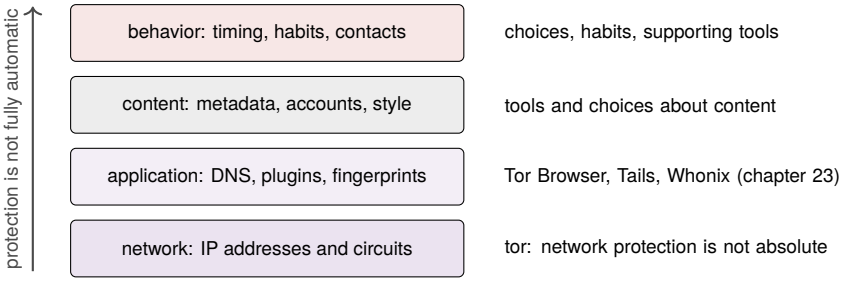


Figure 28.1: Teaching diagram of four layers at which identity may be exposed. Tor mitigates some network risks; application protections and content tools help, while what users disclose and how they behave matters at every layer. No single layer guarantees anonymity.

28.2 Application Mistakes

Traffic outside Tor. tor only protects applications configured to use it [Thef]. Software making direct connections, even for updates or previews, can disclose a real IP address. Tor Project support highlights two familiar cases. BitTorrent applications may ignore proxy settings and may disclose their addresses to trackers anyway. Downloaded documents opened while online present another risk: DOC or PDF files may contain remote resources fetched by applications that do not use Tor.

DNS leaks. A subtler case involves SOCKS. An application can pass the destination *name* to tor, letting the exit resolve it (section 11.1.4), or resolve the name itself using the system DNS service and pass tor only an IP address. In the second case, the subsequent connection still uses Tor, but name resolution might already have disclosed the domain to a resolver outside the Tor circuit, potentially linking it to the client's network connection. A plaintext DNS query is not inevitable: an answer might come from a local file or cache, or from encrypted DNS. tor cannot establish that a leak already happened, but it can detect the warning sign of an IP-only SOCKS request, as the laboratory demonstrates.

28.3 Laboratory: tor Detects a DNS-Leak Warning Sign

Laboratory 18: SOCKS With and Without a Hostname

Objective. Observe how tor distinguishes an application that delegates name resolution from one that performs resolution itself, and how tor can reject the latter.
Prerequisites. Laboratory 11; the laboratory network running.
Estimated time. 30 minutes.

`curl` supports both behaviors. With `--socks5`, it resolves the name locally and passes tor an IP address; with `--socks5-hostname`, it passes the name. The `labs/en/lab/dns_leak.py` script connects to client `008c`'s ControlPort and enables `TestSocks`, which makes tor log the type of each SOCKS request. The script listens for log messages and tests both forms against `http://localhost:8080/`.

This laboratory demonstrates the *mechanism*, not an actual DNS leak observed on the wire. `localhost` resolves locally without contacting a resolver, so no query leaves the computer. tor sees an IP address instead of a hostname: a possible sign of earlier local resolution, not proof of a DNS leak (the application might have used an IP literal). With an ordinary domain instead of `localhost`, similar local resolution could generate an external query that might be

plaintext, depending on DNS configuration. We deliberately avoid generating such traffic and remain on the private network. The script also forces use of SOCKS for `localhost`, overriding any `NO_PROXY` bypass. Finally, it enables `SafeSocks`, which rejects IP-only requests even when no DNS leak occurred:

```
--socks5           HTTP 200  tor: Your application (using socks5 to port
                   8080) is giving Tor only an IP address. Applications
                   that do DNS resolves themselves may leak information...
--socks5-hostname  HTTP 200  tor: Your application (using socks5 to port
                   8080) instructed Tor to take care of the DNS resolution
                   itself if necessary. This is good.
--socks5 SafeSocks HTTP 000
```

In the illustrative output, both initial requests work, but only the second delegates name resolution to tor; the first does not establish that DNS leaked because the target is `localhost`. In tor's code (`src/core/proto/proto_socks.c`), the first warning is logged at `warn` level, rate-limited to avoid flooding logs, and accompanied by a `STATUS_CLIENT` event with action `DANGEROUS_SOCKS` on the ControlPort. A controller can therefore warn the user. With `SafeSocks` enabled, the same request is rejected and `curl` receives no response. This laboratory is included in `make verifica-lab`.

28.4 Content Mistakes

Metadata. Many files contain information their authors never notice: photographs may reveal location or camera model; documents may reveal authors and creation dates. Tails includes a tool to remove metadata, and its warnings mention cases in which photo metadata disclosed where the photographer had been [Taib].

Accounts and identity. Signing in to a personal account tells a website who you are regardless of the connection's apparent IP address [Thef]. Using two identities on the same website in one session may also link them through cookies, session data, application identifiers, or other clues. A website cannot directly inspect the Tor circuit identifier. Tails recommends restarting between activities that must remain separate [Taib]. Tor Browser's unlinkability requirement (section 23.2) applies between different websites, not between different identities used on one website.

Writing style. Even without metadata, writing habits, subjects, recurring mistakes, and publication times may link anonymous text to signed writing. No automatic tool can eliminate every such clue.

28.5 Behavioral Mistakes: An Anonymity Set That Is Too Small

Chapter 1 defined anonymity in terms of being indistinguishable within a set. Tor can reduce some distinguishing information at the network layer, but cannot guarantee that enough people are using Tor at a specific place and time. If very few do, knowing *who used Tor* can substantially narrow the possibilities.

An incident. In December 2013, a US university received emailed false bomb threats shortly before an examination. The sender had used a temporary email service and Tor. According to the FBI affidavit accompanying the complaint, as reported by multiple news outlets, the

university identified who had used Tor on its wireless network in the hours before the messages arrived [WBU13; San13]. The student later admitted sending the messages during questioning. Breaking Tor's encryption was not necessary to narrow the suspect pool: local observers could see Tor usage, without that implying access to connection contents or destinations. They could, however, see *who* was using Tor during that interval, reducing the suspect pool to a few names.

The incident illustrates three principles from the threat model (chapter 4):

- Tor without a bridge does not hide the fact that it is being used: local network observers may recognize it [Taib]. Bridges with pluggable transports (chapter 16) can reduce this exposure.
- Correlation need not involve a global adversary when the candidate set is small and the event occurs in a narrow time window. This is a basic intersection problem.
- Anonymity loves company [DM06]. The more people with different reasons to use Tor do so in the same environment, the less informative any one person's use becomes. Tor Project support explicitly encourages wider Tor adoption [Thef].

28.6 A Checklist

This is not exhaustive, but it covers the layers in order:

1. Use Tor Browser or a system such as Tails or Whonix, not an ordinary browser with a manually configured proxy.
2. Avoid installing extensions or altering settings that make your browser stand out.
3. For other applications, verify that they pass hostnames to tor (`SafeSocks`, `TestSocks`).
4. Do not open downloaded documents while online, or use an isolated environment for them.
5. Prefer HTTPS and onion services, because an exit can see unencrypted content.
6. Remove metadata from files before sharing them.
7. Do not combine identities that must remain separate in one session.
8. If merely being seen using Tor is risky, consider a suitable obfuscated bridge; it does not guarantee invisibility.
9. Remember that schedules, habits, and writing styles also convey information.

Summary

- Identity can leak at the network, application, content, and behavioral layers. tor directly protects only the first.
- Applications bypassing Tor, downloaded documents opened online, and DNS leaks are application-level mistakes. tor can warn about and reject SOCKS requests containing only IP addresses.
- Metadata, accounts, and writing style can link content to people.
- When the set of Tor users is small, knowing who used Tor and when may be enough to narrow identification. Bridges and a broad community of users help reduce this risk.

Exercises

Exercise 28.1 ⋆ For each mistake discussed here, identify its layer in fig. 28.1 and the tool or habit that helps prevent it.

Exercise 28.2 ⋆ Explain why a DNS leak discloses information even when the subsequent connection travels through Tor. Who sees that information?

Exercise 28.3 ⋆⋆ In Laboratory 18, subscribe to `STATUS_CLIENT` events through Stem and display the `DANGEROUS_SOCKS` action generated by an IP-only request. How could a graphical interface use it?

Exercise 28.4 ⋆⋆ Revisit the 2013 incident assuming the student had used an obfs4 bridge from the same network. What might local network observers have seen, and what other information could still have narrowed the candidate set?

Exercise 28.5 ⋆⋆⋆ Choose a real or fictional person with a legitimate need for anonymity, such as a journalistic source, activist, or researcher, and write an operational plan covering threats, relevant layers, tools and habits, and residual risks.

Further Reading

- Tor Project support guidance on anonymity [Thef].
- Tails warnings, *Tails is safe but not magic* [Taib].
- R. Dingledine and N. Mathewson, *Anonymity Loves Company* [DM06].

Part V

Research and the Future

Chapter 29

Academic Research on Tor

Last verified: 23 September 2026 — papers cited in this chapter; Tor Project research portal; consensus of 23 September 2026

Chapter objectives

By the end of this chapter, you will be able to:

- find the principal venues for Tor research and the resources provided by the Tor Project;
- measure anonymity quantitatively, from entropy to time to first compromise;
- select an appropriate experimental method: analytical modeling, simulation, emulation, or measurement;
- simulate thousands of clients over months using a real consensus (Laboratory 19).

Tor has been driven by research from the beginning. Onion routing emerged from a research laboratory (chapter 3); the Tor design was presented at an academic conference [DMS04]; and many mechanisms covered in this book, from guards to congestion control, emerged in response to published research. This chapter is intended for readers who want to conduct research themselves: where to publish, how to measure anonymity, which experimental tools to use, and which safety principles to observe.

29.1 Where Tor Research Is Published

Research on anonymity networks is chiefly published at two kinds of venues:

Privacy Enhancing Technologies Symposium (PETS) and its journal, *Proceedings on Privacy Enhancing Technologies* (PoPETs), a specialized venue: the foundational anonymity-measurement work discussed below appeared at the PET workshop in 2002, before the PoPETs journal existed [Día+02].

Major security conferences USENIX Security, ACM CCS, IEEE Symposium on Security and Privacy, and NDSS. Many of the papers in chapter 26 appeared at these conferences.

Free Haven's *Anonymity Bibliography* has collected publications in this field for more than twenty years and is a natural starting point for a literature review.

The Tor Project also maintains a research portal containing data, tools, technical reports, proposed research topics, and information about the Research Safety Board. Its guidance stresses that the protocol and network are continually evolving: measurements made without understanding their internal operation can lead to incorrect conclusions. Researchers are

therefore encouraged to contact the project early, to learn about concurrent experiments and planned changes.

29.2 Measuring Anonymity

In chapter 1, we defined anonymity in terms of indistinguishability within a set. Yet not every member is equally likely from an adversary's perspective. After observing traffic, the adversary might assign much higher probabilities to some users than to others. The number of users in the set is not enough.

Degree of anonymity. In 2002, Díaz and coauthors proposed measuring anonymity through the entropy of the probability distribution an adversary assigns to possible senders [Día+02]. If p_i is the probability the adversary assigns to user i being the sender, the entropy is

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i,$$

and it reaches its maximum, $H_M = \log_2 N$, when all N users are equally likely. The **degree of anonymity** is the ratio

$$d = \frac{H(X)}{H_M}, \qquad 0 \leq d \leq 1 :$$

for $N \geq 2$, it is 1 when the posterior distribution is uniform (which does not by itself prove that the adversary learned nothing) and 0 when a single sender has probability 1. For $N = 1$, the ratio is undefined because $H_M = 0$.

One intuitive way to interpret entropy is $2^{H(X)}$: the number of equally likely users who would provide the same uncertainty. A set of 1,000 users in which the adversary has narrowed the suspects to just two, each with probability 0.5, provides as much uncertainty as a set of two: $H = 1$ bit.

Limits of one-time measurements. Entropy describes an individual observation, but Tor's exposure accumulates over time, as we saw in section 26.2: every circuit creates another opportunity for an adversary. Johnson and colleagues therefore proposed *longitudinal* measures, including the probability of deanonymization at least once within a given period and median time to first compromise, calculated by simulating users' behavior against real network data [Joh+13]. We will follow a similar approach in the laboratory.

29.3 Four Methods

Analytical modeling A mathematical model, such as degree of anonymity or the model of Sybil attacks in section 26.5. This is fast and general, but depends on assumptions that real systems inevitably violate to some extent.

Behavioral simulation Reproduce a simplified component of a system, such as path selection, using real-world data. TorPS, the path simulator developed by Johnson and colleagues [Joh+13], and our Laboratory 19 use this approach. It is fast, but cannot account for anything omitted from the model.

Network emulation and simulation Run the actual software either in a private network like the Chutney laboratory used throughout this book or inside a discrete-event simulator such as Shadow [JH12], which can execute thousands of relays and clients on one

machine with virtual time (section 19.9). This is especially useful for performance research, because proposed protocol changes can be tested on a realistic network before deployment.

Measurements on the public network Only direct measurement reveals what happens on the real network. It also carries the greatest risk to real users and requires careful experimental design (section 29.5).

Strong research often combines several methods: mathematical analysis suggests a hypothesis, simulation tests it at scale, and carefully designed measurement examines its realism.

In depth: Measuring Without Observing Individuals

PrivCount, developed by Jansen and Johnson, shows how measurement and privacy can coexist [JJ16]. Participating relays count relevant events, but counters are masked with random values shared with other participants. Noise is added according to the principles of *differential privacy*. Only an aggregated, deliberately noisy result becomes visible, limiting what can be inferred about individual users. In 2016, the authors used PrivCount to estimate approximately 710 000 simultaneously connected users, of whom 550 000 were active at a given moment, and found that traffic to web ports accounted for 91% of the data bytes observed at participating exits. These figures describe the 2016 study, not today's network. This takes the same privacy principles as rounding relay statistics (section 24.2) a step further.

29.4 Laboratory: Time to First Compromise

Laboratory 19: Months of Circuits in a Simulation

Objective. Use a real consensus to estimate how many clients an adversary controlling a given bandwidth share could deanonymize over time, with and without persistent guards.
Prerequisites. Laboratories 7 and 15; a running network is not required.
Estimated time. 1–2 hours.

The `labs/en/code/torlab/simulation.py` module reuses the weights from Laboratory 7. The adversary adds a guard and an exit, each representing a fraction q of weighted bandwidth at its circuit position. We simulate 2,000 clients for 180 days, each building 20 circuits daily.

The model replaces a persistent guard on an approximate schedule based on a *confirmed* guard. According to guard-spec and the code in `entrynodes.c`, a confirmed guard is removed from the sample `GUARD_CONFIRMED_MIN_LIFETIME` (60 days) after confirmation, and `CONFIRMED_ON_DATE` is randomly backdated by up to `GUARD_LIFETIME`/10 (12 days). This produces a simulated interval of 48–60 days from initialization. The 120-day `GUARD_LIFETIME` instead governs expiration of unconfirmed guards in the sampled set. Real guard rotation also depends on the sampled/confirmed lists, reachability, and replacements: this exercise does not reproduce the full guard algorithm. An early version incorrectly applied 120 days to confirmed guards as well, understating risk within this model. For comparison, we also simulate clients choosing a new guard for every circuit. A client becomes compromised on the first day one of its circuits includes both an adversarial guard and an adversarial exit.

This model is intentionally simple: it ignores middle relays, families, /16 networks, backup guards, and failures. To remain fast, it does not use `random.choices`, which would recompute thousands of weights on every draw. It instead uses binary search over cumulative weights,

allowing millions of selections without recomputing the distribution; actual runtime depends on the machine.

First obtain the historical public consensus (for example, with `make -C labs/en data`); it is not committed to the repository. The simulation is deterministic: with the same model, consensus, share, and seed, the result is always this one.

```
$ PYTHONPATH=labs/en/code python3 -m torlab.simulation labs/en/data/collector/
    consensus-microdesc-2026-09-23-15 0.05
adversary: 5% of guard bandwidth and 5% of exit bandwidth; 2000 clients, 180 days, 20
    circuits/day
persistent guard      1d:  2.6%    7d:  4.4%   30d:  4.4%   90d:  9.4%  180d: 18.4%
    median: None
per-circuit guard      1d:  4.3%    7d: 27.9%   30d: 76.5%   90d: 98.9%  180d:100.0%
    median: 15
```

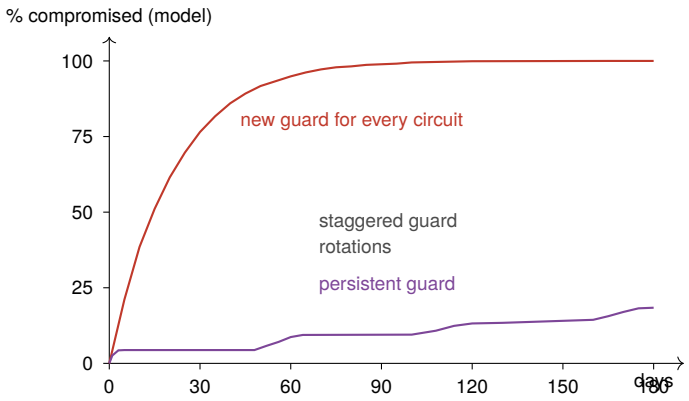


Figure 29.1: Laboratory 19, simplified simulation: 2,000 clients building 20 circuits daily over 180 days, with a 23 September 2026 reference consensus and an adversary holding 5% of weighted guard bandwidth and 5% of weighted exit bandwidth. The cumulative fraction counts clients who used at least one circuit with both adversarial endpoints in the model. Persistent-guard rotations occur at different times for each client (simulated intervals of 48–60 days), not on three shared dates. These are simulated exposures, not observed deanonymizations on the public network.

Figure 29.1 shows both curves. Without persistent guards, half the clients become compromised within fifteen days and almost all within three months, because every new circuit adds another independent opportunity. With persistent guards, a client's exposure is determined largely when the guard is selected. About 4% of clients selecting the adversarial guard are compromised early; others cannot meet this particular compromise condition while keeping an honest guard. As different clients rotate guards at staggered times (approximately every 48–60 days per client in this model), new draws expose additional clients, bringing the total to approximately 18% after six months. When the adversarial bandwidth share is 1%, the corresponding six-month result falls to about 4%, versus 31% without persistent guards.

These results from one September 2026 historical snapshot illustrate persistent guards' role in this simplified model. Rather than necessarily reducing marginal risk per circuit, they redistribute exposure across clients: those retaining an honest guard cannot meet this specific compromise condition, whereas those selecting a malicious guard face higher exposure. Guard lifetime is consequently a trade-off. Slower rotation protects users with honest guards for longer but also prolongs the exposure of those with adversarial guards. A patient adversary who remains in the network for months gains new opportunities at each rotation.

29.5 Conducting Research Safely

Chapter 25 introduced the Tor Research Safety Board's guidelines [Tor]. When designing an experiment, ask first: *does my research question truly require the public network?* Many performance and attack questions can be answered using Shadow or a private network, with reproducible experiments and control over variables. When public-network measurement is essential, follow the principles already discussed: act only on your own traffic, collect the minimum data, aggregate before publication, and consult the Board and Tor Project before beginning.

Recent history provides contrasting examples. Chapter 27 described an experiment, or attack, affecting real users through a signal visible to receiving relays; a passive third party would first need to decrypt the inter-relay TLS connections. PrivCount, by contrast, shows that detailed network measurement is possible without exposing individual users. The difference lies not merely in the tools but in experimental design.

Summary

- Tor research appears at PETS and major security conferences. The Tor Project provides data, tools, and a Research Safety Board.
- Entropy measures anonymity for a single observation; degree of anonymity normalizes it between zero and one. Over time, measures such as probability of compromise within a given interval are needed.
- Analytical models, behavioral simulations, network emulation or simulation, and measurements on the public network address different questions and carry different risks.
- In the reference-consensus simulation, persistent guards substantially reduce the fraction of clients compromised over time by concentrating exposure among fewer clients.

Exercises

Exercise 29.1 $\star$ An adversary observes a four-user system and assigns probabilities $1/2$, $1/4$, $1/8$, and $1/8$. Calculate the entropy, degree of anonymity, and equivalent number of equally likely users.

Exercise 29.2 $\star$ Choose and justify the most appropriate method for each question: (a) How would latency change if circuits used four hops? (b) How many users access Tor from a particular country? (c) Would a new guard-selection algorithm reduce time to first compromise?

Exercise 29.3 $\star\star$ Run Laboratory 19's `simulate` with `guard_lifetime` set to 30 and 180 days instead of the default 60, and compare the fraction of clients compromised after 180 days. Explain your findings.

Exercise 29.4 $\star\star$ Repeat Laboratory 19 with an adversary controlling 10% of exit bandwidth but no guard bandwidth, then one controlling 10% of guard bandwidth but no exit bandwidth. What can each achieve, and why?

Exercise 29.5 $\star\star\star$ Extend the simulator with three features that the current model ignores: clients reusing circuits for ten minutes instead of continually creating new ones, the /16 restriction between guards and exits, and an adversary controlling the entire autonomous system of a major network. Which additions change the results most?

Further Reading

- C. Díaz, S. Seys, J. Claessens, B. Preneel, *Towards Measuring Anonymity* [Día+02].
- R. Jansen, A. Johnson, *Safely Measuring Tor* [JJ16].
- Tor Project research portal, `https://research.torproject.org/`.
- The *Anonymity Bibliography*, `https://www.freehaven.net/anonbib/`.

Chapter 30

Arti: Tor in Rust

Last verified: 23 September 2026 — Arti 2.6.0 (1 September 2026); tor 0.4.9.13; laboratory network

Chapter objectives

By the end of this chapter, you will be able to:

- explain why the Tor Project is rewriting tor in Rust and assess the project's progress;
- understand Arti's architecture and identify the concepts covered in Part II;
- interpret its supported-subprotocol table and understand which features are still missing;
- configure Arti as a client of the book's private network (Laboratory 20).

Throughout this book we have relied on *tor*, the C implementation begun in 2002. The future of the Tor client, however, is **Arti**, a complete rewrite in Rust. The Tor Project has designated Arti as the preferred client for developing new functionality and announced that every new client-side feature will receive an Arti implementation. This chapter presents Arti from the perspective of someone who has just written a small Tor client in Python during the laboratories. Many of Arti's design decisions are the same decisions we faced, implemented at production scale.

30.1 Why Rewrite tor?

The project's introductory document lists four motivations:

Security Safe Rust prevents many classes of memory-safety bugs, although unsafe code, dependencies, and logic errors still require review. According to the Tor Project's estimate, Rust would have ruled out at least half of tor's recorded security vulnerabilities and made many of the rest very unlikely.

Development speed Language-level guarantees make it possible to develop new code with greater confidence and less time.

Flexibility tor started as a SOCKS proxy, with integration features added later. Arti was designed from the outset as a modular library that other applications can embed.

Maintainability Twenty years of development have left complex relationships among the components of tor's codebase. A rewrite offers an opportunity to separate responsibilities more clearly.

Milestones. Arti's first public version, 0.0.1, appeared in October 2021. Version 1.0.0, released on 1 September 2022, declared Arti stable as a client. In August 2023, version 1.1.7 enabled compilation of client-side onion service support by default, although users still needed to enable the feature. Version 2.0.0 followed in February 2026. This chapter's reference version is 2.6.0, released on 1 September 2026, which makes congestion control (section 11.3) and CGO encryption (section 10.4) permanently enabled.

Current scope. According to its project documentation at the reference date, Arti is a complete general-purpose client supporting bridges and pluggable transports; it can both access and publish onion services. It cannot yet operate as a relay, bridge, or directory authority. Those capabilities are under development, and much of the work in recent releases has focused on relays and authorities.

30.2 Architecture

Arti consists of dozens of Rust *crates*, the language's compilation units: more than seventy in version 2.6.0. Naming conventions distinguish their roles. User-facing crates begin with `arti`, Tor protocol-specific crates with `tor-`, and crates useful outside Tor have more general names. A guiding principle shapes the design: keep code that directly reads or writes network data to a minimum; most components should behave "as if the network did not exist", making them easier to test.

Figure 30.1 places the principal crates alongside the `torlab` modules that serve similar purposes. The correspondence is intentional: both follow the protocol's layers, from the lowest-level primitives to the high-level client.

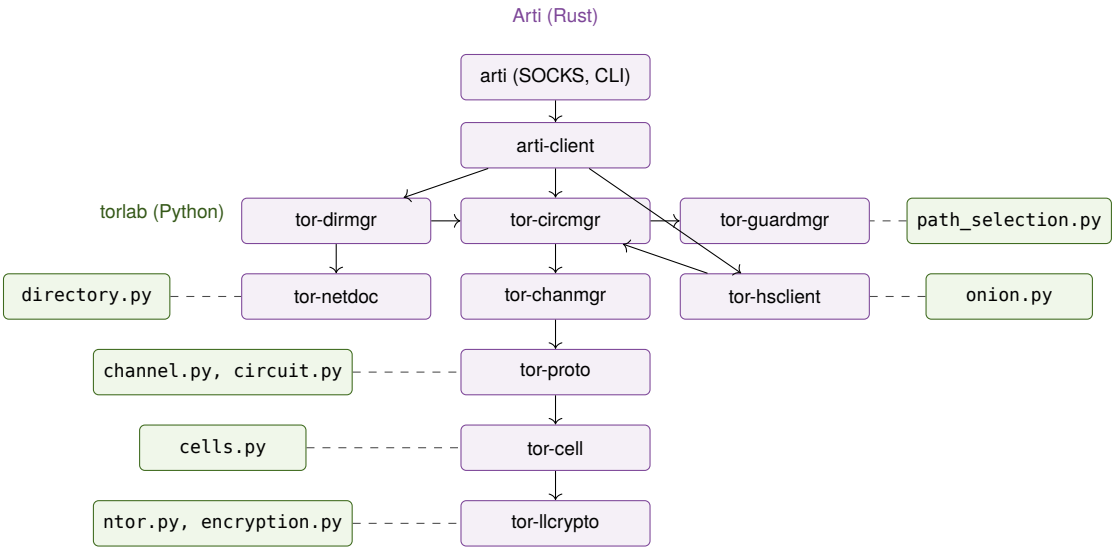


Figure 30.1: Simplified functional map of Arti crates and educational `torlab` modules. Solid arrows summarize architectural relationships, not an exact Cargo dependency graph; dashed links indicate analogous roles, not project dependencies or complete equivalents. For example, `onion.py` only decrypts descriptors and cannot replace an onion-service client; crates such as `tor-bytes` and `tor-hsservice` are omitted.

`tor-llcrypto`, `tor-bytes` Cryptographic primitives and binary parsing/serialization. They provide the foundations that our `ntor.py` and `encryption.py` obtained from Python's `cryptography` and `struct` modules.

tor-cell Cells and relay messages: Arti's equivalent of `cells.py` (chapter 8).

tor-proto Channels, circuits, layered encryption, and streams: everything we built using `channel.py`, `circuit.py`, and `stream.py`.

tor-chanmgr, tor-circmgr, tor-guardmgr Managing multiple channels and circuits, selecting paths, and maintaining guards. Our educational client implemented hardly any of these features, which are among the most complex parts of a real client.

tor-netdoc, tor-dirmgr Reading and updating directory documents, corresponding to `directory.py`.

tor-hsclient, tor-hsservice Both sides of onion services, of which `onion.py` implemented only descriptor decryption.

arti-client, arti The high-level library for embedding into other applications, and the command-line program exposing a SOCKS proxy.

30.2.1 A Comparison: Writing a Cell

In Laboratory 2, we wrote a cell-encoding function. Arti's corresponding function, `write_cell` in `crates/tor-cell/src/chancell/codec.rs`, follows the same steps: write the CircID, then the command. For a variable-length cell, reserve two bytes for its length, encode the body, then go back and write the actual length. For a fixed-length cell, encode the body and verify that it does not exceed 509 bytes (`CELL_DATA_LEN`). Two differences are instructive:

- The CircID is *always* written as four bytes. Arti does not implement link-protocol versions older than version 4, so it does not need the special case of two-byte CircIDs (chapter 8). Only the **`VERSIONS`** cell, which precedes negotiation, is handled separately.
- Violating the length limit does not produce a caller error but an *internal* error: an oversized cell would be a defect in Arti's own code, not in data received from a peer. The error type itself distinguishes an implementation bug from malicious input.

Arti explicitly declines to support obsolete protocols. Its goal is compatibility with *currently recommended* versions, not preservation of every historical format.

30.3 Supported Features

The most precise description of the capabilities of a Tor implementation is its **subprotocol** support ([tor-spec § Subprotocol-based versioning]). Every relay and client announces the versions it supports for each family of features: `Link`, `Relay`, `FlowCtrl`, `HSIntro`, and so forth. Arti's compatibility documentation lists, among other client-side capabilities:

- `Link=4` and `Link=5`: four-byte CircIDs and modern channel padding (chapter 7);
- `Relay=4` (ntor-v3, chapter 9), `Relay=5` (subprotocol negotiation), and `Relay=6` (CGO encryption);
- `FlowCtrl=1` (authenticated SENDME) and `FlowCtrl=2` (congestion control);
- `Desc=4`: key-based families as described in section 22.5.

Warning: An Off-by-One Error in Arti's Table

The referenced Arti compatibility document numbers these two flow-control subprotocols `FlowCtrl=0` and `FlowCtrl=1`. The specification, however, assigns value 1 to authenticated SENDME and 2 to congestion control [tor-spec § Subprotocol-based versioning], and relays in the public consensus announce `FlowCtrl=1-2`. This is an off-by-one error in documentation, not the implementation. It also illustrates a useful practice: when sources disagree, check the specification and look for independent evidence in the consensus.

Two capabilities of the C client remain missing from the Arti client at the source's reference date. `Conflux=1` provides multipath operation: the `tor-proto` crate supports it, but Arti does not yet build Conflux circuits. `Padding=2` refers to padding state machines designed to conceal onion-service circuit construction (section 26.4). While `tor-proto` includes a padding framework based on the *maybenot* library, these machines are not yet implemented. No relay-side subprotocols are marked supported: much of the relay-side code exists, but features will be declared supported only when a functioning relay can provide them.

30.4 Laboratory: Arti on the Private Network

Laboratory 20: A Second Client, in Rust

Objective. Compile Arti, configure it as a client of the book's private network, and use its SOCKS proxy in place of `test008c`.
Prerequisites. The laboratory network running and a Rust toolchain (`cargo`).
Estimated time. 1–2 hours, most of it spent compiling.

Step 1: compile. From the book repository root, check out the reference version of Arti (2.6.0) under `labs/en/vendor/arti` and build the optimized `arti` binary:

```
$ mkdir -p labs/en/vendor
$ git clone https://gitlab.torproject.org/tpo/core/arti.git labs/en/vendor/arti
$ git -C labs/en/vendor/arti checkout arti-v2.6.0
$ (cd labs/en/vendor/arti && cargo build --release -p arti --locked)
$ labs/en/vendor/arti/target/release/arti --version
Arti 2.6.0
```

The first build takes several minutes because it compiles the whole crate tree and its dependencies.

Step 2: configure. Arti does not read `torrc`; it uses TOML configuration. To access a non-public Tor network, Arti must know its directory authorities and the fallback caches from which to fetch its initial directory documents, just as the `DirAuthority` lines do in Chutney-generated `torrc` files (section 19.6). The `labs/en/lab/arti_config.py` script reads each authority's RSA fingerprint, Ed25519 key, v3 certificate identity, and listening ports from its local files. It writes these values to `labs/en/lab/net/arti/arti.toml`. The following abbreviated snippet is illustrative, not valid ready-to-use TOML: the authority identities must be generated locally, and the ellipses are placeholders:

```
[tor_network]
fallback_caches = [
  { rsa_identity = "7B465200C885ED566C6CD4F46DBB0492076D203A",
    ed_identity = "JwhsA9VS01mRVnQseL9xsKZfstoBae7OcgL3jBvDzis",
```

```
    orports = ["127.0.0.1:5100"] },
  ...
]
[tor_network.authorities]
v3idents = ["390998046DEFF1E68AC346D7577319791B32C91A", ...]
```

Three other configuration changes adapt Arti to a network where everything runs on `127.0.0.1`. Setting `allow_local_addrs` allows connections to local addresses, which public-network clients ordinarily reject. Two impossible network prefix lengths (33 bits for IPv4 and 129 for IPv6) disable the same-network constraint that would otherwise prevent every circuit here. Finally, `hsdir_interval` shortens the onion-service period, just as `TestingTorNetwork` does in tor. These settings mirror Chutney's own Arti configurations.

Step 3: start. After generating the private configuration with `python3 labs/en/lab/arti_config.py`, run `labs/en/vendor/arti/target/release/arti proxy -p 9150 -c labs/en/lab/net/arti/arti.toml` from the book's root. Bootstrap time depends on the machine; in one of our runs:

```
INFO tor_dirmgr::bootstrap: 1: Looking for a consensus.
INFO tor_dirmgr::bootstrap: 1: Downloading certificates for consensus
     (we are missing 3/3).
INFO tor_dirmgr::bootstrap: 1: Downloading microdescriptors (we are missing 8).
INFO tor_dirmgr: Marked consensus usable.
WARN tor_netdir: Requested 60 relays, but only 8 were usable, ...  role=Guard
INFO arti::subcommands::proxy: Sufficiently bootstrapped; proxy now functional.
INFO tor_guardmgr::guard: We have found that guard [scrubbed] is usable.
```

Three details are revealing. Bootstrap follows section 12.7 step by step: first the consensus, then certificates for the three authorities, and finally eight microdescriptors. The `tor_netdir` warning reflects our tiny network: the guard manager would prefer a sample of 60 candidates, but only eight are usable. The guard's identity appears in the log as `[scrubbed]`: Arti marks sensitive values with its `safelog` crate and suppresses them in logs, equivalent to tor's `SafeLogging`. On subsequent directory updates, other log output may report `Applying a consensus diff` (not shown in this excerpt). Arti can download a consensus diff when one is available and negotiated using `DirCache=2`; this is not guaranteed for every update.

Step 4: use it. Arti offers a SOCKS proxy on port 9150. `curl` uses it just as it used `test008c`:

```
$ curl -s -o /dev/null -w '%{http_code} in %{time_total}s\n' \
       --noproxy "" --socks5-hostname 127.0.0.1:9150 http://127.0.0.1:8080/
200 in 0.047224s
```

An empty `--noproxy` list ensures that `NO_PROXY` does not bypass SOCKS for the loopback destination. The circuit crosses the C-language relays of the private network: a Rust client and C relays speak exactly the same byte-level protocol studied in Part II. One last observation relates to Laboratory 18: SOCKS requests containing only an IP address (`--socks5`) also succeed, and Arti's default logs do not display a warning analogous to tor's. However, the `arti-client` crate's documentation warns API users about resolving names locally, which might disclose the destination to a DNS resolver, depending on configuration. An IP-only SOCKS request by itself does not prove that a DNS leak occurred (section 28.3).

The laboratory is part of `make verifica-lab` when Arti has been built; otherwise the test is skipped and reported.

Summary

- Arti rewrites tor in memory-safe Rust. It is modular, designed as a library, stable as a client since 2022, and the preferred client for developing new features.
- Its architecture follows the protocol layers: primitives and cells, channels and circuits, then directory, path, and guard management.
- Subprotocol versions describe what Arti supports precisely. Its client implements ntor-v3, CGO, congestion control, and key-based families, while Conflux and padding state machines remain incomplete at this chapter's reference date. Relay support is still being developed.
- Arti connects to a private network through configured authorities and directory caches, much like a Chutney node.

Exercises

Exercise 30.1 ⋆ For each chapter in Part II, identify the Arti crate implementing the main concepts covered there.

Exercise 30.2 ⋆ What are the benefits and drawbacks of Arti's decision not to implement obsolete protocol versions? Who might be excluded?

Exercise 30.3 ⋆⋆ Read `decode_cell` in `crates/tor-cell/src/chancell/codec.rs` and compare it with Laboratory 2's `decode`. How is a variable-length cell recognized, and what happens if the input does not yet contain a complete cell?

Exercise 30.4 ⋆⋆ Repeat Laboratory 11's stream-isolation experiment with Arti as the SOCKS proxy. Do distinct SOCKS credentials prevent streams from sharing circuits under Arti? Distinguish stream isolation from the unsupported assumption that each credential pair permanently maps to exactly one circuit; find the relevant Arti configuration.

Exercise 30.5 ⋆⋆⋆ Write a small Rust program using `arti-client` to retrieve a page from the laboratory network's server, loading Laboratory 20's configuration from a file.

Further Reading

- The project website, `https://arti.torproject.org/`, and source repository, particularly its README, *Architectural overview* (`doc/dev/Architecture.md`), and compatibility guide (`doc/Compatibility.md`).
- Arti's changelog, which documents changes in each monthly release.
- Chutney's generated Arti configuration (`lib/chutney/arti/config.py`), which inspired Laboratory 20.

Chapter 31

Open Problems and Ongoing Projects

Last verified: 23 September 2026 — torspec proposals through number 369; tor 0.4.9.13; Arti 2.6.0

Chapter objectives

By the end of this chapter, you will be able to:

- follow the process by which Tor's protocol changes and interpret a proposal's status;
- explain why post-quantum cryptography requires changes to message formats, not just algorithms;
- describe Conflux, Walking Onions, and other projects addressing performance and scalability;
- identify concrete ways to contribute to the project.

Nothing described in this book is final. The protocol examined here in 2026 differs from its 2016 counterpart, which lacked ntor-v3, congestion control, and CGO; it will differ again in 2036. This final chapter looks forward. For each open problem, we ask what does not yet work well enough, what has been proposed, and how far the work has progressed. Everything in the following sections is necessarily provisional: proposal statuses are those recorded in the specifications repository at this chapter's reference date.

31.1 How the Protocol Changes

Since 2007, protocol changes have been developed through numbered **proposals**, somewhat like IETF RFCs. A contributor discusses an idea on the developers' mailing list, writes it in the required format, and, once there is broad agreement that the idea deserves consideration, the editors add it to the repository under a proposal number. The proposal is discussed and refined until it is detailed enough to implement. Following implementation, its normative content is incorporated into the specifications. Unlike an RFC, a proposal may change while retaining its number, and it is never the definitive document for an implemented feature: that role belongs to the specifications themselves (Proposal 001).

A proposal's status (fig. 31.1) should be the first thing readers check. This book has encountered proposals in many states. ntor-v3 (Proposal 332) is *Closed*: implemented and incorporated into the specifications. Congestion control (Proposal 324) and Conflux (Proposal 329) are *Finished*: implemented but not yet fully incorporated. CGO (Proposal 359) is *Open* despite already being present in both tor and Arti. The plan for removing TAP (Proposal 350) is *Accepted*, while Walking Onions (Proposal 300) is *Informational*.

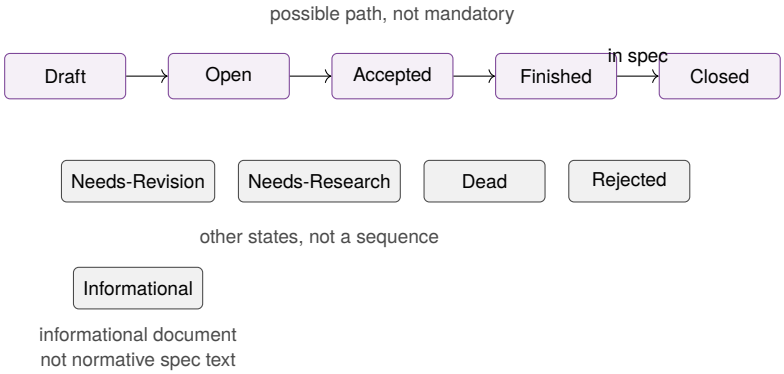


Figure 31.1: Tor proposal statuses under Proposal 001 (simplified, non-exhaustive diagram). The path *Draft–Open–Accepted–Finished–Closed* is illustrative, not required: *Finished* denotes implementation and *Closed* also denotes incorporation into specifications. *Needs-Revision* and *Needs-Research* name different blockers; *Dead* may be reopened, while *Rejected* rejects the feature as proposed. *Informational* denotes a final informational document, not itself normative specification text. Other statuses, including *Meta*, *Reserve* and *Obsolete*, are not drawn.

31.2 Quantum Computers

The problem. All of Tor's public-key mechanisms, including X25519 for handshakes, Ed25519 for signatures, and the remaining uses of RSA, would be broken by a sufficiently powerful quantum computer (chapter 5). Nobody knows whether or when such a computer will exist. For an anonymity system, however, the threat can begin earlier: an adversary might record encrypted traffic today and decrypt it in the future.

Two scenarios. Proposal 355, from 2025, distinguishes two situations. In a *transition* environment, protection against a *future* quantum computer is enough: traffic captured now must resist later decryption, while traditional signatures remain acceptable because an active signature-forgery attack would require a quantum computer today. In a *next-generation* environment, protection must also cover a quantum computer that *already exists* and can forge signatures or actively attack handshakes.

The cost. In 2024, NIST standardized ML-KEM, a post-quantum key-establishment mechanism, together with the signature algorithms ML-DSA and SLH-DSA [NIS24]. The proposal compares their sizes with today's algorithms:

Algorithm	Public key	Ciphertext or signature
X25519	32 bytes	32 bytes
ML-KEM-768	1 184 bytes	1 088 bytes
Ed25519	32 bytes	64 bytes
ML-DSA-65	1 952 bytes	3 309 bytes

In chapter 8, we saw that a relay message can carry no more than 498 bytes. A single ML-KEM-768 public key would therefore require three cells before accounting for the rest of a handshake. Post-quantum cryptography is not simply a matter of replacing algorithms: it changes how messages fit inside cells. The current relay format cannot directly carry a message of that size: three cells is only a lower bound on payload capacity, assuming that a future format can fragment the message.

Current direction. The proposal's conclusions are cautious. During the transition, adding a post-quantum mechanism to ntor, as earlier proposals suggested, may suffice. For a next-generation system, the preferred design uses two post-quantum key-establishment mechanisms to provide forward secrecy as well, if the cost is acceptable. Adding a post-quantum key to microdescriptors would also offset some of the bandwidth savings achieved by removing TAP keys (Proposal 350) and introducing key-based families (section 22.5). Decisions depend on measurements still needed: how often are circuits built from scratch, and how much relay bandwidth and CPU time do handshakes consume? This remains an open research problem and an opportunity to contribute measurements (chapter 29).

31.3 Larger Messages, and Smaller Ones

Proposal 340, currently *Open* at the reference date, directly addresses the one-message-per-cell constraint. It would separate relay messages from cells: long messages could span several cells, and several short messages could share one cell. It is motivated both by messages exceeding 498 bytes, needed for post-quantum handshakes, UDP transport, and Walking Onions, and by very short messages, such as `SENDME`, `XON`, and `XOFF`. These currently consume an entire cell and could instead be concealed within the padding space of other messages, making them invisible to an observer. The new format is intended to work with updated relay encryption, whose CGO design was discussed in section 10.4; coordinating the changes would avoid proliferating incompatible formats.

31.4 Performance: Multiple Paths

Congestion control (section 11.3) made individual circuits more efficient. The next step is **Conflux** (Proposal 329, *Finished*). A client builds two circuits to the same exit with different guards and middle relays, links them, and distributes traffic according to each circuit's measured round-trip time. The idea originated in research by AlSabah, Bauer, Elahi, and Goldberg; Tor's version adapts it using congestion-control information and experience from multipath TCP. We observed Conflux circuits in the laboratory network (section 19.7.3) and through Stem (chapter 18). The proposal recommends that the two legs share no relays apart from the exit, avoiding shared bottlenecks. This is a recommendation rather than a strict rule and is often violated on the laboratory's small network. tor has used Conflux since version 0.4.8; Arti, as noted in section 30.3, does not yet do so at this reference date. OnionPerf measurement points operating with and without Conflux allow its real-network effects to be compared (section 24.3).

31.5 Scalability: Walking Onions

Every Tor client currently knows *every* relay: it downloads the consensus and microdescriptors, and directory traffic grows with the number of relays. Microdescriptors and consensus diffs reduce the cost but do not change its fundamental shape. The initial cost is also concentrated during bootstrap: a new client must download a full directory before constructing its first circuit (Proposal 300).

Obvious solutions have known drawbacks. Showing each client only part of the network creates *epistemic* attacks: the differences between clients' views can identify them. Allowing each relay to select the next relay, as in some decentralized systems, exposes paths to capture by malicious relays. Walking Onions proposes another approach: authorities publish a signed structure associating each relay with a numeric interval. The client chooses a number

randomly, and the previous relay in the circuit provides a signed proof identifying the relay occupying that interval, together with the extension. The client verifies the signature without ever needing the complete list. The original proposal is *Informational*; a more detailed follow-up (Proposal 323) tries to address its remaining gaps. Recent proposals, including the post-quantum work, take it into account.

31.6 Other Open Fronts

UDP Tor only transports TCP. Video calls, games, and QUIC, the transport underlying HTTP/3, are consequently unsupported or require potentially unsafe workarounds. Proposal 348 (*Open*) explores options ranging from tunnels to exits to native datagram transport, along with new risks: application fingerprints, peer-to-peer metadata collection, and port exhaustion.

Fingerprinting We ended section 26.4 with an open problem: no general website-fingerprinting defense has yet achieved acceptable overhead. Padding state machines, deployed by tor for onion-service circuits and under development in Arti using the *maybenot* library, provide a framework for future defenses.

Correlation The fundamental issue in chapter 26 remains unresolved: a low-latency network can preserve correlatable timing and volume patterns despite buffering and competing traffic. Known defenses such as constant-rate padding and added delays are costly; research continues into more efficient compromises and path selection that accounts for Internet topology.

DoS Proof of work protects onion services (chapter 15); automatically tuning this mechanism is the subject of Proposal 362.

Arti as a relay Once Arti can serve as a relay and authority, the C codebase may eventually be retired. This is one of the Tor Project's longer-term undertakings (chapter 30).

31.7 How to Contribute

Readers who have reached this point have several ways to contribute, not all of them requiring code:

- **Contribute bandwidth**: operate a middle relay or bridge or, with the precautions in chapter 25, an exit (chapter 22); alternatively, run a Snowflake proxy, even from a browser.
- **Read and discuss proposals**: many remain *Open* because they need comments, analysis, and measurements. The developers' discussion list is public.
- **Write code**: tor and Arti are developed on the Tor Project's GitLab, with issues labeled for newcomers. Arti is especially important for new features.
- **Conduct research**: use Shadow, Chutney, and Metrics data while following Research Safety Board guidance (chapter 29).
- **Translate and document**: volunteers translate Tor Browser, its manual, and the website. More documentation is always useful in languages including Italian.
- **Use Tor**: a larger community may increase the anonymity set, but user count alone does not guarantee indistinguishability when timing, behavior, or other metadata differ.

Summary

- The protocol evolves through numbered proposals, whose status moves from *Draft* toward *Closed* as changes are implemented and integrated into specifications.
- Post-quantum cryptography matters even before powerful quantum computers exist; it requires messages larger than a single cell, motivating proposal 340 on message fragmentation and packing.
- Conflux distributes traffic across two circuits; Walking Onions could eliminate the need for each client to learn the complete network.
- UDP, fingerprinting and correlation defenses, DoS tuning, and Arti's transition to relay operation all remain open challenges.

Exercises

Exercise 31.1 ⋆ Choose three proposals cited in this book. For each, identify its present status and what that status means: is the feature already deployed, and is it described in the specifications?

Exercise 31.2 ⋆ How many cells are needed to carry one ML-KEM-768 public key and one ML-KEM-768 ciphertext if each relay message carries at most 498 bytes and separate messages cannot share a cell? What would change under Proposal 340? Distinguish the theoretical lower bound on cell capacity (excluding framing overhead) from the actual number of cells, which would depend on the proposal's encapsulation rules.

Exercise 31.3 ⋆⋆ Illustrate an epistemic attack: if different clients knew different subsets of the network, how could an adversary exploit those differences?

Exercise 31.4 ⋆⋆ Using Stem on the laboratory network, identify pairs of Conflux circuits sharing a `CONFLUX_ID`. Verify that both legs end at the same exit and whether they avoid all other shared relays, as recommended by Proposal 329. What do you observe, and how do you explain it?

Exercise 31.5 ⋆⋆⋆ Choose an open problem from this chapter and write a short proposal in the Tor Project format: motivation, design, security analysis, compatibility, and implementation plan.

Further Reading

- The proposal index in the `torspec` repository, organized by status, and the proposal process document (Proposal 001).
- N. Mathewson, *Options for postquantum circuit extension handshakes* (Proposal 355).
- *Walking Onions: Scaling and Saving Bandwidth* (Proposal 300) and its follow-up (Proposal 323).
- The Tor Project blog, which announces and explains protocol changes.

Appendix A

Glossary of Tor Terms

This glossary defines the technical terms used throughout the English edition. Each entry also points to the chapter where the term is introduced.

Term	Meaning	Ch.
traffic analysis	Inferring information from the timing, volume, and addressing of communications without reading their contents.	1
anonymity	Remaining unidentifiable within a set of subjects, called the anonymity set.	1
Sybil attack	Presenting many identities, such as multiple relays, to a network to increase the chances of being selected.	26
directory authority	One of the small number of servers that vote on and sign the consensus.	12
bridge	A relay absent from the public consensus that serves as an entry point for users facing censorship.	16
channel	A TLS connection between two adjacent Tor nodes, carrying cells from multiple circuits.	7
cell	The transmission unit on a channel: ordinarily 514 bytes, with variable lengths for certain commands.	8
blinded key	A key derived from an onion service's identity key for a given period, concealing the service's address from HSDirs.	14
family key	A key shared by relays operated by the same organization, certifying their family membership.	22
circuit ID	A circuit identifier local to one channel; it changes at every hop.	8
circuit	A layered-encrypted route through multiple relays, constructed one hop at a time.	9
consensus	The authority-signed document listing relays, their flags, and their weights.	12
congestion control	Adjusting the quantity of in-flight circuit data according to observed round-trip time.	11
flow control	Preventing a sender from overwhelming a receiver by means of windows and SENDME messages.	11
traffic correlation	Recognizing that two observed flows at different network points belong to the same communication.	26
descriptor	A document by which a relay or onion service publishes its keys, addresses, and properties.	12
exit relay	The last relay on a circuit, opening the connection to the destination.	6

Term	Meaning	Ch.
family	A group of relays under common management that must not appear together in one circuit.	13
flag	A property assigned to a relay by directory authorities, such as `Guard`, `Exit`, or `Stable`.	12
forward secrecy	A property under which compromise of a long-term key does not allow past traffic to be decrypted.	5
guard, entry guard	The first relay of a client's circuits, selected persistently and reused over an extended period.	13
handshake	An exchange of messages through which two parties agree on keys, such as ntor or ntor-v3.	9
hidden service directory	A relay that stores and serves onion-service descriptors.	14
middle relay	A relay in the middle of a circuit that knows neither the originating client nor the destination.	6
stream isolation	Assigning different streams to different circuits, for example on the basis of SOCKS credentials.	18
link protocol	The versioned protocol used over a channel, negotiated with a **`VERSIONS`** cell.	7
relay message	The content of a **`RELAY`** cell addressed to a particular hop, such as `BEGIN` or `DATA`.	8
microdescriptor	A compact descriptor containing only the relay information a client needs.	12
threat model	An explicit description of which adversaries and threats a system is intended to resist.	4
unlinkability	The inability of an observer to determine whether two actions belong to the same subject.	1
onion service (formerly hidden service)	A service reachable through Tor whose address encodes its public key; formerly called a hidden service.	14
padding	Filler data in cells or on a channel, used to conceal lengths and timing.	7
bandwidth weights	Consensus coefficients balancing relay selection across different circuit positions.	13
pluggable transport (PT)	A separate program transforming Tor traffic to resist recognition or blocking, such as obfs4, Snowflake, or WebTunnel.	16
exit policy	An ordered rule set describing which destinations an exit relay will accept.	22
proof of work (PoW)	A computational challenge a client solves to gain priority when accessing an onion service under attack.	15
proposal	A numbered document used to discuss and design a change to the Tor protocol.	31
pseudonym	An identifier different from a person's real name, allowing actions to be linked without disclosing identity.	1
introduction point	A relay chosen by an onion service to receive initial contact requests.	14
rendezvous point	A relay chosen by the client at which the client's and onion service's circuits join.	14
relay, onion router	A Tor network node that forwards cells.	6
SENDME	A message acknowledging received circuit data and reopening a flow-control window.	11
subprotocol	A versioned family of protocol features advertised by relays and clients, such as `Link` or `Relay`.	30

Term	Meaning	Ch.
stream	An application data flow, such as a TCP connection, carried over a circuit.	11
vanguard	A middle-layer relay held fixed for a period to help protect an onion service's guard.	13
website fingerprinting	Recognizing which page was visited using only sizes and timing of encrypted traffic.	26

Appendix B

Further Cryptography

Last verified: 23 September 2026 — dir-spec and tor-spec (torspec commit `8e1f245`); the `cryptography` library; laboratory network

Chapter 5 introduced Tor's cryptographic building blocks with minimal mathematical background. This appendix goes into three topics only briefly mentioned there, using runnable examples throughout: X25519 keys and their clamped bits, conversion between the Montgomery and Edwards forms of the same curve, verified against an actual certificate, and stream-cipher malleability, which enables the marking attack described in section 10.3.

B.1 One Curve, Two Forms

X25519 and Ed25519 use the *same* elliptic curve over the field of integers modulo $p = 2^{255} - 19$, expressed in two forms:

Montgomery form $v^2 = u^3 + 486662\,u^2 + u$. X25519 uses this form: a public key consists only of a point's u coordinate, encoded in 32 little-endian bytes. This form permits a straightforward, fast scalar-multiplication algorithm, the "Montgomery ladder", with execution time independent of the secret key.

Twisted Edwards form $-x^2 + y^2 = 1 + d\,x^2y^2$, where $d = -121665/121666$. Ed25519 uses this form for signatures: a public key is the y coordinate, with the sign of x stored in the highest bit of the final byte.

An explicit transformation relates the two forms; we use it in B.2.

B.1.1 X25519 Key Clamping

An X25519 private key begins as 32 bytes, but the implementation forces certain bits to specified values before using it: it clears the three least-significant bits, clears the highest bit, and sets the next-highest bit. Clearing the three low bits makes the scalar a multiple of 8, the curve's *cofactor*, so that small-order components of a received point do not leak information about the scalar. Fixing the high bits also fixes the number of Montgomery ladder steps. We can verify this with the same library used in the laboratories:

```
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
from cryptography.hazmat.primitives import serialization as s
import os
def public_key(k):
    return X25519PrivateKey.from_private_bytes(k).public_key().public_bytes(
        s.Encoding.Raw, s.PublicFormat.Raw)
```

```
a = bytearray(os.urandom(32)); b = bytearray(a)
b[0] ^= 0x07                    # only the three low bits change
print(public_key(bytes(a)) == public_key(bytes(b)))      # True
```

Two private-key byte strings differing only in bits overwritten during clamping produce the same public key.

B.1.2 The Point at Infinity

In an ntor handshake, both client and relay check that none of their EXP() operations produces the point at infinity. The specification notes that, for Curve25519, this check adequately replaces validating that Y belongs to the group [tor-spec § The "ntor" handshake]. Without it, an adversary supplying a small-order public point could produce a predictable shared secret, such as an all-zero string. The `cryptography` library performs this check itself: calling `exchange` with an all-zero 32-byte public key raises an exception (`Error computing shared key`) instead of returning a zero secret.

B.2 From Montgomery to Edwards: The ntor Cross-Certificate

Each relay publishes its ntor public key, an X25519 key, in its descriptor and proves possession of the associated private key with `ntor-onion-key-crosscert`: a type-`0a` Ed25519 certificate certifying the relay's main Ed25519 identity, but *signed with the ntor key* [dir-spec § ntor-onion-key-crosscert]. That raises a problem: the ntor key uses Montgomery form, whereas Ed25519 signatures need Edwards form. The specification defines a conversion [dir-spec § Converting a curve25519 public key to an ed25519 public key]:

$$y = \frac{u - 1}{u + 1} \bmod p.$$

The u coordinate determines y but not the sign of x: two symmetric Edwards public keys correspond to each Montgomery coordinate. That sign cannot be recovered from the public Montgomery key, so the descriptor supplies it explicitly as the cross-certificate field's first argument: 0 for positive and 1 for negative. The relay computes it from its private key.

The `labs/en/code/torlab/conversion.py` module implements the conversion in a few lines:

```
P = 2**255 - 19
def montgomery_to_edwards(u_bytes, sign):
    u = int.from_bytes(u_bytes, "little") & ((1 << 255) - 1)
    y = (u - 1) * pow(u + 1, P - 2, P) % P      # division via the inverse modulo p
    compressed = bytearray(y.to_bytes(32, "little"))
    compressed[31] |= (sign & 1) << 7          # x-coordinate sign in the high bit
    return bytes(compressed)
```

It then verifies the cross-certificate with the certificate decoder from Laboratory 1. Applied to the laboratory network's descriptors (key abbreviated for readability; keys change with every new network):

```
$ PYTHONPATH=labs/en/code python3 -m torlab.conversion labs/en/lab/net/nodes/000a/
    cached-descriptors test004r
test004r: ntor-onion-key DaxPE+Grnjex2Hmv+5qutl5dIX4La2jF6MIL7IK6+3E
  Edwards key (sign 0): c369d08f8083002a15fe22f65dca14bf...
  valid crosscert; certified identity: kjPIgDB2OFsGJzuwsT+KMS/El9qrlE2/4P5WvIvmzC8
```

In the reference transcript, the signature is valid and the certified identity matches the one recorded in the relay's `fingerprint-ed25519` file. The same applies to the other laboratory relays. Verification fails if we supply the wrong sign, one of the module's automated negative tests. Division modulo p is calculated using Fermat's little theorem, by multiplying by $(u + 1)^{p-2}$.

B.3 Stream-Cipher Malleability

AES in counter mode (section 5.2) generates a byte stream combined with the message using XOR. Anyone changing a ciphertext bit changes *the corresponding plaintext bit* without knowing the key; they need only know or guess the original content. Here is a Python example:

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os
k, n = os.urandom(16), bytes(16)
cipher = lambda d: Cipher(algorithms.AES(k), modes.CTR(n)).encryptor().update(d)
c = bytearray(cipher(b"BEGIN www.example.org:443"))
delta = bytes(x ^ y for x, y in zip(b"www.example.org", b"www.example.net"))
for i, d in enumerate(delta):
    c[6 + i] ^= d                # targeted modification, without the key
print(cipher(bytes(c)))           # b'BEGIN www.example.net:443'
```

Tor's original layered relay encryption, tor1, uses AES-CTR with integrity checked only at the circuit's endpoint by a four-byte digest. A malicious relay may therefore alter a cell predictably and have an accomplice further along the route recognize the alteration. This is the marking attack discussed in section 10.3, and one motivation for CGO (section 10.4), whose design makes an altered cell, and subsequent cells, unrecoverable.

Exercises

Exercise B.1 ⋆ How many distinct values can an X25519 private scalar take after its bits have been clamped? Compare the answer with 2^{256}.

Exercise B.2 ⋆⋆ Derive and implement the inverse transformation from an Ed25519 public key's y coordinate to the Montgomery u coordinate. Preserve the original Ed25519 key's x-sign bit; then use `montgomery_to_edwards` from `conversion.py` to check that the two conversions recover the original Ed25519 public key (for $y \neq 1$).

Exercise B.3 ⋆⋆⋆ In tor1, a cell has a four-byte digest and a two-byte all-zero `recognized` field. What is the probability that a randomly modified cell is accepted as valid by its final hop? Why can marking nevertheless succeed?

Appendix C

A Guide to the torspec Specifications

Last verified: 23 September 2026 — torspec commit `8e1f245`

The Tor Project's specifications are the primary source for almost every chapter in this book. They are comprehensive but were not written as a textbook. Originally maintained as separate text files over more than twenty years, they were brought together on one website only comparatively recently. Their own conventions acknowledge that consistency remains a work in progress. This appendix provides a map for readers who want to study the specifications after reading the book.

C.1 Where to Find Them

The specifications are published at `https://spec.torproject.org/` and maintained alongside proposals in the Tor Project's `torspec` GitLab repository. There, the specifications are Markdown files under `spec/`: individual files for smaller subjects and directories for more extensive documents. The `permalinks.md` file lists stable document addresses, such as `https://spec.torproject.org/tor-spec`; use these when citing the specifications. Throughout the book we cite documents as [document § section], preserving the section titles as they appear on the site, and proposals as "Proposal NNN".

C.2 Conventions

The introductory conventions document establishes rules that apply to the specifications unless otherwise noted:

- `MUST`, `SHOULD`, `MAY`, and their negative forms have their RFC 2119 meanings: respectively a requirement, strong recommendation, or permitted option.
- Lengths are measured in eight-bit bytes, and binary prefixes such as "kibibyte" denote multiples of 1024.
- Multibyte integers use big-endian, or "network", byte order: decimal 4660 in two bytes becomes `12 34`.

Two important exceptions appeared earlier in the book: Curve25519 and Ed25519 keys and points are little-endian (appendix B), while some directory formats use readable text with base64-encoded binary data.

C.3 Document Map

Document	Contents	Chapters
`tor-spec`	Main protocol: channels, cells, circuits, handshakes, relay encryption, streams, flow control and congestion control	7–11
`cert-spec`	Tor's Ed25519 certificates	7
`dir-spec`	Authorities, votes, consensus, descriptors, microdescriptors and extra-info documents	12, 24
`path-spec`	Path selection and constraints	13
`guard-spec`	The guard algorithm	13
`vanguards-spec`	Full and lightweight vanguards	13
`rend-spec`	Onion services v3: keys, descriptors, introduction and rendezvous	14, 21
`hspow-spec`	Proof of work for onion services	15
`dos-spec`	Defenses against denial of service	15
`pt-spec`	The interface between tor and pluggable transports	16
`padding-spec`	Channel and circuit padding	7, 26
`control-spec`	The ControlPort protocol	18
`socks-extensions`	Tor-specific SOCKS extensions	11, 28
`param-spec`	Consensus network parameters and their defaults	11, 15
`address-spec`	Special names, such as `.onion` addresses	14
`srv-spec`	The authorities' shared random value	14

C.4 How to Read a Section

A few suggestions drawn from writing the book:

1. **Start with the format.** Binary-message sections give field names and sizes in tables. Reconstruct the format on paper, as we did in the book's format diagrams, before reading the rules.

2. **Distinguish current from obsolete material.** Many documents preserve historical features such as TAP, link-protocol versions 1–3 and onion services v2. Obsolete material is often grouped in sections or files such as `obsolete-channels.md` or `historical/`, but not always. When uncertain, inspect the code to see whether a branch still exists.

3. **Compare with the implementations.** Specifications express intent; the tor and Arti codebases reveal actual behavior. Several discrepancies proved informative during the book's research: channel padding absent from single-hop circuits (chapter 7), and the closure reason observed at an exit (section 22.4).

4. **Check proposal status.** A proposal may be implemented before it is integrated into a specification, or still be under discussion without being implemented (section 31.1).

5. **Verify experimentally.** A private network like the book's laboratory allows readers to test whether a field behaves as documented. This method underlies all the laboratories.

C.5 Arti and the Specifications

Arti's code frequently refers to the specifications in comments and documentation and identifies ambiguous requirements or differences from tor. Its supported-subprotocol table (section 30.3) indicates which features both implementations support and therefore which parts of the specifications describe the "living" protocol.

Appendix D

Exercise Solutions

Chapter 1: Anonymity, Surveillance, and Metadata

Solution 1.1. With HTTPS, the Internet provider sees the website's IP address and potentially its domain name, from DNS resolution and often from the TLS SNI field. It also sees timing, connection duration, and traffic volume. With a VPN, the provider sees only the connection to the VPN server; the VPN provider, however, can see everything the Internet provider could previously see. With Tor, the provider sees a connection to a Tor relay or bridge, along with its timing and volume. No single point within the Tor network can see both the user and the destination.

Solution 1.2. An account under a fictional name is pseudonymity. An unsigned message on a noticeboard is anonymity, provided nothing else identifies its author. A letter signed with initials is weak pseudonymity and may often permit identification. An anonymous cash donation is an example of anonymity.

Solution 1.3. The anonymity set shrinks from fifty Tor users to those speaking Italian who are also awake at 3 a.m., potentially only a handful. Tor hides an IP address, not behavior. Language, timing, writing style, and personal information can identify a user even when network traffic cannot.

Solution 1.4. A VPN provider sees both the user and their destinations. All trust is concentrated in one operator, which may log activity, be legally compelled to hand over data, or be compromised. Tor distributes trust across three independently operated relays, none of which sees both endpoints. Linking the user and destination would require compromising or simultaneously observing both the first and last hops.

Chapter 2: The Roots

Solution 2.1. Randomly, the probability is 1 in 100. If an observer supplied 99 of the 100 messages in a batch, they could recognize their own outgoing messages and identify the only remaining message as the victim's. Anonymity disappears. This is the $n - 1$ *attack*, which real mix networks address using more sophisticated batches, dummy traffic, and controls on incoming messages.

Solution 2.2. Use an *anonymous return address* in Chaum's sense. The user prepares an encrypted, layered "return onion" for a chain of mixes. Its innermost layer contains the user's address and a key for encrypting the reply. The respondent attaches their message to the

onion; each mix removes one layer, and no individual mix learns the final destination. No central routing table is required.

Solution 2.3. At each step, the request is forwarded to another crowd member with probability $3/4$ and sent to the website with probability $1/4$. The number of members traversed follows a geometric distribution with mean $1/(1/4) = 4$, counting the first forwarding step. A member cannot know whether the member forwarding the request originated it, because intermediate forwarding follows the same rule.

Chapter 3: From NRL Onion Routing to the Tor Project

Solution 3.1. With three hops, a malicious central node immediately knew the two organizational networks at the endpoints, because entry and exit nodes were located within those organizations' networks. With four hops, someone compromising a node near one endpoint knew which additional node to attack to reconstruct the circuit; with five hops, one more compromise was needed. Tor uses three hops because its relays do not reside in the users' networks, and because every additional hop increases latency without greatly reducing the principal threat of entry–exit correlation. Guards instead help address that threat.

Solution 3.2. A single onion encrypted under the nodes' long-term public keys provides no forward secrecy. Someone obtaining those keys later can decrypt recorded onions. With telescoping construction, each hop performs an ephemeral key exchange and the ephemeral secrets are discarded after use. The client also builds the circuit one hop at a time and can immediately identify an unresponsive relay and replace it. With a single onion, the failure becomes apparent only after attempting the full route.

Solution 3.3. This is an open-ended argumentation exercise. Verifiable elements include published code and specifications, reproducible builds, independent analysis in the scientific literature, and a network run by independent operators. Elements requiring trust include whether distributed builds match the published source, although this can partly be verified; whether directory authorities collude; and whether unknown vulnerabilities exist. Security should be evaluated through verifiability and distributed trust, not presumed from a project's funding sources.

Chapter 4: The Threat Model

Solution 4.1. (a) Yes: the network operator sees only a connection to a Tor relay. (b) Yes: the website sees the exit's address. (c) No, with respect to identity: the user identifies themselves by signing into the account. Tor hides only where they connect from. (d) Probably not: an adversary observing both endpoints, the user and the newspaper, could attempt timing correlation, precisely the scenario excluded from Tor's threat model.

Solution 4.2. A difficult system attracts only a few highly motivated users, often precisely those most in need of protection. Merely using it distinguishes them, leaving a small anonymity set. An easy-to-use system adopted by millions for ordinary reasons helps conceal those with more sensitive needs.

Solution 4.3. This is an open-ended exercise. Possible adversaries include the source's government or employer observing the local network, the email or messaging provider, and entities controlling networks in either country. Tor prevents the local network from learning whom the journalist communicates with and prevents the service from seeing the journalist's IP address. It does not protect message content without end-to-end encryption,

metadata visible to the service such as accounts and timestamps, compromised devices, or traffic correlation by an adversary observing both ends. An onion service such as SecureDrop reduces several of these risks.

Chapter 5: Essential Cryptography

Solution 5.1. $C_1 \oplus C_2 = P_1 \oplus P_2$, because the reused keystream cancels out. Given P_1, recover $P_2 = C_1 \oplus C_2 \oplus P_1$.

Solution 5.2. The 16-byte output equals the first 16 bytes of the 32-byte output: SHAKE generates one stream that can be read to any desired length. A KDF can therefore request a long stream and divide it into parts. However, requesting different lengths for different purposes does *not* provide domain separation; distinct domain labels are still needed.

Solution 5.3. Both shared secrets match. An all-zero public key represents a small-order point and would produce an all-zero shared secret regardless of the private key: a secret known to everyone. The `cryptography` library refuses to return it. ntor explicitly requires this check (section 9.6); otherwise, an adversary could force a predictable shared secret.

Solution 5.4. Without domain separation, a value calculated in one context might be accepted in another. For example, a MAC generated to authenticate a handshake message might be presented as a key or proof in another phase, or in a different protocol using the same function and keys. Domain labels make inputs for different uses disjoint by construction.

Chapter 6: Network Architecture

Solution 6.1. Guard: the user's IP address, traffic timing, and volume, but not the destination. Middle: neither the user nor the destination. Exit: the destination and any content not encrypted by the application, but not the user. HSDir: no information about a user visiting an ordinary website. Directory authority: normally no information about a particular website visit. It may see the IP address of a client fetching directory documents *directly*, especially during bootstrap. If the client fetches them from a fallback directory or cache, that server rather than the authority sees the request.

Solution 6.2. `Link=3-5` announces link protocols up to version 5 (chapter 7). `Relay=2-6` includes ntor, ntor-v3 (`Relay=4`), and CGO (`Relay=6`). `FlowCtrl=1-2` means authenticated SENDME and congestion control. The relay can serve as a CGO exit provided it also carries the `Exit` flag and negotiates congestion control together with CGO (section 9.7).

Solution 6.3. 100 KB of application data requires approximately $102400/498 \approx 206$ DATA cells, or roughly $206 \times 514 \approx 106$ KB on the link. Add a few cells for the request, opening the stream, and SENDME messages (about seven, one for every 31 data cells), plus TLS-record overhead. Overall, this is roughly 5–7% above the useful payload, excluding initial handshakes.

Chapter 7: Channels and the Link Protocol

Solution 7.1. A client always initiates its connections: to its guard and to relays providing directory information. No one needs to contact the client, and listening on a port would make it easier to identify. A relay initiates channels chiefly when it receives an `EXTEND2` request

and must reach the circuit's next hop, or when, like any node, it creates its own circuits to download directory documents or check its reachability.

Solution 7.2. `0x000772F7` $= 488183$ hours, corresponding to 9 September 2025 at 23:00 UTC. A 32-bit unsigned count of seconds would overflow in 2106. Counting hours instead gives hundreds of thousands of years, while hour-level resolution is sufficient for certificates lasting days or weeks. One specification note deserves caution: `cert-spec` says four bytes suffice until the year 10136. That is an error explicitly identified in Proposal 338; it arose from interpreting the counter as measuring *minutes*, since 2^{32} minutes are approximately 8,166 years.

Solution 7.3. $38 = 32 + 2 + 2 \times 2$ describes a challenge with two methods. If N_Methods is 2, the body must contain $32 + 2 + 4 = 38$ bytes. A 36-byte cell body declares more methods than it contains and is malformed, so it must be rejected. The specification permits ignoring *extra* bytes, not missing bytes.

Solution 7.4. The comparison between the key certified by certificate type 5 and the SHA-256 fingerprint of the adversary's TLS certificate fails: it differs from the genuine relay's certificate. If the adversary presented the identical genuine TLS certificate, they could not complete the TLS handshake without the corresponding private key.

Solution 7.5. `695661c0` (1 January 2026, 12:00:00 UTC); `04 04 c6336407` (the client's address, 198.51.100.7); `01`; and `04 04 cb007105` (the relay's address, 203.0.113.5), followed by zero padding up to 509 bytes.

Solution 7.6. First, the guard would see the same identity when the user moved between home, work, and mobile networks, linking sessions and IP addresses across all those environments. Second, the identity would also be supplied to relays from which the client fetched directory information. Colluding relays, or an adversary observing them, could link the user's activity across different relays. In either case, a persistent identifier would destroy Tor's intended unlinkability.

Solution 7.7. Outline: parse the type-2 certificate with an X.509 library such as `cryptography.x509`. Extract its RSA public key, verify its self-signature, and check that its exponent is 65537. Next, parse the type-7 cross-certificate according to its format. Ensure that the certified Ed25519 key matches the already verified identity and check the RSA signature using PKCS#1 v1.5 without a hash-algorithm OID over the SHA-256 hash of the prescribed prefix and fields. The relay fingerprint is the SHA-1 digest of the RSA public key's DER-encoded PKCS#1 representation.

Solution 7.8. No. Client `test008c`'s traffic passes through *its own* channel to its guard, not through our channel. Our channel has never been used for a circuit, so `channelpadding_decide_to_pad_channel` decides not to add padding. To observe channel padding, we must carry a multihop circuit over our own channel, as in chapter 10.

Solution 7.9. TLSSECRETS is derived from the secrets of the TLS session between A and the intermediary, and A's signature covers it. B recalculates TLSSECRETS from the secrets of *its own* session with the intermediary. Those secrets differ, so verification fails even when the intermediary forwards every cell faithfully, making SLOG and CLOG match. Without TLSSECRETS and SCERT, the intermediary could present itself to B as A and forward A's signature, misleading B into believing it had an authenticated channel to A.

Chapter 8: Cells

Solution 8.1. **`CREATE2`** is a fixed-length cell: 512 bytes under link protocol 3 with a two-byte CircID, or 514 bytes under version 5 with a four-byte CircID. The **`VERSIONS`** cell always uses a two-byte CircID, giving $2 + 1 + 2 + 4 = 9$ bytes.

Solution 8.2. 0 (**`PADDING`**) is fixed-length; 7 (**`VERSIONS`**) is variable-length; 9 (**`RELAY_EARLY`**) is fixed-length; 12 (**`PADDING_NEGOTIATE`**) is fixed-length; and 129 (**`CERTS`**) is variable-length. Command 200 would be variable-length under the ≥ 128 rule, but it is undefined and therefore not permitted. A recipient may discard it and close the channel.

Solution 8.3. Only `0x92345678`. A CircID of zero is reserved for cells outside a circuit. Under link protocol ≥ 4, the side initiating the channel, here the client, must select CircIDs with their most significant bit set to 1; `0x12345678` has its MSB set to 0.

Solution 8.4. First cell: CircID `00000000`, command `0x81` = 129 (**`CERTS`**), so variable-length; length `0003`, body `aa bb cc`. Second cell: CircID `0x80000007`, command `0x04` (**`DESTROY`**), fixed-length, so it has a 509-byte body. The first body byte, `00`, means reason NONE. The `aabbcc` body is not itself a valid CERTS object: the exercise concerns cell framing only.

Solution 8.5. `02 0000 0005 00000000 012c`: DATA command (2), Recognized 0, StreamID 5, Digest provisionally zero, and length $300 =$ `0x012c`. The remaining $509 - 11 - 300 = 198$ bytes are padding: the first four bytes are zero and the other 194 are random.

Solution 8.6. For a cell still encrypted in another layer, Recognized is zero with probability 2^{-16}: on average, about $10^6/65536 \approx 15$ such cells per million. Recognized alone is insufficient because false positives occur. Only comparison with the running digest, which a non-target relay cannot make match except by chance (another factor of 2^{-32}), confirms that the relay is the cell's destination.

Solution 8.7. Suppose an adversary controls the guard and exit. When the exit wants to mark a circuit, it closes it using a rare reason such as `NOSUCHSERVICE`. If the middle and guard forwarded the reason unchanged toward the client, the guard could observe that unusual value on one of its circuits at the same moment, linking the client known to the guard with the destination known to the exit. If all relays transmit and propagate only NONE, the reason byte cannot carry such a signal. Correlation through the timing of circuit closure remains possible, but that channel is much noisier.

Solution 8.8. Traditional format: $\lceil 1048576/498 \rceil = 2106$ cells, occupying $2106 \times 514 = 1\,082\,484$ bytes on the channel, or 3.2% overhead. CGO: $\lceil 1048576/488 \rceil = 2149$ cells, occupying $1\,104\,586$ bytes, or 5.3% overhead. SENDME and stream-establishment messages are excluded.

Solution 8.9. Outline: use Hypothesis to generate lists of tuples containing CircID, command, and body. Choose commands from `CellCommand` and limit fixed-length cell bodies to 509 bytes. Concatenate the results of `encode`, split the result into fragments of random lengths, and feed them incrementally into a buffer. Repeatedly call `decode` to extract cells until no complete cell remains. Compare fixed-length cells against bodies zero-padded to 509 bytes.

Solution 8.10. In Arti 2.6.0, the channel codec is `ChannelCodec` in `crates/tor-cell/src/chancell/codec.rs`, initialized with the link-protocol version using `ChannelCodec::new`. Its methods `write_cell` and `decode_cell` serialize and extract cells. Like our `decode`, `decode_cell` handles incomplete cells without raising an error, returning `Ok(None)` until enough bytes have arrived and consuming buffer data only for complete cells. Arti instead operates on a mutable `BytesMut` buffer and is generic over channel message type (`M: ChanMsg`), allowing

a narrower set of accepted messages. Another notable difference: `decode_cell` reads the command at offset 4 and assumes 514-byte fixed-length cells, meaning four-byte CircIDs. Arti only supports link protocols 4 and 5, as specified by `LINK_PROTOCOLS` in `crates/tor-proto/src/channel/handshake.rs`; the initial two-byte-CircID **`VERSIONS`** cell is handled separately during the handshake.

Chapter 9: Circuit Construction

Solution 9.1. ntor requires $20 + 32 + 32 = 84$ bytes. ntor-v3 with a three-byte message requires $32 + 32 + 32 + 3 + 32 = 131$ bytes. HDATA can carry at most $509 - 4 = 505$ bytes, so CM can occupy at most $505 - 128 = 377$ bytes.

Solution 9.2. The secret for **`CREATE_FAST`** is $X \| Y$, sent in plaintext inside the channel. For a second hop, X and Y would cross the circuit through the first relay. That relay knows its own layer keys and could read those values, derive the second hop's keys, and decrypt everything the client exchanged with that hop, destroying separation between the hops.

Solution 9.3. AES-CTR encrypts by XORing plaintext with a keystream depending only on a key and counter. With the same key in both directions, the client's cell number n and the relay's cell number n would use the same keystream. An intermediate relay observing both ciphertexts could XOR them to obtain the XOR of the plaintexts, often allowing both messages to be recovered.

Solution 9.4. Relay `test004r` uses ORPort 5104 = `0x13f0`: `03` means three link specifiers; the IPv4-address-and-port specifier is `00 06 7f000001 13f0`. The full message contains $1 + (2 + 6) + (2 + 20) + (2 + 32) = 65$ specifier bytes, plus $2 + 2 = 4$ bytes for HTYPE and HLEN and $32 \cdot 3 + 1 + 32 = 129$ bytes of HDATA. The total is 198 bytes, comfortably within a relay message's 498-byte capacity.

Solution 9.5. Without EXP(B, x), the relay is not authenticated. An intermediate relay forwarding the handshake could generate its own Y and $AUTH$ using its own y, impersonate the next hop, and read subsequent traffic destined for it: a *man-in-the-middle* attack. Without EXP(Y, x), forward secrecy is lost. Anyone later obtaining the long-term onion key b could recompute EXP(X, b) from recorded handshakes and derive all the session keys.

Solution 9.6. `VER` binds the handshake to its context, `"circuit extend"` for Tor. Because it enters every derivation, a handshake generated in one context produces neither keys nor $AUTH$ values valid in another. Without `VER`, an adversary might replay a handshake message from an unrelated protocol to a Tor relay, or vice versa, if both accepted the same computations. These are *cross-protocol* attacks.

Solution 9.7. On the laboratory network, `test003r`, which supports `Relay=2-6`, rejects a request consisting only of `[02 06]` using a **`DESTROY`** cell with reason `PROTOCOL`. It accepts the combined request containing `CC_FIELD_REQUEST` and responds with `CC_FIELD_RESPONSE`. Tor's code enables CGO at a hop only when congestion control is enabled too: `enable_cgo = supports_cgo && use_congestion_control` in `src/core/or/extendinfo.c`. Proposal 359 explains why the new relay-message format, in which only some commands have a StreamID, is incompatible with the previous stream-level SENDME flow-control mechanism. It requires the flow control of Proposal 324, or `FlowCtrl=2`.

Solution 9.8. This is a hands-on exercise. `ntor_ref.py` contains the client and server functions. Using fixed keys instead of randomly generating new ones produces reproducible values for comparison against `ntor.py`.

Solution 9.9. Outline: `server_part1` receives HDATA, verifies `ID` and B, computes $X_b = \mathrm{EXP}(X, b)$, derives phase-one keys, verifies the MAC, and decrypts CM. Then `server_part2` generates (y, Y), calculates *key_seed* and $verify$, derives the key for SM, and calculates $AUTH$. When y is fixed to the proposal's test-vector value, the response must match `server_handshake`.

Chapter 10: The Onion

Solution 10.1. The client updates the second hop's forward running digest, encrypts with Kf_2, and then encrypts with Kf_1. The third hop sees nothing: the second hop recognizes the cell and does not forward it.

Solution 10.2. Each cell consumes 509 bytes of keystream. After 1,000 cells, the keystream position is $509 \times 1000 = 509\,000$ bytes, past AES block $31\,812$, whose blocks are 16 bytes. Reusing the key for a new circuit whose counter restarted at zero would reuse the keystream; XORing two ciphertexts encrypted with it would reveal the XOR of their plaintexts. Each circuit therefore derives fresh keys through a handshake with ephemeral values.

Solution 10.3. The relay's running digest would include a cell the client had not counted for that hop. From then on, their digest states would diverge and no subsequent cell would be recognized, destroying the circuit. The problem occurs with probability 2^{-16} for each transiting cell not addressed to the relay: once every $65\,536$ such cells on average, corresponding to approximately 33 MB of traffic.

Solution 10.4. An adversary should alter bytes whose expected plaintext is known or predictable, so an accomplice at the exit can detect a specific modification: for example, the relay command, `02` for `DATA`, the StreamID, or the four zero bytes at the start of padding. Before checking the digest, the complicit exit examines whether those bytes match the expected alteration. An honest exit sees only a cell with an invalid digest and destroys the circuit.

Solution 10.5. Under CGO, after removing the exit's encryption layer, an altered cell produces a pseudorandom tag and contents. There is no recognizable mark: the modification propagates unpredictably across the cell. Because the tag feeds into processing of the next cell, subsequent cells on that circuit also become undecryptable and the circuit must be discarded. An adversary can therefore only destroy circuits, which it could do anyway.

Solution 10.6. Observed intervals: 4.1, 3.0, 5.7, 8.7, and 6.8 seconds, averaging 5.7. For the maximum of two independent uniform variables on $[a, b]$, the expected value is $a + \frac{2}{3}(b - a) = 1.5 + \frac{2}{3} \cdot 8 \approx 6.8$ seconds. Five samples are too few for the observed difference to be surprising. Also note that the timer restarts whenever *any* cell crosses the channel, not just padding cells.

Solution 10.7. On the laboratory network, the middle relay does not reply to `DROP`; the message is recognized and silently discarded. Indirect evidence comes from the circuit remaining usable: after two `DROP` messages, the client can still extend it to the exit. If the middle had not recognized the cells, being the circuit's final hop at that point, it would have destroyed the circuit. If client and middle had counted cells differently, their running digests would have fallen out of sync and the subsequent extension would have failed.

Solution 10.8. In the private-network test, changing one bit in an `EXTEND2` cell intended for the guard, then the only hop in the circuit, makes the digest check fail. As the endpoint of that circuit, the guard replies with a **`DESTROY`** cell carrying reason 1 (`PROTOCOL`). On a longer circuit, a modification the client makes to a cell intended for the third hop would be forwarded through the first two hops and detected only by the third.

Chapter 11: Streams, SENDME, and Congestion Control

Solution 11.1. The hostname must be lowercase: `example.com:443` followed by a zero byte, that is, `6578616d706c652e636f6d3a343433 00`, followed by FLAGS with bit 1 set to accept IPv6: `00000001`.

Solution 11.2. IPv4 address `c0000201` is 192.0.2.1; TTL `0000012c` is 300 seconds. The client may ignore the address. If it retains it, it must not reuse it across circuits: the exit could have lied or returned a unique address to recognize the client.

Solution 11.3. Without optimistic data, the client waits for `CONNECTED` before sending the request, costing an extra round trip of approximately 600 ms in an operation taking at least two RTTs. If the connection fails, queued data must be discarded because there is no destination to receive them. They must not be delivered over another connection.

Solution 11.4. A stream can have at most 500 cells in flight, approximately $500 \times 498 \approx 249$ KB per RTT. With a 400 ms RTT, this is approximately 620 KB/s. Even with many streams, a circuit is limited to 1,000 cells per RTT, approximately 1.2 MB/s. With congestion control, the window starts at only 124 cells (about 62 KB) but grows until RTT measurements indicate congestion, without a fixed upper limit.

Solution 11.5. A malicious client opens many streams to a server sending a large amount of data, perhaps its own server, reads very little, and repeatedly sends forged `SENDME` messages. The exit continues transmitting, while cells accumulate in the guard's, middle's, and client's queues, exhausting relay memory. Authenticated `SENDME` requires proof that the client received the cell triggering that message. If the cell contents were predictable, for example a familiar downloaded file, the client could compute the digest without receiving the cell; random padding bytes prevent that shortcut.

Solution 11.6. Outline: send `RESOLVE` on a new StreamID with the zero-terminated hostname, then interpret `RESOLVED` as a sequence of tuples containing type, length, value, and TTL. For a nonexistent hostname, the exit sends an error-type entry (`0xF0` or `0xF1`), whose payload should be ignored.

Solution 11.7. The exit responds with `END`, reason 4 (`EXITPOLICY`), followed by the destination address, four bytes for IPv4, and a four-byte TTL. The client can use this information to avoid retrying the same exit for that destination.

Solution 11.8. We measure the time it takes a `SENDME` to reach the exit plus the time for the unblocked exit to send additional cells and for them to reach the client. When the exit is waiting for `SENDME`, this approximates the circuit RTT as observed by the client. A second simultaneous download lengthens queues and raises the measured values: exactly the signal congestion control uses to reduce its window.

Chapter 12: The Directory System

Solution 12.1. The voting interval is one hour, from `valid-after` to `fresh-until`. The client chooses a random instant between $11{:}00 + \frac{3}{4} \cdot 60' = 11{:}45$ and $11{:}45 + \frac{7}{8} \cdot 75' \approx 12{:}50$. If it still has no new consensus by 13:00, its existing one is formally expired. The client nevertheless continues to use it as *reasonably live*, meaning less than 24 hours past expiration, under `REASONABLY_LIVE_TIME` in `networkstatus.c`. It can continue building circuits while repeatedly attempting to fetch a newer consensus. It stops relying on the old consensus approximately 24 hours after expiration.

Solution 12.2. Probably `Running` and `Valid`, and `Fast` since 500 KB/s exceeds 100 KB/s. Possibly `V2Dir` if the relay also serves as a directory cache. Not `Stable`, `Guard`, or `HSDir`, which require days or weeks of operation. Not `Exit`, since its policy allows only port 443, whereas this flag requires exit traffic on both ports 80 and 443.

Solution 12.3. The cache sends the targeted client a two-hour-old but still valid consensus, which lacks recently added relays. The client therefore selects paths from a slightly different set than everyone else, potentially making its traffic recognizable. The attack is limited by the consensus's three-hour validity, by the need for a valid authority signature (the cache cannot invent a consensus), and by clients selecting caches randomly. Fetching microdescriptors by digest rather than relay identity prevents the cache from giving this particular client a different version of a relay's key, because the digest is fixed by the signed consensus.

Solution 12.4. With three measurements, the median is 5200. One inflated measurement does not change it. If the third authority were the only one measuring this relay, the system would not have the three measurements required to use their median. The consensus would instead use advertised or voted bandwidth, with `Unmeasured=1` and an upper limit, so the inflated measurement would not affect the result.

Solution 12.5. The vote has lines absent from or different in the consensus. Examples include `Measured` bandwidth assessments from bandwidth authorities, `m` lines identifying supported consensus methods, flags assigned by that particular authority, its `flag-thresholds`, and commitments to shared randomness. The consensus retains only the combined results.

Solution 12.6. Hash the text beginning with the `onion-key` line that opens the microdescriptor and ending at the newline before the next microdescriptor or at the end of the document, including the PEM-encoded RSA-key lines. The SHA-256 digest, encoded as base64 without trailing `=` characters, must match the consensus's `m` line.

Solution 12.7. With one authority stopped, the remaining two keep voting and produce a consensus with two signatures, enough for the required majority $\lfloor 3/2 \rfloor + 1 = 2$. With two authorities stopped, the sole survivor cannot reach a majority and no new consensus is produced. The client can nevertheless continue using its last consensus as *reasonably live* for approximately 24 hours after expiration. In the laboratory network, a consensus lasts only 40 seconds but the reasonably-live allowance is still 24 hours, so the disruption is not immediately observable.

Solution 12.8. An `ns` consensus normally carries SHA-1 signatures; the specification allows only those for that flavor. Its `r` line additionally contains the full relay descriptor digest, and every relay entry also includes a `p` line summarizing its exit policy.

Chapter 13: Path Selection

Solution 13.1. The exit has the strictest requirements: it must accept the requested destination. Choosing it first establishes that a suitable exit exists. Selecting the guard first might, because of family or network exclusions, eliminate the only suitable exit or force the selection to be repeated. There is also a statistical reason: exclusions apply to later positions, so they should affect the positions offering the largest candidate sets.

Solution 13.2. As a guard: $20000 \times 0/10000 = 0$ (`Wgd=0`). As a middle: $20000 \times 0 = 0$ (`Wmd=0`). As an exit: $20000 \times 10000/10000 = 20000$ (`Wed=10000`). Under this consensus's weights, the relay is used exclusively as an exit.

Solution 13.3. Without persistent guards, compromise probability exceeds 50% when $1 - (1 - c^2)^C > 0.5$, or $C > \ln 0.5 / \ln(1 - c^2)$: approximately 6932 circuits for $c = 1\%$ and 17

circuits for $c = 20\%$. With one fixed guard, the probability remains near c, or 1% and 20%, respectively. Guards are insufficient against a very powerful adversary, but the difference remains substantial.

Solution 13.4. An adversary must provide real bandwidth: selection probabilities depend on bandwidth measured by authorities rather than merely advertised values. Exits are also closely monitored, including through `BadExit`. The limit of two relays per address forces the adversary to obtain many addresses. Declaring a family prevents their relays from appearing twice in the same circuit; omitting the declaration risks discovery and exclusion by authorities. Finally, weights reserve scarce exit capacity for actual exits, where the adversary's bandwidth counts.

Solution 13.5. Without a sample limit, an adversary controlling the client's local network could block connections to every honest guard. The client would keep sampling new guards until finding a reachable one, eventually selecting an adversarial guard. With a limited sample, the client exhausts its attempts and stops adding guards, potentially losing connectivity rather than automatically selecting an attacker-controlled guard.

Solution 13.6. This is a practical exercise. Obtain the microdescriptors from CollecTor alongside the corresponding consensuses. For each relay, construct its set of family identifiers using `family-ids` or mutual `family` lists, and reject a candidate if it shares a family with any relay already selected.

Solution 13.7. This is a research exercise. Large families generally account for a substantially greater share of exit-selection probability than any single large relay. Actual concentration risk therefore depends on operators, not merely the count of relays. This motivates the community's concern with operator diversity and undeclared relay families (chapter 27).

Chapter 14: Onion Services v3

Solution 14.1. $32 + 2 + 1 = 35$ bytes, or 280 bits. Base32 encodes five bits per character, so $280/5 = 56$ characters are required, without padding. A random typing error has approximately a 2^{-16} chance of accidentally matching the checksum. The version byte must also remain `0x03`, making an undetected random error still less likely.

Solution 14.2. At 09:00 UTC on 1 January 2026, $29\,454\,300$ minutes have elapsed since the Unix epoch. Subtracting the 720-minute offset leaves $29\,453\,580$, so $\lfloor 29453580/1440 \rfloor = 20453$. Period $20\,453$ began at 12:00 UTC on 31 December 2025, because 09:00 is still before noon.

Solution 14.3. The adversary needs the period's blinded key, and thus knowledge of the service address, plus the period's SRV and the positions of controlled relays on the ring. They would use those values to generate relay identities positioned immediately after the service indices. The SRV becomes known only when authorities publish it, approximately half a day before the relevant period begins. The adversary cannot prepare the precisely positioned relays several days earlier, and new relays still need to earn the `HSDir` flag, which requires at least 96 hours of operation.

Solution 14.4. An HSDir sees a blinded key, not the service's address, and receives requests through anonymous circuits. An introduction point sees an authentication key and a circuit arriving from the service, and can count introduction requests. A rendezvous point sees a cookie, two circuits, and encrypted traffic passing between them. The introduction point is especially useful for estimating popularity because it observes each connection attempt, even though it does not know the corresponding address. Someone already knowing the address could, however, download the descriptor and establish that correspondence.

Solution 14.5. On a single-hop circuit, the HSDir is the first relay and sees the client's IP address. It could link *who* is searching for *which* service, at least by blinded key and by address if the address is already known. Rejecting nonanonymous requests forces clients to fetch descriptors through other relays.

Solution 14.6. This is a practical exercise. With more than 2×4 HSDirs, the service no longer uploads its descriptor everywhere. Fetching it from every HSDir through a three-hop circuit should find it precisely on the HSDirs calculated for the current or following period, because the service publishes descriptors for both periods.

Solution 14.7. The base64 value holds a count of link specifiers followed by the specifiers in `EXTEND2` format (section 9.2.2). For example, type `00` supplies address and ORPort, type `02` an RSA fingerprint, and type `03` an Ed25519 identity. Recover the relay from its ORPort, which equals 5100 plus its node number. The `auth-key` line contains a type-9 certificate, signed by the descriptor-signing key, certifying the service's authentication key at that introduction point.

Chapter 15: Defenses Against DoS

Solution 15.1. During the first minute, the full bucket provides 60 tokens and refilling adds $2 \times 60 = 120$, giving approximately 180 circuits, perhaps slightly fewer because the client is marked as soon as its bucket reaches zero. Over an hour, at the sustained rate, the total is approximately $60 + 2 \times 3600 = 7260$. A normal client creates only a few dozen circuits.

Solution 15.2. Only the first relay knows the client's real IP address. At later hops, the channel's apparent "address" belongs to the preceding relay. A middle relay limiting circuits per address would actually limit all circuits from a particular guard, which may carry traffic for thousands of users. It would harm everyone rather than specifically restricting the attacker. That is why per-address limits apply only to unauthenticated client connections.

Solution 15.3. The client sends an `INTRODUCE1` message over a circuit that it can reuse. The service must decrypt the request, perform an hs-ntor handshake, and establish a fresh circuit of at least three hops to the rendezvous point. That involves multiple ntor handshakes with relays plus the cells required to create and extend each hop. One client cell can therefore trigger several public-key operations and dozens of network cells: amplification by one or two orders of magnitude.

Solution 15.4. Requiring k leading zero bits permits only effort levels that are powers of two: 1, 2, 4, and so on. Each additional bit doubles the expected computational cost. The linear test instead permits any integer effort E. A feedback controller adjusting effort in small increments needs this finer scale. With doubling steps, it would tend to oscillate between an effort that is too low and one that is too high.

Solution 15.5. No. Effort E is part of the challenge used to compute both the Equi-X solution and hash R. Changing the claimed value to $E = 5000$ changes the challenge, invalidating the solution found for $E = 50$. The client would have needed to select $E = 5000$ before starting its search.

Solution 15.6. This is a laboratory exercise. Cells accumulate in the middle relay's circuit queue until their total memory use exceeds `MaxMemInQueues`. The memory manager then closes the circuits with the oldest queued data, including ours, and records the quantity of memory freed in the relay's log.

Solution 15.7. This is a programming exercise. Requests offering greater effort are expected to have short, stable waiting times. Zero-effort requests should be served mainly when the

queue is almost empty or be discarded when the queue is trimmed. This is the intended behavior: protecting users willing to pay a modest computational cost.

Chapter 16: Censorship, Bridges, and Pluggable Transports

Solution 16.1. A bridge serves the same role as a guard, the first hop of a circuit, but it is absent from the public consensus. A client learns its address from a privately obtained bridge line. With `UseBridges`, clients use bridges *instead of* public guards. Adding a public guard would require connecting directly to a published address, exactly what a censor blocks or monitors.

Solution 16.2. "Look like nothing recognizable": obfs4, whose suppression would require blocking traffic that cannot be identified as a permitted protocol. "Look like something permitted": meek, requiring blocking a major content-distribution provider; Snowflake, requiring interference with WebRTC, its broker, and many volunteer proxies; WebTunnel, requiring blocking HTTPS to unfamiliar sites or the sites hosting the bridges; and Conjure, requiring blocking broad address ranges of cooperating connectivity providers.

Solution 16.3. A censor could examine long-lived connections to addresses without associated hostnames, unusually high-entropy bytes from the first packet, recognizable size and timing distributions such as bursts associated with Tor cells, and upload/download ratios resembling web browsing. `iat-mode` introduces randomized delays and fragmentation to make timing and size signatures harder to recognize, at a performance cost.

Solution 16.4. A censor that blocks every bridge it receives cannot gain reputation: blocked bridges prevent its credentials from advancing to higher levels or gaining invitation privileges. It could instead leave bridges unblocked until accumulating enough high-level credentials and then block them together. That strategy requires maintaining many patient identities for months, while the discovered bridges continue helping legitimate users. Each high-level credential also grants access to only a limited number of bridges.

Solution 16.5. Both captures should have entropy close to eight bits per byte: after its handshake, TLS also carries encrypted data. The difference lies in a few recognizable bytes, such as TLS record headers (`16 03 01`, `17 03 03`, followed by the length) and the initial unencrypted handshake. Deep-packet inspection can recognize these without needing statistical entropy tests.

Solution 16.6. Laboratory 1 times out (`TimeoutError`) because the bridge does not reply. Using an incorrect `cert` parameter makes client-side `obfs4proxy` fail the handshake. The bridge cannot recognize the client's authentication mark, so it does not answer and closes the connection after a delay. Depending on logging verbosity, its logs may contain only a failed-handshake message, or nothing relevant.

Chapter 17: Installation and torrc

Solution 17.1. tor reports the unrecognized option using a message such as `Unknown option '...'. Failing.` and exits without starting. Running `--dump-config short` on the client's `torrc` shows only values differing from defaults, including laboratory-specific settings such as `TestingTorNetwork`, the `DirAuthority` entries, and listening ports.

Solution 17.2. If email and forum traffic share a circuit, the exit sees connections to the user's email provider, potentially identifying them, and the pseudonymous forum from the

same circuit around the same time. It may link the pseudonym to the person. Solutions include two separate SOCKS ports, one per application, which are isolated by default unless assigned the same `SessionGroup`; different SOCKS credentials with `IsolateSOCKSAuth`; or `IsolateDestAddr` on their shared port.

Solution 17.3. An adversary monitoring Italian networks could see both the user entering Tor and an Italian exit connecting to the Italian destination. Both ends would be in the same jurisdiction or observable network region, enabling the type of correlation excluded from Tor's threat model. Italian exits are also relatively few, and restricting selection makes the user more distinguishable and easier to target with malicious relays.

Chapter 18: Control Port and Stem

Solution 18.1. `552` is a permanent error in response to a command: the key `pippo` does not exist. By contrast, a `650` message is not a command response but an asynchronous event sent because a client subscribed with `SETEVENTS`. It may arrive at any time, including while another command's response is pending.

Solution 18.2. Outline: call the controller's `create_ephemeral_hidden_service`, mapping virtual port 80 to local port 8080 and supplying `await_publication`. The method returns the onion address. Access it using `curl --socks5-hostname 127.0.0.1:9008 http://<address>.onion/`. tor retains an ephemeral service's keys in memory and returns them to the controller, which may save them to recreate the same address later. By default, the service disappears when the control connection closes unless detached.

Solution 18.3. Outline: register a handler for `EventType.BW`. It receives the numbers of bytes read and written each second as `event.read` and `event.written`. Print those values while downloading a large file. The incoming rate rises toward the limit imposed by congestion control and the available system resources.

Solution 18.4. With `__LeaveStreamsUnattached`, tor does not automatically attach streams to circuits. The command `EXTENDCIRCUIT 0 test004r,test005r,test006e` builds a circuit and returns its identifier. When a `STREAM` event arrives with status `NEW`, call `ATTACHSTREAM`. Manual path selection is dangerous because it bypasses weighted selection and guard policies: the unusual path may distinguish the user, or malicious software could deliberately route traffic through adversary-controlled relays.

Chapter 19: A Private Tor Network

Solution 19.1. `test006e` is node number 6, so its ORPort is 5106 and its ControlPort is 8006. `test009h` has no ORPort: an onion service is not a relay and does not accept connections initiated by other network nodes. It builds outbound circuits to its introduction and rendezvous points.

Solution 19.2. The setting is `TestingDirAuthVoteGuard *` in the authorities' torrc files. On the public network, the Guard flag requires demonstrated stability and bandwidth over time (chapter 12). In the laboratory, any relay may serve as a guard. Statistics concerning first-hop selection there are consequently not representative of the public network.

Solution 19.3. With `--socks5`, `curl` resolves `localhost` itself and supplies an IP address to the Tor client. With `--socks5-hostname`, it passes the name along for the exit to resolve. For an ordinary Internet hostname on the public network, local resolution would reveal visited sites

to the client's DNS resolver and anyone observing that resolver, even though the subsequent connection would use Tor. This is a classic DNS leak.

Solution 19.4. Every 20 seconds, as configured by `V3AuthVotingInterval`. One day therefore contains $86400/20 = 4320$ laboratory consensuses, compared with 24 on the public network, where authorities vote hourly.

Solution 19.5. With one authority stopped, the network can continue producing consensuses because a majority of the three authorities, or two signatures, is sufficient. With two stopped, the remaining authority cannot produce a newly valid consensus alone. Clients may nevertheless continue using the previous consensus as *reasonably live* for approximately 24 hours after expiration (chapter 12), so they can continue building circuits for some time. The laboratory consensus is valid for only 40 seconds, but its reasonably-live allowance remains 24 hours.

Solution 19.6. Outline: `c.get_network_status(fingerprint)` returns the corresponding consensus entry with its `address` and `or_port` attributes. Call it for each fingerprint in `circ.path`, taking the first member of each tuple.

Solution 19.7. Use the same structure as the book's network definition, replacing the node counts with `getN(5)`, `getN(10)`, and `getN(4)`. Bootstrap time depends on hardware. As the network grows, much of the additional time is spent waiting for relays to appear in the consensus and for clients to download their microdescriptors.

Solution 19.8. This is an exploratory exercise. Look for log messages covering path selection, sending **`CREATE2`** and `EXTEND2`, receiving their responses, and completing the circuit. *Info* logging does not show every cell. Enable domain-specific *debug* logging in the client's torrc for more detail.

Solution 19.9. No. Once TLS negotiation finishes, Tor cells travel encrypted and `tcpdump` can see only TLS records. Addresses, ports, record sizes, and timing remain observable. These are precisely the kinds of information exploited by traffic-analysis attacks (chapter 26).

Chapter 21: Publishing an Onion Service

Solution 21.1. A six-character prefix requires approximately $32^6 \approx 1.07 \times 10^9$ attempts, or about 107 seconds at ten million attempts per second. Eight characters require $32^8 \approx 1.1 \times 10^{12}$ attempts, approximately 1.1×10^5 seconds, slightly more than 30 hours. Each additional character multiplies the expected effort by 32.

Solution 21.2. The error page reveals the web-server software and version, potentially useful for identifying vulnerabilities, and especially an IP address, 192.0.2.10, that might be the server's real public address. The service could therefore be located. Disable server signatures in headers and error pages, bind the server only to localhost or a Unix socket, and inspect its responses using a laboratory client.

Solution 21.3. Delete `alice.auth` and reload the configuration. The service publishes a new descriptor excluding Alice's entry, encrypted using a fresh *descriptor cookie* that she cannot decrypt. Clients retaining the older descriptor may continue using already known introduction points for as long as those points remain valid. For more complete revocation, rotate introduction points, for example by restarting the service. Beware the edge case: if Alice was the only authorized client, an empty directory remains, and the source laboratory observed HSDirs rejecting the descriptor (section 21.3). To disable restricted discovery entirely, remove `authorized_clients/` and reload the configuration.

Solution 21.4. Set `HiddenServicePort 80 unix:/path/socket` and configure the web server to listen on that Unix socket. Beyond avoiding an open TCP port, a filesystem socket can be protected with file permissions, cannot be reached through a network interface, and prevents accidentally exposing the service on a public interface.

Chapter 20: Observing Traffic

Solution 20.1. TLS does not align records with Tor cells. tor writes cells into a buffer, and the TLS library groups them into records up to 16 KB long, which relates to the 31-cell `sendme_inc` value (section 11.3). TLS adds its own headers and authentication tag. TCP can in turn split or combine TLS records into different segments. Fixed length applies to Tor cells, not network packets.

Solution 20.2. Outline: after **`CREATED2`**, decrypt the circuit's **`RELAY_EARLY`** and **`RELAY`** cells by removing their encryption layers in the correct order using the hop keys stored in `circuit.py`. AES-CTR keystream state must advance in exactly the order in which it was used. During circuit construction, the visible messages include `EXTEND2` and `EXTENDED2`, followed by possible `BEGIN`, `CONNECTED`, `DATA`, and `SENDME`.

Solution 20.3. On a client channel carrying multihop circuits, TLS records may appear every few seconds even without application traffic. These contain channel-level **`PADDING`** cells, transmitted at randomized intervals between 1.5 and 9.5 seconds (section 10.5). By contrast, a channel that has carried no multihop circuits shows no such traffic.

Solution 20.4. This is a programming exercise. A Lua analyzer registered as a *heuristic dissector* for decrypted TLS data can read the first two bytes for the initial **`VERSIONS`** cell, or four bytes afterward, as the CircID. It then reads the command and, for variable-length cells, the length field. Keep separate state for each connection and direction, as in `traffic.py`.

Chapter 22: Running a Relay or a Bridge

Solution 22.1. Yes. The first matching rule, `accept *:80`, accepts the connection, and subsequent rules are not considered. Reverse the first two rules: `reject 192.0.2.0/24:*`, `accept *:80`, `reject *:*`.

Solution 22.2. The summary lists only ports, not addresses, and is necessarily approximate. It includes a port when the exit accepts that port for "most" addresses, a precise technical threshold in dir-spec rather than simply more than 50%: the exit must not block more than 2^{25} IPv4 addresses, one /7, or 2^{112} IPv6 addresses, one /16, excluding nonpublic addresses. When the client receives `END` with reason `EXITPOLICY`, it records that the exit rejects the given address and port for the specified TTL, then retries the stream on a different circuit with another exit.

Solution 22.3. With `MyFamily`, only reciprocal declarations count. The five existing relays remain a family among themselves, but the sixth is not part of it because none of the old relays lists it. Clients may therefore select it together with an existing family member. With a family key, installing the same family key on the new relay is sufficient: its signed certificate binds it to the family without changing the other relays.

Solution 22.4. The logs successively report "in-progress connect failed ...(Connection refused)" and "Didn't recognize errno 115". If the stream-end reason used the error returned by `getsockopt(SO_ERROR)`, the client would instead receive `CONNECTREFUSED`. Similarly, `TIMEOUT`

for `ETIMEDOUT`, `NOROUTE` for `ENETUNREACH` or `EHOSTUNREACH`, and `CONNRESET` could be reported correctly, rather than `MISC`, for connections failing after `EINPROGRESS`. The practical difference for a client is modest: the error message changes but stream failure handling remains the same.

Solution 22.5. At 10 MB/s, the relay transfers roughly 864 GB per day, accounting for the maximum of sent and received traffic. A 1 TB monthly allowance would be exhausted in a little over one day. A relay offline for most of the month loses its `Stable` and `Guard` flags and breaks circuits when it shuts down. A slower relay that remains online all month is much more useful, so configure `RelayBandwidthRate` to make the allowance last through the whole interval.

Chapter 23: Tor Browser, Tails, and Whonix

Solution 23.1. New Circuit for This Site changes only the active tab's site's circuit, without clearing cookies, browsing history, or other sites' state; the site may still recognize the user through stored state. New Identity closes everything, clears browser state, and obtains new circuits for all activity. Neither changes the guard, because rotating it frequently increases the chance of eventually selecting an adversarial guard (section 13.3). The guard cannot, in any event, see which websites the client visits.

Solution 23.2. The user may block additional trackers from loading, but Tor Browser already isolates trackers per site and limits persistent identifiers. Installing an extension sacrifices uniformity: websites may detect its presence or filter lists, distinguishing the user from the larger Tor Browser population. The extension also introduces more code with access to visited pages.

Solution 23.3. With identical SOCKS credentials for two tabs visiting the same website, tor attaches their streams to the same circuit. A third tab with different credentials receives a different circuit. `GETINFO stream-status` reveals the circuit identifier associated with each stream.

Solution 23.4. (a) Tor Browser is sufficient, possibly with a bridge. (b) Tails: it boots from a USB drive, normally leaves no traces on another person's computer, and preserves only data placed in encrypted persistent storage, subject to risks from compromised hardware. (c) Whonix: the Workstation's applications reach the network through tor by architectural design, and the system supports long-running work without requiring amnesia.

Solution 23.5. This is an open-ended exercise. Possible properties include screen and window dimensions, time zone, language, available fonts, and canvas rendering. In an ordinary browser, these may contribute anywhere from a few bits, such as time zone, to many bits, such as font or canvas characteristics. Tor Browser rounds dimensions through letterboxing, reports UTC and a common language, limits fonts, and standardizes or disables revealing interfaces. Residual differences include the operating system, selected security level, and rounded display dimensions.

Chapter 24: Measuring the Network

Solution 24.1. Directory requests would be halved, and with the fixed divisor of ten, the user estimate would also be halved. The charts would display a sudden, broadly simultaneous decline across countries as the new tor version spread. A global, gradual decline correlated

with software adoption would suggest a change in client behavior rather than a comparable loss of users. The estimation method would need a revised divisor.

Solution 24.2. $20\,000\,000/(0.6 \cdot 10) \approx 3\,333\,333$ estimated users. With an 8% reporting fraction, the estimate is discarded because the fraction is below the required 10%.

Solution 24.3. Outline: filter relays carrying both flags and divide their combined bandwidth by the network total. For the exit-selection probability, use Laboratory 7's `probabilities(consensus, "exit")`, summing the selection probabilities of relays whose /16 networks are among the five most frequently represented.

Solution 24.4. This is an open-ended exercise. A documented blocking event typically produces a sharp drop in direct-user estimates on the day blocking begins. Bridge-user estimates, especially for Snowflake or WebTunnel, may grow in the following days as affected users switch transports. Metrics may flag the date of the initial drop.

Solution 24.5. Without rounding, an adversary who knows they are the only user from a given country reaching a relay, or knows the exact number of other users, could inspect its extra-info statistics to confirm whether another person from that country connected: a difference of one in the count. Rounding hides small differences. An adversary controlling many relays does not need these published counts, however, because it can directly observe incoming connections to its own relays.

Chapter 25: Legal and Ethical Aspects

Solution 25.1. (a) An exit initiating requests to websites on its own, perhaps intending to "mix" traffic, would initiate transmissions. (b) An exit redirecting connections to a destination different from the one specified in the `BEGIN` cell would select the recipient. (c) An exit inserting content into HTTP pages or filtering content by meaning would select or modify information. These are also examples of behavior that can result in a `BadExit` flag.

Solution 25.2. A kitchen knife is lawful, while assault is not. A person reading a censored newspaper through Tor may be using a lawful tool lawfully; the same person unlawfully accessing a computer commits an offense with or without Tor. A relay operator offers transmission capacity without selecting content, more like a carrier than an author, provided the operator does not participate in unlawful activity. Specific legal conclusions always depend on jurisdiction and facts.

Solution 25.3. Explain that the address belongs to a Tor exit, verifiable through public Tor Project tools; that the operator does not know or log client identities and cannot identify them; and that the complaining website can, if it wishes, handle connections from public exit addresses differently. In practice, the operator may add an exit-policy rejection for that website's address if the operational benefit outweighs the resulting restrictions.

Solution 25.4. The proposal observes real users' traffic without consent, collects potentially disclosive data unsuitable for publication, such as visited domains, and offers a limited benefit. It conflicts with several research-safety principles and, in Italy, may raise criminal-law and data-protection issues. Safer alternatives include using already public aggregated Metrics data, measuring news-site reachability using only the researcher's own Tor client, or asking the Research Safety Board to review a privacy-preserving counting design using aggregation and noise, as in relay statistics.

Solution 25.5. This is a research exercise. Look for judgments in official legal databases, such as the Italian courts' published decisions or the Court of Justice of the European Union,

rather than relying solely on third-party summaries. For each decision, record the court, date, applicable law, and outcome, and distinguish cases concerning relay operators from those concerning Tor users.

Chapter 26: Correlation, Fingerprinting, and Sybil Attacks

Solution 26.1. Traffic correlation requires observing both ends, whether through relays or the underlying networks; guards, path constraints, and measured bandwidth make this harder. Website fingerprinting requires only the client-to-guard link; limited padding defenses exist, and research continues. Discovering a guard involves a middle relay and the ability to induce circuit construction; guards and vanguards help. A Sybil adversary operates numerous relays; defenses include measured bandwidth, families, /16 constraints, and network monitoring. Routing attacks require influence over an autonomous system or Internet exchange point; responses include BGP monitoring outside the Tor protocol.

Solution 26.2. Among 100 000 visits, expected correct alerts number approximately 0.95, while false positives number approximately $0.001 \cdot 99\,999 \approx 100$. The proportion of correct alerts is $0.95/(0.95 + 100) \approx 0.9\%$. Nearly all alerts are false.

Solution 26.3. Using two-second bins over just five seconds leaves only two or three samples per series, making correlation unstable. Giving all clients the same traffic rhythm, for example the same random seed, makes the series nearly identical and matching impossible. Equalizing traffic rhythms is particularly disruptive: that is the principle behind PipeNet's constant-rate traffic (section 2.4).

Solution 26.4. At a constant transmission rate, entry traffic series no longer vary. Pearson correlation is then mathematically undefined because variance is zero, or is dominated by network noise; this attack fails. The cost is transmitting at the maximum chosen rate even when no useful data exist. In the laboratory, that can mean tens or hundreds of KB/s per client, several times more than the application data.

Solution 26.5. With a fixed guard, the adversary succeeds only if the selected guard is malicious, a probability of approximately 5%. When that happens, it also serves as the exit on approximately 5% of circuits. Over six months of intensive use, probability of at least one compromise therefore approaches 5%, remaining zero for users with honest fixed guards in this simplified model. With a random guard for every circuit, compromise probability per circuit is approximately $0.05 \cdot 0.05 = 0.0025$; after thousands of circuits, compromise is almost certain. This is the rationale for persistent guards, and Laboratory 7's `Adversary` class can verify the behavior by simulation.

Chapter 27: Real Cases Against the Network

Solution 27.1. On the client-bound path, each relay adds its own encryption layer to a cell's payload (chapter 10). The guard receives a payload encrypted under the intervening hops' keys and cannot read what the HSDir wrote into it. The outer cell command, however, is part of a header that every relay reads and forwards unchanged.

Solution 27.2. With three guards, the probability that at least one is adversarial is roughly three times as high, and each adversarial guard observes part of the client's traffic. With one guard, the selection risk is lower and an honest choice protects the client for a long time. The disadvantages are that an adversarial single guard observes all that client's traffic and that a slow or unreachable guard can degrade connectivity.

Solution 27.3. The laboratory already uses `DROP` cells rather than extension requests. The counter in `command.c` checks the outer cell command before decryption, so it does not depend on the enclosed message. In the opposite direction, the code logs a warning and closes the circuit with reason `PROTOCOL`, whether the circuit belongs to a client or a relay.

Solution 27.4. Potential signals include relays entering the consensus at exactly the same minute, an uncommon shared tor version and operating system in descriptors, addresses in the same network or hosting provider, similar ports and nicknames, and missing contact information. False positives include legitimate large operators updating all their relays together, popular providers, and automated deployments using identical configurations. No single signal proves common ownership: combine evidence and contact the operator when possible.

Solution 27.5. An intermediate relay sees plaintext outer metadata: CircID, which changes at every hop and therefore does not pass through unchanged; the cell command, including **`RELAY`** and **`RELAY_EARLY`**, the channel exploited in 2014, now blocked in the direction toward the client and limited to eight cells in the opposite direction; **`DESTROY`** and its reason; and the timing and number of cells. The last two remain available for passive confirmation without a complete defense. Payloads are layered-encrypted, but under tor1 a malicious relay could mark them for another malicious relay to recognize (section 10.3). CGO (section 10.4) instead makes such modifications produce unrecoverable plaintext at later hops.

Chapter 28: Operational Security Mistakes

Solution 28.1. Traffic bypassing Tor, documents opened while online, and DNS leaks belong to the application layer; defensive measures include Tor Browser, Tails or Whonix, `SafeSocks`, and opening files while disconnected. Metadata, signing into accounts, and mixing identities belong to the content layer; use metadata-removal tools and separate sessions. Writing style, publication times, and using Tor within a small anonymity set belong to the behavioral layer; consider habits, bridges, and when to publish.

Solution 28.2. A local DNS query travels from the user's computer to the system's resolver, commonly operated by the connectivity provider, and includes the website's domain name. The provider and anyone observing the path to the resolver can learn which website the user intends to visit, even if the subsequent connection uses Tor.

Solution 28.3. Subscribe to `EventType.STATUS_CLIENT` through Stem. A request containing only an IP address produces a `WARN`-severity event with action `DANGEROUS_SOCKS` and arguments `PROTOCOL` and `ADDRESS`. A graphical interface could identify the application and destination in a warning and explain that the application's SOCKS configuration should be corrected.

Solution 28.4. With obfs4, a local observer would have seen an encrypted, apparently structureless connection to an ordinary-looking address and could not have established with certainty that it was Tor traffic. Other clues remain: connection volume, timing, unusual duration, similar connections, and the messages' contents or possible motives, all of which could narrow the population to people expected to take that examination.

Solution 28.5. This is an open-ended exercise. A strong answer follows the structure of chapter 4: adversaries and their capabilities, what each can observe at every layer of fig. 28.1, chosen protections and reasons for them, and an explicit list of residual risks, such as correlation by an adversary observing both ends or seizure of the user's devices.

Chapter 29: Academic Research on Tor

Solution 29.1. $H = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + 2 \cdot \frac{1}{8} \cdot 3 = 1.75$ bits. $H_M = \log_2 4 = 2$ bits, so $d = 0.875$. The equivalent number of equally likely users is $2^{1.75} \approx 3.4$.

Solution 29.2. (a) Simulate a network with Shadow or, at small scale, emulate it with Chutney: real tor behavior is needed, but no real users need to be involved. (b) Measure the real network using public aggregated data, as Tor Metrics does (section 24.2). (c) Use TorPS-style behavioral simulation with real consensuses spanning a long period, potentially supplemented by theoretical analysis.

Solution 29.3. With `guard_lifetime` set to 30 days, yielding an effective lifetime of 18–30 days after timestamp backdating, a client selects guards approximately seven or eight times in 180 days, and the simulation produces 32.1% compromise. At the default 60 days, an effective 48–60 days produces about three selections and 18.4% compromise. With a 180-day lifetime, one or two selections produce 8.6%. Approximately, with k independent guard selections the compromised fraction approaches $1 - (1 - q)^k$, where $q = 5\%$; $k \approx 7.5$ produces about 31%, consistent with the 30-day simulation. Longer guard lifetimes mean fewer selections and expose fewer new clients, but clients who selected a malicious guard remain exposed longer.

Solution 29.4. Neither adversary compromises any client according to this laboratory's criterion, which requires observing both endpoints. Their observations still differ: an adversarial exit sees destinations and any application traffic left unencrypted, but not client identities. An adversarial guard sees the IP addresses of clients that selected it, but not their destinations, and might attempt website fingerprinting (section 26.4).

Solution 29.5. This is a research exercise. Reusing circuits decreases the number of new selections per day and slows the compromise curve without persistent guards. The /16 restriction has little effect unless adversarial relays are concentrated within a network. An autonomous-system adversary changes the model more fundamentally because it observes traffic without operating relays, requiring Internet-topology data of the kind used in [Joh+13].

Chapter 30: Arti

Solution 30.1. Chapter 5: `tor-llcrypto`. Chapter 7: `tor-proto` for channels and `tor-chanmgr`. Chapter 8: `tor-cell`. Chapters 9–11: `tor-proto` for circuits, encryption, streams, and flow control. Chapter 12: `tor-netdoc`, `tor-dirmgr`, and `tor-dirclient`. Chapter 13: `tor-circmgr`, `tor-guardmgr`, and `tor-relay-selection`. Chapter 14: `tor-hsclient`, `tor-hsservice`, and `tor-hscrypto`. Chapter 15: `equix` and `hashx` for proof of work. Chapter 16: `tor-ptmgr`.

Solution 30.2. Benefits: less code, a smaller attack surface, no special cases such as two-byte CircIDs, and no obsolete weak cryptography to maintain. Drawbacks: Arti cannot communicate with very old relays. The public Tor network already excludes versions that are no longer recommended, however, so the practical impact mainly concerns private networks and outdated installations.

Solution 30.3. `decode_cell` reads the command from the fifth byte and uses it to determine whether the cell is variable-length. In that case, the next two bytes specify its length, and its total length is 7+ the declared value; otherwise the total is 514 bytes. If insufficient input has arrived, the function returns `Ok(None)` rather than treating the incomplete cell as an error. The caller tries again when more bytes are available. This is the same behavior needed in a stream decoder such as the one from Laboratory 2.

Solution 30.4. Yes. Arti's SOCKS proxy isolates streams with different SOCKS credentials by default and also separates distinct listeners. Observe circuits from the relays' side, for example through an exit's ControlPort, or in Arti's debug logs. When using Arti as a library, explicit isolation is available through `TorClient::isolated_client` or `StreamPrefs` isolation tokens. As of Arti 2.6.0, the source's `cfg.rs` still documents equivalent per-port configuration as future work.

Solution 30.5. This is a programming exercise. Load the laboratory TOML file into a `TorClientConfig`, start the client with `TorClient::create_bootstrapped`, and open a stream with `connect(("127.0.0.1", 8080))`. Send an HTTP request over the stream. The configuration must permit local addresses, as does the file generated by Laboratory 20.

Chapter 31: Open Problems

Solution 31.1. Examples at the chapter's reference date: Proposal 332, for ntor-v3, is *Closed*, meaning it is implemented and incorporated into the specifications. Proposal 324, for congestion control, is *Finished*, implemented but awaiting complete integration of its text into the specifications. Proposal 359, for CGO, is *Open*: already present in the code, but its proposal is still under discussion. Check the header of each proposal and the repository's status index for updates.

Solution 31.2. With the traditional format, the simple size calculation requires $\lceil 1184/498 \rceil = 3$ cells for the 1,184-byte ML-KEM-768 public key and $\lceil 1088/498 \rceil = 3$ more for the 1,088-byte ciphertext: six in total. The present protocol, however, cannot even fragment an individual relay message that way. Proposal 340 provides the required framing, allowing one message to span cells and several messages to share a cell. Its proposed relay-cell body is 493 bytes, not 498, because the new format consumes 16 bytes for authentication, $509 - 16 = 493$, as discussed in chapter 8. The first fragment of a message carries at most 490 useful bytes because it also includes a framing header. The proposal currently retains a 493-byte limit for ordinary message bodies, so carrying a larger post-quantum `EXTEND2` will additionally require a new `Relay` subprotocol version lifting that limit. Even with that extension, the public key still occupies three cells and the ciphertext another three. Since those values travel in *opposite* directions, from client to relay and relay back to client, they cannot be packed together. The total remains six cells.

Solution 31.3. If every client downloaded a different subset of relays, an adversary knowing or influencing a particular client's subset could recognize its circuits. A circuit using relays available only to that client would disclose who constructed it. Even without advance knowledge, observing which relays a client uses over time can progressively narrow the set of plausible clients.

Solution 31.4. Filter circuits with purpose `CONFLUX_LINKED` and group them by `CONFLUX_ID` (chapter 18), then compare paths within each pair. Both legs always end at the same exit, as expected. They often share other relays on the laboratory network. For example, the legs `[test007e, test003r, test006e]` and `[test003r, test007e, test006e]` use exactly the same relays in different orders. This is not a tor defect. The private network has few eligible relays, and Proposal 329 expresses its recommendation with `SHOULD NOT`, not a strict prohibition. Such sharing is much rarer on the public network with thousands of relays.

Solution 31.5. This is an open-ended exercise. The reference format is given in Proposal 001: a header containing number, title, author, date, and status, followed by motivation, design, security and compatibility analyses, an implementation plan, and a deployment plan.

Appendix B: Further Cryptography

Solution B.1. Of the 256 bits, five are fixed: the lowest three, the highest, and the next-highest bit. There are therefore 2^{251} distinct values compared with 2^{256} before clamping. This reduction does not weaken the scheme: the curve's security level is approximately 128 bits regardless.

Solution B.2. From $y = (u-1)/(u+1)$, obtain $u = (1+y)/(1-y) \bmod p$. In Python, read y from the 32 little-endian bytes after clearing the sign bit, calculate `(1 + y) * pow(1 - y, P - 2, P) % P`, and encode the result as 32 little-endian bytes. Applying `montgomery_to_edwards` with the original x-sign bit recovers the original Edwards key (for $y \neq 1$).

Solution B.3. A randomly modified cell passes both checks with approximate probability $2^{-16} \cdot 2^{-32} = 2^{-48}$: two zero `recognized` bytes and four correct digest bytes. But a marking attack does not require the altered cell to be *accepted*. An adversarial accomplice that observes it, or the endpoint that rejects it, can recognize the modification through its effects, such as circuit closure or predictable contents. That observation can be enough to link two points on the circuit.

Bibliography

[03] *Decreto legislativo 30 giugno 2003, n. 196, Codice in materia di protezione dei dati personali*. Consolidated text published by the Italian data protection authority (Garante per la protezione dei dati personali). June 30, 2003. URL: https://www.garanteprivacy.it/ (visited on 09/23/2026).

[17] *Legge 20 novembre 2017, n. 167, art. 24 (termine di conservazione dei dati di traffico per finalità di accertamento e repressione dei reati)*. Gazzetta Ufficiale, Serie Generale no. 277, 27.11.2017. Nov. 20, 2017. URL: https://www.normattiva.it/uri-res/N2Ls?urn:nir:stato:legge:2017-11-20;167.

[22] *Regolamento (UE) 2022/2065 del Parlamento europeo e del Consiglio, del 19 ottobre 2022, relativo a un mercato unico dei servizi digitali (regolamento sui servizi digitali)*. OJ L 277, 27.10.2022. Oct. 19, 2022. URL: https://eur-lex.europa.eu/eli/reg/2022/2065/oj.

[Bau+07] Kevin Bauer, Damon McCoy, Dirk Grunwald, Tadayoshi Kohno, and Douglas Sicker. "Low-Resource Routing Attacks Against Tor." In: *ACM Workshop on Privacy in the Electronic Society (WPES)*. 2007.

[Boc+24] Cecylia Bocovich, Arlo Breault, David Fifield, Serene, and Xiaokang Wang. "Snowflake, a censorship circumvention system using temporary WebRTC proxies." In: *Proceedings of the 33rd USENIX Security Symposium*. 2024.

[BPW13] Alex Biryukov, Ivan Pustogarov, and Ralf-Philipp Weinmann. "Trawling for Tor Hidden Services: Detection, Measurement, Deanonymization." In: *IEEE Symposium on Security and Privacy*. 2013.

[Cha81] David L. Chaum. "Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms." In: *Communications of the ACM* 24.2 (1981), pp. 84–88.

[Cha88] David Chaum. "The Dining Cryptographers Problem: Unconditional Sender and Recipient Untraceability." In: *Journal of Cryptology* 1.1 (1988), pp. 65–75.

[DDM03] George Danezis, Roger Dingledine, and Nick Mathewson. "Mixminion: Design of a Type III Anonymous Remailer Protocol." In: *Proceedings of the 2003 IEEE Symposium on Security and Privacy*. 2003, pp. 2–15.

[Día+02] Claudia Díaz, Stefaan Seys, Joris Claessens, and Bart Preneel. "Towards Measuring Anonymity." In: *Privacy Enhancing Technologies (PET 2002)*. LNCS. 2002.

[Din14] Roger Dingledine. *Tor security advisory: "relay early" traffic confirmation attack*. July 30, 2014. URL: https://blog.torproject.org/tor-security-advisory-relay-early-traffic-confirmation-attack/ (visited on 09/23/2026).

[DM06] Roger Dingledine and Nick Mathewson. "Anonymity Loves Company: Usability and the Network Effect." In: *Proceedings of the Fifth Workshop on the Economics of Information Security (WEIS 2006)*. 2006.

[DMS04] Roger Dingledine, Nick Mathewson, and Paul Syverson. "Tor: The Second-Generation Onion Router." In: *Proceedings of the 13th USENIX Security Symposium*. 2004, pp. 303–320.

[EDG09] Nathan S. Evans, Roger Dingledine, and Christian Grothoff. "A Practical Congestion Attack on Tor Using Long Paths." In: *Proceedings of the 18th USENIX Security Symposium*. 2009.

[Ele] Electronic Frontier Foundation. *Legal FAQ for Tor Relay Operators*. URL: `https://community.torproject.org/relay/community-resources/eff-tor-legal-faq/` (visited on 09/23/2026).

[Fif+15] David Fifield, Chang Lan, Rod Hynes, Percy Wegmann, and Vern Paxson. "Blocking-resistant communication through domain fronting." In: *Proceedings on Privacy Enhancing Technologies* 2015.2 (2015), pp. 46–64.

[Fro+19] Sergey Frolov, Jack Wampler, Sze Chuen Tan, J. Alex Halderman, Nikita Borisov, and Eric Wustrow. "Conjure: Summoning Proxies from Unused Address Space." In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2019.

[GRS96] David M. Goldschlag, Michael G. Reed, and Paul F. Syverson. "Hiding Routing Information." In: *Information Hiding, First International Workshop*. Vol. 1174. LNCS. Springer, 1996, pp. 137–150.

[GSU13] Ian Goldberg, Douglas Stebila, and Berkant Ustaoglu. "Anonymity and One-Way Authentication in Key Exchange Protocols." In: *Designs, Codes and Cryptography* 67.2 (2013), pp. 245–269.

[Jan+14] Rob Jansen, Florian Tschorsch, Aaron Johnson, and Björn Scheuermann. "The Sniper Attack: Anonymously Deanonymizing and Disabling the Tor Network." In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2014.

[JH12] Rob Jansen and Nicholas Hopper. "Shadow: Running Tor in a Box for Accurate and Efficient Experimentation." In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2012.

[JJ16] Rob Jansen and Aaron Johnson. "Safely Measuring Tor." In: *ACM Conference on Computer and Communications Security (CCS)*. 2016.

[Joh+13] Aaron Johnson, Chris Wacek, Rob Jansen, Micah Sherr, and Paul Syverson. "Users Get Routed: Traffic Correlation on Tor by Realistic Adversaries." In: *ACM Conference on Computer and Communications Security (CCS)*. 2013.

[Jua+14] Marc Juarez, Sadia Afroz, Gunes Acar, Claudia Diaz, and Rachel Greenstadt. "A Critical Evaluation of Website Fingerprinting Attacks." In: *ACM Conference on Computer and Communications Security (CCS)*. 2014.

[Jus14] Just Security. *Video clip of former director of NSA and CIA: "We kill people based on metadata"*. from the Johns Hopkins Foreign Affairs Symposium, April 1, 2014. 2014. URL: `https://www.justsecurity.org/10318/video-clip-director-nsa-cia-we-kill-people-based-metadata/` (visited on 09/24/2026).

[MD05] Steven J. Murdoch and George Danezis. "Low-Cost Traffic Analysis of Tor." In: *IEEE Symposium on Security and Privacy*. 2005.

[NBH18] Milad Nasr, Alireza Bahramali, and Amir Houmansadr. "DeepCorr: Strong Flow Correlation Attacks on Tor Using Deep Learning." In: *ACM Conference on Computer and Communications Security (CCS)*. 2018.

[NIS24] NIST. *FIPS 203* (*ML-KEM*), *FIPS 204* (*ML-DSA*), *FIPS 205* (*SLH-DSA*). the three post-quantum standards, published together. Aug. 13, 2024. URL: `https://csrc.nist.gov/pubs/fips/203/final` (visited on 09/24/2026).

[OON] OONI. *Specification: Tor* (*ts-023*) *and Tor using Snowflake* (*ts-030*). URL: `https://github.com/ooni/spec/tree/master/nettests` (visited on 09/23/2026).

[ØS06] Lasse Øverlier and Paul Syverson. "Locating Hidden Servers." In: *Proceedings of the 2006 IEEE Symposium on Security and Privacy*. 2006, pp. 100–114.

[Pan+11] Andriy Panchenko, Lukas Niessen, Andreas Zinnen, and Thomas Engel. "Website Fingerprinting in Onion Routing Based Anonymization Networks." In: *ACM Workshop on Privacy in the Electronic Society* (*WPES*). 2011.

[Per+18] Mike Perry, Erinn Clark, Steven Murdoch, and Georg Koppen. *The Design and Implementation of the Tor Browser*. Draft. June 15, 2018. URL: `https://2019.www.torproject.org/projects/torbrowser/design/` (visited on 09/23/2026).

[PH10] Andreas Pfitzmann and Marit Hansen. *A terminology for talking about privacy by data minimization: Anonymity, Unlinkability, Undetectability, Unobservability, Pseudonymity, and Identity Management*. Version 0.34. 2010.

[RR98] Michael K. Reiter and Aviel D. Rubin. "Crowds: Anonymity for Web Transactions." In: *ACM Transactions on Information and System Security* 1.1 (1998), pp. 66–92.

[San13] Runa A. Sandvik. *Harvard Student Receives F For Tor Failure While Sending "Anonymous" Bomb Threat*. Dec. 18, 2013. URL: `https://www.forbes.com/sites/runasandvik/2013/12/18/harvard-student-receives-f-for-tor-failure-while-sending-anonymous-bomb-threat/` (visited on 09/24/2026).

[Sir+18] Payap Sirinam, Mohsen Imani, Marc Juarez, and Matthew Wright. "Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning." In: *ACM Conference on Computer and Communications Security* (*CCS*). 2018.

[Sun+15] Yixin Sun, Anne Edmundson, Laurent Vanbever, Oscar Li, Jennifer Rexford, Mung Chiang, and Prateek Mittal. "RAPTOR: Routing Attacks on Privacy in Tor." In: *USENIX Security Symposium*. 2015.

[Syv11] Paul Syverson. "A Peel of Onion." In: *Proceedings of the 27th Annual Computer Security Applications Conference* (*ACSAC*). 2011, pp. 123–137.

[Taia] Tails. *How Tails works*. URL: `https://tails.net/about/` (visited on 09/23/2026).

[Taib] Tails. *Warnings: Tails is safe but not magic!* URL: `https://tails.net/doc/about/warnings/` (visited on 09/23/2026).

[TG23] Lindsey Tulloch and Ian Goldberg. "Lox: Protecting the Social Graph in Bridge Distribution." In: *Proceedings on Privacy Enhancing Technologies* 2023.1 (2023), pp. 494–509.

[Thea] The Tor Project. *History*. URL: `https://www.torproject.org/about/history/` (visited on 09/23/2026).

[Theb] The Tor Project. *Onionbalance: documentazione e guida per gli onion service v3*. URL: `https://onionservices.torproject.org/apps/base/onionbalance/` (visited on 09/23/2026).

[Thec] The Tor Project. *Reproducible Metrics*. URL: `https://metrics.torproject.org/reproducible-metrics.html` (visited on 09/23/2026).

[Thed] The Tor Project. *Tor Browser User Manual*. URL: `https://tb-manual.torproject.org/` (visited on 09/23/2026).

[Thee] The Tor Project. *Tor Specifications*. URL: https://spec.torproject.org/ (visited on 09/23/2026).

[Thef] The Tor Project. *Tor Support: Staying anonymous*. URL: https://support.torproject.org/faq/staying-anonymous/ (visited on 09/23/2026).

[Theg] The Tor Project. *Tor Support: Using Bridges; Unblocking Tor*. URL: https://support.torproject.org/little-t-tor/circumvention/using-bridges/ (visited on 09/23/2026).

[The22] The Tor Project. *Malicious relays and the health of the Tor network*. Apr. 25, 2022. URL: https://blog.torproject.org/malicious-relays-health-tor-network/ (visited on 09/23/2026).

[The23a] The Tor Project. *Introducing Proof-of-Work Defense for Onion Services*. Aug. 23, 2023. URL: https://blog.torproject.org/introducing-proof-of-work-defense-for-onion-services/ (visited on 09/23/2026).

[The23b] The Tor Project. *Tor is slow right now. Here is what is happening*. Feb. 7, 2023. URL: https://blog.torproject.org/tor-network-ddos-attack/ (visited on 09/23/2026).

[The24a] The Tor Project. *Hiding in plain sight: Introducing WebTunnel*. https://blog.torproject.org/introducing-webtunnel-evading-censorship-by-hiding-in-plain-sight/. March 12, 2024. 2024.

[The24b] The Tor Project. *Making new connections: from BridgeDB to Rdsys*. 2024. URL: https://blog.torproject.org/making-connections-from-bridgedb-to-rdsys/ (visited on 09/24/2026).

[The24c] The Tor Project. *Uniting for Internet Freedom: Tor Project & Tails Join Forces*. Sept. 26, 2024. URL: https://blog.torproject.org/tor-tails-join-forces/ (visited on 09/24/2026).

[Tor] Tor Research Safety Board. *Tor Research Safety Board*. URL: https://research.torproject.org/safetyboard/ (visited on 09/23/2026).

[Tor25] Tor Project Forum. *Feedback from China [April 2025]: increased GFW censorship; obfs4/meek/snowflake unusable, webtunnel connects but is quickly blocked*. 2025. URL: https://forum.torproject.org/t/feedback-from-china-april-2025-increased-gfw-censorship-obfs4-meek-snowflake-unusable-webtunnel-connects-but-is-quickly-blocked/18233 (visited on 09/24/2026).

[WBU13] WBUR News. *Student Charged For Bomb Threats At Harvard*. Dec. 17, 2013. URL: https://www.wbur.org/news/2013/12/17/harvard-bomb-hoax-student-charges (visited on 09/23/2026).

[Who] Whonix. *onion-grater*. URL: https://www.whonix.org/wiki/Onion-grater (visited on 09/23/2026).

[WL14] Philipp Winter and Stefan Lindskog. "Spoiled Onions: Exposing Malicious Tor Exit Relays." In: *arXiv:1401.4917*. later included, with further authors, in *Privacy Enhancing Technologies Symposium* (PETS 2014). 2014.

Index

AUTH_CHALLENGE, 37, 42, 44, 47, 54, 58, 159
AUTHENTICATE, 37, 42, 46, 47, 54
AUTHORIZE, 54

CERTS, 31, 37, 39, 41, 42, 44, 46, 47, 54, 58, 159, 255
CREATE, 54
CREATE2, 31, 53, 54, 58, 62, 65–67, 69, 73, 74, 255, 264
CREATE_FAST, 25, 54, 68, 74, 126, 256
CREATED, 54
CREATED2, 54, 58, 65–67, 73, 265
CREATED_FAST, 54

DESTROY, 54, 58, 62, 67, 73, 79, 86, 87, 90, 91, 126, 210, 255–257, 269

initiator, 35

NETINFO, 37, 43–47, 52, 54, 58, 159

PADDING, 44, 47, 52, 54, 73, 83, 84, 255, 265
PADDING_NEGOTIATE, 54, 255
Proposals
  001, 235, 239, 271
  214, 53
  216, 68, 72
  220, 36
  224, 112
  236, 105
  271, 106
  289, 56, 81, 88, 91, 92
  291, 106
  300, 235, 237, 239
  302, 206
  323, 238, 239
  324, 57, 70, 88, 89, 92, 235, 256, 271
  327, 124
  329, 57, 158, 235, 237, 239, 271
  332, 69, 72, 74, 235, 271
  333, 107
  338, 43, 254
  339, 57
  340, 56, 237, 239, 271
  346, 32
  348, 238
  350, 68, 235, 237
  355, 236, 239
  359, 58, 74, 80, 235, 256, 271
  362, 238

RELAY, 44, 54, 55, 58, 81, 165–167, 209, 242, 265, 269
RELAY_EARLY, 44, 54, 55, 58, 62, 63, 67, 81, 83, 84, 209–212, 255, 265, 269
responder, 35

torrc
  __LeaveStreamsUnattached, 147, 150
  AccountingMax, 176, 211
  AssumeReachable, 155, 156
  AutomapHostsOnResolve, 142
  BridgeDistribution, 130, 180, 181
  ClientOnionAuthDir, 171, 172
  ContactInfo, 176
  ControlPort, 142, 145
  CookieAuthentication, 142
  DirAuthority, 155, 262
  DirPortFrontPage, 197
  DNSPort, 87, 142
  EntryNodes, 141
  ExcludeNodes, 141
  ExitNodes, 141
  ExitPolicy, 181
  ExitPolicyRejectPrivate, 177
  ExitPolicyRejectPrivate 0, 177
  ExitRelay 0, 176
  FamilyId, 181
  HashedControlPassword, 142
  HiddenServiceDir, 169, 170, 173, 174
  HiddenServiceNonAnonymousMode, 173

HiddenServicePort, 117, 170, 174
HiddenServiceSingleHopMode, 173
HSLayer2Nodes, 107
HSLayer3Nodes, 107
IsolateSOCKSAuth, 184, 205
KeepalivePeriod, 36
Log, 167
MaxCircuitDirtiness, 141
MaxMemInQueues, 122, 127, 128, 261
MyFamily, 179–181, 265
OfflineMasterKey, 181
OfflineMasterKey 1, 176
ReducedExitPolicy, 177, 181
RelayBandwidthBurst, 176
RelayBandwidthRate, 122, 176, 266
SafeLogging, 142, 143, 166, 197, 233
SafeSocks, 217, 218, 269
SessionGroup, 141, 263
SigningKeyLifetime, 41
SocksPort, 140, 141, 143
StrictNodes, 141
TestingDirAuthVoteGuard, 155, 263
TestingLinkCertLifetime, 41
TestingTorNetwork, 155, 160, 233, 262
TestSocks, 216, 218
TransPort, 142
UseBridges, 262
V3AuthDistDelay, 96
V3AuthNIntervalsValid, 96
V3AuthVoteDelay, 96
V3AuthVotingInterval, 96, 155, 264

VERSIONS, 37, 44, 45, 51, 52, 54, 58, 60–62, 158, 159, 166, 231, 242, 255, 256, 265
VPADDING, 37, 45, 54